

OpenSG-2.0

Mechanics of Structure Genome homogenization in JAX
User Tutorial

Akshat Bagla
Purdue University

Abstract

OpenSG-2.0 computes the macroscopic constitutive law of a heterogeneous structure – a beam 6×6 , a plate ABD, or an equivalent 3-D solid 6×6 – from a structure gene (SG): the smallest piece of the structure that carries its heterogeneity. It also runs the calculation backwards, recovering pointwise 3-D stress inside that SG from macroscopic loads. Two engines share the theory and the file conventions: `msg-shell` works on shell structure genes (a composite cross-section contour, a thin sheet meshed as a surface, a tapered blade segment), and `msg-solid` works on solid structure genes in one, two or three dimensions. Every run has the same shape: a YAML (or SwiftComp `.sc`) file goes in, and a timed output file in the SwiftComp `.K` layout comes back.

This tutorial covers the architecture of both engines, the input file formats and sign conventions, the constitutive relations each capability produces, a worked tutorial for every example shipped with the code, and the full wind turbine blade pipeline from a windIO file to BeamDyn and back to recovered wall stress.

Contents

1	Overview and Architecture	2
1.1	What OpenSG-2.0 is	2
1.1.1	The universal contract	2
1.1.2	One command: <code>opensg</code>	3
1.1.3	The cores hold no paths	4
1.2	The theory in one page	5
1.2.1	The structure gene and the strain split	5
1.2.2	The quadratic form	5
1.2.3	The zeroth-order solve	5
1.2.4	The first-order step: transverse shear	6
1.2.5	One factorization for two solves	6
1.2.6	Dehomogenization: running the chain backwards	7
1.3	Module map	7
1.4	The pipelines	7
1.4.1	<code>msg-shell</code> : 1-D ring $\rightarrow$ Timoshenko 6×6	7
1.4.2	<code>msg-shell</code> : 3-D segment $\rightarrow$ aperiodic Timoshenko 6×6	11
1.4.3	<code>msg-shell</code> : 3-D shell SG $\rightarrow$ equivalent solid C_{3D}	12
1.4.4	<code>msg-solid</code> : one driver, three macro models	14
1.5	GPU readiness	16
2	Input Formats and Conventions	17
2.1	The 1-D shell SG YAML	17
2.1.1	<code>nodes</code>	18
2.1.2	<code>elements</code>	19
2.1.3	<code>sets</code> and <code>sections</code>	19
2.1.4	<code>elementOrientations</code>	20
2.1.5	<code>materials</code>	20
2.1.6	<code>reference</code>	21
2.1.7	Running it	21
2.2	The 3-D shell SG YAML	22
2.3	The solid SG YAML	24
2.3.1	The header contract	25
2.4	The SwiftComp <code>.sc</code> input and the converter	27
2.4.1	From a gmsh surface mesh: <code>msh_to_yaml</code>	29
2.5	The through-thickness <code>layup_db.yaml</code>	29
2.6	Finite elements used by each route	32
2.6.1	Shell route	32

2.6.2	Solid route	32
2.7	Sign and order conventions	33
2.7.1	Solid (3-D) Voigt order	33
2.7.2	Beam order	33
2.7.3	Plate order	34
2.7.4	Changing the reference surface	34
2.8	Material orientation	34
2.8.1	Where the ply angle lives	36
3	Constitutive Relations Recovered by OpenSG	37
3.1	The .out contract	38
3.2	Beam: the Timoshenko 6×6	39
3.2.1	Running it on a shell cross-section	40
3.2.2	Running it on a solid cross-section	41
3.2.3	The classical 4×4 that rides along	42
3.2.4	Which routes produce the beam law	43
3.3	Plate: classical ABD 6×6 and Reissner–Mindlin ABDG 8×8	43
3.3.1	Classical (Kirchhoff–Love) plate law	43
3.3.2	Reissner–Mindlin plate law	44
3.4	Solid: the equivalent 3-D law	47
3.4.1	The engineering-constants block	47
3.4.2	Which entry points produce it	47
3.4.3	Periodicity, and the aperiodic alternative	48
3.4.4	A worked solid law: the TPMS Sample 1 cell	49
3.5	The wall law: per-section 8×8 ABDG	51
3.6	Where each law is produced and written	53
3.7	Running the laws backwards: two-step recovery	54
3.7.1	The shell chain: beam force to plate resultants to 3-D stress	54
3.7.2	What the shell chain gets right, and where it does not	55
3.7.3	The solid chain	55
4	Tutorials: the Shell Engine (msg-shell)	58
4.1	Beam properties from a shell cross-section	58
4.1.1	What this capability computes	58
4.1.2	What a 1-D shell SG is	59
4.1.3	The input file the user edits	59
4.1.4	Orientation frames	60
4.1.5	The element and the wall law	61
4.1.6	Run it	62
4.1.7	The one call	62
4.1.8	What comes out	63
4.1.9	The numbers	64
4.2	Two-step recovery to 3-D wall stress	64
4.2.1	What this capability computes	64
4.2.2	The input the user edits	65
4.2.3	Run it	65
4.2.4	Junction-aware recovery	66
4.2.5	The two-step chain	67

4.2.6	What comes out	68
4.2.7	Reading the result	68
4.2.8	What this route recovers, and what it does not	70
4.3	Equivalent 3-D solid from a cross-section SG	70
4.3.1	What this capability computes	70
4.3.2	The driver	71
4.3.3	Validation: Deo & Yu (2023)	72
4.3.4	Why C_{23} vanishes on axis-aligned lattices	77
4.4	Equivalent 3-D solid from a 3-D shell SG	77
4.4.1	What this capability computes	77
4.4.2	The input the user edits	78
4.4.3	Run it	80
4.4.4	What comes out	80
4.4.5	Boundary treatment: periodic default versus aperiodic	81
4.5	Tapered and aperiodic segments	82
4.5.1	What this capability computes	82
4.5.2	Seeing the capability	84
4.5.3	The input the user edits	86
4.5.4	Run it	86
4.5.5	What comes out	86
4.5.6	Acceptance test: the prismatic identity	87
4.5.7	Tapered demonstration: BAR-URC segment 12	87
4.5.8	Shear-scheme rule for segments	88
5	Tutorials: the Solid Engine (msg-solid)	89
5.1	Plate properties from a through-thickness 1-D SG	90
5.1.1	What it computes	90
5.1.2	The input you edit	91
5.1.3	Run it	92
5.1.4	What comes out	92
5.1.5	What the numbers mean	92
5.1.6	What it is validated against	93
5.2	Through-thickness stress recovery	94
5.2.1	What it computes	94
5.2.2	Driver A: one station, resultants typed into the script	95
5.2.3	Driver B: a whole field from a .ff station table	95
5.2.4	What comes out	96
5.2.5	What it is validated against	97
5.3	Plate properties from a 2-D cell SG	99
5.3.1	What it computes	99
5.3.2	The example SG and the input you edit	99
5.3.3	Run it	101
5.3.4	What comes out	101
5.3.5	What the numbers mean	101
5.3.6	The Reissner-Mindlin option	103
5.3.7	The 1-D gene in the same folder, and a naming trap	103
5.4	Plate recovery from a 2-D SG	103
5.4.1	What it computes	103

5.4.2	The input you edit	104
5.4.3	Run it	104
5.4.4	What comes out	104
5.4.5	Reading the result	105
5.5	Equivalent 3-D solid from a 2-D SG	106
5.5.1	What it computes	106
5.5.2	The example SG	108
5.5.3	Step 1: build the SG yaml from the gmsh mesh	108
5.5.4	Step 2: homogenize	108
5.5.5	What comes out	109
5.5.6	What the numbers mean, and what they were validated against	109
5.6	Equivalent 3-D solid from a 3-D SG	110
5.6.1	What it computes	110
5.6.2	The samples and the runners	111
5.6.3	Normalization: the .out is already per unit cell	113
5.6.4	The SwiftComp digit-parity benchmark	113
5.6.5	Periodic versus aperiodic	114
5.7	Beam properties from a 2-D SG	114
5.7.1	What it computes	114
5.7.2	The example SG and the input you edit	115
5.7.3	Run it	117
5.7.4	What comes out	117
5.7.5	What the numbers mean	117
5.7.6	The built-in cross-check: Euler-Bernoulli against Timoshenko	118
5.8	Beam recovery	118
5.8.1	What it computes	118
5.8.2	The input you edit	119
5.8.3	Run it	120
5.8.4	What comes out	120
5.8.5	What the numbers mean	120
5.9	Files at a glance	123
6	Full Blade Pipeline: windIO to BeamDyn and Back	124
6.1	The worked example	124
6.2	Stage 1: windIO blade to per-station 1-D shell SG YAMLS	126
6.3	Stage 2: each station to a Timoshenko 6×6	128
6.4	Stage 3: handing the table to BeamDyn	133
6.5	Stage 4: the loads come back as the FF table	136
6.6	Stage 5: dehomogenization back to 3-D wall stress	137
6.7	Where each tool ends	142

Chapter 1

Overview and Architecture

1.1 What OpenSG-2.0 is

OpenSG-2.0 is an implementation of Mechanics of Structure Genome (MSG) homogenization written in JAX. You give it a *structure gene* (SG) — the smallest piece of a structure that carries its heterogeneity — and it returns the macroscopic constitutive law of the structure that SG builds: a Timoshenko beam 6×6 , a plate ABD (or the shear-refined 8×8), or an equivalent 3-D solid 6×6 . It also runs the same calculation backwards (*dehomogenization*), rebuilding the pointwise 3-D stress and strain inside the SG from macroscopic beam or plate loads.

The repository is organised in four layers. Only the top layer knows where files live on disk.

Table 1.1: The four layers of OpenSG-2.0. Paths and `main` blocks exist only in the top row.

Layer	Package	Role
user	<code>examples/</code>	YAML in $\rightarrow$ <code>.out</code> out; owns all paths
msg-shell	<code>src/opensg_shell</code>	Reissner–Mindlin shell SGs: ring (cross-section), aperiodic segment, 3-D shell cell
msg-solid	<code>src/opensg_solid</code>	solid SGs: plate ABD, beam KKT, 3-D solid law
FE core	<code>src/fe_jax</code>	JAX FE architecture: basis/quadrature, periodic assembly map, jitted element kernels, element-by-element (EBE) Chebyshev-CG, sparse direct

1.1.1 The universal contract

Every capability in both engines obeys the same contract: **a YAML file (or a SwiftComp `.sc` file) goes in, and a timed `.out` file in the SwiftComp `.K` layout comes back.** There is exactly one writer for that file, `opensg_solid.sg_homo.write_sc_K`, and it is called from inside the solver source, not from the example scripts, so no run can forget it. A minimal session looks like this:

```
from opensg_shell import build_rm_bundle

B = build_rm_bundle("iea_s10_shell.yaml") # writes iea_s10_shell_Timo.out
B["Timo"]                                # the Timoshenko 6x6 as a numpy array
```

Listing 1.1: The whole contract: the `.out` file is written as a side effect of the call.

The file that comes back always has the same shape: an **OpenSG** banner naming the model and the SG measure ω on the first line, the effective stiffness matrix, its inverse (the compliance), an engineering-constants block for 3-D laws only, and **Time taken** on the last line. Numbers are printed in a 19-character right-justified Fortran field (`%.7E` mantissa, signed three-digit exponent) so the file drops straight into existing SwiftComp comparison tooling:

```
OpenSG msg-shell beam model [ext sh2 sh3 twist bend2 bend3]

The Effective Timoshenko Beam Stiffness Matrix
-----
      <6 numbers per row>
...
The Effective Timoshenko Beam Compliance Matrix
-----
...
Time taken: 12.34 sec
```

The suffix on the output file names the model, so two different routes run on the same input YAML do not silently overwrite each other (Table 1.2).

Table 1.2: Output file names. There is one file per model; the `_Timo` vs `_C3D` suffix disambiguates when two routes can read the same YAML.

Model	Entry function	Output file
msg-solid beam / plate / 3-D	<code>plate_homo_2d</code>	<code><base>.out</code>
msg-shell beam (Timoshenko)	<code>build_rm_bundle</code>	<code><yaml>_Timo.out</code> , <code><yaml>_ABDG.out</code>
msg-shell aperiodic segment	<code>segment_timo_from_3dyaml</code>	<code><yaml>_Timo.out</code>
msg-shell solid props, cross-section SG	<code>build_solid_bundle</code>	<code><yaml>_C3D.out</code> , <code><yaml>_ABDG.out</code>
msg-shell 3-D shell SG	<code>shell_sg3d</code>	<code><yaml>_C3D.out</code> , <code><yaml>_ABDG.out</code>

1.1.2 One command: `opensg`

You do not have to know which engine owns a file in order to run it. The package installs one dispatcher, `opensg`, alongside the two engine-specific commands `opensg_solid` and `opensg_shell`; all three take the same one or two arguments:

```
opensg <sg.yaml>      # homogenization -- the default
opensg <sg.yaml> H     # homogenization, stated explicitly
opensg <sg.yaml> D     # dehomogenization: homogenize, then recover
```

`opensg (src/opensg/cli.py)` is a pure dispatcher and prints nothing of its own. It resolves the engine with `opensg_solid.sg_mesh.resolve_msg` — the YAML’s `msg:` key (`shell` or `solid`) or, when that key is absent, the mesh *dialect* (`nodes/cells/mat_id` is `solid`, `nodes/elements/sets/sections/elementOrientations` is `shell`) — and hands the untouched argument list to `opensg_shell.cli.main` or `opensg_solid.cli.main`. A `msg:` that contradicts the dialect is an error, not a silent override, and a file written before the key existed still runs. The engine-specific commands remain and behave identically, as do `python -m opensg_solid <yaml>`

and `python -m opensg_shell <yaml>`. Everything else — the macro model, the classical/shear-refined switch, the macro state for a recovery — lives in the YAML header, which is why the command line needs no flags.

Every run prints the same header, so the terminal states what was actually read before any number appears:

```
=====
OpenSG -- a multiscale structural analysis tool based on the Mechanics of
Structure Genome (MSG), developed by the Multiscale Structural Mechanics
Group led by Prof. Wenbin Yu at Purdue University
=====

input      : /abs/path/to/iea_s10_shell.yaml
msg        : shell
SG dim     : 2D
analysis   : homogenization
macro model: beam, shear-refined

Timoshenko Beam Stiffness Matrix [eps11 gam12 gam13 kappa1 kappa2 kappa3]:
[[ 2.76606e+10 ... ]]
Homogenization stored in iea_s10_shell_Timo.out
Time taken: 2.58 sec
```

Three lines of that header repay reading. `msg` is the engine that was resolved, so a mis-tagged file is visible on the first line of output. `SG dim` is the **ambient** dimension — the space the SG occupies, `node_span_dim`, not the intrinsic dimension of the manifold — so a Schwarz-P shell surface prints 3D while a cross-section ring and a 2-D solid cell both print 2D. `macro model` echoes `n_model` and `refined`, and gains a trailing `,` `aperiodic` when the header sets `aperiodic`. For a dehomogenization the two closing lines are instead `Local field files are computed and stored.` and a single `Time taken`, covering the whole run.

1.1.3 The cores hold no paths

Neither `opensg_shell` nor `opensg_solid` contains a `main` block, a hard-coded directory, or an assumption about where you run from. Every path decision lives in the example script, which names its input file and calls exactly one entry function on it. That is why the tutorials in the later chapters can be reduced to two lines of user input and one call: to run a different cross-section you change the string, not the library.

Two pieces of process-level setup do live in the package `__init__` files, and both matter to you:

- **Double precision is load-bearing.** Both packages execute `jax.config.update("jax_enable_x64", True)` at import. Without it JAX silently degrades to float32 and the digits of the 6×6 are wrong. Import the package (`import opensg_shell`) before you import anything else that touches JAX.
- **A persistent compile cache.** `opensg_solid/__init__.py` points JAX's compilation cache at `$JAX_COMPILATION_CACHE_DIR`, defaulting to `~/.cache/opensg_jax`, with `jax_persistent_cache_min_compile_time_secs` set to 0. A cold run pays each XLA compile once; warm runs load the binaries back. The cache is wrapped in a `try/except` — it is an optimisation, and the engine must run without it. The same file honours `$FE_JAX_CORE`,

which prepends an external `fe_jax` checkout to `sys.path` to override the bundled copy at `src/fe_jax`.

1.2 The theory in one page

Every pipeline in this manual is the same variational statement, specialised. It is worth reading this section once, because the symbols used here are the variable names used in the source: `Dee`, `Dhe`, `Dhh`, `V0`, `V1`, `omega`.

1.2.1 The structure gene and the strain split

A structure gene is the smallest piece of the structure that carries the heterogeneity: a laminate stacking sequence through the thickness (a 1-D SG), a cross-section contour or a 2-D unit cell (a 2-D SG), a lattice or TPMS unit cell (a 3-D SG). The displacement field inside the SG is split into a macroscopic part driven by the macro strains $\bar{\varepsilon}$ and a fluctuation (warping) field w . In strain form,

$$\varepsilon = \Gamma_e \bar{\varepsilon} + \Gamma_h w, \quad (1.1)$$

where Γ_e is the operator that embeds the macro strains into the SG and Γ_h is the SG's own strain operator acting on the warping. The two are not independent: Γ_e is Γ_h evaluated on the macro embedding, a consistency requirement written down in `Rules/gamma_e_gamma_h_consistency.md` and enforced by building both from the same element kernel.

1.2.2 The quadratic form

Substituting (1.1) into the strain energy gives the quadratic form the code assembles, in exactly the block names used throughout the source:

$$2U = \bar{\varepsilon}^T \mathbf{D}_{ee} \bar{\varepsilon} + 2w^T \mathbf{D}_{he} \bar{\varepsilon} + w^T \mathbf{D}_{hh} w. \quad (1.2)$$

Read it as three integrals over the SG: $\mathbf{D}_{ee}$ is the macro-macro block (what you would get with no warping at all, the rule-of-mixtures answer), $\mathbf{D}_{hh}$ is the SG stiffness seen by the warping alone, and $\mathbf{D}_{he}$ couples them.

1.2.3 The zeroth-order solve

Minimising (1.2) over w , subject to the SG's boundary treatment (periodicity, Dirichlet boundary data, or a rigid-body constraint that removes the null space) gives one linear system with as many right-hand sides as there are macro strains,

$$\mathbf{D}_{hh} \mathbf{V}_0 = -\mathbf{D}_{he}, \quad (1.3)$$

and the effective law follows by substituting the minimiser back:

$$\mathbf{D}_{\text{eff}} = \mathbf{D}_{ee} + \mathbf{V}_0^T \mathbf{D}_{he}, \quad \mathbf{C} = \mathbf{D}_{\text{eff}} / \omega. \quad (1.4)$$

Here ω is the SG measure that stays in the model — the thickness of a 1-D laminate SG, the area of a cross-section SG, the volume of a 3-D solid cell, the perimeter (or, in three dimensions, the midsurface area) of a shell SG. The banner of every `.out` states ω because a bare matrix

is ambiguous: a whole-matrix ratio that is uniform across all entries (exactly 0.5000, say) is a normalisation difference, not a physics discrepancy.

$\mathbf{V}_0$ is stored, not discarded. Its columns are the warping fields per unit macro strain, and they are what the dehomogenization reads back.

Equation (1.4) is the **zeroth-order** answer: the plate ABD, the Euler–Bernoulli 4×4 , the equivalent 3-D solid law. It contains no transverse shear, because the classical macro models it corresponds to have none.

1.2.4 The first-order step: transverse shear

Recovering transverse shear — the Reissner–Mindlin G block of a plate, the GA_{12} and GA_{13} entries of a Timoshenko 6×6 — takes a **first-order** step: a second solve for $\mathbf{V}_1$ against a right-hand side built from $\mathbf{V}_0$, followed by an energy transformation. Two functions do this, and they are shared by every beam route in the repository:

- `prepare_v1_rhs(V0, Dh1, D11, D1e_dense, Psi, Dc)` forms the first-order right-hand side and the auxiliary blocks $\mathbf{V}_0^T \mathbf{D}_{1l} \mathbf{V}_0$, $\mathbf{D}_{hl} \mathbf{V}_0$ and $\mathbf{D}_{hl}^T \mathbf{V}_0 + \mathbf{D}_{le}$, projecting the right-hand side onto the range of $\mathbf{D}_{hh}$ so the singular system stays consistent.
- `finalize_v1_and_compute_deff(V1s_raw, V0, D_eff, V0D11V0, Dh1V0, Dh1TV0D1e, Psi, Dc)` projects $\mathbf{V}_1$, symmetrises $\mathbf{D}_{\text{eff}}$, and performs the generalized-Timoshenko transformation of the energy blocks (A, B, C, D) into the sorted 6×6 .

Both live in `opensg_shell/fe_jax/msg_solver.py` and are `@jax.jit`-decorated. The output ordering of the 6×6 is `[EA, GA12, GA13, GJ, EI2, EI3]`, i.e. the strain measures $[\varepsilon_{11} \ \gamma_{12} \ \gamma_{13} \ \kappa_1 \ \kappa_2 \ \kappa_3]$, which is the VABS diagonal order.

The symmetrisation inside `finalize_v1_and_compute_deff` is not cosmetic. $\mathbf{D}_{\text{eff}}$ is a Hessian and therefore symmetric by construction, but a Dirichlet-constrained tapered-segment solve produces a small (2 to 6 percent) antisymmetric artifact, because with $\mathbf{V}_0$ pinned at the end rings $\mathbf{D}_{ee} + \mathbf{V}_0^T \mathbf{D}_{he}$ is no longer the exact symmetric Schur complement. Left alone, that artifact propagates through the inverse into the antisymmetric shear–bending coupling (C_{26}, C_{35}) and inflates it by up to 78 percent on anisotropic tapers while leaving the symmetric terms untouched. Enforcing the invariant is a no-op for a prismatic SG.

1.2.5 One factorization for two solves

Equations (1.3) and the $\mathbf{V}_1$ system share the same operator. Every beam route therefore factorizes once and back-substitutes twice. In `sg_homo.ring_indep` the augmented KKT matrix

$$\begin{bmatrix} \mathbf{D}_{hh} & \mathbf{D}_c \\ \mathbf{D}_c^T & \mathbf{0} \end{bmatrix} \begin{Bmatrix} \mathbf{V} \\ \lambda \end{Bmatrix} = \begin{Bmatrix} \mathbf{R} \\ \mathbf{0} \end{Bmatrix}$$

is built once, passed to `scipy.linalg.lu_factor`, and then `lu_solve` is called with $\mathbf{R} = -\mathbf{D}_{he}$ for $\mathbf{V}_0$ and with the `prepare_v1_rhs` output for $\mathbf{V}_1$. In the tapered-segment route the same idea appears as `sg_assembly.dirichlet_factor` followed by two `dirichlet_solve_fac` calls; the second call falls back to a fresh factorization only if the $\mathbf{V}_1$ Dirichlet set differs from the $\mathbf{V}_0$ one. In the msg-solid beam engine the equivalent is `solve_fluctuation_field`, which returns the assembled KKT matrix alongside $\mathbf{V}_0$ precisely so the caller can reuse it; there the constraint rows are scaled by 10^8 before assembly to keep the saddle-point system well conditioned, and empty rows are given a unit diagonal with a zeroed right-hand side.

1.2.6 Dehomogenization: running the chain backwards

Given macro strains (or macro forces, converted through $\bar{\varepsilon} = \mathbf{C}^{-1}\mathbf{F}$), the warping is rebuilt from the stored $\mathbf{V}_0$ and $\mathbf{V}_1$, equation (1.1) is evaluated pointwise, and the material law of each element turns that strain into 3-D stress. Nothing is re-solved: homogenize once, recover as often as you like. For the shell engine this is a *two-step* recovery — step 1 rebuilds the six shell strains plus the two wall transverse shears $[\varepsilon_{11} \ \varepsilon_{22} \ 2\varepsilon_{12} \ \kappa_{11} \ \kappa_{22} \ 2\kappa_{12}]$ and $[2\gamma_{13} \ 2\gamma_{23}]$ on the ring, and step 2 pushes them through the through-thickness plate SG to get ply-level 3-D stress. The entry points are `opensg_shell.stress_at_points` and `opensg_shell.disp_at_points` for the shell engine, and `opensg_solid.sg_dehom.dehom_fields / plate_dehom_2d` for the solid engine.

1.3 Module map

Both packages are split file-per-concern with the same naming, so a routine you know in one engine is in the same-named file in the other.

The shared FE core `src/fe_jax` is not an OpenSG concern but a general JAX finite-element architecture. The pieces the two engines actually import are `basis_quadrature` (`FiniteElementType`, `get_quadrature`, `eval_basis_and_derivatives`, all backed by `basix`), `setup.mesh_to_jax` and `setup.mesh_to_periodic_sparse_assembly_map`, `solve_cg` (a conjugate-gradient solver that returns convergence information), `linear_elasticity`, and `profiling`.

1.4 The pipelines

Four routes are described here in the order a new user meets them. Each is a sequence of calls that the entry function makes on your behalf; you only ever call the entry function. Figure 1.1 puts all four side by side: one input file, one entry function, one `.out` file per branch.

1.4.1 msg-shell: 1-D ring $\rightarrow$ Timoshenko 6×6

This is the composite blade cross-section route. The input is a 1-D shell SG YAML: nodes and two-node elements tracing the wall contour, `elementOrientations`, `sections` carrying a layup per element set, and `materials`. The entry function is `build_rm_bundle(shell_yaml, ref=None, shear="mitc4_g23", g_source=None)`, and it proceeds as follows.

1. `sg_mesh.load_ring_ref` reads the YAML and returns the contour arrays — nodal coordinates `rx`, connectivity `cells`, the per-element section index `rsub`, the element normals `re3`, the axial index `ax` and the two cross-section indices `cross`. The laminate reference surface comes from the YAML's own `reference` field (absent $\rightarrow$ `center`, the mid-surface), so homogenization and recovery cannot disagree; pass `ref` explicitly only to override. The reference is converted once into a thickness fraction `frac` (`center` $\rightarrow$ 0.5, `oml` $\rightarrow$ 0.0, `oml_flip` and `iml` $\rightarrow$ 1.0) and that single number then drives the ring laminate reference, the plate-SG z reference, the emitted ABD and the recovery depth conversion.
2. `sg_materials._material_by_section` builds, per section, the wall plate ABD 6×6 at the chosen reference and the 2×2 transverse-shear G .
3. The wall G of every section is then replaced by `rm_plate_msg`, the MSG/VAM second-order-energy (Yu-2002 least-squares) projection in `opensg_solid.rm_plate_1D`, run at the same

Table 1.3: `src/opensg_shell` — the MSG shell engine.

File	Concern
<code>sg_mesh.py</code>	SG input loaders and mesh utilities: 1-D ring YAML loaders (<code>load_ring</code> , <code>load_ring_ref</code>) with explicit laminate-reference conventions; 3-D segment loading and topological boundary extraction (<code>extract</code>) into solver <code>.npz</code> bundles; orientation-frame numeric reports and PNG prechecks.
<code>sg_materials.py</code>	Section laws: the per-section wall plate ABD 6×6 plus the 2×2 transverse-shear G (<code>_material_by_section</code>), and the per-station wall-law emitter <code>emit_station_abd</code> / <code>load_station_abd</code> that writes one windIO-style YAML of 8×8 Reissner–Mindlin plate laws per cross-section.
<code>sg_assembly.py</code>	Shell element operators and assembly: the production 6-DOF drilling element, the 5-DOF general taper element, the prismatic MITC4 element, MITC assumed-strain tying, the <code>compute_k22</code> / <code>compute_kg</code> initial-curvature corrections, the Dirichlet factorizations (<code>dirichlet_factor</code> , <code>dirichlet_solve_fac</code>), and the six-block MSG energy assembly (<code>assemble_segment_indep</code> , <code>assemble_constraint</code>).
<code>sg_periodicity.py</code>	Periodic local-to-global sparse assembly map for a shell mesh, 6 DOF per node ($w_1, w_2, w_3, \omega_1, \omega_2, \omega_3$) instead of the solid engine’s three translations. The tie is a plain index map; no transformation matrix is formed.
<code>sg_homo.py</code>	Homogenization drivers: <code>ring_indep</code> (ring $\rightarrow$ Timoshenko 6×6), <code>build_rm_bundle</code> (the packaged bundle the dehom consumes), <code>ring_solid</code> / <code>build_solid_bundle</code> (cross-section $\rightarrow$ equivalent 3-D solid), <code>shell_sg3d</code> (3-D shell SG $\rightarrow$ C3D), <code>segment_timo_from_3dyaml</code> (aperiodic / tapered segment), <code>elastic_constants</code> .
<code>sg_dehom.py</code>	Reissner–Mindlin two-step dehomogenization and recovery: <code>stress_at_points</code> , <code>disp_at_points</code> , built on the same element operators as the homogenization so it is the energy-consistent adjoint.
<code>sg_junction.py</code>	Level-2 junction law: local corner and micro-cell solves (<code>corner_micro_law</code> for L/T/X topologies, <code>microcell_law</code> for a periodic mini-lattice) that correct the shell lattice for sub-shell junction physics.
<code>fe_jax/</code>	The OpenSG-authored <code>msg_*</code> kernels: <code>msg_materials</code> (ABD and 6×6 tables, reference shifts), <code>msg_transverse_shear</code> (wall G , the 8×8 plate law), <code>msg_mesh</code> , <code>msg_rm/msg_rm_timo</code> (RM operators and the 5-DOF Timoshenko assembly), <code>msg_solver</code> (KKT solve, $\mathbf{V}_1$ RHS, Timoshenko finalization), <code>msg_hermite</code> (the Hermite C^1 Kirchhoff–Love thin-walled pipeline), <code>msg_dehom</code> , <code>orient_plot</code> .
<code>helper/</code>	Format converters (<code>ff_to_yaml.py</code>).

Table 1.4: `src/opensg_solid` — the general structure-gene engine.

File	Concern
<code>sg_materials.py</code>	The 6×6 stiffness tables, per material and per element: <code>build_material_C</code> and <code>get_heterogeneous_C_matrix</code> , plus the in-plane rotation. Voigt order everywhere is SwiftComp (xx, yy, zz, yz, xz, xy) .
<code>sg_mesh.py</code>	SG input parsing and mesh utilities: <code>load_sg_input</code> (a SwiftComp <code>.sc</code> or the helper’s YAML), <code>_cell_basis</code> (SG dimension plus nodes-per-element $\rightarrow$ basix cell type and basis degree), and <code>plot_sg_mesh</code> , which draws the actual mesh coloured by material.
<code>sg_assembly.py</code>	Element kernels, periodic assembly, preconditioners and linear solvers: <code>calculate_RHS_and_Ke_batch_periodic</code> , <code>compress_periodic_cells_jax</code> , <code>compute_block_inv_diag</code> , <code>apply_block_precond</code> , <code>apply_chebyshev_precond</code> , <code>ebe_jacobian_product_periodic</code> , <code>estimate_max_eigenvalue</code> , <code>_sparse_direct_solve</code> , <code>solve_fluctuation_field</code> , <code>plate_ladder_element_blocks</code> .
<code>sg_homo.py</code>	Homogenization drivers and the one public entry <code>plate_homo_2d</code> : the periodic CG pipeline (<code>full_homogenization_pipeline</code> , <code>_homo_direct</code>) for <code>n_model</code> 2 and 3, the beam KKT path (<code>_beam_homo_kkt</code>) for <code>n_model</code> 1, the Reissner–Mindlin first-order plate ladder (<code>plate_shear_ladder</code>), and the single output writer <code>write_sc_K</code> .
<code>sg_dehom.py</code>	Dehomogenization: the Gauss-point recovery kernel for <code>n_model</code> 2 and 3, the generalized-Timoshenko beam recovery chain for <code>n_model</code> 1, the public <code>dehom_fields</code> and <code>plate_dehom_2d</code> , and the Gauss exports (<code>export_gauss</code>). The batched recovery is wrapped in a module-level <code>jit</code> so repeated calls in one process do not re-trace.
<code>rm_plate_1D/</code>	The 1-D through-thickness plate stack used by both engines: <code>segment_plate.plate_sg_yaml</code> (build the through-thickness SG from a stacking sequence), <code>msg_rm_plate.rm_plate_msg</code> (the 8×8 ABDG and the warping ladders), <code>rm_dehom</code> (ply-level recovery), <code>rm_homo</code> , <code>cli_rm_plate</code> .

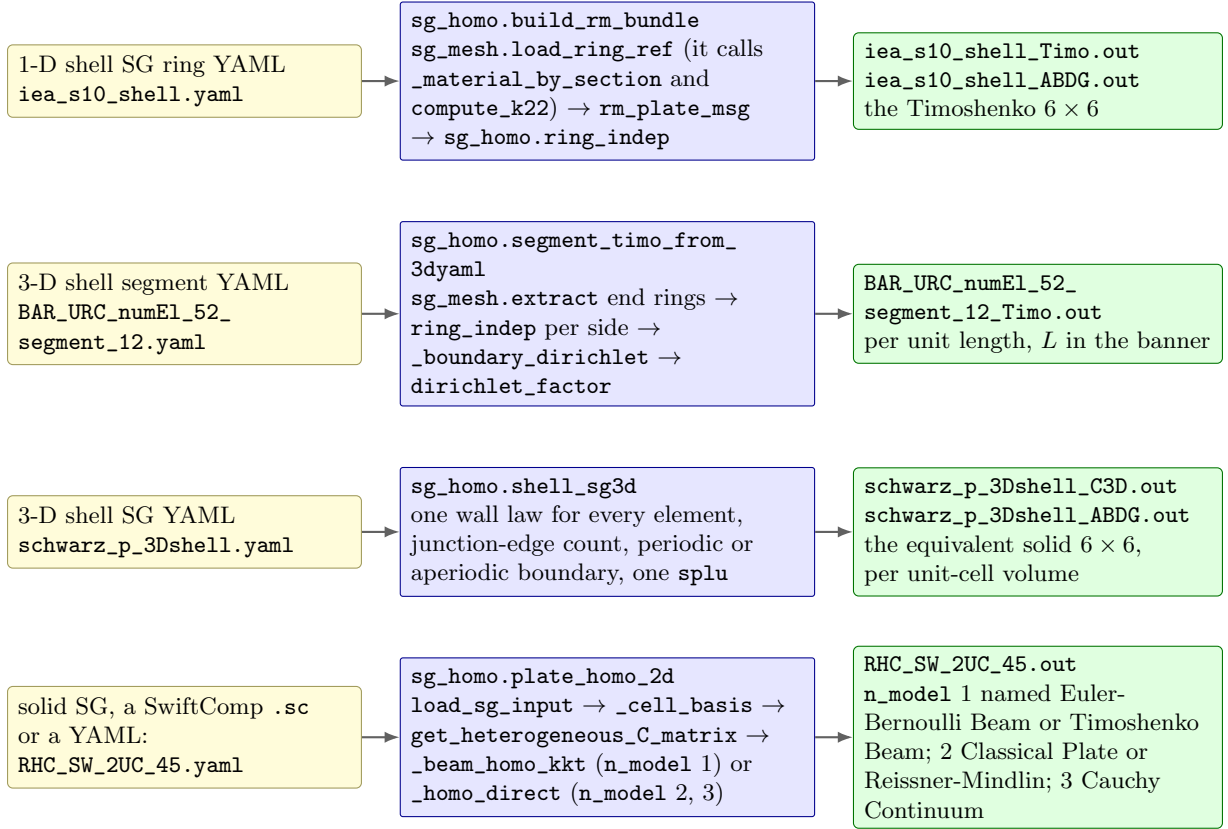


Figure 1.1: The four pipelines, one row each: the three msg-shell routes and the msg-solid driver whose `n_model` argument selects the macro model. Yellow is the file you edit, blue the entry function with the calls it makes in order, green the `.out` file it leaves behind. All four `.out` files are written by the one writer `opensg_solid.sg_homo.write_sc_K`, which is why they share a single layout; the `_ABDG.out` companion — written by every route that reduces a layup to a wall plate law, the cross-section rings and the 3-D shell SG alike — is the one exception, written by `write_abdg_out`.

`fraction=frac`. **This is the only route and there is no switch.** The retired `g_source` selector is still accepted by the signature so existing callers keep working, but any value other than `msg` raises a `DeprecationWarning` and is *ignored* (`sg_materials.check_g_source`), and an emitted ABD cache records `g_source: msg`. The complementary-energy (Whitney-1973) shear flow survives only as the internal numerical fallback on the rare laminate whose MSG U^* fit is not positive definite — a guard, not a user-selectable alternative. The per-section laws are written to `<yaml>_ABDG.out`.

4. `sg_assembly.compute_k22` computes the hoop curvature per edge, the initial-curvature correction that makes a curved wall stiffer than a faceted approximation of it.
5. `sg_homo.ring_indep` assembles the ring as a one-element-deep strip of quads: the 6-DOF drilling element $(w_1, w_2, w_3, \theta_1, \theta_2, \omega_3)$ with an element-wise Lagrange multiplier enforcing the drilling constraint, and MITC tying of γ_{23} only. (Under span invariance γ_{13} is algebraic in the directors and carries no fluctuation gradient, so only γ_{23} pairs a differentiated displacement with a rotation.) The six energy blocks `Dhh`, `Dhe`, `Dee`, `Dhl`, `Dll`, `Dle` come out of `assemble_segment_indep` and the drilling rows `Gc`, `G1`, `Ge` out of `assemble_constraint`; all are divided by the artificial strip depth h .
6. The KKT matrix is factorized once; $\mathbf{V}_0$ and $\mathbf{V}_1$ are obtained by two back-substitutions, and `finalize_v1_and_compute_deff` returns the generalized-Timoshenko 6×6 , symmetrised.
7. `write_sc_K` writes `<yaml>_Timo.out`, and the returned bundle carries `Timo`, `V0`, `V1`, the contour geometry, the strip, the per-element layup names and the layup and material databases — everything the two-step dehomogenization needs.

The shipped example is `examples/OpenSG_shell/1_get_beam_props_from_shell_cross_section`, whose input is `iea_s10_shell.yaml` (the IEA-22 blade at $r/R = 0.2$):

```
cd examples/OpenSG_shell/1_get_beam_props_from_shell_cross_section
opensg iea_s10_shell.yaml
```

The YAML carries `n_model: 1`, `refined: 1` and `msg: shell`, so that one command runs the Timoshenko route and writes `iea_s10_shell_Timo.out` (the Timoshenko stiffness and compliance) and `iea_s10_shell_ABDG.out` (the per-section wall laws). The printed diagonal is $[EA, GA_2, GA_3, GJ, EI_2, EI_3]$. The shipped driver `beam_homo_shell.py` makes the same call through `build_rm_bundle` and additionally draws the two figures the CLI does not — `iea_s10_mesh.png` (the contour coloured by layup) and `iea_s10_orient.png` (the compulsory $e_1/e_2/e_3$ orientation figure, emitted by `auto_emit`) — so run the script when you want those:

```
python beam_homo_shell.py
```

1.4.2 msg-shell: 3-D segment $\rightarrow$ aperiodic Timoshenko 6×6

The ring pipeline assumes the beam is prismatic: periodic along the axis, so one cross-section suffices. A tapered segment is not periodic, so the periodicity condition has to be replaced. The entry function is `segment_timo_from_3dyaml(seg_yaml, workdir=None, lam_space="elem", shear="full")` and the input is a 3-D shell segment YAML (nodes, quads, sections, materials).

1. `sg_mesh.extract` finds the two end cross-sections *topologically*: a mesh edge used by exactly one quad is a free edge, and the connected components of the free-edge graph are the end

rings. Each is written out as a standalone 1-D SG YAML (so you can inspect or re-run it on its own) together with an orientation PNG, and the whole bundle — segment coordinates, cells, subdomains, the three element frame vectors, and the node-to-segment map — is saved to `<base>_boundaries.npz`.

2. Each end ring is solved as a ring in its own right by `ring_indep`, giving `C6`, `V0` and `V1` per side.
3. The full 3-D quad mesh is assembled with the same `assemble_segment_indep` and `assemble_constraint`, with the element hoop curvature `k22_e` and geometric curvature `kg_e` computed from the element centroids and frames. The drilling multipliers are appended as extra rows and columns by `_augment_drilling`, giving a saddle-point system with zero penalty.
4. `_boundary_dirichlet` maps the ring warping fields onto the segment's end nodes. This is the boundary condition that replaces periodicity: $\mathbf{V}_0 = \mathbf{V}_{0,\text{ring}}$ and $\mathbf{V}_1 = \mathbf{V}_{1,\text{ring}}$ on both end sections.
5. `dirichlet_factor` factorizes the constrained interior once; $\mathbf{V}_0$ and then $\mathbf{V}_1$ are solved against it. The Euler–Bernoulli block is $A_{EB} = (\mathbf{D}_{ee} + \mathbf{V}_0^T \mathbf{D}_{he})/L$ with L the axial extent of the nodes, and every energy block handed to `finalize_v1_and_compute_deff` is likewise divided by L , so the result is a per-unit-length stiffness.
6. `<yaml>_Timo.out` is written with the segment length in its banner; the returned dict carries `S6`, `C6L`, `C6R`, `L` and `solve_time`.

Two acceptance checks come with this route. On a *prismatic* segment the segment 6×6 must reproduce its own boundary ring 6×6 ; measured on an isotropic and a ± 45 tube this identity holds to $\leq 5 \times 10^{-5}$ relative. On a genuinely tapered segment no identity holds, and the check is instead that each diagonal term of the segment lies between the two ring values — the ratio $(S_{ii} - R_{ii})/(L_{ii} - R_{ii})$ printed by the example must fall in $[0, 1]$. The shipped case is a BAR-URC 100 m blade segment:

```
cd examples/OpenSG_shell/5_taper_segment_aperiodic_boundaries
python run_bar_urc_segment.py
```

which reads `BAR_URC_numEl_52_segment_12.yaml` and writes `BAR_URC_numEl_52_segment_12_Timo.out`, `BAR_URC_numEl_52_segment_12_boundaries.npz`, the two boundary YAMLs `boundary..._L.yaml` and `boundary..._R.yaml`, three orientation PNGs, and the comparison table `bar_urc_segment12.dat`.

1.4.3 msg-shell: 3-D shell SG $\rightarrow$ equivalent solid C_{3D}

Here the SG is a surface in three dimensions — a triply periodic minimal surface, a lattice of sheets — and the answer wanted is the 6×6 of the equivalent homogeneous 3-D solid. The entry function is `shell_sg3d(yaml_path, omega=None, drill_pen=1.0e-3, g_source=None, solver="direct", boundary=None, shear="mitc")`. `boundary=None` means *read it from the YAML header*, where the optional `aperiodic: 1` key is the opt-out — the same key `opensg_solid.plate_homo_2d` reads, so the two CLIs share one header contract; an explicit argument always wins.

1. The YAML's nodes, elements and `elementOrientations` are read; the element normal e_3 is columns 7 through 9 of the orientation rows. Triangles and quads may be mixed: they are assembled as two separate element batches (MITC3 and MITC4) scattered into the same global system, never padded into one another.
2. The single wall law (ABD at the mid-surface reference plus the 2×2 G , again from `rm_plate_msg`, the only G route) is built once and applied to every element.
3. Junction lines are counted: any shell edge shared by more than two elements. The count appears in the `.out` banner, because it tells you how much of the cell's stiffness comes from junction physics the plain shell lattice does not resolve.
4. The boundary treatment. With `boundary="periodic"`, the default, the periodic map ties opposite faces, edges and corners through the sparse assembly map, and the three rigid translations are removed by three area-weighted Lagrange rows. With `boundary="aperiodic"` the macro (affine) field is mapped onto the boundary instead, prescribing zero *translational* fluctuation $w_1 = w_2 = w_3 = 0$ on every node of the bounding-box faces while the rotational DOFs stay free. Clamping the rotations as well over-stiffens the cut edges; freeing them was measured to move C_{11} from +12 to +7 percent. The aperiodic result is a kinematic upper bound on a single cell and converges to the periodic one as the SG grows.
5. The batched Γ_h and Γ_e operators are evaluated at the Gauss points, contracted into the sparse $\mathbf{K}$, $\mathbf{D}_{he}$ and $\mathbf{D}_{ee}$, with a drilling stabilisation of `drill_pen` $\times A_{11}$ on the ω_3 DOF. One `scipy.sparse.linalg.splu` factorization then serves all six macro strain columns.
6. $\mathbf{D}_{\text{eff}} = \mathbf{D}_{ee} + \mathbf{V}_0^T \mathbf{D}_{he}$ is symmetrised and divided by ω . The default ω is the *volume the equivalent continuum occupies*, i.e. the node bounding-box volume measured from the mesh — the same convention `opensg_solid` uses, and the same cell the periodic assembly map ties — so the returned C3D, the printed matrix and `<yaml>_C3D.out` are one and the same matrix, directly comparable with a solid `.K`. Pass `omega` explicitly (or set the YAML `omega:` header key) only when the continuum occupies something else; the midsurface *surface area*, for a per-unit-area wall law, is returned alongside as `surface_area`.
7. The step-1 record — the 8×8 Reissner–Mindlin wall plate law and its compliance, one block per section — is written to `<yaml>_ABDG.out` by the same `write_abdg_out` the cross-section routes use.

Because the wall law is a Reissner–Mindlin plate law, the sheet thickness is a parameter of the material model rather than of the mesh: the same surface mesh serves any thickness, which is what makes a thickness sweep cheap. Figure 3.5 shows the Schwarz-P unit cell that the shipped example uses.

```
cd examples/OpenSG_shell/4_get_solid_props_from_shell_3D_SG
python make_schwarz_yaml.py          # generates schwarz_p_3Dshell.yaml
opensg schwarz_p_3Dshell.yaml        # homogenizes it
```

The first command is the mesh generator and has no CLI equivalent; the second is the plain homogenization, which the YAML's own header (`n_model: 3, msg: shell`) fully describes. The run prints the junction-edge count, the number of DOF and the solve time, then the 6×6 ; it writes `schwarz_p_3Dshell_C3D.out` and `schwarz_p_3Dshell_ABDG.out`. The driver `schwarz_solid_props.py` makes the same call through `shell_sg3d` and additionally writes `schwarz_p_mesh.png`.

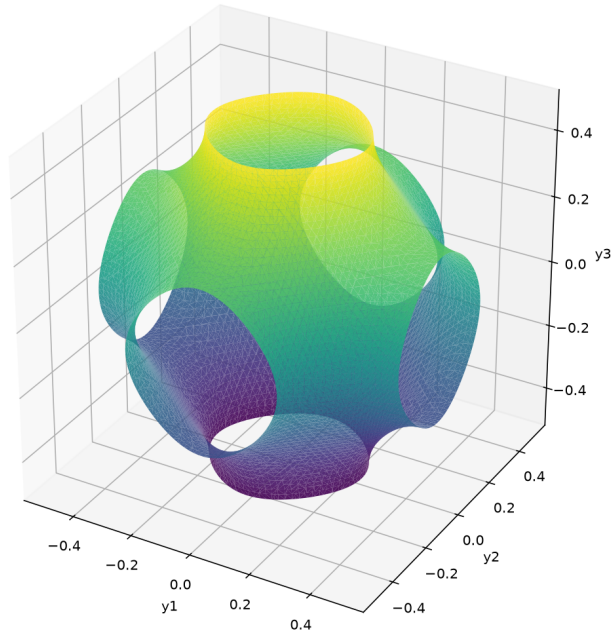


Figure 1.2: The Schwarz-P triply periodic minimal surface meshed as a 3-D shell structure gene. The surface carries a Reissner–Mindlin wall law, so sheet thickness is a material parameter and one mesh serves an entire thickness sweep. Produced by `schwarz_solid_props.py` in `examples/OpenSG_shell/4_get_solid_props_from_shell_3D_SG`.

The companion script `compare_boundary_modes.py` runs the same cell under both boundary treatments and tabulates them in `boundary_compare.dat`. This route is validated against its own solid counterpart: for the Schwarz-P cell the shell SG sits within -3 percent on the normal moduli, -6 percent on the shears and $+0.3$ percent on Poisson’s ratio relative to the SwiftComp-exact msg-solid answer.

1.4.4 msg-solid: one driver, three macro models

The msg-solid engine has a single public entry,

```
from opensg_solid.sg_homo import plate_homo_2d

r = plate_homo_2d(sc_path,          # a .sc or .yaml SG file
                  material_param=None, # (n_mat, 9) engineering override
                  angles=None,        # deg per material
                  n_model=None,       # 1 beam, 2 plate, 3 3-D solid
                  refined=None,       # 0 classical, 1 shear-refined
                  workdir=None,
                  elem_rotation=None, # (E, 9) per-element direction cosines
                  solver="direct",    # "direct" | "cg"
                  shear_refined=False, # legacy alias of refined=1 (plate)
                  plot=True,          # writes <base>_mesh.png
                  boundary=None,      # "periodic" (default) | "aperiodic"
                  density=None,       # (n_mat,) beam mass rho
                  omega=None)         # user SG measure (3-D solid)
```

Listing 1.2: The msg-solid driver signature: `n_model` selects which macro strain set is built.

and the argument that selects the physics is `n_model`: it decides which macro strain set Γ_e is built. `n_model`, `refined`, `boundary` and `omega` all default to `None`, meaning *take it from the YAML header*, so a self-describing SG file needs no arguments at all — that is what makes `opensg <yaml>` equivalent to the call. The steps shared by all three macro models are:

1. `sg_mesh.load_sg_input` parses the `.sc` or `YAML` into a dict carrying `dim` (the SG dimension, 1, 2 or 3), `nodes`, `cells`, `mat_id` and the materials. That `dim` is a *parse result*, not something you write: the input contract has no `dim` key and no `scale` key, and both the dimension and the SG measure ω are read from the mesh itself. With `plot=True` it also writes `<base>_mesh.png` of the actual mesh, elements coloured by material. Mixed element types in one SG are rejected — the solver runs one homogeneous batch.
2. `sg_mesh._cell_basis` maps (SG dimension, nodes per element) to a basix cell type and basis degree; `fe_jax.basis_quadrature` then supplies the quadrature points `xi_qp`, weights `W_q`, basis values `phi_qn` and parametric gradients `dphi_dxi_qnp`. The quadrature degree is 6 for higher-order elements and 2 for linear ones.
3. `fe_jax.setup.mesh_to_jax` produces the element-node coordinate array `x_end` of shape (E, N, d) , and `mesh_to_periodic_sparse_assembly_map` produces the master connectivity and the DOF map that carries periodicity. Periodicity rides entirely in the assembly map; no transformation matrix is formed. With `boundary="aperiodic"` the map is the identity and instead every bounding-box-face node gets $w = 0$ Dirichlet; that option is available for the plate and solid models with `solver="direct"` only.
4. `sg_materials.get_heterogeneous_C_matrix` builds the per-element 6×6 table `C_ess` of shape $(E, 6, 6)$, applying `angles` and `elem_rotation` (per-element frames are honoured by all three models, not only the beam).

From there the routes diverge:

n_model=1, beam. Control goes to `_beam_homo_kkt`: the four Euler–Bernoulli macro modes $[\varepsilon_{11} \ \kappa_1 \ \kappa_2 \ \kappa_3]$ are solved under four rigid-body Lagrange constraints, then the first-order chain runs and the generalized-Timoshenko reduction produces the 6×6 . The result dict carries `C_eff` (the Timoshenko 6×6) and `C_eff_EB` (the classical 4×4 from the same run). The output is `<base>.out`, named *Timoshenko Beam* with `refined: 1` and *Euler-Bernoulli Beam* with `refined: 0`, with ω in the banner. This path needs an SG of dimension ≥ 2 .

n_model=2, plate. The macro strain set is $\bar{\varepsilon} = [\varepsilon_{11} \ \varepsilon_{22} \ 2\varepsilon_{12} \ \kappa_{11} \ \kappa_{22} \ \kappa_{12}]$ and the answer is the plate ABD, written as *Classical Plate*. With `refined: 1` (or the legacy `shear_refined=True`) the first-order warping ladder `plate_shear_ladder` runs as well, the result gains `G_msg` (2×2), `ABDG` (the 8×8 with the shear block on the diagonal) and `A6_ladder`, and the file is named *Reissner-Mindlin*.

n_model=3, solid. The macro strain set is the six 3-D strains and the answer is the equivalent solid 6×6 , named *Cauchy Continuum* and written with the orthotropic-approximated engineering constants appended ($E_1, E_2, E_3, G_{12}, G_{13}, G_{23}, \nu_{12}, \nu_{13}, \nu_{23}$).

Models 2 and 3 share the assembly and solve code (`_homo_direct`), so switching between a plate law and a solid law on the same mesh is a one-character change. The shipped examples are numbered by capability under `examples/OpenSG-solid/`: `1_get_plate_props_from_1DSG` (a stacking

sequence in `layup_db.yaml` → `layup_db_plate_homo.out`, run with `python 1d_sg.py layup_db.yaml`, `3_get_plate_props_from_2D_SG`, `6_get_solid_props_from_3D_SG`, and `7_get_beam_props_from_SG`, whose script is:

```
cd examples/OpenSG-solid/7_get_beam_props_from_SG
python beam_homo_sg.py
```

That one reads `RHC_SW_2UC_45.yaml` (a two-unit-cell honeycomb cross-section) with `n_model = 1` and three materials overridden through `material_param` and `angles`, and writes `RHC_SW_2UC_45.out` plus `RHC_SW_2UC_45_mesh.png`.

1.5 GPU readiness

The msg-solid engine has two interchangeable linear solvers, and the choice is the `solver` argument of `plate_homo_2d`.

`solver="direct"` is the default. The jitted element kernels build the sparse matrix, which is handed as a CSR to `_sparse_direct_solve` — pypardiso (Intel MKL PARDISO) when it is importable, SciPy’s SuperLU otherwise. One factorization serves all macro strain columns. This is the fastest route on a workstation, and it is the only stage of the calculation that leaves the accelerator.

`solver="cg"` is the GPU route. The element kernels (`calculate_RHS_and_Ke_batch_periodic` and the jitted `jacfwd` stiffness), the element-by-element operator (`ebe_jacobian_product_periodic`), the block and Chebyshev preconditioners (`compute_block_inv_diag`, `apply_block_precond`, `apply_chebyshev_precond`, `estimate_max_eigenvalue`) and the CG solver itself (`fe_jax.solve_cg`) are pure JAX and run device-resident: no global matrix is ever formed, so memory scales with the element batch rather than with the bandwidth of an assembled operator. On a GPU host, `plate_homo_2d(..., solver="cg")` therefore keeps the entire solve on the device.

Switching solvers is digit-safe. The effective law $\mathbf{C}_{\text{eff}}$ is *stationary* in $\mathbf{V}_0$ — equation (1.4) evaluates the energy at its own minimiser — so an error δ in the fluctuation field enters the answer at $O(\delta^2)$. A solver tolerance loose enough to be visible in $\mathbf{V}_0$ is still invisible in the printed matrix. Both routes are documented to produce the same digits.

The msg-shell routes are one step behind. Their operators are assembled as batched einsum contractions in the two-step form $\mathbf{CB}$ then $\mathbf{B}^T(\mathbf{CB})$, and those batches are written against the NumPy/jnp API and are portable to the device; the sparse factorization (SuperLU for the ring and the 3-D shell SG, pypardiso for the KKT solve in `msg_solver`) is the remaining CPU stage. For the SG sizes these routes handle — a cross-section contour is a few thousand DOF — that factorization is not the bottleneck.

Chapter 2

Input Formats and Conventions

OpenSG-2.0 reads three kinds of structure-gene (SG) description, plus one laminate shorthand and one legacy SwiftComp converter; a second converter turns a gmsh surface mesh into the shell dialect (Section 2.4.1). Which dialect you write is decided by the geometry you have, not by the answer you want: a thin-walled cross-section contour is a 1-D shell SG, a sheet meshed as a surface in space is a 3-D shell SG, and anything meshed as a filled domain (a laminate through its thickness, a honeycomb unit cell, a lattice block) is a solid SG. The five dialects are listed in Table 2.1.

Table 2.1: The five input dialects, the package that reads each, and the example file shipped with the code.

dialect	read by	example file
1-D shell SG (cross-section contour)	<code>opensg_shell</code>	<code>iea_s10_shell.yaml</code>
3-D shell SG (surface in space)	<code>opensg_shell</code>	<code>schwarz_p_3Dshell.yaml</code>
solid SG (1-D / 2-D / 3-D mesh)	<code>opensg_solid</code>	<code>RHC_SW_2UC_45.yaml</code>
SwiftComp <code>.sc</code>	converted to the solid SG YAML	<code>RHC_SW_2UC_45.sc</code>
through-thickness layup database	<code>opensg_solid.rm_plate_1D</code>	<code>layup_db.yaml</code>

The two shell dialects share one schema — the same six top-level keys, differing only in element dimension — while the solid dialect is a separate, deliberately minimal schema whose material blocks mirror SwiftComp’s.

Every run, in every dialect, writes a timed `.out` in the SwiftComp `.K` layout. The writer is `opensg_solid.sg_homo.write_sc_K` and it always emits, in order: an `OpenSG` banner line naming the model, `The Effective <Name> Stiffness Matrix`, `The Effective <Name> Compliance Matrix`, and a `Time taken: <s> sec` footer. The `<Name>` infix is *always* the macro law’s own console title, so the file and the terminal read the same; the five names are listed in Table 2.2. Only the 3-D solid law is written with `constants=True` and therefore also prints `The Engineering Constants (Approximated as Orthotropic)`. The full contract is `Rules/output_format_and_timing.md` at the repository root.

The per-section *wall* plate laws are a different writer, `opensg_shell.sg_homo.write_abdg_out`, and keep the title `The Effective Reissner-Mindlin Plate Stiffness Matrix` — one 8×8 block per section in `<base>_ABDG.out`.

2.1 The 1-D shell SG YAML

This is the cross-section of a thin-walled beam: the wall midline (or mold line) meshed with 2-node line segments, each segment carrying a laminate. It is the input to `opensg_shell.build_`

Table 2.2: The five <Name> infixes `write_sc_K` emits. The name is the macro law’s console title, so the terminal and the `.out` agree word for word.

macro law	<Name>
3-D solid (Cauchy continuum) 6×6	Cauchy Continuum
beam 6×6 , shear-refined	Timoshenko Beam
beam 4×4 , classical	Euler-Bernoulli Beam
plate ABD 6×6 , classical	Classical Plate
plate ABDG 8×8 , shear-refined	Reissner-Mindlin

`rm_bundle`, which returns the Timoshenko 6×6 .

The example `iea_s10_shell.yaml` in `examples/OpenSG_shell/1_get_beam_props_from_shell_cross_section/` is station $r/R = 0.2$ of the IEA-22 blade: 307 nodes, 310 line elements, 6 layups, 6 materials. Its top-level keys, in the order OpenSG’s own writers emit them, are listed in Table 2.3.

Table 2.3: Top-level keys of the 1-D shell SG YAML.

key	content
<code>nodes</code>	one bracketed, space-separated triple per node: the two cross-section coordinates then the beam-axis coordinate
<code>elements</code>	one bracketed, space-separated pair per line element, 1-based node ids
<code>sets</code>	<code>element</code> : a list of { <code>name</code> , <code>labels</code> } groups; <code>labels</code> are 1-based element indices
<code>sections</code>	one entry per layup, tied to a set through <code>elementSet</code> , carrying the <code>layup</code> ply table
<code>elementOrientations</code>	one row of nine direction cosines per element, a genuine YAML list of floats
<code>materials</code>	named orthotropic materials with <code>density</code> and an <code>elastic</code> block of <code>E</code> , <code>G</code> , <code>nu</code> triples
<code>reference</code>	one scalar naming the surface the wall laws are referenced to: <code>center</code> , <code>oml</code> , <code>oml_flip</code> or <code>iml</code>

2.1.1 nodes

One row per node, three coordinates. In the shell dialect the row is written as a bracketed, *space-separated* triple — a single YAML scalar inside a flow sequence — and not as a comma list:

```
nodes:
- [4.70852238 0.09722431 0.00000000]
- [4.64563825 0.11238283 0.00000000]
```

Listing 2.1: The first two node rows of `iea’s10’shell.yaml`.

The space-separated form is required, not merely conventional: the orientation plotter `fe_jax/orient_plot.py` parses each row with `str(row[0]).split()`, so a comma-separated list of floats will not plot. The loader `sg_mesh.load_ring_ref` is more forgiving — its `_row` helper replaces commas by spaces before splitting — but writing the space form keeps every consumer happy at once.

The component order is fixed and is the source of most first-run confusion. The loader sets `ax = 2` and `cross = [0, 1]`, meaning the **third column is the beam axis** and the first two columns are the in-plane cross-section pair (y_2, y_3) . For a plane cross-section the beam-axis column is identically zero, exactly as in the excerpt above. If you are writing a cross-section by hand and your third column is not all zeros, you have written a segment, not a cross-section.

2.1.2 elements

One row per element, the node ids of a 2-node line segment, 1-based, again space-separated inside brackets:

```
elements:
- [1 2]
- [2 3]
```

Listing 2.2: The first two element rows; element k here is element label k everywhere else in the file.

The loader detects the base with `if cells.min() == 1: cells = cells - 1`, so a 0-based mesh is also accepted, but every generator in the repository writes 1-based. Element k in this list is element label k in `sets`, and row k of `elementOrientations` belongs to it. The contour need not be a single loop: the last element of the IEA station is `- [307 65]`, which closes a shear web back onto a skin node.

2.1.3 sets and sections

`sets` groups element labels by name; `sections` gives each named group its laminate. The tie between them is the `elementSet` field, which must equal a set name:

```
sets:
  element:
    - name: layup_0
      labels:
        - 1
        - 2
        - 3
sections:
- type: shell
  elementSet: layup_0
  layup:
    - - gelcoat
      - 0.0005
      - 0.0
    - - glass_triax
      - 0.00300294
      - 0.0
    - - glass_uniax
      - 0.02609467
      - 0.0
    - - glass_triax
      - 0.00300294
      - 0.0
```

Listing 2.3: Abridged sets and sections blocks. In the real file `layup_0` lists 25 labels (elements 1-12 and 209-221) and there are six sets and six sections.

Labels are 1-based element indices into `elements`. `sg_materials._material_by_section` builds one ABD per entry of `sections` and keys it by the section's *position in the list*; the loader then maps every element through `rsub[label - 1] = section_index`. An element that appears in no set silently falls to section 0, so every element should appear in exactly one set. `type: shell` is descriptive metadata — the shell readers key only off `elementSet` and `layup`.

Each `layup` row is `[material, thickness, angle_deg]`: a material name that must exist in `materials`, a ply thickness in metres, and a ply angle in degrees. The list is ordered **outer face first** and stacks inward along e_3 : `compute_ABD_matrix` documents its input as layer thicknesses bottom to top with the reference surface at that bottom (outer) face, and `emit_station_abd` records the same order in its `convention` string as `plies OML->IML`. The sum of the row thicknesses is the wall thickness used for the reference shift; for `layup_0` above that is $0.0005 + 0.00300294 + 0.02609467 + 0.00300294 = 0.03260055$ m, which is the `thickness` the cached ABD file reports for that `layup`.

2.1.4 elementOrientations

One row of nine direction cosines per element, $[e_1(3) \ e_2(3) \ e_3(3)]$, in the same component order as the node rows. Unlike `nodes` and `elements`, these rows are true YAML lists of floats:

```
elementOrientations:
- [0.0, 0.0, 1.0, -0.9721541292349604, 0.2343423756204075, 0.0, -0.2343423756204075,
  -0.9721541292349604, 0.0]
```

Listing 2.4: out of plane, then the in-plane e_2 and e_3 .]One orientation row: $e_1 = [0,0,1]$ out of plane, then the in-plane e_2 and e_3 .

The meaning of the triad is covered in Section 2.8.

2.1.5 materials

A list of named materials, each with a `density` and an `elastic` block holding three-component E , G and ν :

```
materials:
- name: glass_triax
  density: 1940.0
  elastic:
    E:
      - 28211400000.0
      - 16238800000.0
      - 15835500000.0
    G:
      - 8248220000.0
      - 3491240000.0
      - 3491240000.0
    nu:
      - 0.497511
      - 0.18091
      - 0.27481
```

Listing 2.5: One material block from `iea's10'shell.yaml`.

The triples are $[E_1, E_2, E_3]$, $[G_{12}, G_{13}, G_{23}]$ and $[\nu_{12}, \nu_{13}, \nu_{23}]$ — the argument order of `fe_jax.msg_materials.build_stiffness_6x6`, which fills the orthotropic compliance

$$S_{11} = 1/E_1, \quad S_{12} = -\nu_{12}/E_1, \quad S_{13} = -\nu_{13}/E_1, \quad S_{23} = -\nu_{23}/E_2,$$

$$S_{44} = 1/G_{23}, \quad S_{55} = 1/G_{13}, \quad S_{66} = 1/G_{12},$$

and inverts it to the 6×6 stiffness. Units follow the input; the examples are SI, so moduli are in Pa, thicknesses in m and **density** in kg/m^3 . **density** feeds the section mass per unit area that **emit_station_abd** reports as **mass_per_area** (62.892567 kg/m^2 for **layup_0** of this station); it is not needed for the stiffness.

2.1.6 reference

```
reference: center
```

Listing 2.6: The whole reference block – one scalar, the last line of the file.

This single scalar names the surface the wall ABD — and therefore the whole cross-section model — is referenced to, and it is the single source of truth. **build_rm_bundle(shell_yaml)** reads it when its **ref** argument is left **None** via **ref = d.get("reference", "center")**, and the value is immediately turned into a thickness fraction **frac = {"center": 0.5, "oml": 0.0, "oml_flip": 1.0, "iml": 1.0}** that then drives the ring laminate reference, the plate-SG **z_ref**, the emitted ABD file, and the depth conversion used in stress recovery. Absent, it defaults to **center**. The four accepted values and the fraction each maps to are listed in Table 2.4.

Table 2.4: Values of **reference** and the thickness fraction each maps to.

value	meaning	fraction from the outer face
center	contour is the laminate mid-surface; ABD shifted by $t/2$	0.5
oml	contour is the outer mold line; laminate stacks inward, no shift	0.0
oml_flip	diagnostic: reference moved the full thickness the other way	1.0
iml	contour is the inner mold line; laminate stacks outward	1.0

Mid-surface meshes (tubes, ellipses, generated cross-sections) use **center**; airfoil YAMLS written on the OML use **oml**. Getting this wrong shifts **B** and **D** by a parallel-axis term of the wall thickness and moves every bending and torsion entry of the beam 6×6 , which is why it is recorded in the file rather than passed at the call site.

2.1.7 Running it

```
cd examples/OpenSG_shell/1_get_beam_props_from_shell_cross_section
opensg_iea_s10_shell.yaml
```

The file is the whole problem: its header carries **n_model: 1**, **refined: 1** and **msg: shell**, so that one command picks the msg-shell engine and the Timoshenko route without a flag, and writes **iea_s10_shell_Timo.out** and **iea_s10_shell_ABDG.out**. The shipped driver **beam_homo_shell.py** makes the same call through the API and adds the two figures the CLI does not draw. It sets **name = "iea_s10"**, calls **build_rm_bundle(name + "_shell.yaml")**, and prints the 6×6 with the diagonal labelled EA GA2 GA3 GJ EI2 EI3:

```
from opensg_shell import build_rm_bundle, auto_emit

name = "iea_s10"
B = build_rm_bundle(name + "_shell.yaml")
C6 = np.asarray(B["Timo"])
```

```

LBL = ["EA", "GA2", "GA3", "GJ", "EI2", "EI3"]
print("diagonal: " + " ".join("%s=%.5g" % (LBL[i], C6[i, i]) for i in range(6)))

auto_emit(name + "_shell.yaml", out_png=name + "_orient.png")

```

Listing 2.7: The whole compute part of beam'homo'shell.py.

Four files come back. `iea_s10_shell_Timo.out` holds the Timoshenko 6×6 and its compliance under the banner `OpenSG msg-shell beam model [ext sh2 sh3 twist bend2 bend3]`; for this station the diagonal reads $EA = 2.766 \times 10^{10}$ N, $GA_2 = 7.186 \times 10^8$ N, $GA_3 = 4.217 \times 10^8$ N, $GJ = 2.438 \times 10^9$ N m², $EI_2 = 3.514 \times 10^{10}$ N m², $EI_3 = 6.813 \times 10^{10}$ N m², and the footer records `Time taken: 2.58 sec.` `iea_s10_shell_ABDG.out` holds one 8×8 RM wall law per section, written by `sg_homo.write_abdg_out` with the header rows `[eps11 eps22 2eps12 | K11 K22 K12+K21 | 2g13 2g23]`, so you can read off, for instance, the $A_{11} = 1.370 \times 10^9$ N/m and the transverse-shear $G_{11} = 8.283 \times 10^7$ N/m of `layup_0` directly. `iea_s10_mesh.png` is the contour coloured by `layup` and `iea_s10_orient.png` is the orientation check discussed in Section 2.8. On the first run `build_rm_bundle` additionally caches the per-station wall laws as `abd/iea_s10_shell_abd.yaml` at the same reference the YAML declared (here written out as `reference: mid`); the write is guarded by an existence test on that path, so later runs reuse the cached file rather than recompute it.

2.2 The 3-D shell SG YAML

Same six keys, but the mesh is a surface embedded in 3-D: `nodes` are genuine three-dimensional points and `elements` are 3-noded triangles or 4-noded quads. There is no `reference` key — `shell_sg3d` calls `_material_by_section` with `center_ref=True` unconditionally, so every wall law is referenced to its mid-surface. This is the input to `opensg_shell.sg_homo.shell_sg3d`, which returns the equivalent 3-D solid 6×6 .

Two files come back, not one. `<base>_C3D.out` carries the equivalent solid law, and `<base>_ABDG.out` carries the *step-1* record that produced it: the 8×8 Reissner–Mindlin wall plate law and its compliance, one block per section, written by the same `write_abdg_out` the cross-section routes use. Every route that reduces a layup to a wall plate law now emits it — the cross-section rings (beam `refined: 1` and the `n_model: 3` equivalent solid) and the 3-D shell SG alike — so the wall law is always auditable next to the answer it fed.

`schwarz_p_3Dshell.yaml` is a Schwarz-P triply-periodic minimal-surface cell: 13536 nodes and 26360 triangles, one aluminium layup at $T = 0.036457$ m with $E = 69$ GPa and $\nu = 0.3$. The surface may hold triangles and quads at once: they are assembled as two *separate* element batches scattered into the same global system, never padded into one another (the old collapsed quad `[n1 n2 n3 n3]` is rejected by `check_element_edges`, because its zero-length edge made the MITC tying point singular). Ids are 1-based as in the 1-D dialect. The mesh itself is shown in Figure 2.1.

`make_schwarz_yaml.py` in the same folder is the canonical way to write one, and its emission loop shows every row format at once. The frame is built per triangle from the geometry: e_3 is the unit facet normal, e_2 is the unit $p_1 \rightarrow p_2$ edge, and $e_1 = e_2 \times e_3$.

```

o = ["nodes:"]
o += ["- [%12f %12f %12f]" % tuple(p) for p in nd]
o.append("elements:")
o += ["- [%d %d %d]" % tuple(e) for e in el]
o.append("elementOrientations:")
for k in range(ne):

```

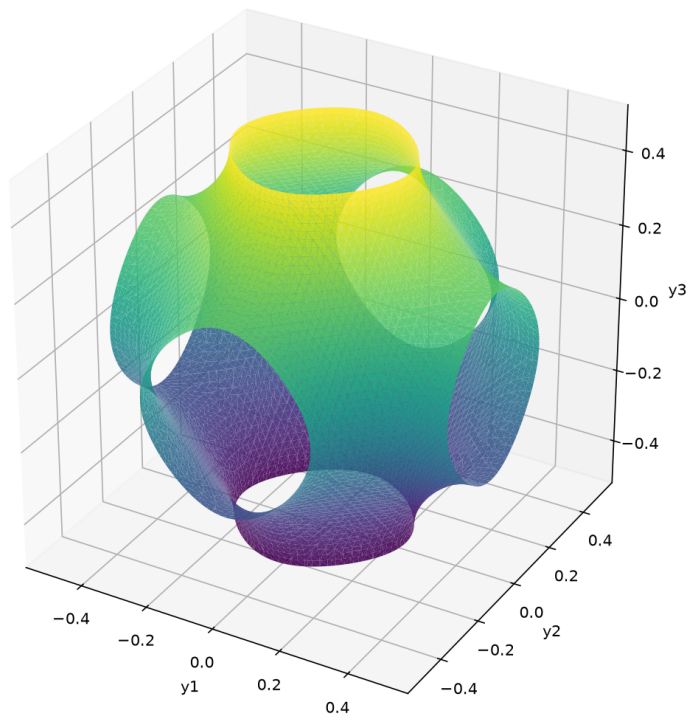


Figure 2.1: The Schwarz-P TPMS 3-D shell SG: 13536 nodes, 26360 triangles, one aluminium layup. The mesh figure is drawn by the driver from the same YAML the solver reads.

```

e1 = np.cross(e2[k], e3[k])
o.append("- [%f, %f, %f, %f, %f, %f, %f, %f, %f]"
         % (*e1, *e2[k], *e3[k]))
o += ["sections:", "- type: shell", " elementSet: layup_0", " layup:",
      " - - alu", " - %.6e" % T, " - 0.0",
      "materials:", "- name: alu", " density: 2700.0", " elastic:",
      " E: [%f, %f, %f]" % (E, E, E),
      " G: [%f, %f, %f]" % (G, G, G),
      " nu: [%f, %f, %f]" % (nu, nu, nu),
      "sets:", " element:", " - name: layup_0", " labels:"]
o += [" - %d" % (e+1) for e in range(ne)]
open("schwarz_p_3Dshell.yaml", "w").write("\n".join(o) + "\n")

```

Listing 2.8: The emission loop of `make_schwarz_yaml.py`.

Note the asymmetry the loop makes explicit and that trips up hand-written files: `nodes` and `elements` rows are space-separated inside brackets, `elementOrientations` rows are comma-separated, and the `sets` labels are 1-based. The generator prints the surface area and the relative density area $\times T$ so the shell idealisation can be checked against the solid cell it replaces.

```

cd examples/OpenSG_shell/4_get_solid_props_from_shell_3D_SG
python make_schwarz_yaml.py          # writes schwarz_p_3Dshell.yaml
opensg schwarz_p_3Dshell.yaml        # homogenizes it

```

The first command is the mesh generator, which has no command-line equivalent; the second is the plain homogenization, and the YAML's header (`n_model: 3, msg: shell`) describes it completely. It writes `schwarz_p_3Dshell_C3D.out`, whose banner records the mesh size and element mix, the junction-edge count, the boundary treatment and the bounding-box volume the law is normalized by. For this cell it reads `OpenSG msg-shell equivalent 3D solid (3-D shell SG, 13536 nodes, 26360 elems [26360 tri3/MITC3], 0 junction edges, periodic in 3 dirs, per unit cell 1)`. Because the 3-D solid law is written with `constants=True`, the file ends with the orthotropic-approximated engineering constants — here $E_1 = 9.2745 \times 10^8$, $E_2 = 9.2749 \times 10^8$, $E_3 = 9.2712 \times 10^8$ Pa and $G_{12} = 1.0395 \times 10^9$ Pa — followed by `Time taken: 10.85 sec`. The near-equality of the three Young's moduli is the physical check: a Schwarz-P cell is cubic-symmetric, so any appreciable spread means the mesh or the frames are wrong. The companion `schwarz_p_3Dshell_ABDG.out` holds the single wall law that fed it: $A_{11} = 2.7643220 \times 10^9$, $A_{12} = 8.2929659 \times 10^8$, $A_{66} = 9.6751264 \times 10^8$, $D_{11} = 3.0617465 \times 10^5$ and the transverse shear $G = 8.5193106 \times 10^8$ N/m. The driver `schwarz_solid_props.py` makes the same `shell_sg3d` call and additionally writes `schwarz_p_mesh.png`.

2.3 The solid SG YAML

The solid dialect describes a meshed *domain* rather than a laminated *surface*, so it carries no layups and no orientations — each material block is already a 6×6 stiffness or a set of engineering constants with a ply angle. It is the **native user input**: you author it directly, and a SwiftComp `.sc` is only one optional way to generate one. `RHC_SW_2UC_45.yaml` is a two-unit-cell honeycomb plate SG with 4251 nodes, 6640 triangles and 3 materials (Figure 2.2). The excerpt below keeps the analysis header, two rows per list and two of the three material blocks; the header keys are collected in Table 2.5, the mesh keys in Table 2.6 and the material types in Table 2.7.

```

n_model: 2          # 1 = beam, 2 = plate, 3 = solid -- the macro model

```

```

refined: 1          # 0 = classical (plate ABD / beam EB); 1 = shear-refined
msg: solid          # the ENGINE that owns the file; 'opensg <yaml>' dispatches on it
nodes:
- [17.6240005, 23.3500004, 0.0]
- [-17.6240005, 23.3500004, 0.0]
cells:
- [0, 66, 345]
- [345, 3, 0]
mat_id: [1, 1, 1, 1, 1]
materials:
  1:
    type: 1
    engineering: [108000.0, 8000.0, 8000.0, 4000.0, 4000.0, 3000.0, 0.32, 0.32, 0.30]
    angle: 45.0
  3:
    type: 1
    engineering: [69000.0, 69000.0, 69000.0, 26540.0, 26540.0, 26540.0, 0.30, 0.30, 0.30]
    angle: 0.0

```

Listing 2.9: Abridged RHC`SW`2UC`45.yaml. The leading scalar keys are the analysis request; the mesh blocks follow.

2.3.1 The header contract

The header is the leading block of scalar keys above the first mesh block. It is the *analysis request* — what to run, not what the SG is — and it is shared by both engines, so `opensg`, `opensg_solid` and `opensg_shell` all read the same keys. **Every key is defaulted**, and a headerless mesh still runs (as a classical plate homogenization).

There is no dim: key and no scale: key. The SG dimension is read from the mesh — the element node count, with ties broken by the leading node-coordinate columns that actually vary — and the SG measure ω is measured from the mesh as well. A legacy `dim:` or `scale:` left in an old file is still tolerated by the loader, but neither is part of the input contract and neither should be written into a new file.

Each material block carries a `type` that selects how the 6×6 is built, matching the SwiftComp material types.

Any block may additionally carry `angle` (degrees — the in-plane ply rotation, applied when the material is built from constants, and the way a ± 45 laminate lives in the file) and `density` (used by the beam route’s mass 6×6). `aux` is the auxiliary line carried through verbatim when a file was converted from an `.sc`; the elastic path does not consume it, and a hand-written YAML simply omits it.

The recommended pattern keeps the ply data in exactly one place — the YAML itself: `type: 1` blocks carrying the nine engineering constants plus their `angle`, and the analysis request in the header. Nothing then has to be passed at the call site, and the run is one command:

```

cd examples/OpenSG-solid/3_get_plate_props_from_2D_SG
opensg RHC_SW_2UC_45.yaml

```

or, through the API, one call:

```

from opensg_solid.sg_homo import plate_homo_2d

name = "RHC_SW_2UC_45"          # reads <name>.yaml

```

Table 2.5: The YAML header contract. Every key is optional; the last three are read by the msg-shell D route only.

key	values	meaning
<code>msg</code>	<code>shell</code> <code>solid</code>	which engine owns the file; omit it and the mesh dialect decides
<code>n_model</code>	<code>1</code> <code>2</code> <code>3</code>	the macro model: 1 beam, 2 plate, 3 3-D solid
<code>refined</code>	<code>0</code> <code>1</code>	0 classical (plate ABD 6×6 / beam EB 4×4), 1 shear-refined (plate ABDG 8×8 / beam Timoshenko 6×6); ignored for the solid macro model
<code>analysis</code>	<code>H</code> <code>D</code>	homogenization or dehomogenization; the command-line argument wins
<code>epsilon_bar</code>	6 floats	the macro state a D run recovers from
<code>omega</code>	float	a user SG measure, overriding the one measured from the mesh
<code>aperiodic</code>	<code>1</code>	zero fluctuation on every bounding-box-face node instead of periodicity; omit the key for the periodic default
<code>junction</code>	<code>off</code> <code>flag</code> <code>exclude</code>	msg-shell D only: the junction-aware recovery tier
<code>junction_bl</code>	float	msg-shell D only: boundary-layer radius in wall thicknesses (default 1.0)
<code>junction_ang</code>	float (deg)	msg-shell D only: tangent-grouping tolerance of the junction detector (default 1.0)

Table 2.6: Mesh keys of the solid SG YAML, as consumed by `opensg_solid.sg_mesh.load_sg_input`.

key	meaning
<code>nodes</code>	comma-separated coordinate rows, always three components. Only the leading <code>n_sg</code> columns are used (<code>points = nodes[:, 0:n_sg]</code>), so a column that never varies — the trailing zeros above — is padding.
<code>cells</code>	element connectivity, 0-based in this dialect (note the 0 in the first row). All elements in one SG must have the same node count; a mixed batch raises mixed element types in one SG are unsupported .
<code>mat_id</code>	one material id per cell, 1-based (the reader forms <code>mat_id - 1</code> internally).
<code>materials</code>	a mapping from that id to a material block.

Table 2.7: Material types in the solid SG YAML and the `.sc` it mirrors.

type	fields	interpretation
0	<code>E</code> , <code>nu</code>	isotropic; $G = E/[2(1 + \nu)]$
1	<code>engineering</code>	the nine constants $E_1 E_2 E_3 G_{12} G_{13} G_{23} \nu_{12} \nu_{13} \nu_{23}$
2	<code>C</code>	the 6×6 stiffness stored directly — typically an already-rotated ply, so no angles should be supplied with it

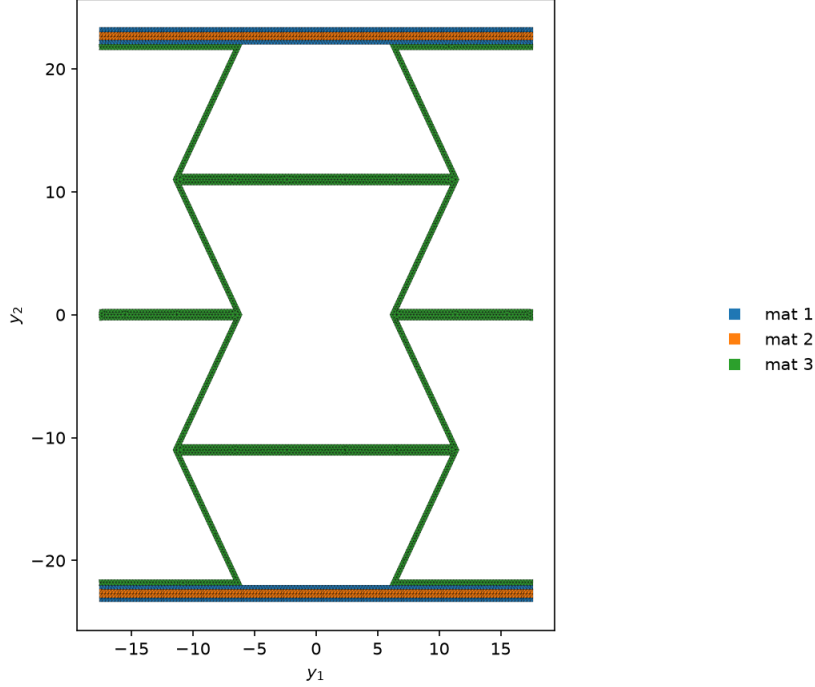


Figure 2.2: The RHC honeycomb 2-D solid SG, elements coloured by material id. This figure is written automatically as `RHC_SW_2UC_45_mesh.png` by `plot_sg_mesh` on every run with `plot=True` (the default), and is drawn from the actual parsed mesh.

```
r = plate_homo_2d(name + ".yaml")      # header + materials come from the file
print(r["law_title"] + ":")
print(r["law"])
```

Listing 2.10: `plate'homo'2dsg.py`, the whole user input and the one call.

(The `material_param=` / `angles=` override arguments remain for scripting over pre-rotated `type: 2` files, as examples 4 and 7 do.) The header's `n_model` picks the macro model — 1 beam, 2 plate, 3 equivalent 3-D solid — and `refined` upgrades the plate ABD to the ABDG, or the beam EB to Timoshenko. This file sets `refined: 1`, so the run writes `RHC_SW_2UC_45.out` — The Effective Reissner-Mindlin Stiffness Matrix, rows [N11 N22 N12 M11 M22 M12 Q1 Q2], under the banner `OpenSG msg-solid plate model, omega 35.248001, periodic` — plus `RHC_SW_2UC_45_mesh.png`. The printed $\omega = 35.248001$ is the in-plane span measured from the mesh: the YAML carries no normalization key, because the homogenizer always measures its own SG. For a cell converted from SwiftComp that measurement agrees with the `.sc` trailing line (35.248 here) to the digits that line carries, and that agreement is the check that the mesh and the `.sc` header describe the same cell. For this run $A_{11} = 3.3586 \times 10^5$, $D_{11} = 9.0368 \times 10^7$ in the file's own units, and the footer reads `Time taken: 1.89 sec`.

2.4 The SwiftComp `.sc` input and the converter

A `.sc` file can be used directly — `load_sg_input` dispatches on the extension and converts — or converted once and kept as YAML. The anatomy the reader expects is documented in the `opensg_`

`solid.helper.sc_to_yaml` module docstring, implemented in its `read_sc`, and summarized in Table 2.8. (The historical `opensg_solid.rm_plate_1D.helper.sc_to_yaml` path still imports: it is now a thin shim that re-exports the module above. New code should use the canonical path.)

Table 2.8: Blocks of a SwiftComp `.sc` file, in file order.

block	content
model header	1 to 3 short lines (submodel flags and similar)
META line	<code>dim n_nodes n_elems n_mats</code> followed by further integers; located as the first line holding at least five integers
node records	<code>n_nodes</code> lines of <code>id x [y [z]]</code>
element records	<code>n_elems</code> lines of <code>id mat_id conn</code> , the connectivity zero-padded to a fixed slot count; the padding zeros are not nodes
material blocks	per material: a header <code>mat_id mat_type n</code> , an auxiliary line such as <code>0 0</code> , then the properties — type 0 is <code>E nu</code> , type 1 is <code>E1 E2 E3 / G12 G13 G23 / v12 v13 v23</code> , type 2 is the 21 upper-triangular constants of the 6×6 over six lines
trailing line	the <code>scale</code> (volume or thickness normalization). It is read and kept on the parsed dict for the cross-check above, but it is <i>not</i> written into the YAML as a <code>scale:</code> key — ω is measured from the mesh. The converter re-emits it as the optional <code>omega:</code> header key in the one case the solver would consume it, a 3-D SG homogenized to a 3-D solid, and then only when it differs from the measured bounding-box volume by more than 10^{-6} relative

In `RHC_SW_2UC_45.sc` the META line is `2 4251 6640 3 0 0` — a 2-D SG with 4251 nodes, 6640 elements and 3 materials — and the file ends with the lone line `35.248`. The file’s last triangle, element 6640 of material 3 on nodes 4251/2988/3079, shows the zero padding in full:

```
6640 3 4251 2988 3079 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

The documented way to run the conversion is the module’s own `convert`:

```
from opensg_solid.helper.sc_to_yaml import convert

sc = convert("RHC_SW_2UC_45.sc")
print(sc["dim"], len(sc["nodes"]), len(sc["cells"]), sc["scale"])
```

Listing 2.11: Converting a `.sc` by hand.

`convert` writes **two** files next to the input — `<base>.yaml`, the solid SG YAML documented above, and `<base>.msh`, a gmsh v2.2 mesh carrying the material id as both physical and elementary tag — and returns the parsed dict. The module defines no `__main__` block, so import `convert` rather than invoking the module with `python -m`. Because `plate_homo_2d` performs the same conversion automatically when handed a `.sc` path, converting by hand is only needed when you want to inspect or edit the YAML before homogenizing.

Two failure modes are worth knowing. The tet10 slot convention is remapped explicitly (four corners, then the six midsides from slots 7–12), and any 3-D record whose nonzero slot count is not 4, 8 or 10 is refused loudly rather than converted into phantom nodes.

2.4.1 From a gmsh surface mesh: msh_to_yaml

The shell twin of the `.sc` converter is `opensg_shell.helper.msh_to_yaml`, which turns an ASCII gmsh `.msh` (format 2.2 or 4.1) *surface* mesh into an msg-shell SG YAML. A gmsh mesh knows the geometry and nothing else, so the converter writes the whole mesh side and never invents the constitutive side. It emits the header, **nodes**, **elements**, one **elementOrientations** row per facet, and one **layup_<k>** element set per gmsh physical tag. The per-facet frame is the one `make_schwarz_yaml.py` established: e_3 is the facet normal, e_2 the first edge, and $e_1 = e_2 \times e_3$, right-handed. Only 3-node triangles (gmsh type 2) and 4-node quads (type 3) are accepted; the 0- and 1-D entities gmsh writes for physical points and curves are skipped, and any other element type is an error.

What you get depends on how much of the layup you supply (Table 2.9).

Table 2.9: The three calling forms of `opensg_shell.helper.msh_to_yaml.convert`.

what you pass	result
nothing (the default)	a marked FILL_IN template : the mesh is complete, sections: and materials: are placeholders. <code>opensg_shell</code> refuses to run it, and check_filled names the fields still unfilled
<code>thickness=t</code>	a runnable single-ply YAML; the default material is aluminium, $E = 69$ GPa, $\nu = 0.30$, density 2700 kg/m ³
<code>layup=[[mat, t, angle], ...]</code> plus <code>materials=[...]</code>	the general multi-ply form

```
from opensg_shell.helper.msh_to_yaml import convert

convert("schwarz_p_D2_shell.msh")           # FILL_IN template
convert("schwarz_p_D2_shell.msh", thickness=0.036457) # runnable, aluminium
```

Listing 2.12: Converting a gmsh surface mesh to an msg-shell SG YAML.

The **sections:** and **materials:** blocks are written **first**, above the mesh, because they are the part a human edits: a TPMS cell runs to 10^4 nodes and 10^5 mesh lines, and a **sections:** key buried at line 66262 of a 4 MB file looks missing. A YAML mapping is unordered by specification, so every reader in the stack is indifferent to the order.

2.5 The through-thickness layup_db.yaml

For a plain laminate there is no mesh to write: the 1-D SG is one element stack through the thickness, and `layup_db.yaml` is the whole user input. The driver `1d_sg.py` reads it, generates the SG mesh (`1dsg.yaml` plus `1dsg.png`) through `opensg_solid.rm_plate_1D.segment_plate.plate_sg_yaml`, homogenizes with `msg_rm_plate.rm_plate_msg`, and writes `layup_db_plate_homo.out`. The generated mesh is shown in Figure 2.3 and the file's keys in Table 2.10.

The Nayak sandwich in the example is a nine-ply (0/90/0/90/core)_s stack of graphite/epoxy faces on a HEREX C70.130 PVC foam core:

```
mesh:
```

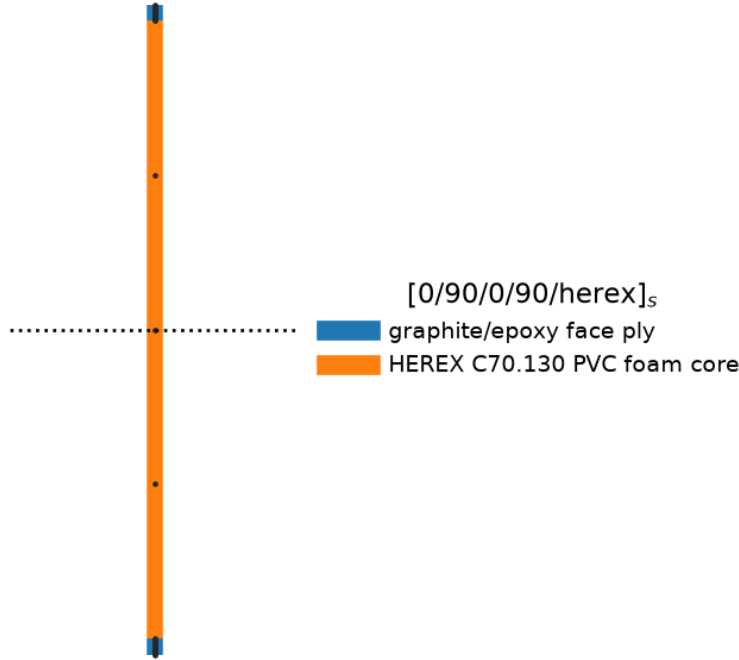


Figure 2.3: The generated through-thickness 1-D plate SG, elements coloured by material: eight thin graphite/epoxy plies bracketing the thick PVC foam core. The abscissa is y_3 , running from $-h/2$ to $+h/2$ because `fraction: 0.5` put the origin at mid-thickness.

Table 2.10: Keys of `layup_db.yaml` read by `1d_sg.py`.

key	meaning
<code>model</code>	0 prints the classical 6×6 ABD, 1 the shear-refined 8×8 ABDG. This also names the matrix in the <code>.out</code> : Classical Plate or Reissner-Mindlin .
<code>fraction</code>	the reference surface as a fraction of the total thickness: 0 = bottom/OML face, 0.5 = mid-surface, 1 = top. This is the solid-side analogue of the shell YAML’s reference .
<code>mesh.n_per_layer</code>	SG elements per physical ply.
<code>mesh.elem_order</code>	element polynomial order p ; nodes per element are $p + 1$, and the engine supports 1 to 4. 4 — the five-noded quartic — is the default in the example and represents the whole warping ladder exactly (V_0 quadratic, V_1 cubic, V_2 quartic).
<code>materials</code>	a mapping from short name to E (3), G (3), nu (3) and rho , with an optional full_name . Every number must parse as a float; the example’s comment about 128.0e9 is a real PyYAML trap, which is why <code>1d_sg.py</code> coerces with <code>float(v)</code> on read.
<code>layup</code>	the stacking sequence, bottom ply first , each entry { material , thickness , angle }. An optional divisions key is read only by the Abaqus 3-D deck generator and ignored by the homogenizer.

```

n_per_layer: 1
elem_order: 4
fraction: 0.5
layup:
- {material: ge,    thickness: 0.0009525, angle: 0.0}
- {material: ge,    thickness: 0.0009525, angle: 90.0}
- {material: ge,    thickness: 0.0009525, angle: 0.0}
- {material: ge,    thickness: 0.0009525, angle: 90.0}
- {material: herex, thickness: 0.1447800, angle: 0.0, divisions: 8}
- {material: ge,    thickness: 0.0009525, angle: 90.0}
- {material: ge,    thickness: 0.0009525, angle: 0.0}
- {material: ge,    thickness: 0.0009525, angle: 90.0}
- {material: ge,    thickness: 0.0009525, angle: 0.0}

```

Listing 2.13: The mesh, reference and stacking blocks of layup.db.yaml.

```

cd examples/OpenSG-solid/1_get_plate_props_from_1DSG
python 1d_sg.py

```

1d_sg.py accepts an alternative database as its one positional argument, and the output file is named after that database rather than after the mesh. The .out banner echoes back every number that mattered, which is the fastest way to confirm the file was read as intended: `OpenSG msg-solid plate model from 1dsg.yaml: 9 plies, h = 0.152400 m, fraction = 0.5, rho*h = 30.251400 kg/m^2, rows [e11 e22 g12 k11 k22 k12 2g13 2g23]`. With model: 1 the matrix printed is the 8×8 , whose transverse-shear block for this sandwich is $G_{11} = 7.7010 \times 10^6$ and $G_{22} = 7.5424 \times 10^6$ N/m, and whose B block is numerically zero (entries of order 10^{-6} against a 10^8 membrane term) as a symmetric stack requires.

The generated 1dsg.yaml records the same decisions in its own header,

```

sg: {type: plate_1d, elem_order: 4, n_per_layer: 1, reference_fraction: 0.5,
    thickness: 0.15239999999999998, n_ply: 9}
nodes:
- [-0.07619999999999999]
elements:
- [1, 2, 3, 4, 5]
- [5, 6, 7, 8, 9]

```

Listing 2.14: Header, first node and first elements of the generated 1dsg.yaml (37 node rows and 9 element rows in full).

and read_plate_sg_yaml cross-checks the stored ply thicknesses and reference_fraction against the mesh itself, rejecting a hand-edited file whose header and coordinates disagree instead of silently trusting it.

Note the element rows carefully, because this is the one place where the two solid routes genuinely differ. segment_plate.plate_sg_yaml numbers each element's nodes *sequentially*, np.arange(p*e, p*e + p + 1), which is why the rows read [1, 2, 3, 4, 5] and [5, 6, 7, 8, 9] — consecutive integers, adjacent elements sharing their end node. rm_plate_msg then homogenizes that mesh with its own equispaced Lagrange basis (nodes_xi = np.linspace(-1.0, 1.0, p + 1), max(3, p + 1)-point Gauss-Legendre), so the layup_db.yaml route never touches the general SG engine's element table at all. The ends-first gmsh/basix line ordering — [seq[0], seq[-1]] + seq[1:-1] — belongs instead to opensg_solid.sg_mesh.laminate_to_sg, the parity route that feeds the general engine, and it is that engine's plate_homo_2d which consults _cell_basis (Section 2.6).

2.6 Finite elements used by each route

The two packages discretize differently, and knowing which element you are getting explains both the DOF count and the failure modes.

2.6.1 Shell route

The shell SG never assembles a line element directly. The contour is extruded one quad deep along the beam axis by `sg_homo._strip`, with extrusion length h equal to the mean contour element length, giving quads `[a, b, m+b, m+a]`; the top node row is then DOF-mapped onto the bottom row (`dof_map = np.concatenate([np.arange(m), np.arange(m)])`) so the field is exactly span-invariant. That is the operator's own prismatic reduction. The element is therefore a 4-node bilinear quad integrated at 2×2 Gauss points ($\pm 1/\sqrt{3}$ in each direction), and the 1-D cross-section problem is recovered as its span-invariant limit. Two element families are built on that quad, and they differ only in what they do with the drilling rotation (Table 2.11).

Table 2.11: The two shell element families and how each handles the drilling rotation.

entry point	DOF/node	DOF/elem	drilling
<code>sg_homo.ring_indep</code> (production)	6: <code>[w1, w2, w3, om1, om2, om3]</code>	24	independent nodal ω_3 , tied back by a finite drilling residual (<code>NDOF6 = 6</code>)
<code>sg_assembly.ring_general</code>	5: <code>[w1, w2, w3, om1, om2]</code>	20	eliminated algebraically through $\omega_3 = S/(2C_{33}) - (C_{3b}/C_{33})\omega_b$ (<code>NDOF6 = 5</code>)

The independent-drilling element exists because the elimination divides by $C_{33} = n \cdot b_3$, which vanishes on a flat wall. In the 6-DOF element ω_3 is a genuine nodal DOF and the in-plane symmetry that defined it is re-imposed in its finite, undivided form as the drilling residual

$$DR = C_{33} \omega_3 + C_{3b} \omega_b - S/2,$$

which equals $C_{33}(\omega_3 - \omega_3^{\text{eliminated}})$ and stays finite even at $C_{33} = 0$. It is enforced by an element-wise Lagrange multiplier (`lam_space="elem"`, the production setting) or by a penalty. Transverse shear uses MITC assumed-strain tying in the Dvorkin–Bathe sense: the production ring scheme is `shear="mitc4_g23"`, which ties only γ_{23} , because under span invariance γ_{13} is algebraic in the directors and carries no fluctuation gradient.

The 3-D shell SG uses the same quad element for its quads, and a genuine 3-node triangle (MITC3) for its triangles: the two are assembled as separate element batches into one global system, and the old collapsed quad `[n1 n2 n3 n3]` is rejected rather than accepted.

2.6.2 Solid route

`opensg_solid.sg_mesh._cell_basis` maps the SG dimension and the node count per element onto a basix cell type and Lagrange degree. This table is the list of supported elements, and it is consulted by exactly one function: `sg_homo.plate_homo_2d`, the general SG engine entry point (and therefore also by anything that reaches it, including a `.sc` handed to it directly and the `laminates_to_sg` parity route). The supported combinations are given in Table 2.12.

Anything else raises `unsupported <dim>D element with <n> nodes`. Basis functions are equispaced Lagrange (`LagrangeVariant.equispaced`) with quadrature degree 6 for degree > 1 and 2 otherwise. Two connectivity conventions are applied on the way in: `_to_basix_order` permutes

Table 2.12: The `_cell_basis` table: every element the general solid SG engine supports.

SG dim	nodes/elem	basix cell	degree
1	2	interval	1
1	3	interval	2
1	4	interval	3
1	5	interval	4
2	3	triangle	1
2	4	quadrilateral	1
3	4	tetrahedron	1
3	8	hexahedron	1
3	10	tetrahedron	2

quad4 to [0, 1, 3, 2] and hex8 to [0, 1, 3, 2, 4, 5, 7, 6], while triangles, tetrahedra and intervals pass through unchanged.

The through-thickness laminate of Section 2.5 does *not* go through this table. Its route — `plate_sg_yaml` to `rm_plate_msg` — carries its own equispaced Lagrange basis and its own sequential node numbering, as described there. Only `laminate_to_sg`, the alternative route that packages a laminate as a generic 1-D SG dict for `plate_homo_2d`, writes the ends-first gmsh/basix line ordering that the `_cell_basis` elements expect.

2.7 Sign and order conventions

Three orderings appear across the codebase. Every one of them is stated in a module docstring rather than inferred, and every one of them is a common source of a plausible-looking but wrong answer.

2.7.1 Solid (3-D) Voigt order

The solid order is the SwiftComp order

$$[11 \ 22 \ 33 \ 23 \ 13 \ 12]$$

with **engineering shears**. The compliance therefore carries $S_{44} = 1/G_{23}$, $S_{55} = 1/G_{13}$, $S_{66} = 1/G_{12}$, and the shear rows of a recovered strain are 2γ , not γ . This is the order of `write_sc_K`'s output, of `build_stiffness_6x6`, and of every `type: 2` material `C` block in a solid SG YAML.

2.7.2 Beam order

The beam order is

$$[\varepsilon_{11} \ \gamma_{12} \ \gamma_{13} \ \kappa_1 \ \kappa_2 \ \kappa_3],$$

whose diagonal is the familiar VABS ordering

$$[EA \ GA_2 \ GA_3 \ GJ \ EI_2 \ EI_3],$$

the LBL list used by the shell drivers. **Axis 1 is the beam axis**; axes 2 and 3 are the cross-section axes.

The solver indexes global axes as $(1, 2, 3) = (\text{axial}, y_2, y_3)$ while an SG YAML writes each 3-vector as (y_2, y_3, axial) . That is why `opensg_solid.sg_materials.elem_rotation_from_yaml` cycles

$$[a, b, c] \longrightarrow [c, a, b]$$

(implemented as `o[:, :, [2, 0, 1]]`) before a YAML orientation can be used as a direction-cosine matrix in the solid beam path. Skipping that cycle points e_1 along y_3 : the run still completes and still looks plausible, but it destroys the section's symmetry, and a square tube comes out with $EI_2 \neq EI_3$.

2.7.3 Plate order

The plate order is $[N_{11} \ N_{22} \ N_{12} \ M_{11} \ M_{22} \ M_{12}]$ for the classical 6×6 , conjugate to the strain vector $[\varepsilon_{11} \ \varepsilon_{22} \ 2\varepsilon_{12} \ \kappa_{11} \ \kappa_{22} \ \kappa_{12}]$. The Reissner–Mindlin 8×8 appends the two transverse shears, giving the rows `[e11 e22 g12 k11 k22 k12 2g13 2g23]` that both `write_abdg_out` and `1d_sg.py` print in their banners, and the block structure

$$\mathbf{ABDG} = \begin{bmatrix} \mathbf{A} & \mathbf{B} & \mathbf{0} \\ \mathbf{B} & \mathbf{D} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{G} \end{bmatrix}.$$

For a plate SG the **last SG coordinate is the thickness direction** — the y_3 of the solid Voigt order above. For an SG of dimension 2 that is node coordinate index 1, since only the leading two coordinates are read; for a through-thickness 1-D SG it is the single coordinate. This is why the RHC cell reports $\omega = 35.248$, the span of coordinate 0 (the in-plane periodic direction), and not its larger extent in coordinate 1.

2.7.4 Changing the reference surface

Moving the reference surface by z_0 maps the membrane strain ε to $\varepsilon + z_0 \kappa$, so with $\mathbf{T} = \begin{bmatrix} \mathbf{I} & z_0 \mathbf{I} \\ \mathbf{0} & \mathbf{I} \end{bmatrix}$ the plate law transforms as

$$\mathbf{ABD}_{\text{new}} = \mathbf{T}^{-\top} \mathbf{ABD} \mathbf{T}^{-1},$$

which is exactly what `fe_jax.msg_materials.shift_abd_reference(ABD, z0)` computes. Use it rather than reversing the layout: reversal flips e_3 and breaks agreement with the material orientation. The transverse-shear block $\mathbf{G}$ is reference-independent, which is why `compute_ABD_matrix` can shift the 6×6 and leave the 8×8 's $\mathbf{G}$ untouched.

2.8 Material orientation

Each row of `elementOrientations` is nine direction cosines forming three stacked unit vectors,

```
[ e1x e1y e1z | e2x e2y e2z | e3x e3y e3z ]
```

with the roles fixed across both shell dialects:

- e_1 is the **beam axis** — the prismatic direction, which for a plane cross-section meshed in (y_2, y_3) is out of plane, so `e1 = [0, 0, 1]`.
- e_2 is the **in-plane ply-flow direction**, the wall tangent along the contour.

- $e_3 = e_1 \times e_2$ is the **wall normal**, in plane for a cross-section SG, and the direction the layup stacks along from the reference surface.

The triad must be orthonormal and right-handed. `opensg_shell.sg_mesh.frame_report(nodes, cells, e1, e2, e3)` checks this numerically. It **returns** the pair (ok, txt) and prints nothing — callers such as `sg_mesh.extract` do the printing — and its one-line summary ends in OK or CHECK. Its pass/fail gate is

$$\begin{aligned} |\overline{|e_i|} - 1| &< 10^{-6} \quad (i = 1, 2, 3), & \max_{i \neq j} |e_i \cdot e_j| &< 10^{-6}, \\ \min(e_1 \times e_2 \cdot e_3) &> 0.99, & \min(e_1 \cdot \hat{x}) &> 0.999. \end{aligned}$$

The first three are universal; the fourth — e_1 axial — is cylinder-specific, as the docstring says. The report also prints a fifth number, the mean of $e_3 \cdot (-\hat{r})$, i.e. how nearly e_3 is the inward radial normal, but that value is **reported and not gated**: it appears in the text so you can read it, and a poor value alone will not turn an OK into a CHECK.

Because any mesh handling can renumber nodes and cells, an orientation PNG is a compulsory deliverable of every OpenSG run. It is emitted by the *example driver* calling `opensg_shell.auto_emit` — see the last line of `beam_homo_shell.py` — and not by any homogenization entry point: `build_rm_bundle`, `shell_sg3d` and `plate_homo_2d` never call it. `auto_emit` is cached once per (shell, solid) path per process and swallows any exception with a [orient_plot] skipped message, so it never raises and is safe to call from inside a compute path; a new driver of your own should call it explicitly.

The colour convention of `fe_jax/orient_plot.py` is e_2 **blue**, e_3 **green**, drawn at element centroids over a light mesh; e_1 is out of the page and so not drawn. (The separate 3-D plotters in `sg_mesh.py` use red/blue/black for $e_1/e_2/e_3$, since there e_1 is visible.) The canonical example is the IEA-22 $r/R = 0.2$ station of Figure 2.4.

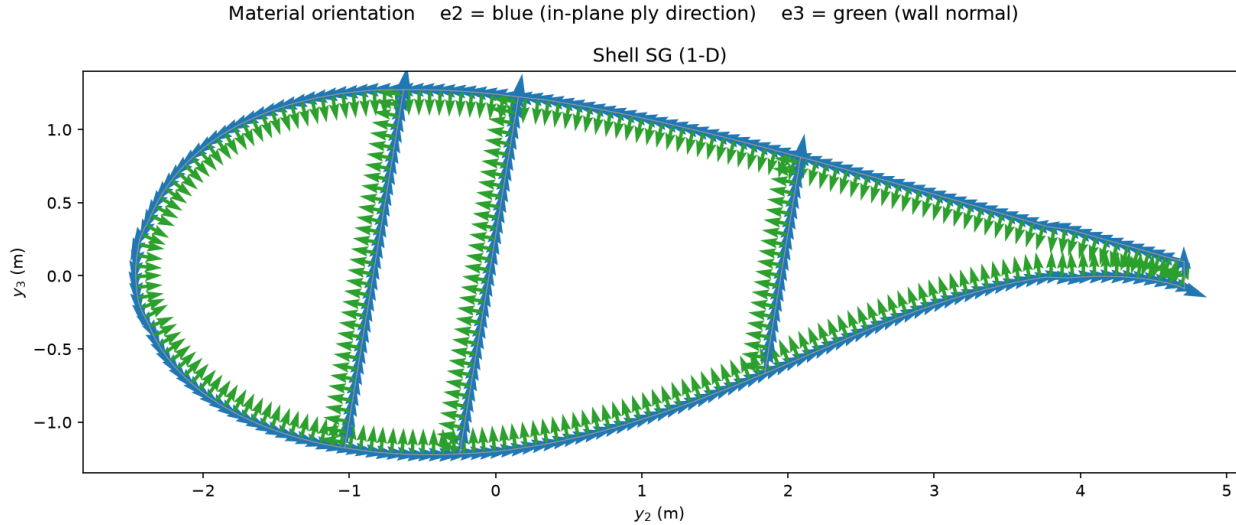


Figure 2.4: Material orientation of the IEA-22 $r/R = 0.2$ shell cross-section: e_2 in blue (in-plane ply direction), e_3 in green (wall normal). This is `iea_s10_orient.png`, written by the `auto_emit` call at the end of `beam_homo_shell.py`.

Read it as a physical check, not decoration. On this station the green e_3 arrows point inward everywhere on the skin, which is what an outward-first layup stacked from the mid-surface requires,

and the blue e_2 arrows are tangent to every wall — running consistently around the closed skin loop and along each of the three shear webs. An e_3 that flips between neighbouring skin elements, or a web whose e_2 opposes its neighbours, means the layup is being stacked in two directions at once: visible here in seconds, and invisible in the resulting 6×6 .

2.8.1 Where the ply angle lives

The ply angle is **not** baked into e_1 . It lives in the third column of each `layup` row (shell dialect) or in the `angles` argument (solid dialect), and it rotates the material about e_3 , away from e_1 . The rotation `rotation_6x6(theta)` leaves the Voigt-33 row untouched and mixes the 11/22/12 rows, and the reduced in-plane stiffness built by `msg_transverse_shear._ply_Q_and_G` is the textbook

$$\bar{Q}_{11} = Q_{11} \cos^4 \theta + 2(Q_{12} + 2Q_{66}) \sin^2 \theta \cos^2 \theta + Q_{22} \sin^4 \theta,$$

so `angle: 0.0` places the fibre along e_1 — the beam axis. A wind-blade unidirectional spar cap therefore carries `angle: 0.0`, and a ± 45 ply carries `45.0` and `-45.0`.

The governing repository rule is `Rules/orientation_e1_out_of_plane.md`, which also records the failure mode: a YAML whose e_1 already contains the fibre rotation (so $|e_{1z}| < 1$) is a *fibre* frame, and combining it with a non-zero `angle` applies the rotation twice. The rule supplies a two-line check worth running on any YAML you did not write yourself:

```
ori = np.array(d["elementOrientations"], float)
e1, e3 = ori[:, 0:3], ori[:, 6:9]
assert np.allclose(np.abs(e1[:, 2]), 1.0)    # e1 out of plane
assert np.allclose(e3[:, 2], 0.0)           # e3 in plane
```

Listing 2.15: The orientation precheck from `Rules/orientation_e1_out_of_plane.md`.

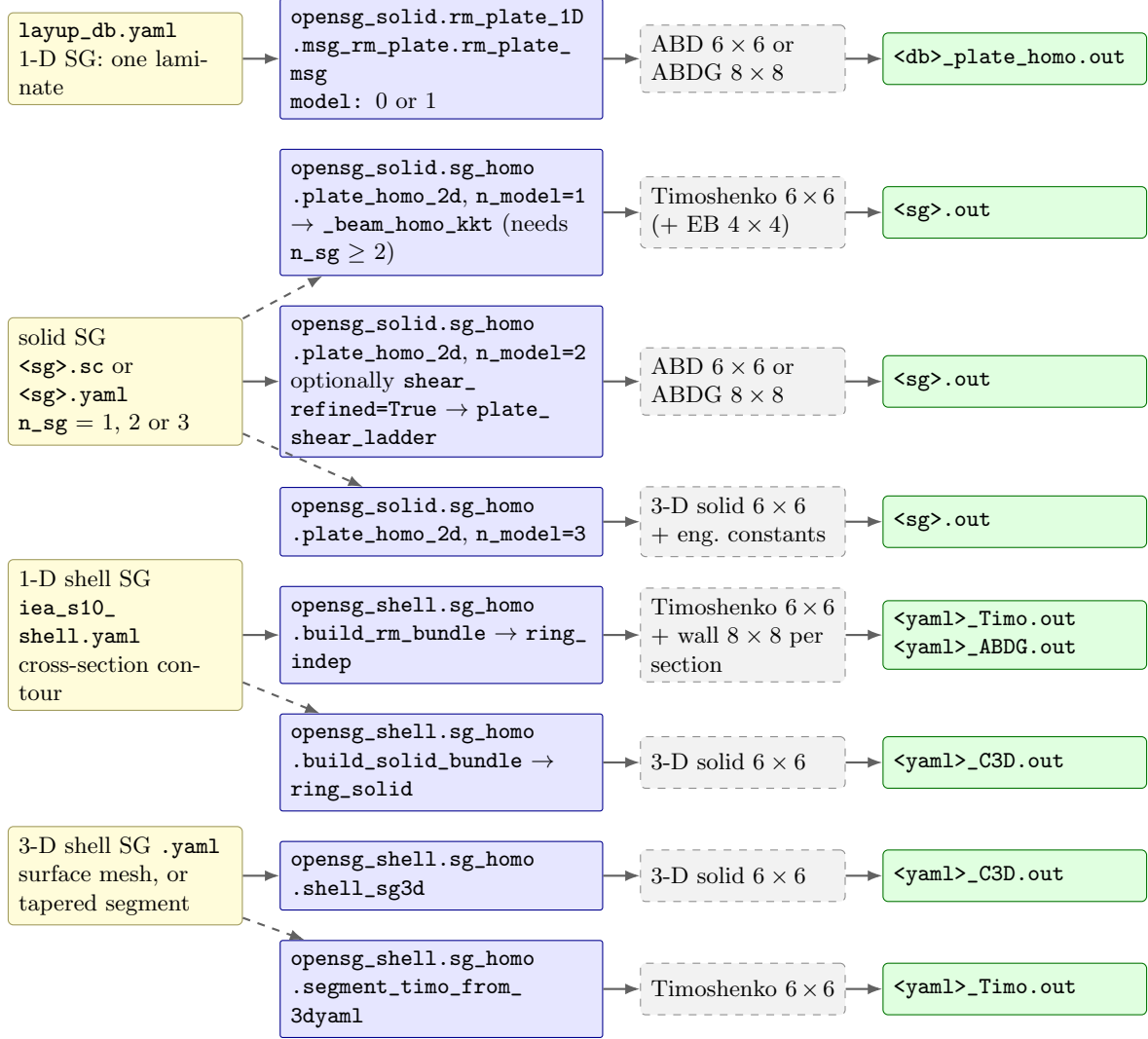
If the first assertion fails, the file carries a fibre frame; either switch to the frame variant built with the axial rotation only, or set the fibre angle to zero downstream so the rotation is applied exactly once.

Chapter 3

Constitutive Relations Recovered by OpenSG

Every OpenSG run answers the same question. Given a structure gene (SG) — a one-dimensional through-thickness stack, a two-dimensional cross-section or cell, or a three-dimensional unit cell — what is the constitutive law of the macroscopic model that replaces it? The SG problem is solved once, and its output is a small dense stiffness matrix relating *generalized forces* to *generalized strains*. This chapter lists every such law the code produces, the exact ordering of the strain and force components in each, the function that computes it, the input file you edit, the command you type, and the file the answer is written to.

The map is short. Every law in this chapter is one path through the tree below: an SG of a given dimension, a model choice made either by the `n_model` argument of the solid engine or by which `opensg_shell` entry point is called, the macro law that comes back, and the `.out` file it is written to. Solid lines are the default path of each entry point; dashed lines are the alternative model choices that reuse the same input file.



Every one of those files is printed by `write_sc_K`, except the per-section wall law `<yaml>_ABDG.out`, which `write_abdg_out` writes block by block in the same number format. In both cases the title above the matrix names the law, so the tree above and Table 3.1 together identify any output file the code produces.

Two conventions hold throughout and are worth internalising before reading any matrix.

- **Units follow the input.** The single writer `write_sc_K` performs no unit conversion, so an SG given in metres with moduli in pascals returns N, N·m and Pa, while an SG given in millimetres with moduli in MPa returns the corresponding mixed units (N/mm, N·mm).
- **Shears are engineering shears.** Every off-diagonal strain component carries the factor of two ($2\varepsilon_{12}$, $2\bar{\Gamma}_{23}$, $2\gamma_{13}$), which is why each shear stiffness appears once, not four times, in the quadratic strain energy.

3.1 The .out contract

Every homogenization in OpenSG-2.0 writes a timed text file in the SwiftComp .K layout through one function, `opensg_solid.sg_homo.write_sc_K`:

```
write_sc_K(path, C, solve_time=None, model="", constants=True, name="")
```

Listing 3.1: The single output writer. Every law in this chapter is printed by this call.

The file opens with an `OpenSG <model>` banner, prints the effective stiffness $\mathbf{C}$, then its exact inverse $\mathbf{S} = \mathbf{C}^{-1}$ (the effective compliance), and closes with `Time taken: <t> sec`. Numbers are written by the helper `_knum`, a 19-character right-justified `%.7E` mantissa with a signed three-digit exponent, which is exactly the SwiftComp `.K` column format.

The `name` argument is the only thing that changes the matrix title, and it identifies which macro law you are looking at (Table 3.1). **name is always the macro law’s console title**, so the `.out` header and the line the terminal prints are word-for-word the same.

Table 3.1: The `name` argument of `write_sc_K` and the title it prints. The title is the fastest way to tell which law an `.out` file holds, and it is also the line the run prints to the terminal.

name passed to <code>write_sc_K</code>	title printed in the <code>.out</code>	macro law
"Cauchy Continuum"	The Effective Cauchy Continuum Stiffness Matrix	3-D solid 6×6
"Timoshenko Beam"	The Effective Timoshenko Beam Stiffness Matrix	beam 6×6 , shear-refined
"Euler-Bernoulli Beam"	The Effective Euler-Bernoulli Beam Stiffness Matrix	beam 4×4 , classical
"Classical Plate"	The Effective Classical Plate Stiffness Matrix	plate ABD 6×6
"Reissner-Mindlin"	The Effective Reissner-Mindlin Stiffness Matrix	plate ABDG 8×8

Only the first of these also prints `The Engineering Constants (Approximated as Orthotropic)`. That is the `constants=True` branch, and it is meaningful only for a three-dimensional elasticity law; the beam and plate calls pass `constants=False`.

Two footnotes to the table. An empty `name` would give the anonymous `The Effective Stiffness Matrix`, but no macro law uses it any more. And the per-section *wall* laws written by the other writer, `opensg_shell.sg_homo.write_abdg_out`, keep the title `The Effective Reissner-Mindlin Plate Stiffness Matrix`, one block per section (Section 3.5) — a wall plate law, not the macro law of the run. Older documentation used `Timoshenko`, `Reissner-Mindlin Plate` and an anonymous `The Effective Stiffness Matrix` for the 3-D solid law; those spellings are stale, and the 25 baseline `.out` files under `examples/` were regenerated to the titles above.

3.2 Beam: the Timoshenko 6×6

The beam macro model carries six generalized strains: one axial extension, two transverse shears, one twist and two bending curvatures. In code order — the order stated in the module header of `src/opensg_solid/sg_homo.py` — these are

$$\bar{\epsilon} = \{ \bar{\epsilon}_{11} \ \gamma_{12} \ \gamma_{13} \ \kappa_1 \ \kappa_2 \ \kappa_3 \}^T, \quad (3.1)$$

and the conjugate generalized forces are the axial force, the two transverse shear forces, the torque and the two bending moments — the VABS ordering $\{F_1 \ F_2 \ F_3 \ M_1 \ M_2 \ M_3\}$ in which the shipped blade load table `ff51_rmc_reform.dat` is written (its header reads `# eta F1 F2 F3 M1 M2 M3 (RM CENTER-ref BeamDyn, VABS order)`). The law is

$$\begin{Bmatrix} F_1 \\ F_2 \\ F_3 \\ M_1 \\ M_2 \\ M_3 \end{Bmatrix} = \begin{bmatrix} S_{11} & S_{12} & S_{13} & S_{14} & S_{15} & S_{16} \\ S_{12} & S_{22} & S_{23} & S_{24} & S_{25} & S_{26} \\ S_{13} & S_{23} & S_{33} & S_{34} & S_{35} & S_{36} \\ S_{14} & S_{24} & S_{34} & S_{44} & S_{45} & S_{46} \\ S_{15} & S_{25} & S_{35} & S_{45} & S_{55} & S_{56} \\ S_{16} & S_{26} & S_{36} & S_{46} & S_{56} & S_{66} \end{bmatrix} \begin{Bmatrix} \bar{\epsilon}_{11} \\ \gamma_{12} \\ \gamma_{13} \\ \kappa_1 \\ \kappa_2 \\ \kappa_3 \end{Bmatrix}.$$

The matrix is fully populated in general — a composite blade section couples extension to bending and twist — but its diagonal carries the familiar engineering meaning

$$\text{diag}(\mathbf{S}) = [EA \quad GA_2 \quad GA_3 \quad GJ \quad EI_2 \quad EI_3], \quad (3.2)$$

which is how the drivers label their printout: the list `LBL = ["EA", "GA2", "GA3", "GJ", "EI2", "EI3"]` appears verbatim in `examples/OpenSG_shell/IEA_blade_beam/1_homo_beam_props.py`. Read Eq. (3.2) as a naming convention, not as a claim of decoupling. S_{44} is the torsional stiffness only to the extent that the twist channel is uncoupled; when S_{45} or S_{46} is large, GJ as conventionally understood is not S_{44} .

3.2.1 Running it on a shell cross-section

The `msg-shell` ring route homogenizes a cross-section contour with the constrained six-DOF shell element (independent drilling ω_3 enforced by element-constant Lagrange multipliers, γ_{23} tied by MITC) and takes its wall transverse shear from the MSG least-squares plate solution. The user edits one file, the one-dimensional shell SG YAML, which carries the contour nodes, the line elements, the per-element orientations, the layup sections, the materials and the `reference:` field. From `examples/OpenSG_shell/IEA_blade_beam`:

```
opensg iea_s10_shell.yaml
```

`opensg` is the unified entry point: it resolves the engine from the YAML's `msg:` key (or, if the key is absent, from the mesh dialect) and runs the homogenization. The driver `1_homo_beam_props.py` does the same solve from Python and adds the figures. Its user block is three lines long; only the station name changes:

```
name = "iea_s10" # IEA-22 station at r/R = 0.20
B = build_rm_bundle(name + "_shell.yaml")
C6 = np.asarray(B["Timo"])
```

Listing 3.2: The complete user input of the shell beam driver. `build_rm_bundle` reads `iea_s10_shell.yaml`.

`build_rm_bundle` writes two files by itself: the SwiftComp-layout `<yaml base>_Timo.out` (here `iea_s10_shell_Timo.out`) with banner `OpenSG msg-shell beam model [ext sh2 sh3 twist bend2 bend3]` and title `The Effective Timoshenko Beam Stiffness Matrix`, and the per-section wall law `<yaml base>_ABDG.out` described in Section 3.5. The driver additionally saves the bare 6×6 as `iea_s10_beam_Timo.out` and the ring mesh figure `iea_s10_ring_mesh.png`.

For the IEA-22 blade station at $r/R = 0.20$ — the SG of Figure 3.1 — that 6×6 has the diagonal

$$\text{diag}(\mathbf{S}) = [2.7661 \times 10^{10}, 7.1861 \times 10^8, 4.2174 \times 10^8, 2.4376 \times 10^9, 3.5144 \times 10^{10}, 6.8130 \times 10^{10}]$$

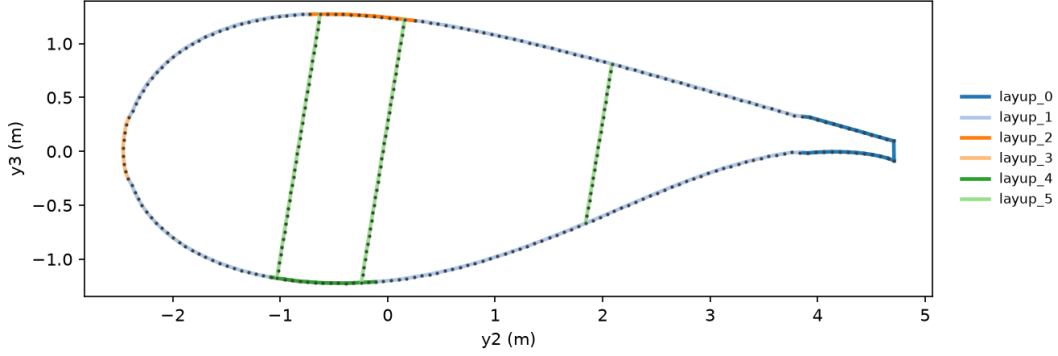


Figure 3.1: The one-dimensional shell SG solved by `build_rm_bundle`: the IEA-22 blade cross-section at $r/R = 0.20$, 307 contour nodes and 310 two-node line elements along the wall mid-surface, coloured by `sets: element` entry. The Timoshenko 6×6 beam law whose diagonal is quoted in this section is this mesh's answer, written to `iea_s10_shell_Timo.out`.

in SI units, so $EA = 27.7$ GN, $EI_3 = 68.1$ GN·m² edgewise against $EI_2 = 35.1$ GN·m² flapwise, and $GJ = 2.44$ GN·m². The largest off-diagonal terms in that file are $S_{15} = 1.66864 \times 10^9$ and $S_{16} = -2.07730 \times 10^9$ (extension–bending, i.e. the section reference axis is not the tension centre) and $S_{56} = -5.55712 \times 10^9$ (the two bending channels are not principal). Terms that should vanish by symmetry sit at 10^{-5} against diagonals of 10^{10} , which is the numerical zero of this solve.

3.2.2 Running it on a solid cross-section

The `msg-solid` route runs the KKT beam driver: a zeroth-order V_0 solve under four rigid-body Lagrange constraints, then a first-order V_{1s} solve reusing the same factorization, then the 6×6 reduction in `finalize_v1_and_compute_deff`. The entry point is `plate_homo_2d(..., n_model=1)`, which requires `n_sg >= 2` (a two- or three-dimensional solid SG). From `examples/OpenSG-solid/7_get_beam_props_from_SG`:

```
opensg RHC_SW_2UC_45.yaml
```

The same dispatcher serves the solid engine: this YAML's header carries `msg: solid` and `n_model: 1`, so `opensg` hands it to `opensg_solid`. The driver `beam_homo_sg.py` runs the same model from Python with the material table written out in full:

```
n_model = 1                # 1: Beam; 2: Plate; 3: 3D elastic
name = "RHC_SW_2UC_45"    # reads <name>.yaml

material_param = jnp.array([
    (108e3, 8e3, 8e3, 4e3, 4e3, 3e3, 0.32, 0.32, 0.30),    # ply +45
    (108e3, 8e3, 8e3, 4e3, 4e3, 3e3, 0.32, 0.32, 0.30),    # ply -45
    (69e3, 69e3, 69e3, 26.54e3, 26.54e3, 26.54e3, 0.30, 0.30, 0.30)])
angles = jnp.array([45.0, -45.0, 0.0])

r = plate_homo_2d(name + ".yaml", material_param=material_param,
                  angles=angles, n_model=n_model)
```

Listing

3.3:

The user block of `beam_homo_sg.py`. The material rows are (E1,E2,E3,G12,G13,G23,nu12,nu13,nu23) in MPa; angles rotates each material about the SG normal.

This writes `RHC_SW_2UC_45.out` with banner `OpenSG msg-solid beam model, omega 1` and the mesh figure `RHC_SW_2UC_45_mesh.png` (Figure 3.2). Its diagonal is

$$\text{diag}(\mathbf{S}) = [9.8993 \times 10^6, 1.9075 \times 10^6, 9.3619 \times 10^5, 2.9775 \times 10^8, 2.1896 \times 10^9, 9.2899 \times 10^8]$$

in the input units (N and N·mm², since the SG is in millimetres and the moduli in MPa).

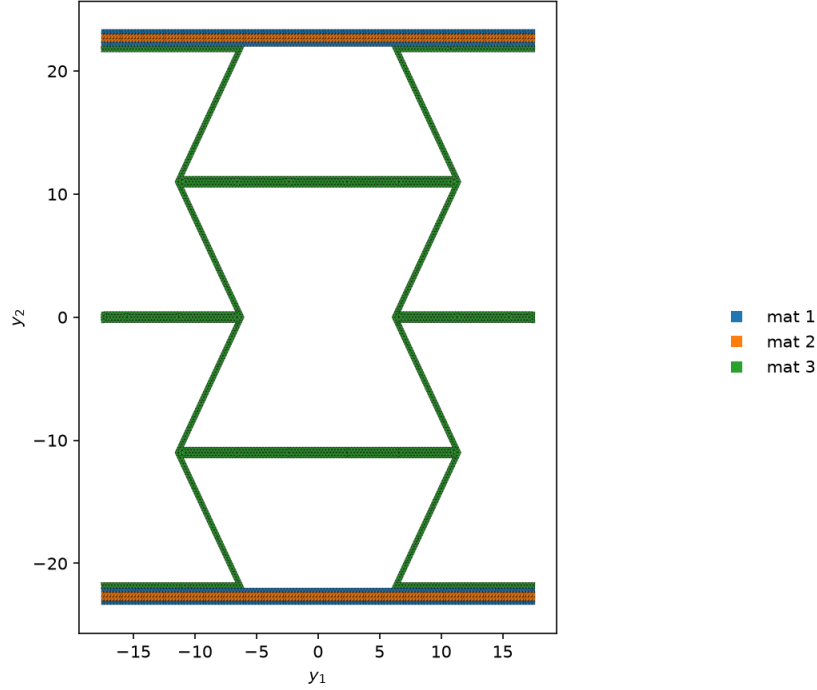


Figure 3.2: The two-dimensional honeycomb solid SG `RHC_SW_2UC_45`, run here with $n_{\text{model}} = 1$: the Timoshenko 6×6 beam law whose diagonal is quoted in this section is this mesh's answer, written to `RHC_SW_2UC_45.out`. The same mesh is the SG of the plate ($n_{\text{model}} = 2$) and solid (3) examples. Colours are the three material sets of the `.yaml`: the +45 and -45 composite plies of the cell walls and the isotropic face sheets.

3.2.3 The classical 4×4 that rides along

The Timoshenko reduction is built on top of a zeroth-order (Euler–Bernoulli) solve, and that intermediate law is a genuine classical beam stiffness in its own right. It drops the two transverse shears:

$$\begin{Bmatrix} F_1 \\ M_1 \\ M_2 \\ M_3 \end{Bmatrix} = \mathbf{A}_{\text{EB}} \begin{Bmatrix} \bar{\epsilon}_{11} \\ \kappa_1 \\ \kappa_2 \\ \kappa_3 \end{Bmatrix}, \quad \mathbf{A}_{\text{EB}} \in \mathbb{R}^{4 \times 4}. \quad (3.3)$$

On the solid route it is returned as `r["C_eff_EB"]` alongside the Timoshenko `r["C_eff"]`. In `_beam_homo_kkt` it is formed as $\mathbf{C}_{\text{eb}} = (D_{ee} + D_1^{V_0})/\omega$ immediately after the V_0 solve, before the first-order ladder runs, and it is then passed straight into `finalize_v1_and_compute_deff` as the **A** block of the Timoshenko transformation. Which of the two lands in the `.out` is the header's `refined:` key, exactly as on the plate side: `refined: 1` writes the Timoshenko 6×6 , and

refined: 0 writes this 4×4 under the title **The Effective Euler-Bernoulli Beam Stiffness Matrix**. Either way both matrices are in the result dict, so a driver can print the other one, as `beam_homo_sg.py` does.

3.2.4 Which routes produce the beam law

Table 3.2: The three entry points that return a Timoshenko 6×6 . The msg-solid entry point lives in `opensg_solid.sg_homo`, the two msg-shell ones in `opensg_shell`.

route	entry point	SG input
msg-solid beam	<code>plate_homo_2d(..., n_model=1)</code>	2-D or 3-D solid SG (<code>n_sg >= 2</code>)
msg-shell ring	<code>build_rm_bundle</code> (calls <code>ring_indep</code>)	1-D shell SG (cross-section contour)
msg-shell segment	<code>segment_timo_from_3dyaml</code>	3-D shell segment YAML (tapered or prismatic)

The segment route replaces axial periodicity by Dirichlet data. Each end cross-section is extracted topologically by `sg_mesh.extract`, solved on its own with `ring_indep` to give that ring's $\mathbf{C}_6$, V_0 and V_1 , and those warping fields are then imposed on the segment's boundary nodes; the segment energy is divided by the segment length L_z . It writes `<yaml base>_Timo.out` with the same Timoshenko title and the banner `msg-shell tapered/aperiodic segment` (boundary V_0/V_1 Dirichlet, $L=<L_z>$). For a prismatic segment the result must reproduce the boundary ring's own 6×6 ; that identity is the standing verification of the route.

3.3 Plate: classical ABD 6×6 and Reissner–Mindlin ABDG 8×8

3.3.1 Classical (Kirchhoff–Love) plate law

The classical plate model pairs three membrane stress resultants and three moment resultants with three membrane strains and three curvatures:

$$\begin{Bmatrix} N_{11} \\ N_{22} \\ N_{12} \\ M_{11} \\ M_{22} \\ M_{12} \end{Bmatrix} = \begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{B}^\top & \mathbf{D} \end{bmatrix} \begin{Bmatrix} \varepsilon_{11} \\ \varepsilon_{22} \\ 2\varepsilon_{12} \\ \kappa_{11} \\ \kappa_{22} \\ \kappa_{12} \end{Bmatrix}, \quad \mathbf{A}, \mathbf{B}, \mathbf{D} \in \mathbb{R}^{3 \times 3}.$$

The row order [N11 N22 N12 M11 M22 M12] is stated in the module header of `src/opensg_solid/sg_homo.py` and echoed by the driver in `examples/OpenSG-solid/3_get_plate_props_from_2D_SG`. The $\mathbf{B}$ block is the extension–bending coupling; it vanishes only for a laminate that is symmetric about the chosen reference surface, so it is a property of the SG *and* of where you put the reference. Run it with

```
opensg RHC_SW_2UC_45.yaml
```

which writes `RHC_SW_2UC_45.out` with banner

```
OpenSG msg-solid plate model, omega 35.248001, periodic
```


Which of the two plate laws comes back is the header's `refined: key: refined: 0` returns the classical ABD 6×6 of this section, under the title **The Effective Classical Plate Stiffness Matrix**; the shipped file sets `refined: 1`, so the run as shipped returns the Reissner–Mindlin ABDG 8×8 of the next section, whose leading 6×6 is exactly the ABD quoted here. (`plate_homo_2dsg.py` is the same solve from Python, and it also regenerates the `.yaml` and `.msh` from the shipped `.sc`.) Here $\omega = 35.248001$ is the in-plane SG measure (the cell width), and the leading entries are $A_{11} = 3.3586 \times 10^5$, $A_{22} = 1.1291 \times 10^5$, $A_{66} = 1.0552 \times 10^5$ N/mm and $D_{11} = 9.0368 \times 10^7$, $D_{22} = 5.7011 \times 10^7$, $D_{66} = 4.6026 \times 10^7$ N·mm. The **B** block is at the 10^0 level against an **A** of 10^5 , i.e. the cell is symmetric about its own mid-surface to solver precision.

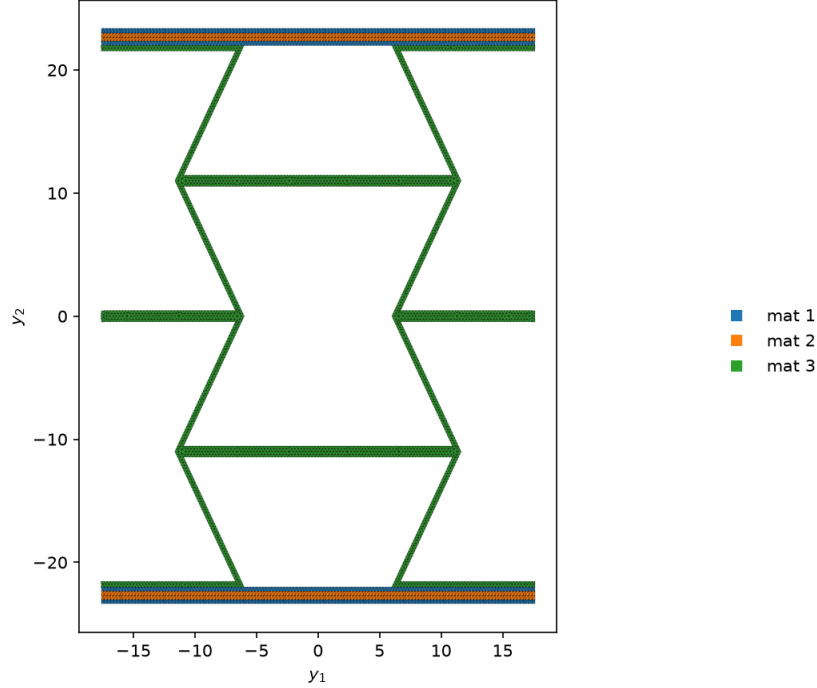


Figure 3.3: The same two-dimensional honeycomb cell SG `RHC_SW_2UC_45` as Figure 3.2, run here with $n_{\text{model}} = 2$: the classical plate ABD 6×6 whose leading entries are quoted in this section is this mesh's answer, written to `RHC_SW_2UC_45.out` under the title **The Effective Classical Plate Stiffness Matrix** at `refined: 0`. Only the model choice changes between this run and the beam run of Section 3.2; the SG is byte-identical.

3.3.2 Reissner–Mindlin plate law

The shear-refined model adds the transverse-shear pair. The strain vector grows to eight components and the resultant vector gains the two transverse shear forces Q_1 and Q_2 :

$$\begin{Bmatrix} \mathbf{N} \\ \mathbf{M} \\ \mathbf{Q} \end{Bmatrix} = \begin{bmatrix} \mathbf{A} & \mathbf{B} & \mathbf{0} \\ \mathbf{B}^T & \mathbf{D} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{G} \end{bmatrix} \begin{Bmatrix} \boldsymbol{\varepsilon} \\ \boldsymbol{\kappa} \\ \boldsymbol{\gamma} \end{Bmatrix}, \quad \boldsymbol{\gamma} = \{2\gamma_{13} \ 2\gamma_{23}\}^T, \quad \mathbf{Q} = \{Q_1 \ Q_2\}^T.$$

Here $\boldsymbol{\varepsilon}$ and $\boldsymbol{\kappa}$ are exactly the first and second triples of the classical law above.

The zero off-diagonal blocks are structural, not an approximation of convenience. The code assembles the 8×8 as `ABDG[:6,:6] = A6` and `ABDG[6:,6:] = G_msg`, so the in-plane law and

the transverse-shear law come from two separate variational problems and are then stacked. The 2×2 block $\mathbf{G}$ is obtained as $\mathbf{G} = \mathbf{X}^{-1}$ from the $\mathbf{U}^*$ least-squares reduction of the first-order warping ladder (function `_rm_ls_reduction`), a 78-equation, 27-unknown system solved by a column-equilibrated truncated-SVD minimum-norm solve with cutoff `_LS_RCOND = 1e-12`. Two diagnostics come out with it: the relative residual of the fit, reported as `r["Ustar_rel"]`, and the smallest eigenvalue of $\mathbf{X}$. If that eigenvalue is not positive the SPD gate fails and the block is returned as `None` — in which case no 8×8 exists and the code falls back to printing the classical 6×6 . Always check that you got an 8×8 before trusting a transverse-shear number.

3.3.2.1 The 1-D SG route

`opensg_solid.rm_plate_1D.msg_rm_plate.rm_plate_msg` solves the through-thickness SG of a single laminate and returns `A6` (the 6×6 ABD), `G_msg` and `ABDG`, with rows `e11 e22 g12 k11 k22 k12 2g13 2g23`. Which of the two matrices is printed is selected by `model`: in the user's `layup_db.yaml`: `model: 0` prints the classical 6×6 , `model: 1` prints the full 8×8 . From `examples/OpenSG-solid/1_get_plate_props_from_1DSG`:

```
python 1d_sg.py
```

The input the user edits is `layup_db.yaml` alone. It carries the reference surface (`fraction: 0.5` is the mid-surface, 0 the bottom or OML face, 1 the top), the materials with their densities, the stacking sequence bottom-ply-first, and the SG discretisation (`n_per_layer: 1` element per physical ply, `elem_order: 4` for the five-noded quartic element that represents the warping ladder exactly). The script writes the SG mesh `1dsg.yaml` plus `1dsg.png` (Figure 3.4) and the law in `layup_db_plate_homo.out`.

The shipped case is the Nayak Example-5 sandwich, $(0/90/0/90/\text{core})_s$, nine plies, total thickness $h = 0.1524$ m, graphite/epoxy faces ($E_1 = 128$ GPa) over a HEREX C70.130 PVC foam core, each face ply 0.9525 mm and the core 144.78 mm. Its `.out` banner records 9 plies, $h = 0.152400$ m, `fraction = 0.5`, $\rho \cdot h = 30.251400$ kg/m² and the leading entries are

$$A_{11} = A_{22} = 5.49165 \times 10^8 \text{ N/m}, \quad D_{11} = 3.00055 \times 10^6, \quad D_{22} = 2.93711 \times 10^6 \text{ N}\cdot\text{m}, \quad (3.4)$$

$$G_{11} = 7.70098 \times 10^6 \text{ N/m}, \quad G_{22} = 7.54237 \times 10^6 \text{ N/m}. \quad (3.5)$$

Read them physically: the thin stiff faces carry the membrane and bending channels, while the thick compliant core sets the transverse shear. The $\mathbf{B}$ block prints at 10^{-6} against an $\mathbf{A}$ of 10^8 , because the stack is symmetric about `fraction: 0.5`; change that to `fraction: 0.0` and $\mathbf{B}$ becomes a leading-order term.

3.3.2.2 The 2-D / 3-D SG route

`plate_homo_2d(..., n_model=2, shear_refined=True)` runs `plate_shear_ladder` on the general SG assembly after the classical solve, re-integrating the value-product ladder blocks on a quadrature exact to degree $2p + 2$. It adds `r["G_msg"]` (2×2), `r["A6_ladder"]`, `r["X_shear"]`, `r["Ustar_rel"]` and `r["ABDG"]` to the result dict, and the `.out` switches its title to `The Effective Reissner-Mindlin Stiffness Matrix` and its matrix to the 8×8 . Passing `shear_refined=True` with any `n_model` other than 2 raises a `ValueError`.

Note the derivation status recorded in the docstring of `plate_shear_ladder`, because it decides how much to trust the $\mathbf{G}$ block. For `n_sg = 1` this is the Yu (2005) one-dimensional formulation,

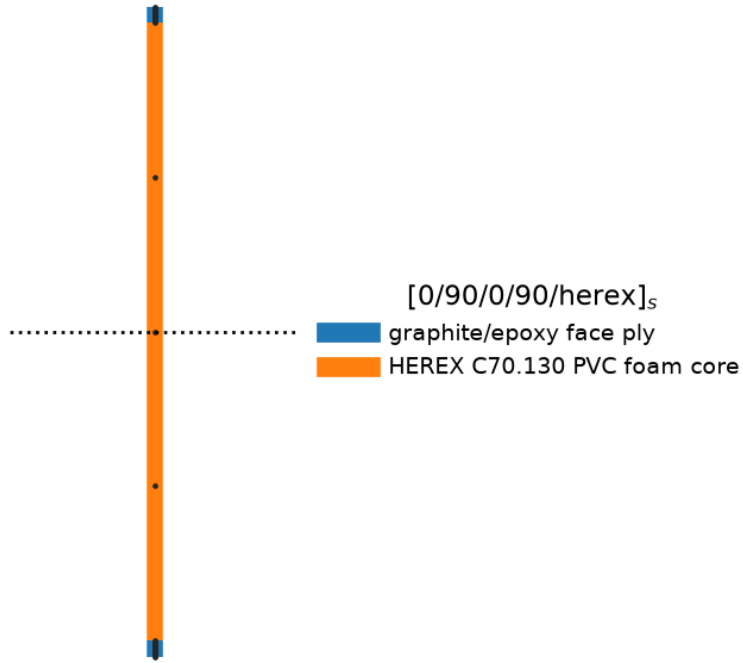


Figure 3.4: The through-thickness one-dimensional structure gene of the Nayak Example-5 sandwich written by `1d_sg.py`: eight graphite/epoxy face plies over a HEREX C70.130 foam core, with the mid-surface reference (`fraction: 0.5`) marked. This is the SG whose solution is the Reissner–Mindlin ABDG 8×8 whose A , D and G entries are quoted in this section, written by `rm_plate_msg` to `layup_db_plate_homo.out`.

gated against the closed-form anchors (an isotropic layer with $\nu = 0$ returns $\frac{5}{6}Gh$). For `n_sg = 2` or `3` it is the SwiftComp-style extension — the same energy functional with Γ_h carrying the resolved-dimension derivatives, periodic master–slave coupling, and the $\langle w \rangle = 0$ average constraint replacing the one-dimensional node constraint. That extension is gated on the laminate-as-strip equivalence (a laminate meshed as a 2-D or 3-D SG must reproduce its 1-D $\mathbf{G}$ under refinement) but is **not** yet validated against an external reference for SGs that are heterogeneous *in plane*. The shear-refined *recovery* is likewise not ported: plate dehomogenization stays the classical Kirchhoff-mode recovery.

3.4 Solid: the equivalent 3-D law

When the macro model is ordinary three-dimensional elasticity, the SG returns a 6×6 anisotropic stiffness in the Voigt order [11 22 33 23 13 12] with engineering shear strains:

$$\begin{Bmatrix} \sigma_{11} \\ \sigma_{22} \\ \sigma_{33} \\ \sigma_{23} \\ \sigma_{13} \\ \sigma_{12} \end{Bmatrix} = \mathbf{C}^{3D} \begin{Bmatrix} \bar{\Gamma}_{11} \\ \bar{\Gamma}_{22} \\ \bar{\Gamma}_{33} \\ 2\bar{\Gamma}_{23} \\ 2\bar{\Gamma}_{13} \\ 2\bar{\Gamma}_{12} \end{Bmatrix}, \quad \mathbf{C}^{3D} \in \mathbb{R}^{6 \times 6}.$$

Internally the solid engine stores strain and stress in the SwiftComp order (xx, yy, zz, yz, xz, xy) , which is the same list. On the `msg-shell` routes axis 1 is the prismatic (beam) direction and axes 2 and 3 span the cross-section, as recorded in the module constant `opensg_shell.sg_homo.GBAR_ORDER`:
`strains [e11 e22 e33 2e23 2e13 2e12] -> stiffness C11..C66`
(1=beam axis, 2/3=cross-section axes).

3.4.1 The engineering-constants block

This is the only law for which the `.out` also prints an engineering-constants block. It is computed from the compliance $\mathbf{S} = (\mathbf{C}^{3D})^{-1}$ by the orthotropic-approximation formulas coded in `write_sc_K` (and mirrored on the shell side by `opensg_shell.sg_homo.elastic_constants`, which also returns `cond(C)`):

$$E_1 = \frac{1}{S_{11}}, \quad E_2 = \frac{1}{S_{22}}, \quad E_3 = \frac{1}{S_{33}}, \quad G_{12} = \frac{1}{S_{66}}, \quad G_{13} = \frac{1}{S_{55}}, \quad G_{23} = \frac{1}{S_{44}},$$

$$\nu_{12} = -\frac{S_{12}}{S_{11}}, \quad \nu_{13} = -\frac{S_{13}}{S_{11}}, \quad \nu_{23} = -\frac{S_{23}}{S_{22}}.$$

The word *approximated* in the printed heading is load-bearing. These nine numbers reproduce $\mathbf{C}^{3D}$ exactly only if that matrix is genuinely orthotropic in the SG axes. A cell with normal–shear coupling still prints them, and they will not rebuild the full matrix. The matrix, not the constants block, is the result.

3.4.2 Which entry points produce it

For `build_solid_bundle` called directly from Python the normalising area falls back to the convex hull of the contour (`area_source = "hull"`); pass `cell_area` explicitly to use a different measure, and the banner records which was used. The `opensg <yaml> CLI` never takes that fallback: it supplies the measure itself (`opensg_shell.cli.sg_cell_area`) as the `omega:` header key

Table 3.3: The three entry points that return an equivalent 3-D solid law, and the measure each divides the assembled energy by.

entry point	SG	normalisation
<code>plate_homo_2d(..., n_model=3)</code>	1-D/2-D/3-D solid SG	$\mathbf{C}_{\text{eff}} = (\bar{\mathbf{D}} + \mathbf{D}_1)/\omega$
<code>opensg_shell.build_solid_bundle</code>	1-D shell SG (cross-section contour)	$\mathbf{C}^{3\text{D}} = \mathbf{D}_{\text{eff}}/A_{\text{cell}}$
<code>opensg_shell.sg_homo.shell_sg3d</code>	3-D shell SG (surface mesh)	$\mathbf{D}_{\text{eff}}/\omega$, $\omega = \text{bounding-box volume}$

when the file declares one, and otherwise as the measured node *bounding box*, which is the periodic cell the assembly map ties and which differs from the convex hull on any non-convex cell. The isotropic square-tube example in the folder `examples/OpenSG_shell/3_get_solid_props_from_shell_cross_section/` `square_section_isotropic` sets `CELL_AREA = 4*a*t_w`, the wall material area, so that the shell route and the solid route are normalised identically and can be compared term by term. Its `square_tube_1Dshell_C3D.out` returns $E_1 = 7.0000000 \times 10^{10}$ Pa and $\nu_{12} = 0.30000$, recovering the wall material's own E and ν along the beam axis exactly, which is the sanity check that fixes the normalisation.

The last row of Table 3.3 is worth stating plainly, because for a three-dimensional SG the measure is a **volume**, not an area. The ω of `shell_sg3d` is the node bounding-box volume `ptp(y1) ptp(y2) ptp(y3)`, measured from the mesh: the volume the equivalent continuum occupies, and the same cell the periodic assembly map ties. It is the convention `opensg_solid` uses too, so the returned `C3D`, the line the CLI prints and `<yaml>_C3D.out` are one and the same matrix, and the printed moduli compare directly against a solid SwiftComp `.K` file with no rescale. An explicit `omega=` argument, or the YAML's `omega:` header key which the CLI forwards, overrides the measure — for instance with the midsurface *surface* area, which the result dict also returns as `"surface_area"`, if a per-unit-area wall law is what you want.

3.4.3 Periodicity, and the aperiodic alternative

A free SG driven by six macro strains is rank one by construction: every mode except $\bar{\Gamma}_{11}$ can be cancelled at zero energy by an affine fluctuation. Something must forbid those fields. All three routes *default* to periodicity, and periodicity is the SwiftComp-parity mode:

- `build_solid_bundle(..., periodic=True)` is the default; it ties opposite bounding-box faces through the shell periodic assembly map, and the repeated `dof_map[dof_map]` resolves the edge and corner chains, so faces, edges and corners are all tied. The tie rides in the assembly map, so no constraint rows are added and the rigid kernel drops from four rows to the three translations.
- `shell_sg3d(..., boundary="periodic")` is the default; the same map is applied on the full three-dimensional coordinates, and the three rigid translations are removed by area-weighted Lagrange rows.
- `plate_homo_2d(..., boundary=None)` resolves to `"periodic"`.

Two of these also accept an aperiodic treatment, and it is a genuine alternative, not a diagnostic. It is the *boundary-solution* treatment: for a unit cell the boundary solution is the macro (affine) field itself, and mapping it onto the boundary prescribes zero fluctuation there.

- `shell_sg3d(..., boundary="aperiodic")` clamps $w_1 = w_2 = w_3 = 0$ on every node of the bounding-box faces and leaves the three rotational DOFs **natural**. Translations are what the

macro solid strain prescribes; clamping the rotations as well over-stiffens the cut edges, and the in-code note records the measured effect of freeing them (+12% down to +7% on C_{11}). No Lagrange border is needed, because the Dirichlet data also fixes the rigid modes.

- `plate_homo_2d(..., boundary="aperiodic")` pins all three fluctuation DOFs of every bounding-box-face node — the solid engine carries no rotational DOFs, so those three are the whole node — instead of the usual first-node pin, and assembles on the raw mesh rather than the periodic map. It is supported for the plate and solid models with `solver="direct"`; requesting it with `n_model=1` or `solver="cg"` raises a `ValueError`.

Kinematic (Dirichlet) homogenization is an *upper bound* on a single cell, and it converges to the periodic answer as the SG grows, because the boundary layer scales like $1/N$. That convergence is the acceptance check.

The head-to-head is shipped. On the shell side, the script `compare_boundary_modes.py` in `examples/OpenSG_shell/4_get_solid_props_from_shell_3D_SG` runs `shell_sg3d` on the same Schwarz-P TPMS cell (Figure 3.5) in both modes and appends a per-unit-cell block to `boundary_compare.dat`:

```
python compare_boundary_modes.py
```

Table 3.4: Schwarz-P shell cell, per-unit-cell stiffness in MPa, from `boundary_compare.dat`. The aperiodic run clamps 720 free-face nodes (81216 DOF, 42.3 s) against 79056 DOF and 66.5 s for periodic.

term	$t = 0.036457$			$t = 0.129330$		
	aperiodic	periodic	diff %	aperiodic	periodic	diff %
C_{11}	2375.89	2225.31	+6.77	9881.00	8916.81	+10.81
C_{22}	2375.92	2225.28	+6.77	9881.00	8916.85	+10.81
C_{33}	2376.22	2225.03	+6.80	9873.34	8916.17	+10.74
C_{12}	1537.47	1564.87	-1.75	4921.09	5352.21	-8.06
C_{44}	1078.28	1040.98	+3.58	4150.25	3961.63	+4.76
C_{66}	1078.61	1040.93	+3.62	4151.27	3961.51	+4.79

Table 3.4 is exactly the expected signature: the direct terms come out stiffer (a kinematic upper bound), the Poisson terms softer, and the gap widens with wall thickness because the clamped boundary layer occupies more of the cell. The solid-side twin, the identically named script in `examples/OpenSG-solid/6_get_solid_props_from_3D_SG`, does the same for the tetrahedral TPMS SGs with `plate_homo_2d(n_model=3)` and shows a larger boundary layer on the denser cell: for Sample'1, C_{11} is 12977.0 MPa aperiodic against 10190.2 MPa periodic (+27.3%) with 17610 boundary nodes, while C_{12} falls 3.94%. Both runs take about 150 s, so the aperiodic mode costs nothing extra — the choice is physical, not computational.

3.4.4 A worked solid law: the TPMS Sample'1 cell

The reference case for the equivalent 3-D law of this section on the solid engine is `examples/OpenSG-solid/6_get_solid_props_from_3D_SG/sample_1`:

```
python run_sample_1.py
```

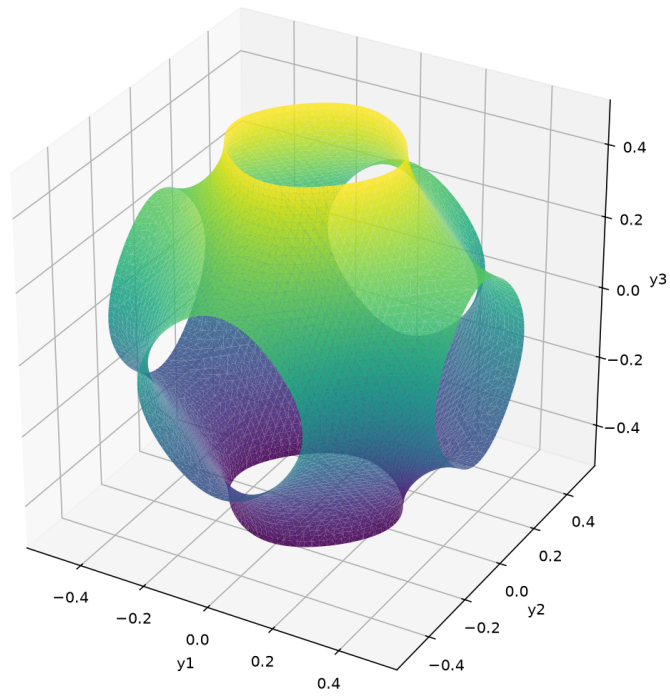


Figure 3.5: The Schwarz-P TPMS three-dimensional shell SG solved by `shell_sg3d`: 13536 nodes, 26360 shell elements, and zero junction edges (a smooth TPMS has every edge interior, so the junction treatment never fires). The banner of the `.out` reports all three counts. This is the SG behind Table 3.4: every number there is an entry of this mesh's equivalent 3-D solid 6×6 , the law `shell_sg3d` writes to `<yaml>_C3D.out`.

```

E, nu = 69.0e9, 0.30
G = E/(2*(1+nu))
r = plate_homo_2d(SC, material_param=jnp.array([(E, E, E, G, G, G,
                                                nu, nu, nu)]),
                  angles=jnp.array([0.0]), n_model=3, workdir=".", plot=True,
                  boundary="periodic") # SwiftComp .K digit-parity mode

```

Listing 3.4: run'sample1.py: a 546k-tet TPMS cell of aluminium, periodic in all three directions. The single material row is isotropic $E = 69$ GPa, $\nu = 0.3$, matched to the SwiftComp .K it is checked against.

This writes `Sample_1.out` (banner `OpenSG msg-solid 3D elastic model, omega 0.3`, where $\omega = 0.3$ is the relative density of the cell), the mesh figure `Sample_1_mesh.png`, the per-unit-cell matrix `Sample_1_msg-solid.out`, and the comparison table `Sample_1_compare.dat`. The engineering-constants block of `Sample_1.out` reads

Table 3.5: The engineering constants printed by `Sample_1.out` for the TPMS Sample1 cell (aluminium, $E = 69$ GPa, $\nu = 0.3$, $\omega = 0.3$).

constant	value (Pa)	constant	value (Pa)	constant	value
E_1	2.0547093×10^{10}	G_{12}	1.5065850×10^{10}	ν_{12}	0.35652386
E_2	2.0547272×10^{10}	G_{13}	1.5065455×10^{10}	ν_{13}	0.35654253
E_3	2.0546693×10^{10}	G_{23}	1.5065467×10^{10}	ν_{23}	0.35653028

Three things to read out of Table 3.5. First, the three Young's moduli agree to five significant figures although they come from three independent unit-strain solves on the same cell — that is the cubic symmetry of the geometry reappearing in the answer, and it is the cheapest available verification that the periodic tie caught all faces, edges and corners. Second, the aluminium is isotropic but the cell is not: $E_1/E = 0.298$ while $G_{12}/G = 0.568$, so a TPMS at 30% relative density retains far more shear stiffness than axial stiffness. Third, the off-diagonal terms of the printed stiffness sit at 10^5 against diagonals of 10^{10} , so the orthotropic approximation behind the constants block is legitimate here. The per-unit-cell comparison in `Sample_1_compare.dat` agrees with the SwiftComp .K to $\pm 0.000\%$ on all nine reported terms ($C_{11} = 10190.158$ MPa for both).

A second, independent validation of the same law is `examples/OpenSG-solid/5_get_solid_props_from_SG` (Figure 3.6), a UD fibre/matrix square cell meshed with 801 nodes and 1536 triangles. It ships a self-contained constant-strain-triangle driver that solves the same 6×6 law under generalized plane strain with periodic faces and writes `UDcomp_2D_Deфф.out` plus `UDcomp_2D_constants_vs_paper.dat`. Its nine constants reproduce the ASC-23 Table II Model-3 values ($E_1 = 167.2553$ GPa, $E_2 = E_3 = 11.4136$ GPa, $G_{12} = G_{13} = 6.8017$ GPa, $G_{23} = 3.0955$ GPa, $\nu_{12} = \nu_{13} = 0.31178$, $\nu_{23} = 0.57574$) to within 0.002% on every entry.

3.5 The wall law: per-section 8×8 ABDG

The msg-shell routes need a plate law for **each wall of the cross-section** before they can solve the shell SG at all, and that intermediate is exported so it can be inspected independently. `opensg_shell.sg_homo.write_abdg_out` writes one 8×8 stiffness and its compliance per layup section to `<yaml base>_ABDG.out`. Both `build_rm_bundle` and `build_solid_bundle` call it, so this file appears next to every shell .out without your having to ask for it.

The file header states the row convention verbatim:

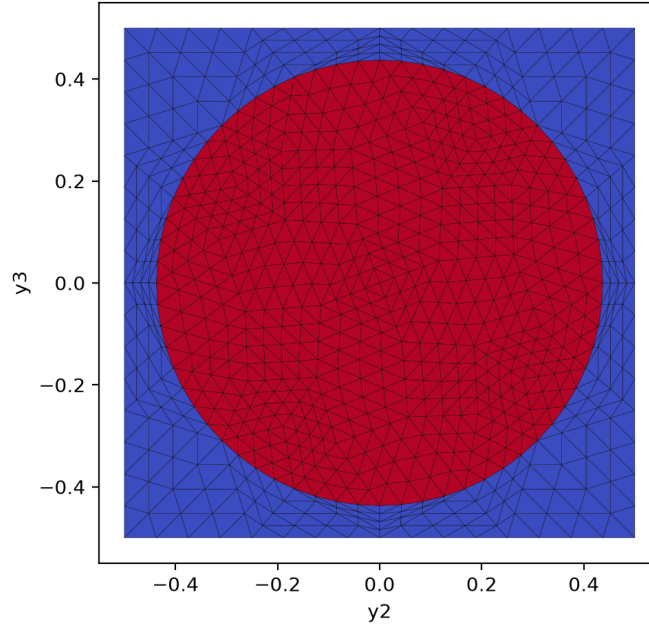


Figure 3.6: The UD fibre/matrix two-dimensional solid SG of `examples/OpenSG-solid/5_get_solid_props_from_SG`, 801 nodes and 1536 constant-strain triangles. The equivalent 3-D solid 6×6 of this section is this mesh’s law, written to `UDcomp_2D_Deфф.out`; the nine constants quoted in this section are its orthotropic-approximation block, and they reproduce the published values to 0.002 %.

```
OpenSG msg-shell wall plate laws, one block per section
rows [eps11 eps22 2eps12 | K11 K22 K12+K21 | 2g13 2g23]
```

so the block structure is the same $[[\mathbf{A}, \mathbf{B}, \mathbf{0}], [\mathbf{B}^\top, \mathbf{D}, \mathbf{0}], [\mathbf{0}, \mathbf{0}, \mathbf{G}]]$ as the Reissner–Mindlin plate law of Section 3.3, with the shell operators’ twisting measure $K_{12} + K_{21}$ in the sixth slot rather than κ_{12} . Each block is preceded by its section identity and full layup, for example the first block of `iea_s10_shell_ABDG.out`:

```
section 0: layup_0 layup [['gelcoat', 0.0005, 0.0], ['glass_triax', 0.00300294, 0.0],
                        ['glass_uniax', 0.02609467, 0.0], ['glass_triax', 0.00300294, 0.0]]
```

Each layup entry is `[material name, thickness (m), angle (deg)]`, bottom ply first. That section returns $A_{11} = 1.3701540 \times 10^9$ N/m, $A_{22} = 5.5764061 \times 10^8$ N/m, $A_{66} = 1.3539842 \times 10^8$ N/m, $D_{11} = 1.0897268 \times 10^5$ N·m, $G_{11} = 8.2828865 \times 10^7$ N/m and $G_{22} = 9.8804474 \times 10^7$ N/m, with a non-zero coupling block ($B_{11} = 3.1172920 \times 10^5$ N) because the stack — gelcoat, triax, uniax, triax — is not symmetric about the mid-surface.

There is no separate mesh figure for these blocks, because there are two meshes behind them and both are already shown. Section 0 is one element set of the cross-section SG of Figure 3.1, and the block itself is the solution of the through-thickness one-dimensional SG of that set’s layup, built by `rm_plate_msg` exactly as in Figure 3.4. One 8×8 per section is the entire wall-law export.

Two facts about how these blocks are built matter when comparing them against hand calculations.

1. The ABD is shifted by the parallel-axis rule to the reference surface the YAML declares. `reference: center` gives the laminate mid-surface and sets `fraction = 0.5`; `oml` gives 0.0,

`iml` and `oml_flip` give 1.0. The **B** block depends on that choice, and so therefore does every extension–bending coupling downstream of it.

2. The transverse-shear block **G** is **always replaced** by the MSG least-squares result from `rm_plate_msg`, evaluated at the same `fraction`. It is not a shear-correction-factor estimate, and there is no switch that selects anything else. The former alternative `g_source="whitney"` — the complementary-energy shear flow — has been retired: the keyword is still accepted by the old signatures, but passing it explicitly raises a `DeprecationWarning` and is ignored, and every emitted ABD cache records `g_source: msg`. The complementary-energy **G** survives only as the internal numerical fallback on a laminate whose MSG **U***-fit is not SPD.

3.6 Where each law is produced and written

Table 3.6 is the index for everything above: given a macro model, it names the function that computes the law, the file the law lands in, and the title printed inside that file.

Table 3.6: Macro model, producing function, output file, and the matrix title printed in the `.out`. Function names are given without their module prefix: `plate_homo_2d`, `_beam_homo_kkt`, `plate_shear_ladder` and `write_sc_K` live in `opensg_solid.sg_homo`, `rm_plate_msg` in `opensg_solid.rm_plate_1d.msg_rm_plate`, and the rest in `opensg_shell`. `<sg>` is the basename of the `.sc/.yaml` input, `<yaml>` the basename of the shell YAML, and `<db>` the basename of the `layup_db.yaml`.

macro law	producing function	output file	title in the <code>.out</code>
Timoshenko beam 6×6	<code>plate_homo_2d(n_model=1)</code> → <code>_beam_homo_kkt</code>	<code><sg>.out</code>	The Effective Timoshenko Beam Stiffness Matrix
Timoshenko beam 6×6	<code>build_rm_bundle</code> → <code>ring_indep</code>	<code><yaml>_Timo.out</code>	The Effective Timoshenko Beam Stiffness Matrix
Timoshenko beam 6×6	<code>segment_timo_from_3dyaml</code>	<code><yaml>_Timo.out</code>	The Effective Timoshenko Beam Stiffness Matrix
Euler–Bernoulli beam 4×4	<code>_beam_homo_kkt</code> → <code>r["C_eff_EB"]</code> (refined: 0)	<code><sg>.out</code>	The Effective Euler–Bernoulli Beam Stiffness Matrix
Classical plate ABD 6×6	<code>plate_homo_2d(n_model=2)</code>	<code><sg>.out</code>	The Effective Classical Plate Stiffness Matrix
RM plate ABDG 8×8	<code>plate_homo_2d(n_model=2, shear_refined=True)</code> → <code>plate_shear_ladder</code>	<code><sg>.out</code>	The Effective Reissner–Mindlin Stiffness Matrix
RM plate ABDG 8×8	<code>rm_plate_msg</code> → <code>r["ABDG"]</code>	<code><db>_plate_homo.out</code>	The Effective Reissner–Mindlin Stiffness Matrix
3-D solid 6×6	<code>plate_homo_2d(n_model=3)</code>	<code><sg>.out</code>	The Effective Cauchy Continuum Stiffness Matrix + engineering constants
3-D solid 6×6	<code>build_solid_bundle</code> → <code>ring_solid</code>	<code><yaml>_C3D.out</code>	The Effective Cauchy Continuum Stiffness Matrix + engineering constants
3-D solid 6×6	<code>sg_homo.shell_sg3d</code>	<code><yaml>_C3D.out</code>	The Effective Cauchy Continuum Stiffness Matrix + engineering constants
Wall plate law 8×8 , per section	<code>sg_homo.write_abdg_out</code>	<code><yaml>_ABDG.out</code>	The Effective Reissner–Mindlin Plate Stiffness Matrix, one block per section

3.7 Running the laws backwards: two-step recovery

Homogenization is only half of MSG. Each law above was obtained together with the fluctuation (warping) fields that produced it, and those fields let the macro answer be pushed back down to pointwise 3-D stress and strain inside the SG. The rule is *homogenize once, recover as often as needed*: every field the recovery needs rides along in the result dict, so no second homogenization is ever required.

3.7.1 The shell chain: beam force to plate resultants to 3-D stress

The shell chain has two interfaces, and naming them is the clearest way to see why it works. It is verified end to end in `examples/OpenSG_shell/IEA_blade_beam/2_dehom_v2_vabs.py`:

```
python 2_dehom_v2_vabs.py
```

Step 2a — beam to wall (opensg_shell). The macro beam strain follows from inverting the Timoshenko law of Section 3.2, $\mathbf{st} = \mathbf{C}_6^{-1} \mathbf{FF}$, with $\mathbf{FF}$ the six beam force and moment components in VABS order read from the load table `ff51_rmc_reform.dat`. For the IEA-22 station at $r/R = 0.20$ that row is

$$\mathbf{FF} = [4.1927 \times 10^4, 8.2674 \times 10^3, 1.4673 \times 10^6, 1.2191 \times 10^6, -6.0185 \times 10^7, 3.0426 \times 10^5]$$

in N and N·m, i.e. a flapwise-moment-dominated state. The SG being recovered is the cross-section of Figure 3.1, the same mesh that produced the 6×6 being inverted here. `_macro_fields` recombines the ring warping modes V_0 and V_1 into the nodal fields $w = V_0 \mathbf{st}_m + V_1 \mathbf{st}_{c1}$ and $w' = V_0 \mathbf{st}_{c1} + V_1 \mathbf{st}_{c2}$, and `_rm_shell_strain` then evaluates, per ring element, the six shell strains $\mathbf{s}_6 = [\varepsilon_{11} \ \varepsilon_{22} \ 2\varepsilon_{12} \ \kappa_{11} \ \kappa_{22} \ 2\kappa_{12}]$ together with the transverse pair $[2\gamma_{13} \ 2\gamma_{23}]$. Multiplying by that wall's own MSG ABD gives the interface quantity handed to the plate SG:

$$\mathbf{F}_6 = \mathbf{A}_6 \mathbf{s}_6 = [N_{11} \ N_{22} \ N_{12} \ M_{11} \ M_{22} \ M_{12}]^T. \quad (3.6)$$

These are exactly the resultants of the classical plate law of Section 3.3: the beam model has handed each wall a plate problem. They are written to `<name>_elem_plate_forces.dat`, one row per ring element, with columns

elem	y2(m)	y3(m)	e11	e22	2e12	k11	k22	2k12	N11	N22	N12(N/m)	M11	M22	M12(N)
------	-------	-------	-----	-----	------	-----	-----	------	-----	-----	----------	-----	-----	--------

The first row of `iea_s10_elem_plate_forces.dat` reads element 0 at $(y_2, y_3) = (4.677, 0.105)$ m with $\varepsilon_{11} = 5.427 \times 10^{-4}$ and $N_{11} = 7.149 \times 10^5$ N/m — a trailing-edge panel in tension under the flapwise moment.

Step 2b — wall to 3-D (opensg_solid.rm_plate_1D). For each distinct layup, `rm_plate_msg` builds the through-thickness SG once. Then `msgrm_strain_at_depth` — or its vectorized twin `msgrm_strain_at_depth_batch`, which is the same algebra evaluated with batched einsums and is what makes a 10^5 -point section sweep tractable — returns the 3-D Voigt strain, the 3-D Voigt stress and the ply angle at any depth z . Passing only the first strain gradients `dE1/dE2` selects the first-order recovery (Yu Eq. 63); adding `dE11/dE12/dE22` selects the second-order recovery (Eq. 66), which is what can carry the transverse pair σ_{33} and σ_{13} that first order cannot produce. Output is in the ply material frame by default (`frame="material"`), since failure criteria live there; pass `frame="plate"` for anything that integrates or differentiates across plies.

The files the D route writes. Run from the CLI as `opensg <yaml> D`, the chain above writes `<base>_dehom.txt` (one row per recovery point: 15 field columns, the six stresses, six strains and three displacements), the matching `<base>_dehom.vtk`, and — unless the header sets `junction: off` — the `<base>_dehom.junc` sidecar. At the default `junction: flag` tier the `.txt` carries two *extra* columns at the very end, `jflag` and `jdist`, which are also written as extra `CELL_DATA` scalars in the `.vtk`: they flag the recovery stations that fall inside a wall-junction boundary layer, where two crossing walls hang their through-thickness columns over the same material. Nothing else changes at `flag`. The full treatment, including the `exclude` tier and the `junction_bl` and `junction_ang` keys, is Section 4.2.4.

3.7.2 What the shell chain gets right, and where it does not

The IEA-22 station at $r/R = 0.20$ is validated against a VABS two-dimensional solid solution of the same section, sampled at every VABS Gauss point. The RMS differences, in MPa, web / skin, are recorded in `iea_s10_report.txt` and reproduced in Table 3.7.

Table 3.7: RMS difference against VABS, in MPa, web / skin, for the IEA-22 station at $r/R = 0.20$. The last column is the RMS of the VABS field itself, so a difference equal to it means the chain returns essentially zero for that component.

component	first order	second order (V2)	VABS RMS
σ_{11}	43.073 / 9.529	43.073 / 9.529	44.440 / 96.676
σ_{22}	0.612 / 0.482	0.612 / 0.482	0.704 / 0.998
σ_{12}	6.799 / 1.255	6.798 / 1.255	22.026 / 6.009
σ_{33}	0.081 / 0.049	0.081 / 0.049	0.081 / 0.049
σ_{13}	0.394 / 0.192	0.394 / 0.192	0.394 / 0.193
σ_{23}	0.035 / 0.028	0.035 / 0.028	0.035 / 0.028

Read the last column together with the others. The dominant σ_{11} is recovered to roughly 10 % of its own RMS in the skin, and the same holds along a through-thickness path in the spar cap (38.355 MPa difference against a VABS path RMS of 285.457 MPa). But the transverse components σ_{33} , σ_{13} and σ_{23} show differences *equal to* the VABS RMS itself, which means the shell chain returns essentially zero there. That gap in the transverse pair is the honest open item on this route. It is also why the second-order drivers exist even though their payload at this near-constant load state is small: V2 changes σ_{11} by 1.95×10^{-4} MPa RMS against a first-order RMS of 8.851×10^1 MPa, and its visible effect is on σ_{33} , where the first-order field is 3.6×10^{-13} MPa (machine zero) and V2 produces 5.19×10^{-5} MPa against a VABS RMS of 5.58×10^{-2} MPa.

The recovered in-plane stress over the full section is shown in Figure 3.7: the spar caps in compression above and tension below under the flapwise moment, with the shear flow visible in σ_{12} around the leading edge and the trailing-edge panels.

3.7.3 The solid chain

For an SG homogenized through `plate_homo_2d`, recovery is a single call in `opensg_solid.sg_dehom`. From `examples/OpenSG-solid/4_get_plate_dehom_from_2DSG`:

```
from opensg_solid.sg_homo import plate_homo_2d
from opensg_solid.sg_dehom import dehom_fields, export_gauss

r = plate_homo_2d("RHC_SW_2UC_45.yaml", material_param=material_param,
```

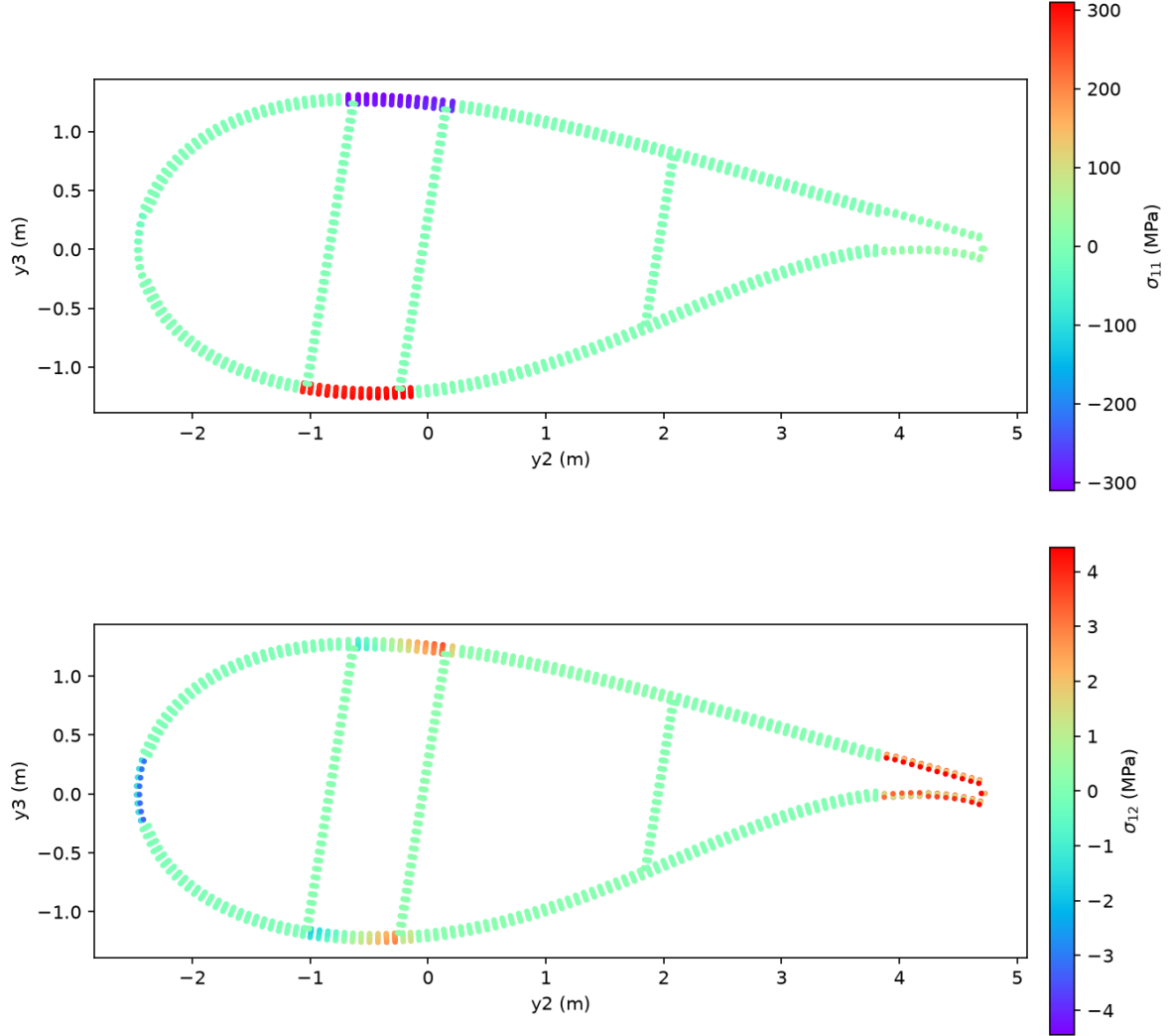


Figure 3.7: Recovered σ_{11} and σ_{12} over the IEA-22 blade cross-section at $r/R = 0.20$, plotted at the recovery points around the contour and the shear webs, from the two-step chain of Eq. (3.6) followed by `msgrm_strain_at_depth_batch`.

```

        angles=angles, n_model=2)
# the MACRO plate strain [e11, e22, 2e12, k11, k22, 2k12]
epsilon_bar = jnp.array([0.0, 0.1, 0.0, 0.1, 0.0, 0.0])
Gamma, Sigma, U = dehom_fields(r, epsilon_bar)
export_gauss(r, Gamma, Sigma, "RHC_SW_2UC_45_dehom", U_eqd=U)

```

Listing 3.5: The plate recovery driver `plate_dehom_2dsg.py`, reduced to its essentials. `epsilon_bar` is the user's macro state at the point of interest.

`epsilon_bar` is whichever macro state matches the model that was homogenized: the six plate strains $[\varepsilon_{11} \ \varepsilon_{22} \ 2\varepsilon_{12} \ \kappa_{11} \ \kappa_{22} \ 2\kappa_{12}]$ for `n_model=2`, the six macro strains of Section 3.4 for `n_model=3`, or the six Timoshenko beam strains of Section 3.2 for `n_model=1`. The beam case is run by `examples/OpenSG-solid/8_get_beam_dehom_from_SG/beam_dehom_sg.py`, whose shipped state is `epsilon_bar = jnp.array([0.001, 0.0, 0.0, 0.0, 0.01, 0.0])`, i.e. a unit axial extension of 10^{-3} combined with a κ_2 curvature of 10^{-2} . For `n_model=1` the valid recovery drivers are the classical-channel states (extension, twist, bending and their combinations); the shear channels do not drive the recovery chain.

The call returns, at every element Gauss point, the local strain `Gamma` of shape $(E, Q, 6)$, the local stress `Sigma` of shape $(E, Q, 6)$ — both in SwiftComp storage order (xx, yy, zz, yz, xz, xy) — and the fluctuation-only displacement `U` of shape $(E, Q, 3)$. `plate_dehom_2d` is a two-output view of the same single pass, and `gauss_coords(r)` returns the physical coordinates of the recovery points as $(E \cdot Q, n_{sg})$.

`export_gauss` writes five files from one recovery: `<prefix>.txt` (the tab-separated Gauss table, one row per point with coordinates, six strains and six stresses), `<prefix>.vtk` (a point cloud for ParaView carrying all twelve components as scalars), and the OpenSG dehom files `<prefix>.SM` (stress), `<prefix>.EM` (strain) and `<prefix>.U` (fluctuation displacement). One convention trap is worth flagging: the printed component order in the `.SM`, `.EM` and `.U` files is 11 22 33 12 13 23, not the SwiftComp order. The reorder happens on write only; the arrays you hold in memory stay in (xx, yy, zz, yz, xz, xy) . In those three files the SG coordinates occupy the *last* n_{sg} of the (x, y, z) columns, with the leading slots zero, so that the last SG coordinate — the plate thickness direction — always lands in the z column.

Chapter 4

Tutorials: the Shell Engine (msg-shell)

The `msg-shell` engine solves structure genes whose heterogeneity lives in a *surface*: the mid-surface of a thin composite wall. Thickness never appears in the mesh. It lives inside the wall’s Reissner–Mindlin plate law and inside the through-thickness recovery ladder, so changing a ply schedule is a change of material data, not a remesh.

Five capabilities ship as runnable examples, all under `examples/OpenSG_shell/`. Every one of them takes a YAML file in and writes a timed output file in the SwiftComp `.K` layout.

example folder	driver	entry point	produces
1_get_beam_props_...	beam_homo_shell.py	build_rm_bundle	Timoshenko 6×6
2_get_beam_dehom_...	beam_dehom_shell.py	build_rm_bundle + recovery	3-D wall stress
3_get_solid_props_...	four sub-folders	build_solid_bundle	equivalent solid 6×6
4_get_solid_props_...	schwarz_solid_props.py	shell_sg3d	equivalent solid 6×6
5_taper_segment_...	run_taper_segment.py	segment_timo_from_3dyaml	tapered-segment 6×6

Table 4.1: The five `msg-shell` capabilities and the example folder that demonstrates each one. Folder names are abbreviated; the full names appear at the head of each section below.

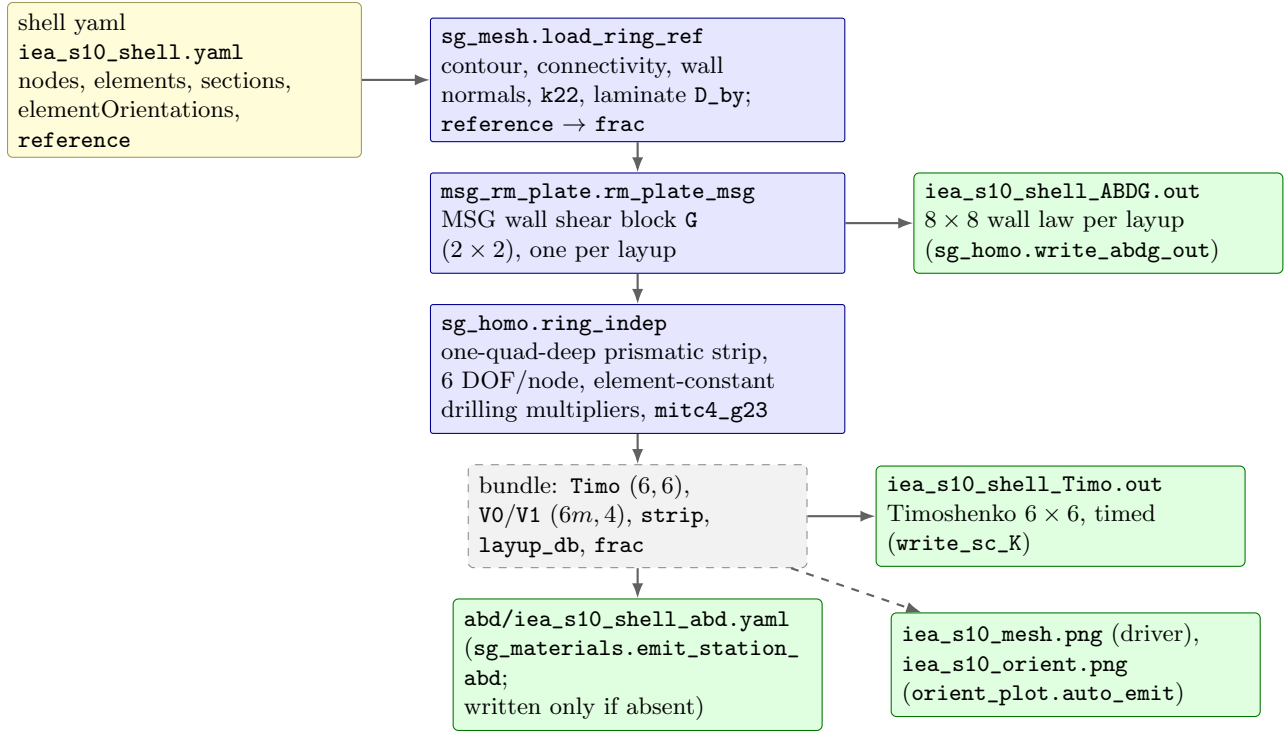
4.1 Beam properties from a shell cross-section

4.1.1 What this capability computes

Given the contour of a thin-walled composite cross-section, this run produces the 6×6 Timoshenko beam stiffness matrix of that section: the constitutive law a beam solver such as BeamDyn or GEBT needs. It also writes the 8×8 Reissner–Mindlin plate law of every distinct wall region, because those are the intermediate quantities the beam matrix is assembled from.

The example is `examples/OpenSG_shell/1_get_beam_props_from_shell_cross_section`, and the input is `iea_s10_shell.yaml`: station $r/R = 0.20$ of the IEA-22 MW reference blade.

The whole capability is the chain below: one YAML in, one ring solve, four files out.



4.1.2 What a 1-D shell SG is

A **1-D shell structure gene** is the contour of the cross-section: a chain of straight two-node line elements along the wall mid-surface, each element carrying a laminate stack as material data rather than as geometry. A 2-D solid cross-section mesh, by contrast, discretizes the wall *through* its thickness with area elements, so every ply interface must be resolved by the mesh.

The consequence for the user is size. This station spans about 7.2 m in y_2 and 2.5 m in y_3 , has spar caps 87.6 mm thick and three shear webs, and is fully described by 307 nodes and 310 line elements. The price is that the walls are modelled as Reissner–Mindlin plates, so wall-thickness effects enter through the plate law and the recovery ladder rather than through the mesh.

4.1.3 The input file the user edits

`build_rm_bundle` reads exactly one file. These are its blocks, with the values found in `iea_s10_shell.yaml`.

block	content	this station
<code>nodes</code>	<code>[y2 y3 0]</code> contour coordinates, metres	307 nodes
<code>elements</code>	1-based two-node line connectivity	310 elements
<code>sets: element</code>	named element sets, one per wall region	6 sets
<code>sections</code>	per set: <code>elementSet</code> plus <code>layup</code> as <code>[material, thickness, angle]</code> triples, outer ply first	6 sections
<code>elementOrientations</code>	per element, the flattened 3×3 frame $[e_1 \ e_2 \ e_3]$	310 rows
<code>materials</code>	<code>name, density, elastic: {E, G, nu}</code> as 3-vectors	6 materials
<code>reference</code>	the reference surface the whole run is referred to	center

Table 4.2: The seven YAML blocks read by `build_rm_bundle`.

The **reference** field is the single source of truth. `build_rm_bundle` reads it and maps it to a thickness fraction (`center` $\rightarrow$ 0.5, `oml` $\rightarrow$ 0.0, `iml` and `oml_flip` $\rightarrow$ 1.0). That one number is then used consistently for the ring laminate reference, the plate-SG z origin, the emitted ABD cache and the recovery depth conversion. Pass `ref=` explicitly only to override it. Because the reference here is the mid-surface, quantities that depend on the reference axis, in particular GJ and the bending-extension couplings, are mid-surface values.

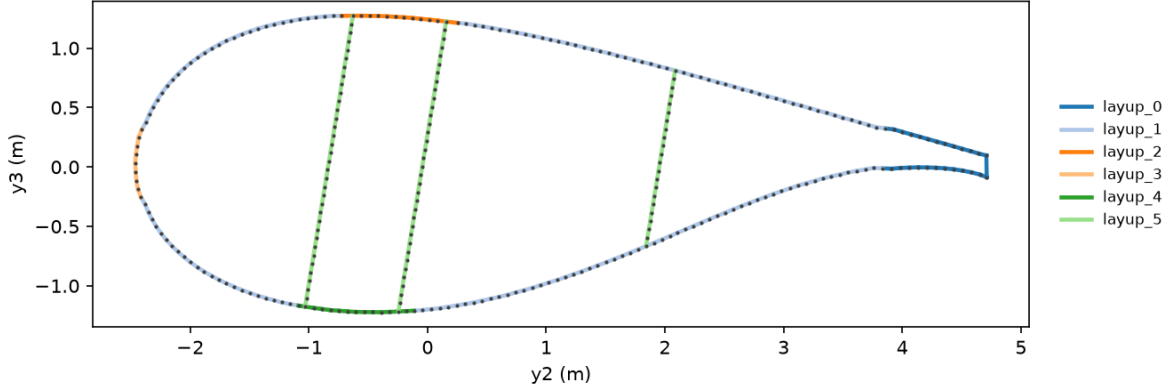


Figure 4.1: The 1-D shell SG of IEA-22 station $r/R = 0.20$: 307 nodes and 310 line elements along the wall mid-surface, coloured by `sets: element` entry. This is the figure written by `beam_homo_shell.py` as `iea_s10_mesh.png`.

Each colour in Figure 4.1 is one element set, and each set carries one laminate stack.

layup	plies, outer ply first	total thickness (m)	elements
layup_0	gelcoat, glass'triax, glass'uniax, glass'triax	0.03260055	25
layup_1	gelcoat, glass'triax, medium'density'foam, glass'triax	0.07711403	161
layup_2	gelcoat, glass'triax, carbon'uniax (0.08106 m), glass'triax	0.08756579	13
layup_3	gelcoat, glass'triax, glass'uniax, glass'triax	0.02808872	8
layup_4	gelcoat, glass'triax, carbon'uniax (0.07707 m), glass'triax	0.08358040	14
layup_5	glass'biax, medium'density'foam, glass'biax	0.04600000	89

Table 4.3: The six laminate stacks of station `iea_s10`, as read from the `sections` block and echoed in the header of each block of `iea_s10_shell_ABDG.out`.

`layup_2` and `layup_4` are the carbon spar caps on the upper and lower surfaces, `layup_1` is the foam-cored skin that covers most of the contour, `layup_5` is the three shear webs, `layup_0` is the trailing-edge reinforcement and `layup_3` the leading edge. Every ply angle in this file is 0° ; the biax and triax fabrics are supplied as smeared orthotropic materials rather than as individual plies, so the material frame of the recovery coincides with the wall frame here.

4.1.4 Orientation frames

A wrong `elementOrientations` block is the most common way to get a plausible-looking but wrong answer, so the example driver calls `auto_emit` explicitly after the homogenization and writes an orientation figure. Note that `auto_emit` is called *by the driver*, not from inside `build_rm_bundle`: if you write your own script around `build_rm_bundle`, add the `auto_emit` call yourself.

e_1 is the beam axis (out of the page), e_2 (blue) is the in-plane ply-flow direction along the contour, and e_3 (green) is the wall normal, which sets the OML $\rightarrow$ IML sense used by the recovery.

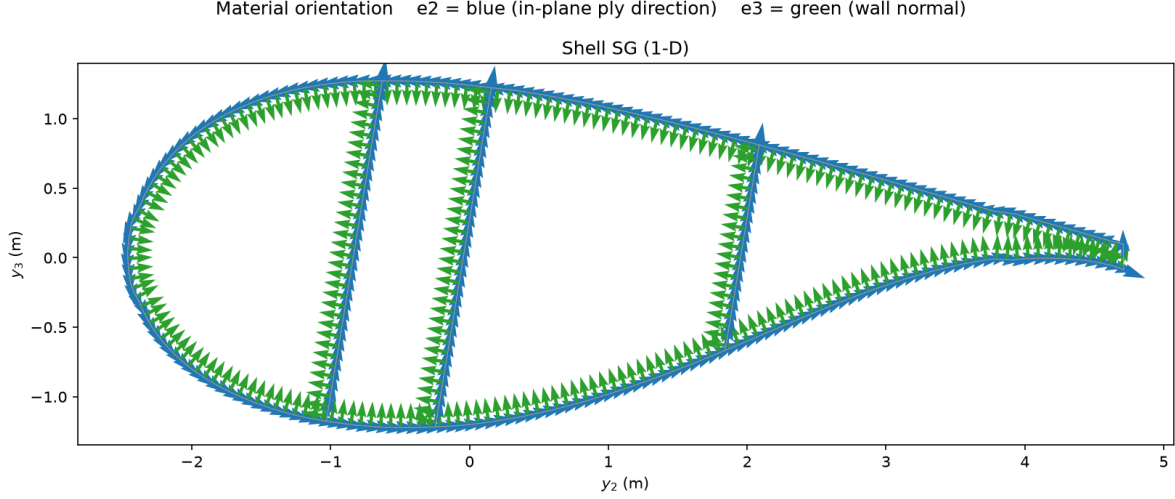


Figure 4.2: Per-element e_2 (in-plane ply direction) and e_3 (wall normal) on the contour, written by `auto_emit` as `iea_s10_orient.png`. Check that the normals point consistently into the section on the skin and consistently to one side on each web before trusting any stiffness number.

4.1.5 The element and the wall law

The ring is assembled as a one-quad-deep prismatic strip: the contour is extruded by the mean element length along the beam axis, and the extruded node row is DOF-mapped back onto the original row, which enforces the prismatic (span-invariant) condition exactly. All matrices are then divided by that depth, so the result is per unit length. The strip actually assembled is returned in the bundle under the key `strip` as `(nodes, quads, h)`.

Each node carries **six** degrees of freedom, $[w_1 \ w_2 \ w_3 \ \omega_1 \ \omega_2 \ \omega_3]$. The drilling rotation ω_3 is an *independent* field rather than an algebraic function of the displacements. Making it independent is what allows adjacent walls meeting at a junction to share a consistent rotation, but it also introduces a spurious mode, which is removed by a weak drilling constraint carried by one element-constant Lagrange multiplier per element (`lam_space="elem"`). Element-constant is the inf-sup-stable choice; equal-order nodal multipliers over-constrain as the mesh is refined.

Transverse shear uses assumed-strain (MITC) tying, and the production ring scheme (`shear="mitc4_g23"`, the default of `build_rm_bundle`) ties **only** the γ_{23} row. The reason is specific to a prismatic strip: under span invariance γ_{13} carries no fluctuation gradient and is algebraic in the directors, so tying it would alias the drilling mode instead of curing shear locking. Only γ_{23} pairs a differentiated displacement with a rotation.

The wall constitutive law is therefore the 8×8 Reissner–Mindlin plate law

$$\mathbf{ABDG} = \begin{bmatrix} \mathbf{A} & \mathbf{B} & \mathbf{0} \\ \mathbf{B} & \mathbf{D} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{G} \end{bmatrix}, \quad (4.1)$$

driven by the eight-component generalized wall strain

$$\bar{\varepsilon} = [\varepsilon_{11} \ \varepsilon_{22} \ 2\varepsilon_{12} \ K_{11} \ K_{22} \ K_{12} + K_{21} \ 2\gamma_{13} \ 2\gamma_{23}]^T,$$

with exactly that row grouping printed in the header of `iea_s10_shell_ABDG.out`: rows `[eps11 eps22 2eps12 | K11 K22 K12+K21 | 2g13 2g23]`.

The 2×2 transverse-shear block $\mathbf{G}$ is built by **one** route, with no switch to set: the MSG/VAM second-order-energy projection (the Yu–2002 least-squares construction, Eq. 61 of Yu, Hodges & Volovoi, *Computers & Structures* **81**:439–454, 2003), computed by `opensg_solid.rm_plate_1D.msg_rm_plate.rm_plate_msg`. The former alternative `g_source="whitney"` — the coupling-aware complementary-energy shear flow — has been retired. The keyword is still accepted by the old signatures, but passing it explicitly raises a `DeprecationWarning` and is **ignored**, and every emitted ABD cache records `g_source: msg`. The complementary-energy $\mathbf{G}$ survives only as the internal numerical fallback on a laminate whose MSG U^* -fit is not SPD. Table 4.4 shows why this is not a detail on a foam-cored blade section.

layup	G_{11} (N/m)	G_{22} (N/m)
layup_0	8.2828865×10^7	9.8804474×10^7
layup_1	4.1457954×10^6	4.1451801×10^6
layup_2	2.8674578×10^8	2.1272143×10^8
layup_3	5.8528099×10^7	8.6668255×10^7
layup_4	2.7301565×10^8	2.0317797×10^8
layup_5	2.4877763×10^6	2.4877763×10^6

Table 4.4: The MSG wall transverse-shear block $\mathbf{G}$ of each layup, rows 7 and 8 of the 8×8 blocks in `iea_s10_shell_ABDG.out`. The foam-cored skin `layup_1` and the foam-cored webs `layup_5` are roughly seventy and a hundred times more shear-compliant than the carbon cap `layup_2`, so the soft core dominates the section’s shear compliance and must not be assigned by a rule of thumb.

4.1.6 Run it

The driver has a single user-input line:

```
##### User Input #####
name = "iea_s10"           # reads <name>_shell.yaml
#####
```

Listing 4.1: The user-input block of `beam_homo_shell.py`.

The homogenization itself needs no driver. `opensg` is the unified entry point: it resolves the engine from the YAML’s `msg:` key (or, when that key is absent, from the mesh dialect) and hands the file to `opensg_shell`.

```
cd examples/OpenSG_shell/1_get_beam_props_from_shell_cross_section
opensg iea_s10_shell.yaml
```

That writes `iea_s10_shell_Timo.out` and `iea_s10_shell_ABDG.out`. Run the driver when you also want the two figures, which the CLI does not draw:

```
python beam_homo_shell.py
```

4.1.7 The one call

Everything is behind one function.

```
import numpy as np
from opensg_shell import build_rm_bundle, auto_emit
```

```

B = build_rm_bundle("iea_s10_shell.yaml") # RM 6-DOF ring + MSG wall G
C6 = np.asarray(B["Timo"])               # (6, 6) Timoshenko stiffness
print(B["ref"], B["g_source"])           # "center msg"
print(np.diag(C6))
auto_emit("iea_s10_shell.yaml", out_png="iea_s10_orient.png")

```

Listing 4.2: The minimal script: build the bundle, take the 6x6, emit the orientation figure.

`build_rm_bundle` returns a **bundle**, not just a matrix, because the dehomogenization of Section 4.2 must reuse the identical discretization. The bundle carries the keys of Table 4.5.

key	content
Timo	the (6, 6) Timoshenko stiffness
V0, V1	zeroth- and first-order warping fields, $(6m, 4)$ each, including the drilling ω_3 boundary values
corners, red_cells	the $(m, 2)$ contour coordinates and $(n_{el}, 2)$ connectivity
rx3, re3, k22	3-D ring coordinates, wall normals and hoop curvature per element
strip	the prismatic strip actually assembled, as (nodes, quads, h)
ax, cross	the beam-axis index and the two cross-section axis indices
layup_per_elem	the layup name of each element
layup_db, material_db	the geometry-free plate-SG databases
frac, ref, g_source	the reference fraction, its name, and the wall- G source

Table 4.5: Keys of the dictionary returned by `build_rm_bundle`.

4.1.8 What comes out

file	written by	content
iea_s10_shell_Timo.out	<code>build_rm_bundle</code> (via <code>write_sc_K</code>)	the effective Timoshenko stiffness and compliance in the SwiftComp <code>.K</code> layout, with an <code>OpenSG</code> banner and a <code>Time taken</code> footer
iea_s10_shell_ABDG.out	<code>build_rm_bundle</code> (via <code>write_abdg_out</code>)	one 8×8 RM plate stiffness and compliance block per section, each headed by its layup
abd/iea_s10_shell_abd.yaml	<code>emit_station_abd</code>	per-layup cache of the 8×8 wall law plus ply lists, thickness and mass per area
iea_s10_mesh.png	the driver	the contour coloured by layup
iea_s10_orient.png	<code>auto_emit</code>	the e_2/e_3 arrows

Table 4.6: Output files of `beam_homo_shell.py`.

Two notes on the ABD cache. It is written **only if it does not already exist**, so delete `abd/iea_s10_shell_abd.yaml` after changing the layups — or after upgrading `OpenSG`, since a cache left over from an older version is never refreshed in place — or the change will not take. And it records `g_source: msg` (with `reference: mid`, the same fraction = 0.5 this station's `reference: center` resolves to): `emit_station_abd` builds the same MSG wall **G** the ring uses, at the station's own reference surface, so the cached 8×8 agrees with the per-section blocks of `_ABDG.out`. It is a cache for downstream station-level tools such as shell buckling.

4.1.9 The numbers

This is the stiffness block of `iea_s10_shell_Timo.out` exactly as written, in SI units, on the strain set $[\varepsilon_{11} \gamma_{12} \gamma_{13} \kappa_1 \kappa_2 \kappa_3]$ (extension, the two transverse shears, twist, and the two bending curvatures, which is the VABS ordering):

The Effective Timoshenko Stiffness Matrix					
2.7660599E+010	-1.7230658E-005	4.6650498E-006	5.4020232E-006	1.6686413E+009	-2.0773040E+009
-1.7230658E-005	7.1861326E+008	5.7520370E+007	-1.0192044E+008	2.3079953E-006	5.2738749E-006
4.6650498E-006	5.7520370E+007	4.2174428E+008	-1.6217837E+008	8.4816767E-006	2.7931393E-007
5.4020232E-006	-1.0192044E+008	-1.6217837E+008	2.4375653E+009	1.6449557E-006	-8.1140553E-006
1.6686413E+009	2.3079953E-006	8.4816767E-006	1.6449557E-006	3.5143887E+010	-5.5571249E+009
-2.0773040E+009	5.2738749E-006	2.7931393E-007	-8.1140553E-006	-5.5571249E+009	6.8129724E+010

The diagonal is what the driver prints.

term	value	units	meaning
EA	2.7660599×10^{10}	N	axial stiffness
GA_2	7.1861326×10^8	N	transverse shear, chordwise
GA_3	4.2174428×10^8	N	transverse shear, flapwise
GJ	2.4375653×10^9	N·m ²	torsion about the mid-surface reference
EI_2	3.5143887×10^{10}	N·m ²	flapwise bending
EI_3	6.8129724×10^{10}	N·m ²	edgewise bending

Table 4.7: The Timoshenko diagonal of station `iea_s10`, read from `iea_s10_shell_Timo.out`.

The off-diagonals are the physics that a beam table without couplings would throw away. The extension–bending entries 1.6686×10^9 and -2.0773×10^9 N·m locate the tension centre away from the mid-surface reference point; -5.5571×10^9 N·m² is the flap–edge bending coupling, which says the section’s principal bending axes are not the y_2/y_3 axes; and -1.0192×10^8 and -1.6218×10^8 N·m are the shear–twist couplings that place the shear centre.

Entries printed at 10^{-5} against diagonals of 10^{10} are exact zeros to about fifteen digits: extension does not couple to transverse shear or to twist on this section. That is a useful self-check that the drilling constraint and the rigid-body kernel are behaving.

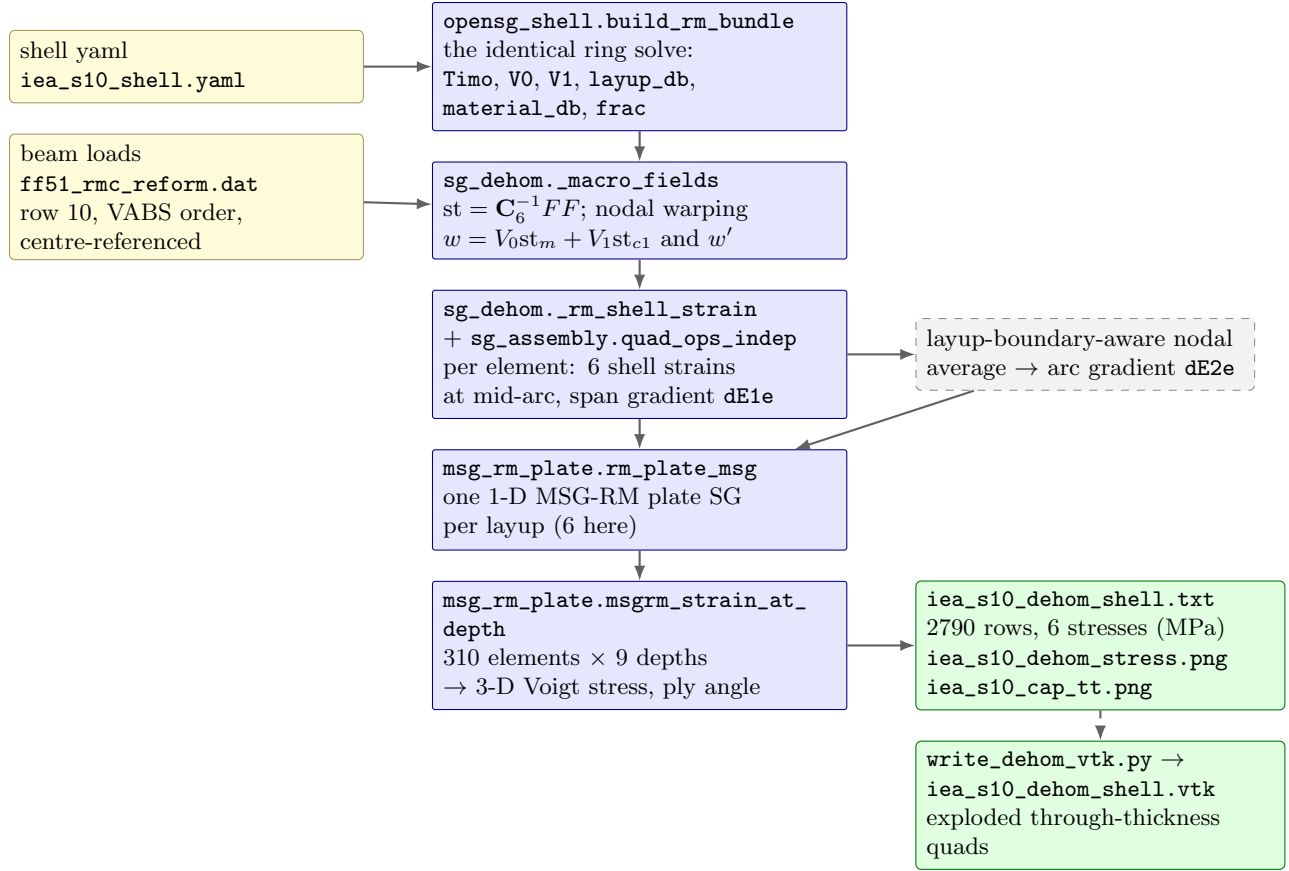
The footer of the same file reads **Time taken:** 8.02 sec for this run. That covers the whole bundle build: reading the YAML, the six per-layup MSG plate solves, the ring KKT factorization and the layup-database build.

4.2 Two-step recovery to 3-D wall stress

4.2.1 What this capability computes

Homogenization gave the beam its stiffness. *Dehomogenization* does the inverse: given the beam’s internal forces at this station, it reconstructs the 3-D stress at every point of every wall, which is what a failure criterion actually needs. The example is `examples/OpenSG_shell/2_get_beam_dehom_from_shell_cross_section` and runs on the same `iea_s10_shell.yaml`.

The chain below is the whole recovery. Note that it starts by rebuilding the *same* bundle as Section 4.1: nothing about the discretization is re-derived for the recovery.



4.2.2 The input the user edits

```

##### User Input #####
name = "iea_s10"           # reads <name>.yaml
ff_file = "ff51_rmc_reform.dat" # beam FF per station (# eta F1 F2 F3 M1 M2 M3)
station_row = 10          # row 10 -> eta = r/R = 0.20 (iea_s10)
n_depth = 9               # through-thickness recovery points per element
stress_frame = "material"  # "material" (ply axes) | "plate" (wall axes)
#####

```

Listing 4.3: The user-input block of beam-dehom-shell.py.

ff51_rmc_reform.dat is a plain table whose header line reads **# eta F1 F2 F3 M1 M2 M3** (RM CENTER-ref BeamDyn, VABS order): one row per spanwise station, the first column $\eta = r/R$ and the remaining six the beam force and moment resultants, in the same VABS order and the same SI units (N, N·m) as the 6 × 6. Row 10 is

$FF = [4.1927 \times 10^4 \quad 8.2674 \times 10^3 \quad 1.4673 \times 10^6 \quad 1.2191 \times 10^6 \quad -6.0185 \times 10^7 \quad 3.0426 \times 10^5]$, a 60 MN·m flapwise moment with a 1.47 MN flapwise shear, which is what dominates everything that follows. The forces must be referred to the same axis as the stiffness: this table is center-referenced, matching **reference: center** in the YAML.

4.2.3 Run it

The recovery is the D argument of the unified entry point: it homogenizes and then recovers, in one timed run.

```
cd examples/OpenSG_shell/2_get_beam_dehom_from_shell_cross_section
opensg iea_s10_shell.yaml D
```

That writes `iea_s10_shell_dehom.txt`, `.vtk` and `.junc`. Run the driver when you also want the two stress figures, which the CLI does not draw:

```
python beam_dehom_shell.py
```

4.2.4 Junction-aware recovery

A midline shell SG hangs a full through-thickness column off *every* element mid-arc. Within about a wall thickness of a junction the columns of two crossing walls therefore cover the **same material twice**, with two different wall laws — which is how a spar-cap ply stress ends up reported at a web station. `src/opensg_shell/sg_dehom_junction.py` does not repair that; it *measures* it, and the YAML header says what to do about it. Three scalar keys, all optional and all read only on the D route of `msg-shell`:

key	default	meaning
<code>junction</code>	<code>flag</code>	the recovery tier: <code>off</code> , <code>flag</code> or <code>exclude</code>
<code>junction_bl</code>	1.0	k_{bl} , the boundary-layer radius in units of the junction's <i>thickest</i> wall
<code>junction_ang</code>	1.0	the tangent-grouping tolerance of the junction detector, in degrees

Table 4.8: The three junction keys of the shell dehomogenization header.

tier	behaviour
<code>off</code>	the pre-junction behaviour exactly: no extra columns, no sidecar, no census, bit-identical files
<code>flag</code>	the default , and non-mutating : two extra columns <code>jflag</code> and <code>jdist</code> at the <i>end</i> of <code><base>_dehom.txt</code> , the same two as extra <code>CELL_DATA</code> scalars in <code><base>_dehom.vtk</code> , the <code><base>_dehom.junc</code> sidecar, and a printed census. It changes no field value
<code>exclude</code>	as <code>flag</code> , plus <code>NaN</code> in the 15 field columns of every duplicate (<code>jflag</code> 3) row, so the point cloud <i>partitions</i> the material instead of double-covering it. The row itself stays, so row i is still element $\lfloor i/n_{\text{depth}} \rfloor$ at depth $i \bmod n_{\text{depth}}$. The <code>.vtk</code> values are never <code>NaN</code> -ed — threshold on <code>jflag</code> instead

Table 4.9: The three implemented junction tiers. Because the two columns are appended at the end, every existing column index is untouched.

A fourth tier, `patch`, is **designed but deliberately not implemented** and raises if requested. The corner micro-solve it would call is admissible only for two straight walls of uniform layup crossing at one node, which most real blade junctions — a web landing on a tapering spar cap, a three-way leading-edge node, coincident doublers — are not.

The two new columns are defined as follows. `jflag` is 0 clean, 1 near, 2 overlapped and this wall owns the block, 3 duplicate (another wall owns it), and 4 is reserved for `patch` and is never

emitted. `jdist` is $|p - J| / \max_i t_i$ at the nearest junction node J (infinite when the section has no junction at all), so `jflag` ≥ 1 holds exactly when `jdist` $\leq k_{bl}$.

Both remaining keys are modelling choices, not derived lengths. `junction_bl` sets k_{bl} : the RM wall solution is polluted within roughly one wall thickness of a crossing, and 1.0 is the conventional Saint-Venant decay scale, nothing more; widen it to be conservative. `junction_ang` is the detector's tangent-grouping tolerance, and its default of 1.0° is the homogenization census value and deliberately tight: on a curved airfoil contour it also calls every $>1^\circ$ contour *kink* a junction. Raise it to 5–15° for a census that lists the wall crossings only.

```
junctions : 97 detected (5 T, 91 L, 1 unresolved)
stations  : 2790 total, 1032 near, 56 overlapped (50 duplicates)
```

That is the measured census of this station, read from `iea_s10_shell_dehom.junc`: of 97 detected junctions only a handful are genuine wall crossings, the rest being discretized curvature at the default 1.0° tolerance, and 1032 of the 2790 recovery stations fall inside some boundary layer while only 56 are truly overlapped.

One warning. This is **not** the same switch as the *homogenization* junction correction `build_solid_bundle(junction="census" | "micro" | "microcell")` of Section 4.3. That one is an energy correction with its own validity domain and it stays off by default; the tier here is recovery bookkeeping, and at `flag` it corrects nothing at all.

4.2.5 The two-step chain

Step 1, along the contour. The macro beam strain is $st = \mathbf{C}_6^{-1} FF$. That strain is fed through `_macro_fields`, which recombines the bundle's warping fields into the nodal warping $w = V_0 st_m + V_1 st_{c1}$ and its axial derivative $w' = V_0 st_{c1} + V_1 st_{c2}$, and then through `_rm_shell_strain`, which applies the *same* element operators used to assemble the stiffness. The result is, per element, the six shell strains $[\varepsilon_{11} \ \varepsilon_{22} \ 2\varepsilon_{12} \ K_{11} \ K_{22} \ 2K_{12}]$ at mid-arc. Because it reuses `quad_ops_indep`, step 1 is the exact energy-consistent adjoint of the homogenization rather than a separate post-processing model.

The recovery also needs the *gradients* of those strains. The span gradient $\partial/\partial y_1$ comes from the operators directly; the arc gradient $\partial/\partial y_2$ is a finite difference of nodally averaged values, and the averaging is **layup-boundary aware**. Two element values are averaged at a node only when both elements carry the same layup, so no difference is ever taken across a section change where the strains legitimately jump.

Step 2, through the thickness. For each of the 6 layups, `rm_plate_msg` builds a 1-D MSG-RM plate structure gene (the warping ladder plus the MSG $\mathbf{G}$), and `msgrm_strain_at_depth` evaluates the first-order recovery at a depth z measured from the reference surface, returning the 3-D Voigt strain, the 3-D Voigt stress $[\sigma_{11} \ \sigma_{22} \ \sigma_{33} \ \sigma_{23} \ \sigma_{13} \ \sigma_{12}]$ and the ply angle at that depth. Both functions are imported from `opensg_solid.rm_plate_1D.msg_rm_plate`, shared verbatim with the plate and solid examples.

The driver evaluates `n_depth = 9` ply-interior stations per element, at $\zeta = (k + \frac{1}{2})/9$ with $\zeta = 0$ at the OML and $\zeta = 1$ at the IML, giving $310 \times 9 = 2790$ recovery points.

```
import numpy as np
from opensg_shell import build_rm_bundle, _macro_fields, _rm_shell_strain
from opensg_solid.rm_plate_1D.msg_rm_plate import rm_plate_msg, msgrm_strain_at_depth

B = build_rm_bundle("iea_s10_shell.yaml")
```



```

FF = np.loadtxt("ff51_rmc_reform.dat")[10, 1:]
st, st_m, aA, aB = _macro_fields(B, beam_force_vabs=FF)
s6, s2 = _rm_shell_strain(B, 75, 0.5, st_m, aA, aB)    # element 75, mid-arc

ldb = B["layup_db"]; mdb = B["material_db"]; frac = float(B["frac"])
warp = rm_plate_msg(ldb["layup_2"]["thick"], ldb["layup_2"]["angles"],
                    ldb["layup_2"]["mat_names"], mdb, fraction=frac)
h = float(sum(ldb["layup_2"]["thick"]))
Gam, Sig, ply = msgrm_strain_at_depth(warp, (0.056 - frac) * h,
                                     np.asarray(s6, float), frame="material")
print(Sig / 1e6)    # MPa, Voigt [S11 S22 S33 S23 S13 S12]

```

Listing 4.4: The recovery chain in miniature: one element, one depth.

4.2.6 What comes out

file	content
iea_s10_dehom_shell.txt	2790 rows, one per recovery point: elem , y2 , y3 , z from the reference surface, zeta , ply angle in degrees, then the six stresses in MPa. The header repeats the frame, η and the full FF.
iea_s10_dehom_stress.png	the section stress cloud, σ_{11} and σ_{12}
iea_s10_cap_tt.png	σ_{11} and σ_{12} against ζ along one through-thickness line at the arc-centre element of the peak-stress cap

Table 4.10: Output files of beam_dehom_shell.py.

The driver reports the peak as

```

peak |sigma11| = 318.18 MPa at (y2, y3) = (-0.677, 1.311), zeta = 0.06 [material frame]

```

4.2.7 Reading the result

The axial stress is confined almost entirely to the two spar caps, exactly as a flap-dominated load state should be.

The VTK file is not written by beam_dehom_shell.py itself. The sibling script write_dehom_vtk.py imports that driver, so the physics is not duplicated, and re-packages its recovered values as the exploded quad grid; docs/latex/render_pv.py then renders the two panels above with pvpython.

quantity	value	where
minimum σ_{11}	-318.177 MPa	element 75, layup_2, $(y_2, y_3) = (-0.677, 1.311)$, $\zeta = 0.056$
maximum σ_{11}	+314.461 MPa	element 150, layup_4, $(y_2, y_3) = (-0.145, -1.249)$, $\zeta = 0.056$
maximum $ \sigma_{12} $	5.573 MPa	element 11, layup_0, $(y_2, y_3) = (3.895, 0.335)$

Table 4.11: Extremes of iea_s10_dehom_shell.txt under the load state FF of row 10 of ff51_rmc_reform.dat.

The upper cap is in compression and the lower cap in tension under this flap-dominated state, at nearly equal magnitude, and both extremes fall at the outermost recovery station $\zeta = 0.056$, which for these layups lies just inside the carbon uniaxial ply. That is where a maximum-stress

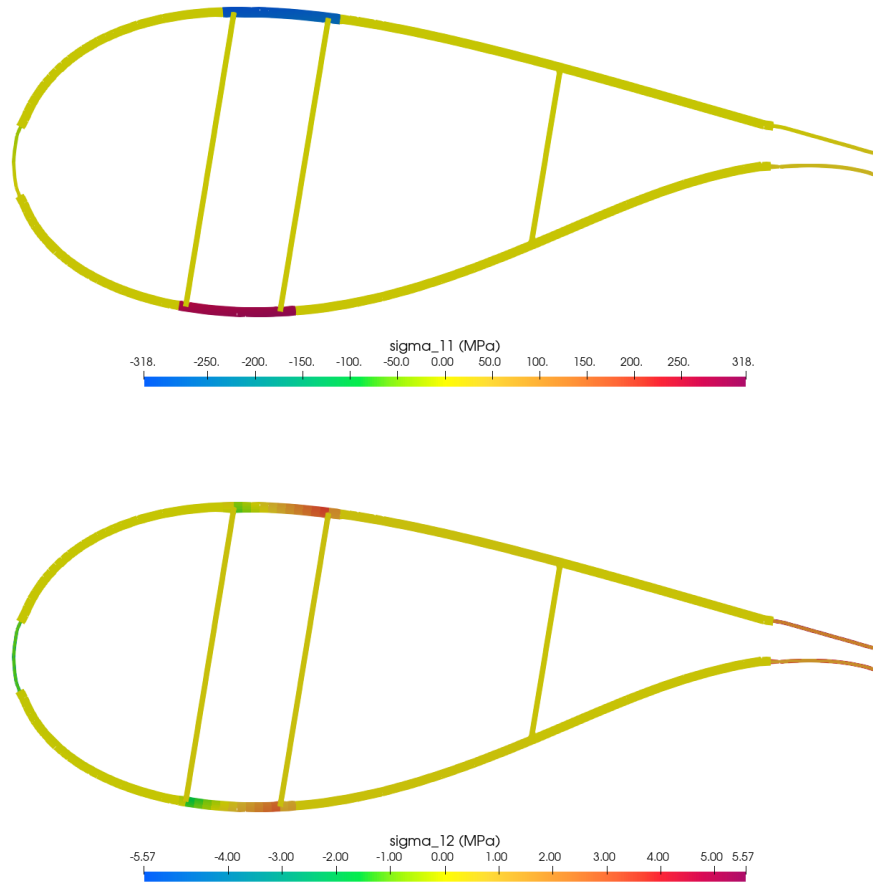


Figure 4.3: Recovered σ_{11} (top) and σ_{12} (bottom) over the cross-section, in MPa, rendered from `iea_s10_dehom_shell.vtk`. These are *conformal exploded-mesh* contours: `write_dehom_vtk.py` emits one filled quad per contour element per through-thickness interval $\zeta \in [k/9, (k+1)/9]$, so the 310 elements become $310 \times 9 = 2790$ patches carrying 11 160 points of their own. Each patch is flat-shaded with the stress of its own recovery station as `CELL_DATA`, so no value is averaged across an element boundary or onto a shared node, and because the nine intervals tile the full layup thickness h the wall is drawn at its true thickness rather than as a line.

check on the carbon belongs. The in-plane shear tells a different story: it peaks not in the caps but in the trailing-edge and leading-edge regions, where the closed cell carries the torque as shear flow.

4.2.8 What this route recovers, and what it does not

The repository's `examples/OpenSG_shell/IEA_blade_beam` benchmark folder compares this same station's recovery against a VABS 2-D solid run of the identical section (`iea_s10.sg.SM`), and records the result in `iea_s10_report.txt` as rms differences alongside the rms magnitude of the VABS field itself.

component	region	rms difference (MPa)	rms of the VABS field (MPa)
σ_{11}	skin	9.529	96.676
σ_{11}	web	43.073	44.440
σ_{12}	web	6.799	22.026
σ_{11}	spar-cap through-thickness path	38.355	285.457
σ_{11}	cap-left junction path	1.265	286.837

Table 4.12: Two-step shell recovery against a VABS 2-D solid run of the same section, from `iea_s10_report.txt`.

The in-plane picture is sound where the load is: about 10 % rms on the skin, 13 % along the cap thickness, and 0.4 % along the reference junction path. The web σ_{11} difference is the same size as the field itself, which is honest to state plainly: the webs carry almost no axial stress at this state, so the comparison there is a ratio of two small numbers.

Two components are **not** delivered by this route and should not be used from the output file:

- σ_{33} is machine-zero in the first-order recovery (rms 3.6×10^{-13} MPa against a VABS rms of 5.6×10^{-2} MPa). This is visible directly in the `S33` column of `iea_s10_dehom_shell.txt`, whose entries are of order 10^{-13} (median 1.96×10^{-13} , maximum 9.18×10^{-13}).
- σ_{13} is recovered at rms 6.3×10^{-3} MPa against a VABS rms of 2.4×10^{-1} MPa. This is structural, not a bug: on a prismatic ring the γ_{13} row is untied by construction, so it carries the flat-wall drilling mode and cannot serve as a recovery target. The companion diagnostic `iea_s10_q_consistency.txt` quantifies this. The tied γ_{23} channel closes its equilibrium identity $\int \sigma_{23} dz = (\mathbf{G}_{\text{msg}} s_2)_2$ to 1.05 % median error, while the untied γ_{13} channel does not, giving a σ_{13} target that is invalid by a factor of order 100. A physical σ_{13} needs the surface-quad route of Sections 4.4 and 4.5, where both rows are tied, not a rescale of this ring.

The same benchmark folder also records that adding the second-order recovery rung changes σ_{11} by an rms of 1.95×10^{-4} MPa against a first-order rms of 88.51 MPa, so the first-order chain used by `beam_dehom_shell.py` is the right default for this class of section.

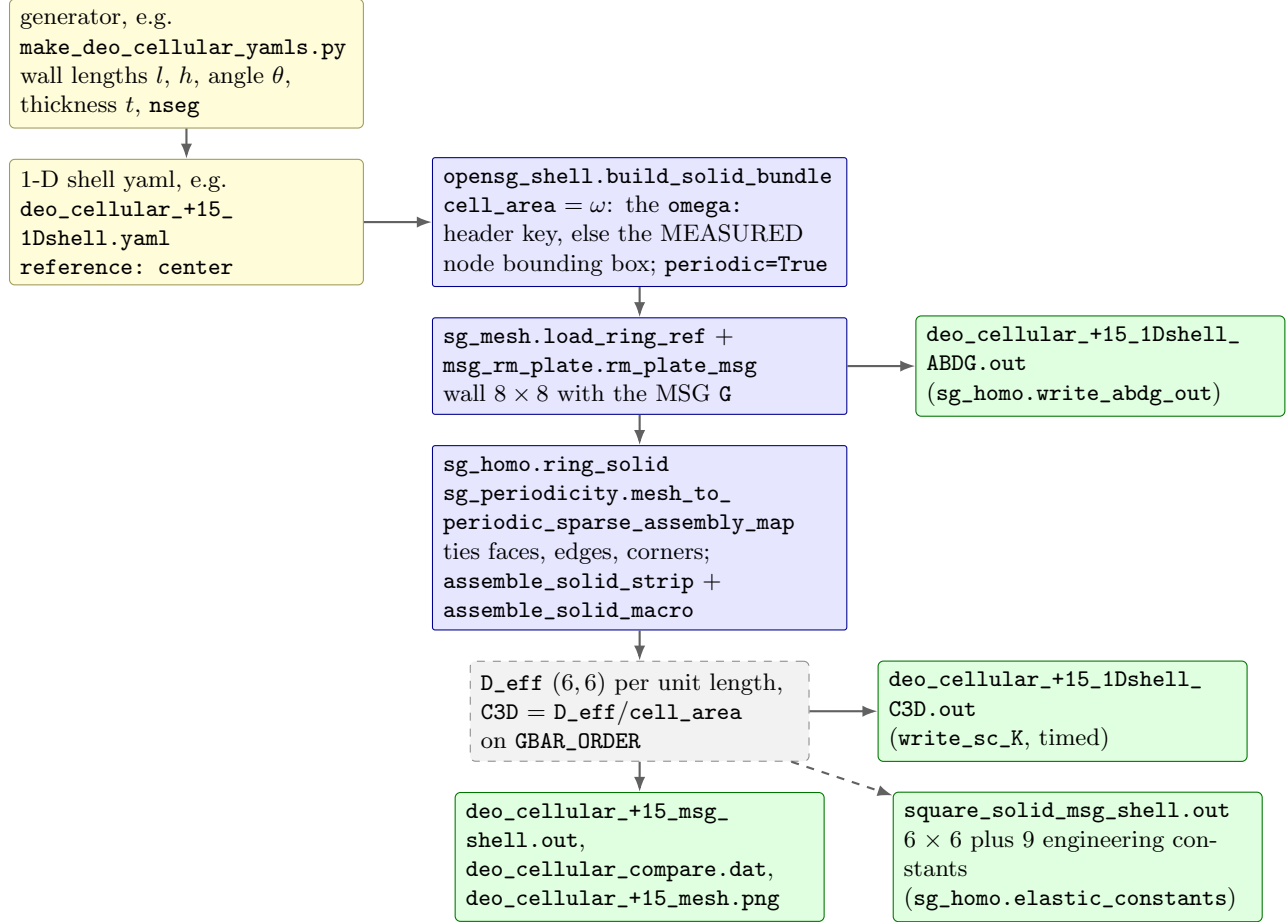
4.3 Equivalent 3-D solid from a cross-section SG

4.3.1 What this capability computes

The same 1-D shell structure gene can be homogenized to an **equivalent 3-D solid** 6×6 instead of a beam 6×6 : the anisotropic continuum stiffness of a cell that is tiled periodically in the two

transverse directions. This is the MSG thin-walled (MSG-TW) equivalent-continuum problem, and it is what you want for a cellular solid, a lattice, or any prismatic microstructure whose walls are thin.

Every case in this section is a two-command run, and the two commands are the two input nodes of the chain below: a generator that writes the 1-D shell YAML, then a driver that calls `build_solid_bundle` on it.



The strain order is the solid Voigt order

$$\bar{\Gamma} = [\bar{\Gamma}_{11} \quad \bar{\Gamma}_{22} \quad \bar{\Gamma}_{33} \quad 2\bar{\Gamma}_{23} \quad 2\bar{\Gamma}_{13} \quad 2\bar{\Gamma}_{12}]^T,$$

with axis 1 along the prismatic direction. That string is returned in the bundle under the key `order` and exported as `GBAR_ORDER`.

Where the beam problem drives the SG with four beam strains, the solid problem drives it with the six macro strains, so **periodicity is always on**: opposite edges and the corners of the 2-D cell are tied (`periodic=True` is the default of `build_solid_bundle`). The wall law is the same Reissner–Mindlin 8 × 8 of Eq. (4.1), so the walls carry transverse shear that a classical (CLPT/Kirchhoff) wall model discards. That distinction is the entire point of the benchmarks below.

4.3.2 The driver

```
from opensg_shell import build_solid_bundle

b = build_solid_bundle("cell_1Dshell.yaml", cell_area=A_cell)
C3D = b["C3D"]          # 6x6 equivalent solid stiffness, Pa
b["cell_area"], b["area_source"] # the normalizing area and where it came from
b["solve_time"]          # seconds; also written into <yaml>_C3D.out
```

Listing 4.5: The one call for equivalent-solid properties from a cross-section SG.

`cell_area` is the one argument you must think about: it is the measure ω the energy is divided by. Pass it explicitly (then `area_source` reads "user"); if you omit it in a direct library call, the convex-hull area of the contour is used as a legacy fallback and `area_source` reads "hull". The `opensg <yaml>` CLI never takes that fallback: it supplies the measure itself (`cli.sg_cell_area`) as the `omega:` header key when the file declares one, and otherwise as the **measured periodic cell** — the node *bounding box*, because the assembly map ties opposite box faces. The bounding box is not the convex hull of the contour; the two differ on any non-convex cell, which is every honeycomb and every lattice in this section. `build_solid_bundle` writes `<yaml>_C3D.out` by default, in the SwiftComp .K layout (effective stiffness, effective compliance, orthotropic-approximated engineering constants) with an OpenSG banner and a Time taken footer, plus `<yaml>_ABDG.out` for the wall laws.

4.3.3 Validation: Deo & Yu (2023)

The reference is A. Deo and W. Yu, *Equivalent 3D properties of thin-walled composite structures using mechanics of structure genome*, *Mechanics of Advanced Materials and Structures* **30**(9), 2023, 1737–1748. Two columns are quoted from that paper: **Deo TW**, its own thin-walled model with a classical wall law, and the reference **SwiftComp** column, which is the paper’s MSG solid result and the equivalent of an OpenSG 2-D-solid run. All values below are MPa, and **ours %** and **Deo TW %** are both measured against the SwiftComp column. Every table is the literal content of the `*_compare.dat` file written by the driver.

Four sub-folders live under `examples/OpenSG_shell/3_get_solid_props_from_shell_cross_section`. Each is a two-command run: a generator that writes the 1-D shell YAML, then the driver that homogenizes it.

```
cd examples/OpenSG_shell/3_get_solid_props_from_shell_cross_section/deo_cellular_solid
python make_deo_cellular_yamls.py
python deo_cellular_solid_props.py
```

The generator is where the geometry is edited. For the cellular solid it is a five-line block at the top of `make_deo_cellular_yamls.py`: `l, h, t = 0.10, 0.10, 0.005`, aluminium `E, nu = 68.9e9, 0.33`, and `nseg = 10` elements per wall of length l . The driver then computes the cell area itself from the wall angle, `area = (2*l*np.cos(th))*(2*(h + l*np.sin(th)))`, and passes it as `cell_area`.

Every driver in this folder redraws the 1-D SG it just solved, straight from the `nodes` and `elements` blocks of its own YAML, so Figures 4.4, 4.5 and 4.6 are the meshes the numbers came from and not sketches.

4.3.3.1 Cellular solid, $\theta = +15^\circ$ (paper Table 1)

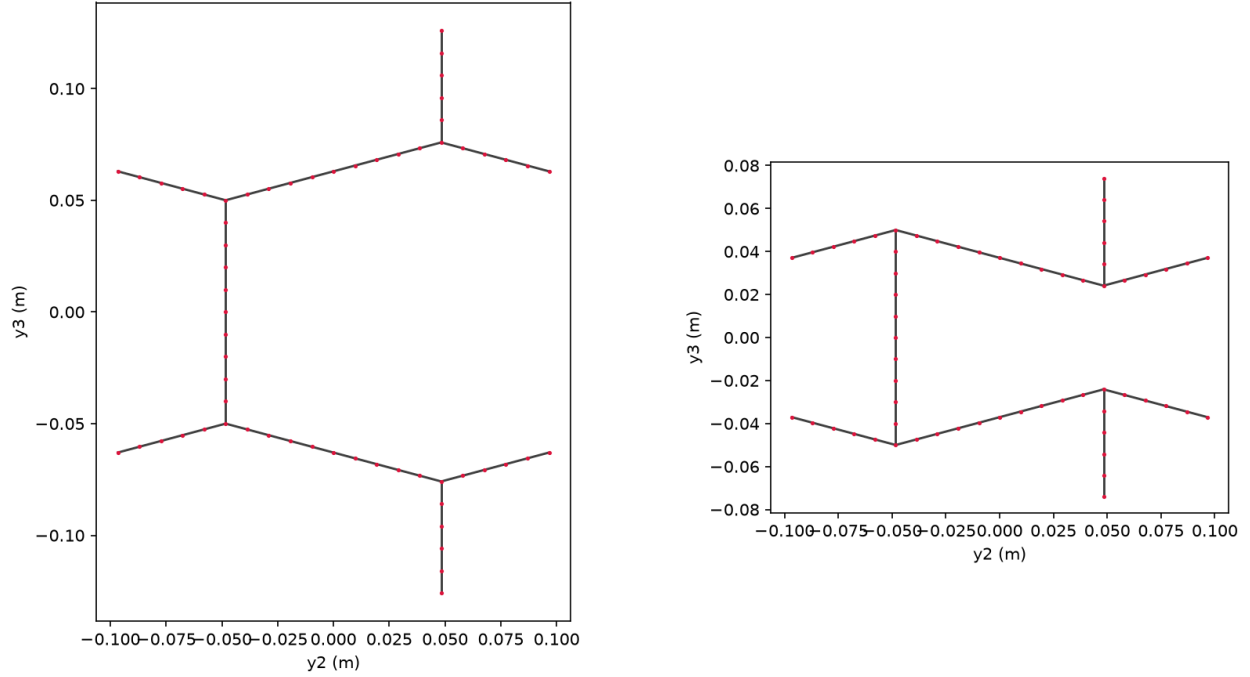


Figure 4.4: The two cellular 1-D shell SGs, written as `deo_cellular_+15_mesh.png` and `deo_cellular_-15_mesh.png` by `deo_cellular_solid_props.py`. Left: the hexagonal cell at $\theta = +15^\circ$, whose inclined walls splay outward. Right: the re-entrant (auxetic) cell at $\theta = -15^\circ$, whose inclined walls fold inward, which is what reverses the sign of C_{13} and C_{23} . Both are the periodic double cell of nine walls: one full vertical column of length $h = 10$ cm, two box-clipped column halves, two full inclined walls of length $l = 10$ cm and four clipped inclined stubs, all $t = 5$ mm aluminium at `nseg = 10` elements per wall of length l . Node dots mark the two-node line elements.

term	OpenSG-TW (RM)	Deo MSG-TW	SwiftComp	ours %	Deo TW %
C_{11}	4733.84	4736.90	4678.90	+1.17	+1.24
C_{12}	1086.01	1089.40	1105.50	-1.76	-1.46
C_{13}	380.63	381.81	386.88	-1.61	-1.31
C_{22}	2444.24	2446.39	2488.90	-1.79	-1.71
C_{23}	846.69	847.44	860.89	-1.65	-1.56
C_{33}	306.74	306.99	311.48	-1.52	-1.44
$\mathbf{C}_{44}$	4.20	4.32	4.19	+0.18	+3.09
C_{55}	562.61	564.15	573.11	-1.83	-1.56
C_{66}	993.78	997.52	1000.10	-0.63	-0.26

Table 4.13: Deo cellular solid at $\theta = +15^\circ$, MPa, from `deo_cellular_compare.dat`. Aluminium $E = 68.9$ GPa, $\nu = 0.33$; $l = h = 10$ cm, $t = 5$ mm. Solve 1.5 s.

4.3.3.2 Re-entrant cellular solid, $\theta = -15^\circ$ (paper Table 2)

The auxetic lattice, with the negative C_{13} and C_{23} couplings.

term	OpenSG-TW (RM)	Deo MSG-TW	SwiftComp	ours %	Deo TW %
C_{11}	7505.26	7507.40	7352.90	+2.07	+2.10
C_{12}	1090.52	1094.10	1075.20	+1.42	+1.76
C_{13}	-219.81	-220.53	-213.94	+2.74	+3.08
C_{22}	4151.29	4154.90	4080.00	+1.75	+1.84
C_{23}	-846.69	-847.45	-821.77	+3.03	+3.12
C_{33}	180.61	180.75	173.48	+4.11	+4.19
C_{44}	2.47	2.68	2.47	+0.17	+8.38
C_{55}	331.26	332.17	346.01	-4.26	-4.00
C_{66}	1687.82	1694.20	1706.40	-1.09	-0.71

Table 4.14: Deo re-entrant cellular solid at $\theta = -15^\circ$, MPa, from `deo_cellular_compare.dat`. Solve 0.2 s.

C_{44} is the in-plane shear channel, the one the paper flags as its largest error and attributes to the classical wall model dropping the segment transverse shear, with a first-order-shear wall model named as future work. That is exactly the RM wall law used here: the +3.09 % and +8.38 % errors drop to +0.18 % and +0.17 %. Every other term tracks the paper’s own thin-walled column to a few tenths of a percent, so the two independent MSG-TW implementations agree where they share physics.

4.3.3.3 Hierarchical square (paper Table 3)

Run from `deo_hierarchical_square`. The generator `make_deo_square_yaml.py` carries R , r , t = 0.10, 0.05, 0.005 and $nseg = 10$, the same aluminium.

term	OpenSG-TW (RM)	Deo MSG-TW	SwiftComp	ours %	Deo TW %
C_{11}	5183.48	5186.39	5099.03	+1.66	+1.71
C_{12}	24.22	24.83	26.75	-9.47	-7.18
C_{13}	24.22	24.83	26.75	-9.47	-7.18
C_{22}	46.23	47.13	50.46	-8.39	-6.60
C_{23}	27.15	27.94	30.59	-11.23	-8.66
C_{33}	46.23	47.13	50.45	-8.37	-6.58
C_{44}	3.20	3.22	3.39	-5.60	-5.01
C_{55}	647.56	650.00	670.92	-3.48	-3.12
C_{66}	647.56	650.00	670.93	-3.48	-3.12

Table 4.15: Deo hierarchical square, MPa, from `deo_square_compare.dat`. $R = 10$ cm, $r = 5$ cm, $t = 5$ mm. Solve 1.8 s.

This lattice is **bending-dominated**: its transverse channels are t^3 -scale ($C_{22} \approx 50$ MPa against $C_{11} \approx 5100$ MPa), so the load path is wall bending, not wall stretching. Two consequences are worth recording. First, the RM result is mesh-converged and the offset is physics: refining from 10 to 40 elements per segment moves C_{23} by 1 %, and running the same model with shear-rigid walls (the Kirchhoff limit) reproduces the paper’s classical column to 0.0–0.3 %. The gap between the two columns is therefore the wall transverse-shear compliance, which is real at the $t/l = 0.1$ ligaments; element order is not involved. Second, both thin-walled models miss the same junction stiffness.

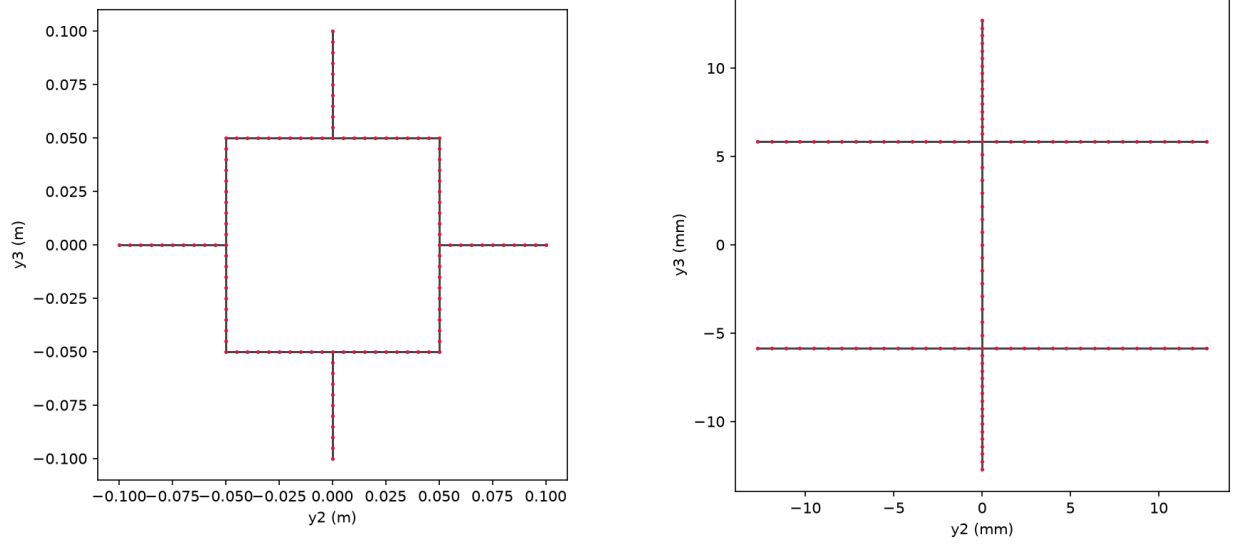


Figure 4.5: The two square-lattice 1-D shell SGs. Left: the hierarchical square (`deo_square_mesh.png`, from `deo_square_solid_props.py`) – a closed inner square of half-side $r = 5$ cm built from eight half-walls, plus four ligaments running out to the cell boundary at $\pm R = \pm 10$ cm, twelve walls in all at $t = 5$ mm and $nseg = 10$ elements per wall. Right: the composite square (`deo_composite_square_mesh.png`, from `deo_composite_square_props.py`) – a 25.4×25.4 mm cell with two horizontal walls at $y_3 = \pm 5.84$ mm and one vertical wall spanning the full cell height through the centre, generated as seven segments of $nseg = 16$ elements each, every wall a $[15]_8$ laminate of $t = 1.016$ mm. Both are axis-aligned 0/90 lattices, which is why both lose the membrane part of C_{23} .

The residual against SwiftComp is the finite rotational stiffness of the joint regions, which no midline model carries. The classical model’s smaller apparent error is a partial cancellation, artificial shear rigidity offsetting missing joint stiffness, and the RM model removes the first error, exposing the second.

4.3.3.4 Composite square (paper Fig. 13 / Table 4)

Run from `deo_composite_square`. The cell is 25.4×25.4 mm, so the driver passes `cell_area = L2*L2` with `L2 = 25.4e-3`; horizontal segments sit at $y_3 = \pm 5.84$ mm with one vertical segment through the centre, and every wall is a $[15]_8$ laminate of $t = 1.016$ mm with $E_1 = 141.96$, $E_2 = E_3 = 9.79$, $G = 6.136$ GPa, $\nu = 0.42$. The right panel of Figure 4.5 is that mesh.

term	OpenSG-TW (RM)	Deo MSG-TW	SwiftComp	ours %	Deo TW %
C_{11}	15263.83	16133.90	15518.11	−1.64	+3.97
C_{12}	936.74	1016.82	1008.76	−7.14	+0.80
C_{13}	468.37	468.38	459.01	+2.04	+2.04
C_{15}	+1191.85	−1192.81	−1135.27	sign	+5.07
C_{16}	2383.70	2589.74	2535.23	−5.98	+2.15
C_{22}	906.11	983.56	988.88	−8.37	−0.54
C_{23}	0.00	0.00	20.44	−100	−100
C_{26}	292.25	317.09	311.80	−6.27	+1.70
C_{33}	453.06	453.06	456.25	−0.70	−0.70
C_{35}	+146.13	−146.06	−140.04	sign	+4.30
C_{44}	0.94	1.08	1.16	−18.57	−6.90
C_{55}	547.31	547.32	545.43	+0.35	+0.35
C_{66}	1094.63	1188.20	1188.30	−7.88	−0.01

Table 4.16: Deo composite square, MPa, from `deo_composite_square_compare.dat`. Solve 1.8 s. The `.dat` file prints the two sign-flipped rows as -204.98 and -204.35 percent, which is the arithmetic of a sign reversal, not a magnitude error.

C_{13} , C_{33} and C_{55} reproduce the paper’s thin-walled column exactly, and C_{11} improves on it against SwiftComp. Two open items are recorded honestly. C_{15} and C_{35} match the published magnitudes to 0.1 % with the *opposite sign*, a ply-angle handedness convention still to be reconciled, and the likely cause of the -6 to -8 % group. And C_{23} is zero in both thin-walled models, for the reason below.

4.3.3.5 A fifth folder: the isotropic square tube

`square_section_isotropic` is the simplest case in the set and the best place to start. It reads `square_tube_1Dshell.yaml`, whose geometry is `a, t_w = 1.0, 0.03`. This is the one case in the section that does *not* want the measured bounding box: the YAML therefore declares `omega: 0.12` in its header, which is $4at_w$, the *wall material area* and the same measure the solid engines use, so the two routes are directly comparable. Because the measure rides in the header, the run is the plain one-liner

```
opensg square_tube_1Dshell.yaml
```

which writes `square_tube_1Dshell_C3D.out`, carrying the 6×6 and the nine engineering constants under the banner

where (user) is `area_source` recording that the measure was supplied rather than defaulted to the hull. The driver `solid_props_msg_shell.py` adds the mesh figure and a second copy of the same law, `square_solid_msg_shell.out`, laid out for the comparison scripts.

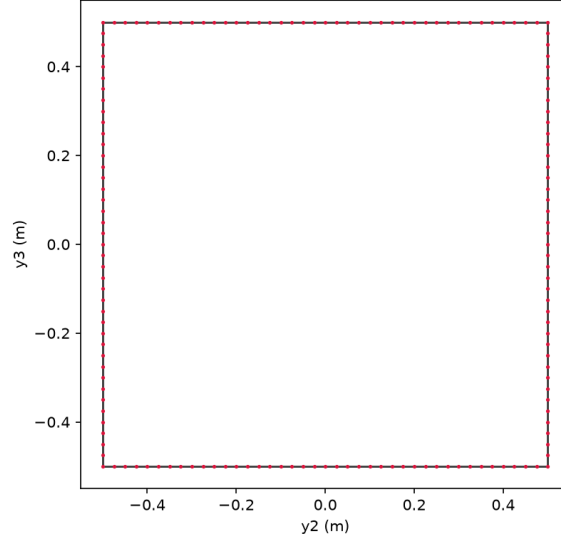


Figure 4.6: The isotropic square-tube 1-D shell SG, written by `solid_props_msg_shell.py` as `square_tube_mesh.png`. A single closed cell: four straight walls on the midline of a square of side $a = 1$ m, wall thickness $t_w = 0.03$ m carried as material data, centre reference. This is the one case whose measure is the wall material area $4at_w$ rather than a bounding-box area, which is what makes it directly comparable with a 2-D solid run of the same tube.

4.3.4 Why C_{23} vanishes on axis-aligned lattices

For a wall at angle φ the only Γ_e entries carrying weight in *both* the $\bar{\Gamma}_{22}$ and $\bar{\Gamma}_{33}$ columns are the membrane ε_{22} row $(\cos^2 \varphi, \sin^2 \varphi)$ and the transverse-shear $2\gamma_{23}$ row $(-\sin \varphi \cos \varphi, +\sin \varphi \cos \varphi)$, giving a per-length integrand

$$(A_{11} - G_{\text{msg}}) \sin^2 \varphi \cos^2 \varphi. \quad (4.2)$$

On a 0/90 lattice every wall has $\sin^2 \varphi \cos^2 \varphi = 0$, so this $O(t)$ membrane path vanishes identically and the whole of C_{23} is the $O(t^2)$ **junction** contribution: the wall-overlap blocks, which have zero measure on a midline mesh. Inclined-wall cells are unaffected, which is why the cellular cases above carry C_{23} correctly and only the square lattices expose the junction term. Optional junction corrections (`junction="census"`, `"micro"`, `"microcell"`) recover it for stretch-dominated cells; they do *not* apply to bending-dominated lattices such as the hierarchical square, and they are off by default.

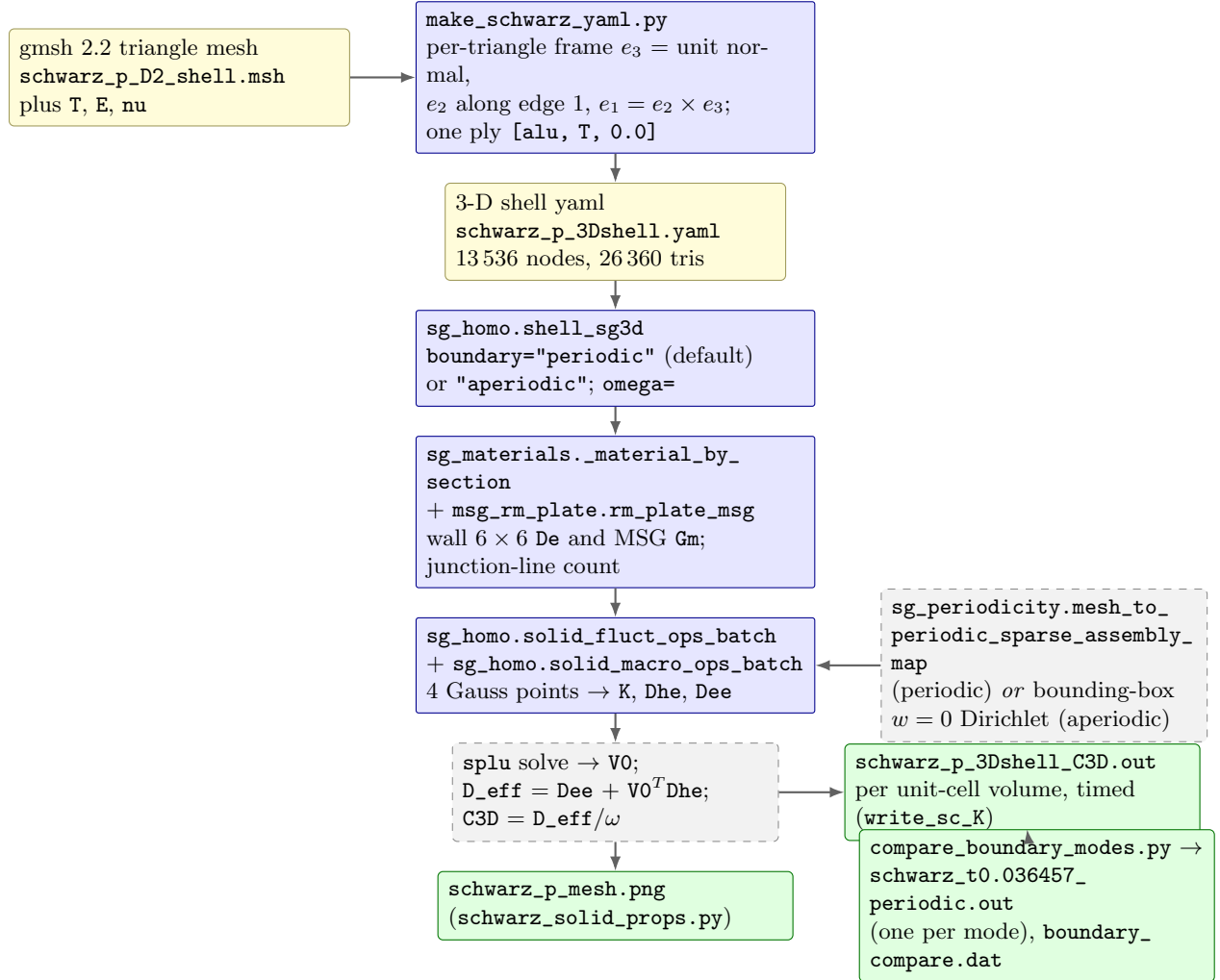
4.4 Equivalent 3-D solid from a 3-D shell SG

4.4.1 What this capability computes

A structure gene periodic in **all three** directions – a triply-periodic minimal surface (TPMS) unit cell, a lattice block, any 3-D microstructure whose material is a thin sheet – homogenizes to the same equivalent 3-D solid 6×6 , on the same solid Voigt order $\bar{\Gamma}$ of Section 4.3. `shell_sg3d` solves it when the microstructure is a thin sheet meshed as a *surface*: the wall carries the RM 8×8 law, so thickness is a parameter rather than a remesh.

It reuses the cross-section Γ_e / Γ_h operators unchanged (they are geometry-general) and adds only the 3-D environment: sparse assembly, the three-direction periodic map, drilling by penalty on the element-constant residual, and a three-translation kernel. Shell edges shared by more than two elements are detected as **junction lines** and reported as `n_junction_edges`; a smooth TPMS has none.

The example is `examples/OpenSG_shell/4_get_solid_props_from_shell_3D_SG`, and its data flow is the chain below. The only branch point is the boundary argument.



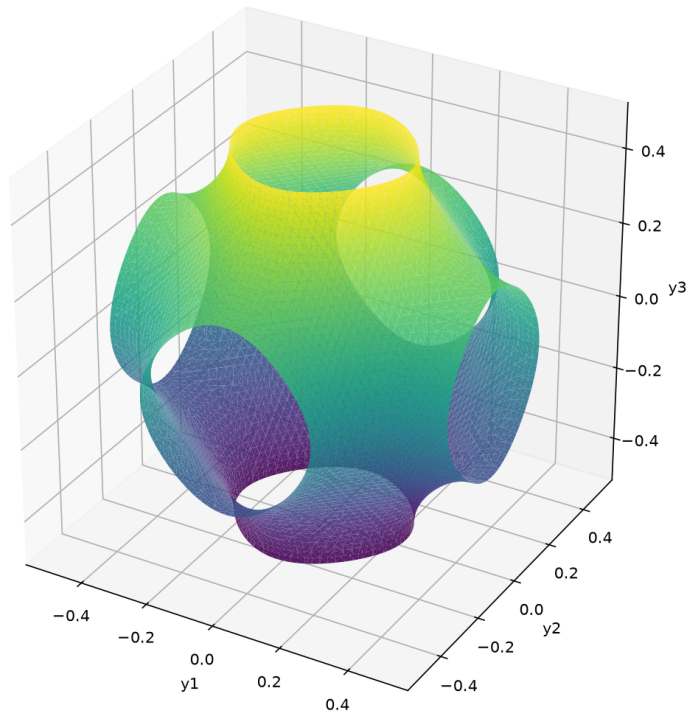


Figure 4.7: The Schwarz-P TPMS unit cell as a 3-D shell SG: 13 536 nodes and 26 360 triangular shell elements on the midsurface, written by `schwarz_solid_props.py` as `schwarz_p_mesh.png`. The cell is the unit cube, and the wall thickness is a number in the YAML, not geometry.

4.4.2 The input the user edits

The mesh comes from a gmsh 2.2 triangle file, and `make_schwarz_yaml.py` converts it. Its user block is four lines:

```
MSH = ("../../../../tests/08072026_square_shell_mesh/TPMS_SC_files/"
       "schwarz_p_D2_shell.msh")
T = 0.036457          # Sample_2, volume-consistent
E, nu = 69.0e9, 0.30
G = E/(2*(1+nu))
```

Listing 4.6: The user block of `make_schwarz_yaml.py`: mesh path, thickness, material.

The converter computes each element's frame from the triangle geometry (e_3 = unit normal, e_2 along the first edge, $e_1 = e_2 \times e_3$), writes one `sections` entry with the single ply `[alu, T, 0.0]`, and reports the surface area and the resulting relative density `area.sum()*T`.

4.4.3 Run it

The generator writes the YAML; the homogenization is then the unified one-liner, because everything the route needs — the layup, the material and the SG measure — is in that file. The third command is the two-mode boundary comparison of Table 4.17, which no single run can produce.

```
cd examples/OpenSG_shell/4_get_solid_props_from_shell_3D_SG
python make_schwarz_yaml.py
opensg schwarz_p_3Dshell.yaml
python compare_boundary_modes.py
```

`schwarz_solid_props.py` runs the same homogenization from Python and adds the mesh figure.

```
from opensg_shell.sg_homo import shell_sg3d

r = shell_sg3d("schwarz_p_3Dshell.yaml")          # boundary="periodic" (default)
r = shell_sg3d("schwarz_p_3Dshell.yaml", boundary="aperiodic") # non-periodic SGs (explicit)
r["C3D"], r["n_junction_edges"], r["ndof"], r["solve_time"]
```

Listing 4.7: The two boundary treatments of `shell_sg3d`.

For a 3-D SG the measure ω is a **volume**, not an area: it defaults to the node **bounding-box volume** `ptp(y1) ptp(y2) ptp(y3)`, measured from the mesh — the volume the equivalent continuum occupies, and the same cell the periodic assembly map ties. It is the same convention `opensg_solid` uses, so the returned `C3D`, the line the CLI prints and `<base>_C3D.out` are one and the same matrix, directly comparable with a solid SwiftComp .K. An explicit `omega=` argument (or the YAML's `omega`: header key, which the CLI forwards here) overrides it — for example with the midsurface *surface* area, which the result dict also returns as `"surface_area"`, for a per-unit-area wall law.

4.4.4 What comes out

`shell_sg3d` writes two files, `schwarz_p_3Dshell_C3D.out` and `schwarz_p_3Dshell_ABDG.out`, and the driver writes `schwarz_p_mesh.png`.

The `_ABDG.out` is the step-1 record: every route that reduces a layup to a wall plate law now emits it, the cross-section rings of Sections 4.1 and 4.3 and the 3-D shell SG alike. It is an 8×8 Reissner–Mindlin ABDG stiffness plus its compliance, one block per section, opening with

```
OpenSG msg-shell wall plate laws, one block per section
rows [eps11 eps22 2eps12 | K11 K22 K12+K21 | 2g13 2g23]
```

The Schwarz-P surface has a single section, headed `section 0: layup_0 layup [['alu', 0.036457, 0.0]]`, and its wall law is $A_{11} = 2.7643220 \times 10^9$, $A_{12} = 8.2929659 \times 10^8$, $A_{66} = 9.6751264 \times 10^8$ N/m, $D_{11} = 3.0617465 \times 10^5$ N·m and $G_{11} = G_{22} = 8.5193106 \times 10^8$ N/m — the plate law of a single 36.457 mm aluminium ply, which is the cheapest possible check that the thickness and the material reached the solver intact.

The banner line of the `_C3D.out` records the whole configuration, and the default periodic run at $t = 0.036457$ gives:

```
OpenSG msg-shell equivalent 3D solid (3-D shell SG, 13536 nodes, 26360 elems,
0 junction edges, periodic in 3 dirs, per unit cell 1)
```

```
The Effective Stiffness Matrix
```

```
-----
      2.2253086E+009      1.5648739E+009      1.5649503E+009  ...
      1.5648739E+009      2.2252770E+009      1.5647791E+009  ...
      1.5649503E+009      1.5647791E+009      2.2250347E+009  ...
```

```
The Engineering Constants (Approximated as Orthotropic)
```

```
-----
E1 = 9.3296450E+008
E2 = 9.3313789E+008
E3 = 9.3284790E+008
G12 = 1.0409258E+009
G13 = 1.0410061E+009
G23 = 1.0409794E+009
nu12= 4.1277916E-001
nu13= 4.1304617E-001
nu23= 4.1288357E-001
```

```
Time taken: 66.43 sec
```

Read the numbers as a health check first. The three normal terms agree to five digits (2.2253, 2.2253, 2.2250 GPa) and the three shear terms likewise (1.04098, 1.04101, 1.04093 GPa), so the effective law is cubic to five digits, as the Schwarz-P symmetry demands. Every symmetry-forbidden coupling sits at 10^3 – 10^5 Pa against diagonals of 10^9 Pa, four to six orders below. That is the health check on the all-directions periodic tie: if the tie map were wrong, those entries would not be small. The repository also records a head-to-head of this same surface against the msg-solid route on the same structure, where the shell result runs about 3% below on the normal terms and about 6% below on the shears, with cubic symmetry intact in both; because the solid column is digit-identical to SwiftComp, those percentages measure the thin-shell reduction itself at a surface reaching $t/R \approx 0.2$ near the saddle necks.

4.4.5 Boundary treatment: periodic default versus aperiodic

`shell_sg3d` takes a `boundary` argument, and the default is "periodic": opposite faces, edges and corners of the bounding box are tied through the sparse assembly map, and the three rigid translations are removed by area-weighted Lagrange rows. This is the exact treatment for a periodic medium, and it is what to report.

`boundary="aperiodic"` is the boundary-solution treatment. For a unit cell the boundary solution is the macro (affine) field itself, so mapping it onto the boundary nodes prescribes zero **translational** warping fluctuation ($w_1 = w_2 = w_3 = 0$, Dirichlet) on every node of the bounding-box faces, with the rotational DOFs left natural. That forbids the affine fluctuations which make a *free* aperiodic SG rank-one, fixes the rigid modes without a Lagrange border, and is a kinematic upper bound. Clamping the rotations as well would roughly double the offset.

`compare_boundary_modes.py` runs both on the current YAML, copies each per-mode `.out` to `schwarz_t<T>_<mode>.out`, and appends a block to `boundary_compare.dat`. Table 4.17 is that file.

term	$t = 0.036457$			$t = 0.129330$		
	aperiodic	periodic	%diff	aperiodic	periodic	%diff
C_{11}	2375.89	2225.31	+6.767	9881.00	8916.81	+10.813
C_{22}	2375.92	2225.28	+6.770	9881.00	8916.85	+10.813
C_{33}	2376.22	2225.03	+6.795	9873.34	8916.17	+10.735
C_{12}	1537.47	1564.87	-1.751	4921.09	5352.21	-8.055
C_{13}	1539.24	1564.95	-1.643	4931.10	5352.71	-7.877
C_{23}	1538.99	1564.78	-1.648	4931.00	5352.55	-7.876
C_{44}	1078.28	1040.98	+3.583	4150.25	3961.63	+4.761
C_{55}	1078.32	1041.01	+3.585	4150.26	3961.63	+4.761
C_{66}	1078.61	1040.93	+3.620	4151.27	3961.51	+4.790
ndof	81216	79056		81216	79056	
solve	42.3 s	66.5 s		42.2 s	65.4 s	

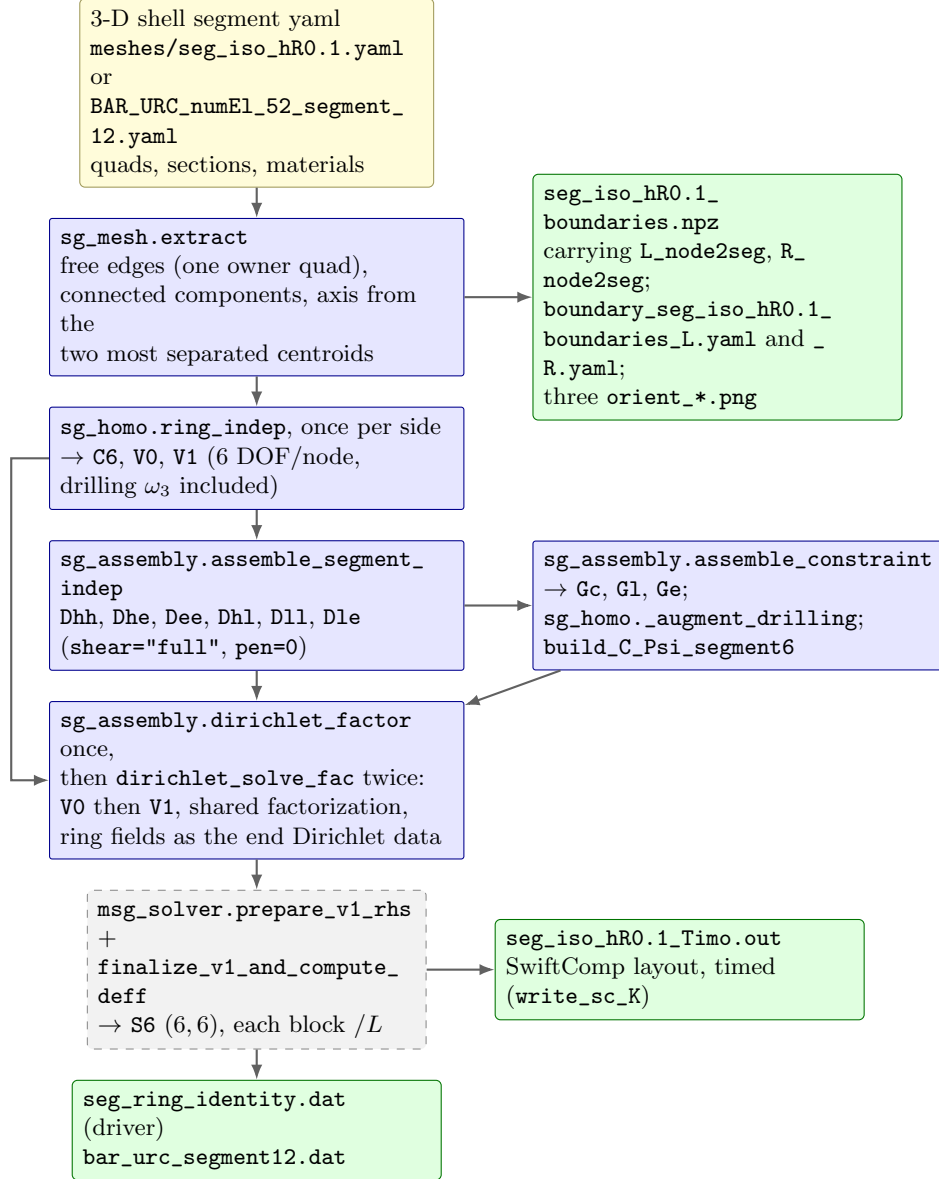
Table 4.17: Aperiodic versus periodic on the same Schwarz-P YAML at two wall thicknesses, per unit cell, MPa. The literal content of `boundary_compare.dat`. In both blocks 720 free-face nodes of 13 536 are clamped by the aperiodic treatment.

The offset is the clamped-face boundary layer of a **single** cell: the expected kinematic bound, not an error in either path. It grows with wall thickness, from +6.8 % on C_{11} at $t = 0.036457$ to +10.8 % at $t = 0.129330$, because a thicker wall makes the clamped skin a larger fraction of the cell. Aperiodic is about 35 % *faster* here, because the Dirichlet rows leave the factorization and there is no tie map or Lagrange border, despite the slightly larger nominal DOF count. Report periodic for a periodic medium; request "aperiodic" only for SGs that genuinely are not periodic.

4.5 Tapered and aperiodic segments

4.5.1 What this capability computes

Everything so far has assumed the beam is prismatic: the classical msg-shell analysis meshes only the cross-section contour and enforces periodicity along the beam axis. A **tapered** segment has no such periodicity, because its two end cross-sections differ. `opensg_shell.sg_homo.segment_timo_from_3dyaml` handles this by replacing the axial periodic condition with Dirichlet data taken from the two ends. The input is no longer a 1-D ring but a genuine 3-D surface mesh of a slice of the beam.



The pipeline has four stages:

1. **Boundary extraction.** The two end cross-sections are found *topologically*: a mesh edge used by exactly one quad is a free edge, and each connected component of the free-edge graph with at least three nodes is one end ring. The axis is the direction between the two most widely separated components, and left/right is decided by the axial coordinate. Each ring is written out as a standalone 1-D YAML together with the ring-node $\rightarrow$ segment-node map.
2. **Boundary solves.** Each ring is homogenized on its own with `ring_indep`, giving its Timoshenko 6×6 and the warping fields V_0 (Euler–Bernoulli) and V_1 (Timoshenko), all six DOFs per node including the drilling rotation ω_3 .
3. **Mapping.** The ring V_0/V_1 are transferred onto the segment’s boundary nodes as Dirichlet data, replacing the periodic boundary condition and fixing the segment’s rigid-body modes.

4. **Segment solve.** One LU factorization of the constrained system serves both orders,

$$\mathbf{D}_{hh} V_0 = -\mathbf{D}_{he}, \quad \mathbf{D}_{hh} V_1 = b(V_0), \quad (4.3)$$

with $V_0 = V_0^{\text{ring}}$ and $V_1 = V_1^{\text{ring}}$ on both ends. The generalized-Timoshenko finalization divides the energy by the segment length L , which is measured as the extent of the nodes along the axis.

The example is `examples/OpenSG_shell/5_taper_segment_aperiodic_boundaries`.

4.5.2 Seeing the capability

BAR_URC_numEl_52_segment_12_boundaries segment e1/e2/e3

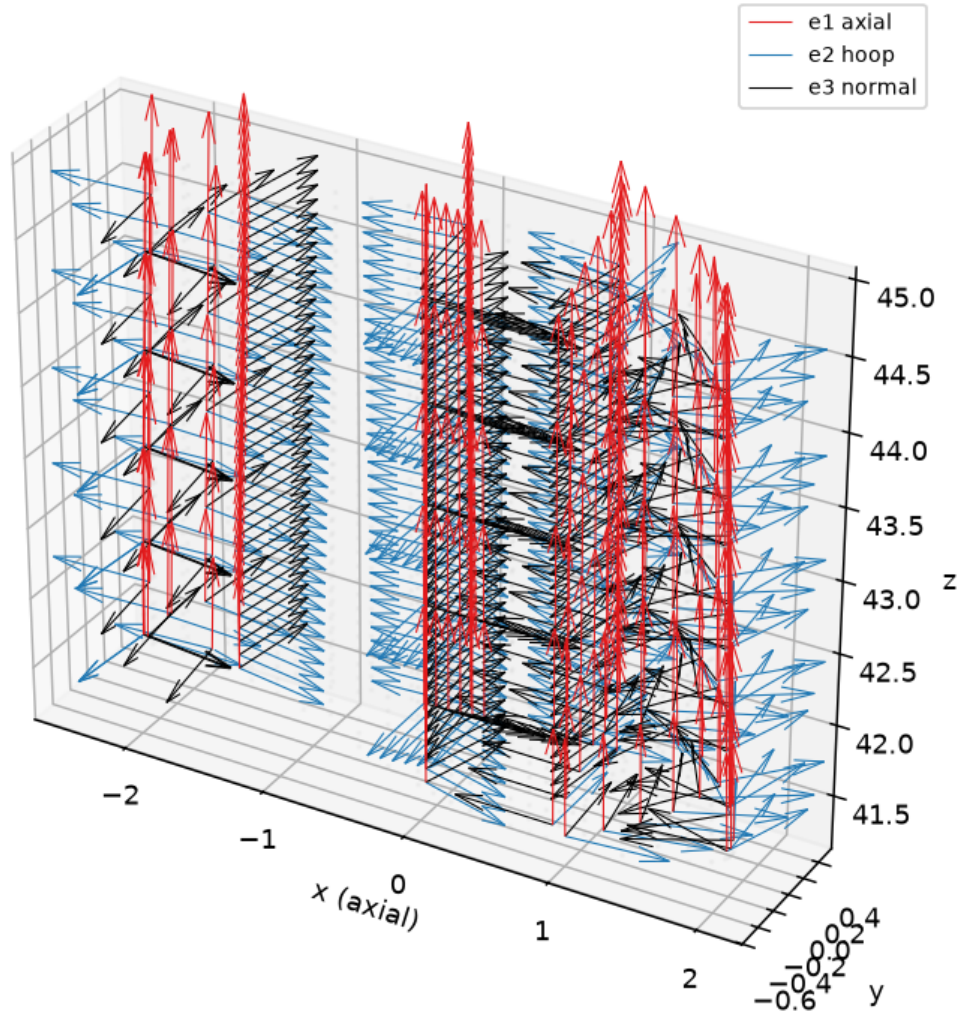


Figure 4.8: The BAR-URC blade segment the example runs, coloured by its per-element material frames. This is the surface on which the segment problem is solved: a 3-D shell mesh, not a cross-section.

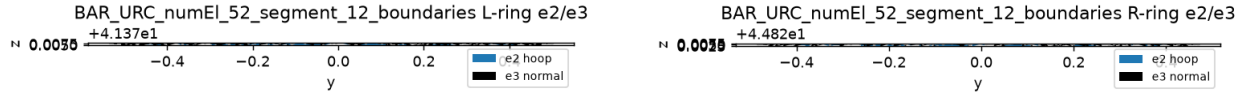


Figure 4.9: The two end cross-sections extracted automatically from the free-edge graph of the segment mesh, left ring and right ring, each written as a standalone 1-D SG and solved on its own. On a tapered segment the two rings differ, and their two 6×6 matrices bracket the segment result.

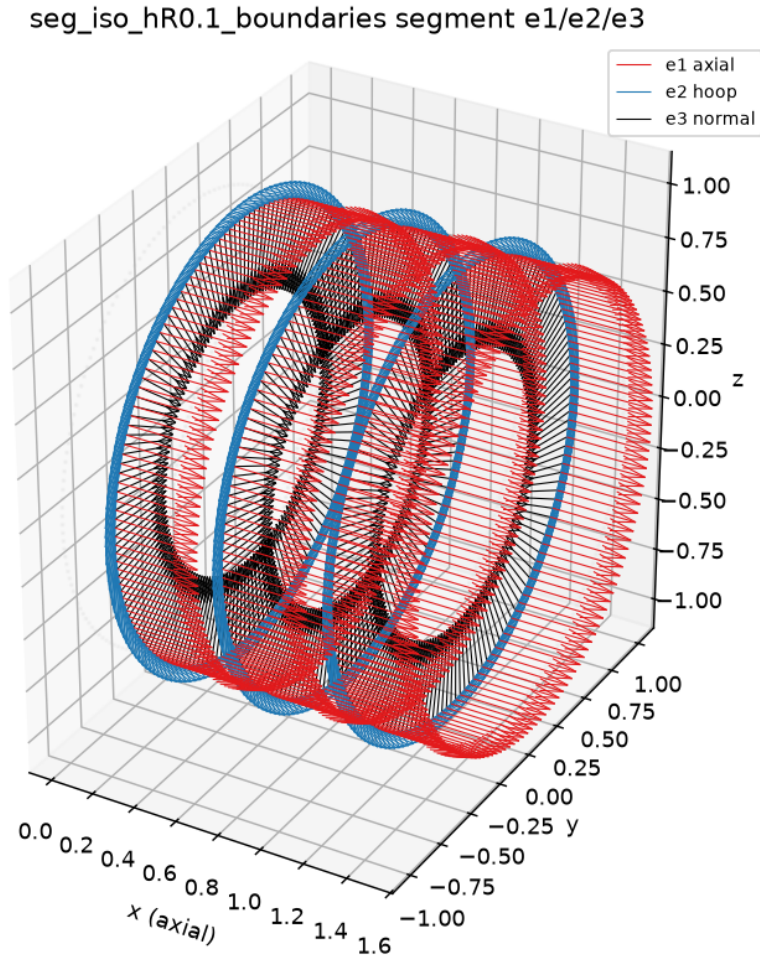


Figure 4.10: The prismatic circular cylinder used for the acceptance test. The same free-edge extraction runs on it, and here both ends are identical by construction, so the segment 6×6 must reproduce the ring 6×6 .

4.5.3 The input the user edits

For the validation case, `make_cylinder_segment.py` generates the surface mesh. Its parameter block is the whole user interface:

```
R = 1.0          # wall mid-surface radius [m]
NC = 160         # circumferential quads
NL = 3           # axial quads over the segment (>=2 so an interior ring exists)
L = 1.5          # segment length [m]
HR = [0.05, 0.1, 0.2] # wall/radius ratios: thin, moderate, thick

ISO = {"name": "iso", "density": 1800.0,
       "E": [70e9, 70e9, 70e9], "G": [26.923e9]*3, "nu": [0.3, 0.3, 0.3]}
ANI = {"name": "ani", "density": 1800.0,
       "E": [37e9, 9e9, 9e9], "G": [4e9, 4e9, 4e9], "nu": [0.3, 0.3, 0.3]}
```

Listing 4.8: The parameter block of `make_cylinder_segment.py`.

It writes six files into a `meshes/` sub-folder, named `seg_<tag>_hR<ratio>.yaml`: an isotropic single-ply tube and a balanced $[+45/-45]$ tube at each of the three thickness ratios. The beam axis is x , the cross-section plane is (y, z) , the reference surface is the wall mid-surface, and each element stores the nine-component triad $[e_1 e_2 e_3]$ with e_1 axial, e_2 the hoop tangent and e_3 the inward normal.

4.5.4 Run it

```
cd examples/OpenSG_shell/5_taper_segment_aperiodic_boundaries
python make_cylinder_segment.py
python run_taper_segment.py
python run_bar_urc_segment.py
```

The whole pipeline is one call on the 3-D shell YAML. There are no boundary files to prepare, because the ends are found from the mesh topology. Note the path: the cylinder meshes live in the `meshes/` sub-folder, so the argument carries that prefix.

```
from opensg_shell.sg_homo import segment_timo_from_3dyaml

r = segment_timo_from_3dyaml("meshes/seg_iso_hR0.1.yaml")
r["S6"]          # segment Timoshenko 6x6
r["C6L"], r["C6R"] # the two boundary-ring 6x6s
r["L"]           # segment length used to normalize the energy
r["solve_time"]  # seconds
```

Listing 4.9: The segment call, with the `meshes/` path used by `run_taper_segment.py`.

4.5.5 What comes out

Each run writes, alongside the input YAML, the two boundary 1-D YAMLs (`boundary_<tag>_L.yaml` and `_R.yaml`), the `<tag>_boundaries.npz` bundle, three orientation PNGs (`orient_<tag>_segment.png`, `orient_<tag>_ring_L.png`, `orient_<tag>_ring_R.png`), and the timed `<yaml>_Timo.out` in the SwiftComp layout. For the isotropic cylinder that file opens with

```
OpenSG msg-shell tapered/aperiodic segment (boundary V0/V1 Dirichlet, L=1.5)
```

and closes with `Time taken: 2.51 sec.`

One caveat on the figures. The extraction is called with `plot="auto"`, which means an orientation PNG is drawn *only if that file is not already on disk*. If you regenerate or edit a mesh and rerun, delete the three `orient_*.png` files for that tag first, or you will be looking at the frames of the previous mesh.

4.5.6 Acceptance test: the prismatic identity

For a **prismatic** segment the taper terms vanish, so the segment 6×6 must equal the boundary ring's own 6×6 . That identity is the acceptance test of the whole boundary extraction plus V_0/V_1 mapping, and `run_taper_segment.py` measures it on the prismatic circular tube ($R = 1$, $t/R = 0.1$, $L = 1.5$) in both the isotropic and the ± 45 layup, writing `seg_ring_identity.dat`.

term	segment S_6	boundary ring C_6	% error	case
EA	4.397947×10^{10}	4.397947×10^{10}	0.0000	iso_hR0.1 $L = 1.5$ solve 2.5 s
GA_2	8.473281×10^9	8.473258×10^9	0.0003	
GA_3	8.473281×10^9	8.473258×10^9	0.0003	
GJ	1.694078×10^{10}	1.694078×10^{10}	0.0000	
EI_2	2.200273×10^{10}	2.200269×10^{10}	0.0002	
EI_3	2.200273×10^{10}	2.200269×10^{10}	0.0002	
EA	7.706160×10^9	7.706160×10^9	0.0000	aniso_hR0.1 $L = 1.5$ solve 1.5 s
GA_2	3.262832×10^9	3.262807×10^9	0.0007	
GA_3	3.262832×10^9	3.262807×10^9	0.0007	
GJ	6.526891×10^9	6.526891×10^9	0.0000	
EI_2	3.855225×10^9	3.855204×10^9	0.0006	
EI_3	3.855225×10^9	3.855204×10^9	0.0006	

Table 4.18: The prismatic identity, diagonal terms, from `seg_ring_identity.dat`. The worst diagonal error is 0.0003% (on GA) for the isotropic tube and 0.0007% for the ± 45 layup. The worst error anywhere in the 6×6 is 0.0052% and it falls on the GA_2 – EI_2 shear–bending coupling of the anisotropic case, whose ring value is -3.684772×10^7 against a segment value of -3.684965×10^7 .

The **% error** matrix in the file reports insignificant ring terms as zero: entries whose magnitude is below 10^{-6} of the largest are divided by infinity rather than by numerical noise, which is why the file shows exact zeros where the ring value is 10^{-5} against diagonals of 10^{10} .

4.5.7 Tapered demonstration: BAR-URC segment 12

`run_bar_urc_segment.py` runs the same pipeline on a real tapered segment of the 100 m BAR-URC blade, `BAR_URC_numEl_52_segment_12.yaml`. Here no identity holds. Each boundary ring has its own 6×6 , and the segment result is the true length-averaged Timoshenko stiffness of the tapered piece. The physical check is that every segment diagonal falls *between* its two ring values, which the driver reports as the ratio $(S - R)/(L - R)$; a value in $[0, 1]$ means the segment lies between its two ends.

Read the ratios as a taper diagnostic. The bending and torsion terms sit near 0.47–0.48, close to the arithmetic mean of the two ends, which is what a nearly linear taper in those channels produces. EA and GA_2 sit at 0.26 and 0.31, noticeably below the mean, which says the axial and chordwise-shear stiffness of this segment is closer to its thin end than a linear interpolation of the two ring values would suggest. A station-by-station beam table built by interpolating ring properties would therefore over-predict those two terms on this segment.

term	segment S	ring L	ring R	ratio
EA	1.487716×10^{10}	1.498819×10^{10}	1.483824×10^{10}	0.260
GA_2	6.098229×10^8	6.248039×10^8	6.030954×10^8	0.310
GA_3	1.398945×10^8	1.457255×10^8	1.345727×10^8	0.477
GJ	3.353538×10^8	3.684346×10^8	3.057355×10^8	0.472
EI_2	2.996686×10^9	3.244829×10^9	2.765385×10^9	0.482
EI_3	8.971707×10^9	9.620926×10^9	8.397279×10^9	0.469

Table 4.19: BAR-URC tapered shell segment 12, from `bar_urc_segment12.dat`. Segment length $L = 3.45293$ m, solve 15.9 s. All six ratios lie in $[0, 1]$, so every segment diagonal is bracketed by its two boundary rings.

4.5.8 Shear-scheme rule for segments

`segment_timo_from_3dyaml` defaults to `shear="full"` (2×2 Gauss), and that is the production setting to keep: MITC tying aliases the algebraic drilling content on flat-walled and webbed sections. The tied variants exposed by the element ("`mitc4_wonly`", which ties both rows at the Dvorkin–Bathe points but keeps every rotation column at its full-integration value; "`mitc4_g23`", which ties only the γ_{23} row; and "`mitc4_both`", naive full tying) are ablation options for studying the element, not recommended production settings for segments. Note the contrast with the ring of Section 4.1, whose production default is "`mitc4_g23`": the ring is a one-quad-deep prismatic strip and the segment is a genuine surface mesh, and the two geometries do not want the same treatment.

Chapter 5

Tutorials: the Solid Engine (msg-solid)

The `msg-solid` engine reduces a *solid* structure gene (SG) — a mesh of ordinary continuum elements in one, two or three dimensions — to a macroscopic constitutive law, and then runs the same construction backwards to recover the pointwise three-dimensional strain and stress inside that gene. Everything in this chapter is reached through two entry points.

entry point	module	structure gene it reads
<code>rm_plate_msg</code>	<code>opensg_solid.rm_plate_1D.msg_rm_plate</code>	a 1-D line mesh through a wall thickness
<code>plate_homo_2d</code>	<code>opensg_solid.sg_homo</code>	a 1-D, 2-D or 3-D solid SG

Table 5.1: The two homogenization entry points of `msg-solid`.

The first is the dedicated through-thickness (laminate) route: the wall is a stack of plies, nothing varies in-plane, so the gene is one-dimensional and the result is a plate law. The second is the general route. Its `n_model` argument selects which macro model the same SG is reduced to.

<code>n_model</code>	macro model	row and column order	title infix
1	beam	[<code>eps11 gam12 gam13 k1 k2 k3</code>]	Timoshenko Beam
2	plate	[<code>N11 N22 N12 M11 M22 M12</code>]	Classical Plate
3	3-D elastic	Voigt [<code>11 22 33 23 13 12</code>]	Cauchy Continuum

Table 5.2: `n_model` on `plate_homo_2d`. The infix goes into **The Effective <infix> Stiffness Matrix** and is always the macro law’s console title, so the file and the terminal read the same. Two of the five titles belong to the refinement switch rather than to `n_model`: the classical beam 4×4 is **Euler-Bernoulli Beam**, and the plate route becomes **Reissner-Mindlin** when the shear-refined 8×8 is produced.

Every `plate_homo_2d` call writes a timed `SwiftComp.K`-layout report — effective stiffness, effective compliance, and, for the 3-D solid law only, an engineering-constants block — opening with an `OpenSG` banner and closing with a `Time taken:` line in seconds. Unless you pass `plot=False` it also writes a `<base>.mesh.png` with the elements coloured by material. One rule governs the names of both: inside `plate_homo_2d` the output base is `base = os.path.splitext(sc_path)[0]`, so the report is *always* `<input basename>.out`, regardless of what the script calls the case.

The eight runnable examples live under `examples/OpenSG-solid`. Each runner is meant to be executed from inside its own folder, because the SG file is named relative to the working directory.

folder	script	what it does
1_get_plate_props_from_1DSG	1d_sg.py	layup $\rightarrow$ 1-D SG $\rightarrow$ plate law
2_get_plate_stress_from_1DSG	rm_dehom.py, rm_dehom_ff.py	plate resultants $\rightarrow$ 3-D ply stress
3_get_plate_props_from_2D_SG	plate_homo_2dsg.py	2-D cell SG $\rightarrow$ plate ABD
4_get_plate_dehom_from_2DSG	plate_dehom_2dsg.py	macro plate strain $\rightarrow$ fields in the cell
5_get_solid_props_from_SG	solid_homo_2dsg.py	2-D cell SG $\rightarrow$ equivalent 3-D solid
6_get_solid_props_from_3D_SG	tpms_solid_props.py and others	3-D SG $\rightarrow$ equivalent 3-D solid
7_get_beam_props_from_SG	beam_homo_sg.py	2-D cross-section SG $\rightarrow$ Timoshenko 6×6
8_get_beam_dehom_from_SG	beam_dehom_sg.py	macro beam strain $\rightarrow$ fields in the section

Table 5.3: The eight msg-solid example folders.

A note on what you will find in those folders before you run anything. Some of the files named as outputs below are regenerated on every run and are simply not checked in; where that is the case it is said explicitly. Nothing in this chapter depends on a pre-computed file being present.

5.1 Plate properties from a through-thickness 1-D SG

5.1.1 What it computes

The structure gene is the wall discretised through its own thickness: one element per ply, quartic by default, with each node carrying the three warping (fluctuation) degrees of freedom. Minimising the 3-D strain energy over that line mesh order by order in h/ℓ gives the warping ladder — the zeroth-order columns V_0 and the first-gradient columns V_{11} , V_{12} — from which the classical plate stiffness follows as

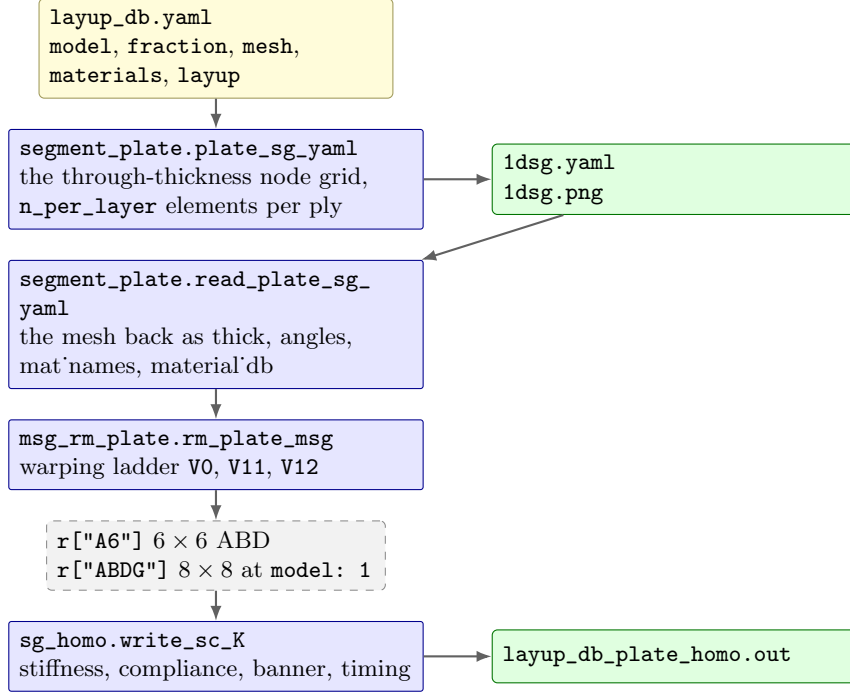
$$\mathbf{A}_6 = \langle (\mathbf{\Gamma}_\varepsilon + \mathbf{\Gamma}_h \mathbf{V}_0)^\top \mathbf{C}(z) (\mathbf{\Gamma}_\varepsilon + \mathbf{\Gamma}_h \mathbf{V}_0) \rangle, \quad (5.1)$$

where $\langle \cdot \rangle$ is the through-thickness integral and $\mathbf{C}(z)$ the ply stiffness rotated by its fibre angle. With `model: 1` the run additionally identifies the 2×2 MSG transverse-shear block $\mathbf{G}$ from the second-order energy and assembles the 8×8 law

$$\mathbf{ABDG} = \begin{bmatrix} \mathbf{A}_6 & \mathbf{0} \\ \mathbf{0} & \mathbf{G} \end{bmatrix}, \quad (5.2)$$

in which the transverse shears are uncoupled from the membrane and bending measures. Row and column order is [`e11 e22 g12 k11 k22 k12 2g13 2g23`], and that string is printed in the banner of every `.out` so a file can always be read without the manual.

The whole capability is one file you edit and five functions that run. The SG mesh is written to disk and read back before the solve, so the mesh file — not the database — is what the homogenization actually sees.



5.1.2 The input you edit

Everything lives in `layup_db.yaml`. The script reads that file and nothing else, so a new laminate means editing the YAML, never the code.

key	what it sets	value in the shipped file
<code>model</code>	0 = classical 6×6 ABD, 1 = 8×8 ABDG	1
<code>fraction</code>	reference surface as a fraction of thickness	0.5
<code>mesh: n_per_layer</code>	SG elements per physical ply	1
<code>mesh: elem_order</code>	element order; 4 = 5-noded quartic	4
<code>materials</code>	per material E: [E1,E2,E3], G: [G12,G13,G23], nu: [nu12,nu13,nu23], rho	ge, herex
<code>layup</code>	one entry per ply, bottom ply first	9 plies

Table 5.4: The six keys `1d_sg.py` reads from `layup_db.yaml`.

For `fraction`, 0 is the bottom (OML) face, 0.5 the mid-surface and 1 the top face. It is a property of the *mesh*, not of the homogenization call: the generated node coordinates are measured from the reference plane, so they run from $-\text{fraction} \cdot h$ to $(1 - \text{fraction}) \cdot h$ and $x_3 = 0$ is the reference. The *B* block of the ABD depends on this choice, so homogenize, analyze and recover with the same `fraction`.

The shipped case is a $[0/90/0/90/\text{core}]_s$ sandwich: eight graphite/epoxy plies of 0.9525 mm ($E_1 = 128$ GPa, $E_2 = E_3 = 11$ GPa, $G_{12} = G_{13} = 4.48$ GPa, $G_{23} = 1.53$ GPa, $\rho = 1500$ kg/m³) around a 144.78 mm HEREX C70.130 PVC foam core ($E = 103.63$ MPa, $G = 50$ MPa, $\nu = 0.32$, $\rho = 130$ kg/m³), total $h = 0.1524$ m.

The same `layup_db.yaml` also carries a `V2` flag, a `plate:` block and a `dehom:` block. Those are read by the recovery scripts of Section 5.2; `1d_sg.py` itself uses only the six keys in Table 5.4.

5.1.3 Run it

```
cd examples/OpenSG-solid/1_get_plate_props_from_1DSG
python 1d_sg.py
```

A different database file may be passed as the single positional argument:

```
python 1d_sg.py my_other_layup.yaml
```

5.1.4 What comes out

file	content
1dsg.yaml	the generated 1-D SG mesh: an <code>sg:</code> header (<code>type: plate_1d</code> , <code>elem_order: 4</code> , <code>n_per_layer: 1</code> , <code>reference_fraction: 0.5</code> , <code>thickness</code> , <code>n_ply</code>), 37 nodes given as single x_3 coordinates from -0.0762 to $+0.0762$ m, 9 five-noded elements, the material blocks with densities, and one element set plus one <code>sections:</code> entry per ply carrying its material, angle and thickness
1dsg.png	the mesh figure, plies coloured by material
layup_db_plate_homo.out	the plate law, named <code><layup_db stem>_plate_homo.out</code>

Table 5.5: Outputs of `1d_sg.py`.

The `.out` opens with a banner that records the whole run, then the matrix whose title follows **model — Reissner-Mindlin here, Classical Plate at model: 0:**

```
OpenSG msg-solid plate model from 1dsg.yaml: 9 plies, h = 0.152400 m, fraction = 0.5, rho*h = 30.251400 kg/m^2, rows [e11 e22 g12 k11 k22 k12
2g13 2g23]

The Effective Reissner-Mindlin Stiffness Matrix
-----
5.4916502E+008    2.6417019E+007    1.5524611E-010    3.6182957E-006    3.6122933E-006    1.0244011E-024    0.0000000E+000
0.0000000E+000
```

The effective compliance follows in the same layout and the file closes with **Time taken: 0.82 sec.** Engineering constants are printed only for the 3-D solid law, never for a plate law, because a plate stiffness is not a material stiffness.

5.1.5 What the numbers mean

The eight diagonal entries of the shipped run, in SI (the code is unit-agnostic and simply follows the input, which is metres and pascals here):

Three checks are worth making on any run of your own, and all three are visible in this one.

- **The section mass.** The banner's $\rho h = 30.2514 \text{ kg/m}^2$ equals $8 \times 0.0009525 \times 1500 + 0.14478 \times 130 = 11.43 + 18.8214 \text{ kg/m}^2$, so the layup that reached the solver is the layup you wrote.
- **The B block.** The stack is symmetric about the mid-surface and `fraction: 0.5` puts the reference plane there, so membrane-bending coupling must vanish. It does: the largest entry of the B block is 3.6183×10^{-6} against an A_{11} of 5.4917×10^8 , fourteen orders down.

term	value	units
A_{11}	5.4916502×10^8	N/m
A_{22}	5.4916502×10^8	N/m
A_{66}	4.1376600×10^7	N/m
D_{11}	3.0005458×10^6	N·m
D_{22}	2.9371144×10^6	N·m
D_{66}	2.0111708×10^5	N·m
G_{11}	7.7009772×10^6	N/m
G_{22}	7.5423712×10^6	N/m

Table 5.6: Diagonal of the 8×8 plate law of the shipped sandwich.

- **The transverse-shear block.** $G_{11} = 7.7010 \times 10^6$ N/m sits 6.4 % above the crude core-only estimate $G_{\text{core}}h_{\text{core}} = 50 \times 10^6 \times 0.14478 = 7.239 \times 10^6$ N/m, which is the expected magnitude for a sandwich whose shear is carried almost entirely by the foam. A_{11} , by contrast, is set by the stiff faces.

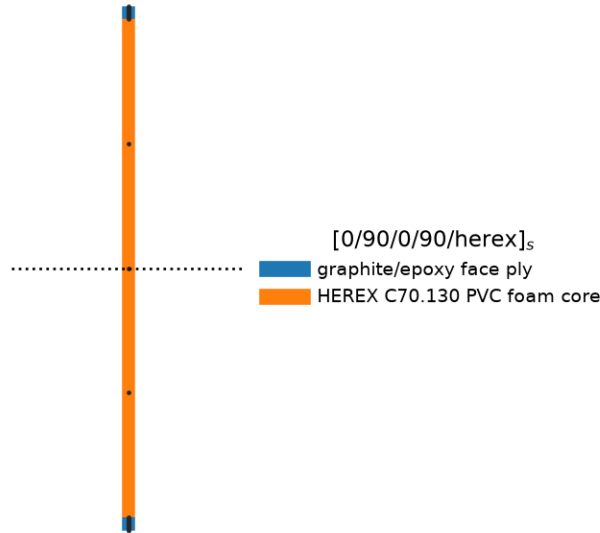


Figure 5.1: The through-thickness 1-D structure gene of the sandwich laminate, as written by the run to 1dsg.png. The gene is a line, drawn with thickness for legibility: the thin graphite/epoxy face plies are at top and bottom, the foam core between them, and the dashed line marks the reference surface at mid-thickness.

5.1.6 What it is validated against

src/opensg_solid/rm_plate_1D/tests/test_msg_rm_plate.py gates the construction on analytic anchors and on an independent plain-NumPy re-assembly of the same SG matrices. For an isotropic plate with $\nu = 0$ the transverse-shear block comes out exactly $\frac{5}{6}Gh$, the textbook shear-correction factor, with the least-squares residual U^* at machine zero (test_isotropic_nu0_gives_five_sixths); $\mathbf{A}_6$ equals the classical lamination-theory msg_materials.compute_ABD_matrix at the matching reference plane (test_A6_matches_compute_ABD); $\mathbf{A}_6(\text{fraction})$ equals

the offset transform of $\mathbf{A}_6(0)$ (`test_reference_plane_transform`); and the warping fields satisfy the Euler-Lagrange equations and the gauge condition $\psi^T H V = 0$ for V_0 , V_{11} and V_{12} directly (`test_euler_lagrange_residuals`, `test_H_identities_and_gauge`). The formulation is that of Yu, Hodges and Volovoi, *Computers & Structures* 81 (2003) 439–454.

5.2 Through-thickness stress recovery

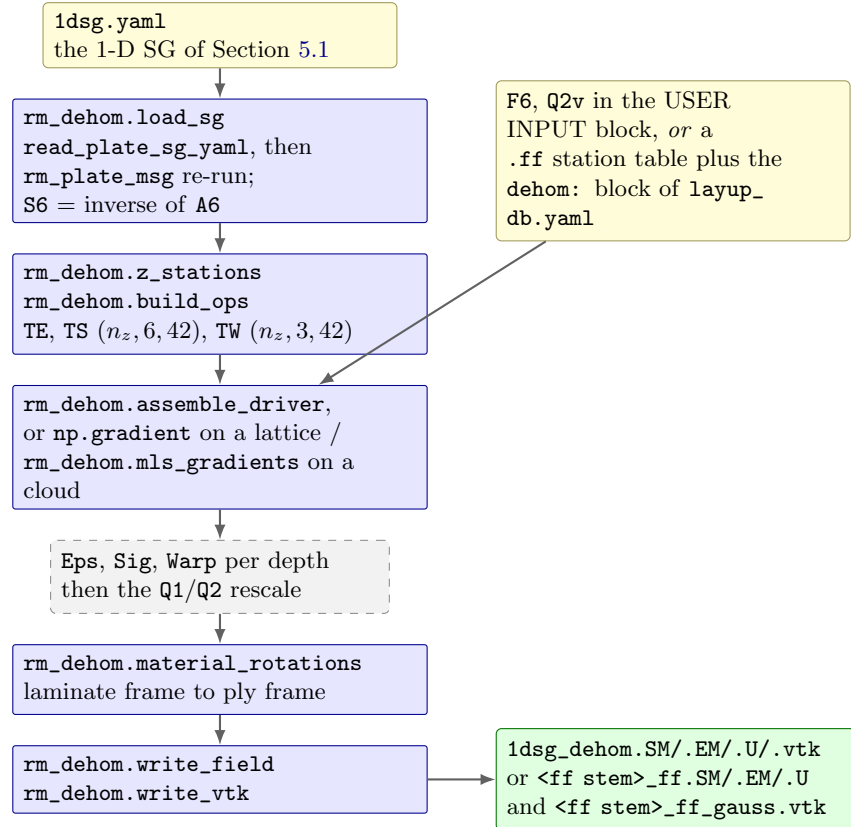
5.2.1 What it computes

Homogenization threw information away; recovery puts it back. Given the section resultants at a point of the plate, the recovery evaluates the 3-D strain, stress, warping and displacement at every depth of the plate. The construction is linear in **42 drivers**,

$$\mathbf{D} = [\mathbf{E}_6, \mathbf{E}_{6,1}, \mathbf{E}_{6,2}, \mathbf{E}_{6,11}, \mathbf{E}_{6,12}, \mathbf{E}_{6,22}, \mathbf{q}\mathbf{t}_6], \quad (5.3)$$

so the code builds three operators once per depth from 42 unit evaluations — $\mathbf{T}_E, \mathbf{T}_S \in \mathbb{R}^{6 \times 42}$ and $\mathbf{T}_W \in \mathbb{R}^{3 \times 42}$ — after which every station is one matrix-vector product, `jax.vmap`-ed over the whole field. The plate strains come from the section law itself, $\mathbf{E}_6 = \mathbf{A}_6^{-1} \mathbf{F}_6$, so a recovery never bypasses the homogenization.

Both drivers of this section — the single station of `rm_dehom.py` and the whole field of `rm_dehom_ff.py` — walk the same core functions in `opensg_solid.rm_plate_1D.rm_dehom`. They differ only in where the 42-driver vector comes from.



Which driver feeds which component is the thing to understand before choosing an input.

driver	where it comes from	what it produces	without it
$\mathbf{E}_6$	$\mathbf{A}_6^{-1}\mathbf{F}_6$ at the station	$\sigma_{11}, \sigma_{22}, \sigma_{12}$ layer by layer	nothing
$\mathbf{E}_{6,1}, \mathbf{E}_{6,2}$	in-plane gradients of the resultants	σ_{13}, σ_{23} (the V_{11}/V_{12} warping)	$\sigma_{13} = \sigma_{23} \equiv 0$
$\mathbf{E}_{6,11}, \mathbf{E}_{6,12}, \mathbf{E}_{6,22}$	second gradients	the σ_{33} interior build-up	σ_{33} keeps only its face terms
$\mathbf{qt}_6$	the <i>applied</i> surface-pressure ladder $[\mathbf{q} \ \mathbf{q},1 \ \mathbf{q},2 \ \mathbf{q},11 \ \mathbf{q},12 \ \mathbf{q},22]$	the face traction $\sigma_{33} = -q$	$\sigma_{33} = 0$ at that face

Table 5.7: The four driver groups of the through-thickness recovery.

Q_1 and Q_2 are deliberately not part of $\mathbf{F}_6$: there is no shear slot among the 42 drivers, because in the asymptotic construction the transverse shears are produced by the gradient brackets. They enter only as the per-station rescale targets $\int \sigma_{13} dz = Q_1$ and $\int \sigma_{23} dz = Q_2$, so the profile *shape* comes from the gradients while the carried force is exactly the one you supplied.

5.2.2 Driver A: one station, resultants typed into the script

`rm_dehom.py` recovers a single station. The input is the `USER INPUT` block at the bottom of the file: `SG_YAML` (the mesh written in Section 5.1), `F6` and `Q2v`, plus the optional gradient and load ladders. `F6` and `Q2v` ship as `None` and the script raises a `ValueError` rather than run, by design — a built-in number would silently masquerade as a result. The block below is the file with the two mandatory entries filled in with an example loading; the values are yours to choose.

```
SG_YAML = os.path.join(HERE, "1dsg.yaml")
F6 = [1.0e6, 0.0, 0.0, 5.0e4, 0.0, 0.0] # [N11, N22, N12, M11, M22, M12]
Q2v = [0.0, 0.0] # [Q1, Q2] [N/m]
DE1 = DE2 = None # d(E6)/dx, d(E6)/dy
D11 = D12 = D22 = None # second gradients
QT6 = None # [q q,1 q,2 q,11 q,12 q,22]
LATTICE = "gauss" # "gauss" (default) or "nodes"
```

Listing 5.1: The `USER INPUT` block of `rm_dehom.py`, with the two mandatory entries supplied.

```
cd examples/OpenSG-solid/2_get_plate_stress_from_1DSG
python rm_dehom.py
```

It writes `1dsg_dehom.SM`, `.EM`, `.U` and `.vtk` — the VTK is part of the default output, not an extra step — and prints `max|S11|` through `max|S23|`. Because the script refuses to run until you supply `F6` and `Q2v`, these four files are produced by your own run and are not checked into the folder.

With every optional input left at `None` this is the honest classical recovery: $\sigma_{11}, \sigma_{22}, \sigma_{12}$ exact layer by layer, $\sigma_{13} = \sigma_{23} = 0$ exactly, σ_{33} at machine zero. One station has no neighbours, so it cannot supply gradients, and the code does not invent them. `LATTICE` selects the output depths: `"gauss"` (the default) puts them at the Gauss abscissae of each element, `"nodes"` at the SG nodes.

5.2.3 Driver B: a whole field from a .ff station table

`rm_dehom_ff.py` is the general driver and the one to use for real work. A `.ff` file is the plate analogue of a VABS `.glb`: whitespace separated, `#` comments, one row per in-plane station, in any

row order and on any station geometry. The shipped table was collected from an Abaqus S8R shell field, but any solver can write it — the driver contains no Abaqus. Its first three lines state the format:

```
# plate dehom FF station table (collected from ex5_shell_S8R_field.dat)
# x[m] y[m] N11 N22 N12[N/m] M11 M22 M12[N] Q1 Q2[N/m] u1 u2 u3[m] wx wy[-]
# 40 x 40 rectangular station lattice, x-fastest
```

Ten columns are required (x y N11 N22 N12 M11 M22 M12 Q1 Q2); the optional columns 11–15 add the mid-surface displacement u_1, u_2, u_3 and slopes w_x, w_y , and without them the .U file carries the warping alone. The loader accepts exactly 10 or 15 columns and raises otherwise. **Every gradient driver is computed from the stations themselves**, by one of two paths chosen automatically: exact tensor-grid finite differences (`np.gradient` on the true non-uniform spacings) when the coordinates form a complete rectangular lattice, or a meshfree KD-tree patch fit ($k = 12$ nearest neighbours, distance-weighted local quadratic least squares, first and second derivatives from one batched solve) for any other cloud. The flag `--scatter` forces the meshfree path.

Unlike the single-station driver, `rm_dehom_ff.py` has no `LATTICE` variable at all: it always calls `z_stations(sg, "gauss")` and therefore always reports at the element Gauss depths. `LATTICE = "nodes"` exists only in `rm_dehom.py`.

The applied face pressure is *not* in the table — loads are input, not solver output — so it lives in `layup_db.yaml`:

```
dehom:
  ff: Abaqus_results/ex5_shell_S8R_field.ff
  loads:
    top: {q0: 68.9476e6, shape: sinsin} # uniform / sin_x / sin_y / sinsin
    bottom: none
```

Listing 5.2: The `dehom:` block of `layup_db.yaml`.

Each `shape` fixes the whole six-term ladder analytically, differentiated with `a` and `b` from the `plate:` block ($a = b = 1.524$ m in the shipped file). Bottom-face loads raise `NotImplementedError`: the 42-driver operator block carries the top ladder only.

```
cd examples/OpenSG-solid/2_get_plate_stress_from_1DSG
python rm_dehom_ff.py ex5_shell_S8R_field.ff
```

Pass the table explicitly, as above. The `dehom: ff:` key in the shipped `layup_db.yaml` points at `Abaqus_results/ex5_shell_S8R_field.ff`, and that folder does not exist in this example: the table sits at the top level of the folder as `ex5_shell_S8R_field.ff`. Leaving the path to the key therefore trips the driver’s own guard, which reads `no .ff table: pass it on the command line or set dehom: ff in` followed by the database path. Either give the file name on the command line, as above, or correct the key. `--db` points at a database elsewhere. The run prints its chosen gradient path (`lattice FD (40 x 40)` for this table), the operator build, the `jax.vmap` over stations, and the peak of each stress component.

5.2.4 What comes out

Outputs are named after the table, `<ff stem>_ff`.

The three text files share a two-line header naming the recovery and its station set. For this run the second line reads `1600 .ff stations [lattice FD (40 x 40)] x the 45 SG-element`

file	content
ex5_shell_S8R_field_ff.SM	3-D stress: columns x y z then S11 S22 S33 S12 S13 S23 in Pa
ex5_shell_S8R_field_ff.EM	3-D strain, same layout, E11 through E23
ex5_shell_S8R_field_ff.U	3-D displacement U1 U2 U3 in m
ex5_shell_S8R_field_ff_gauss.vtk	ASCII STRUCTURED_GRID, DIMENSIONS 40 40 45, 72 000 points, written only on a lattice

Table 5.8: Outputs of `rm_dehom_ff.py` on the shipped table.

Gauss depths; ALL gradient drivers from the stations; S/E in the PLY MATERIAL frame (U global); $z = 0$ is the reference surface. Four conventions matter when you read the files.

- z is in the **SG frame**: $z = 0$ is the reference surface, not the mid-plane, unless `fraction` put them at the same place.
- The depths are the Gauss abscissae of each five-noded element — five per ply here, 45 in all — so every output point is strictly inside one ply, and a material interface, where stress is genuinely double-valued, never appears in a file.
- Stress and strain are written in the **ply material frame** of each depth, which is where failure criteria live. The chain computes in the laminate frame — it must, since the gradients and the Q -rescale integrate across plies — and rotates only at write time. The displacement `.U` stays global.
- Component order in the files is 11 22 33 12 13 23, reordered from the internal SG Voigt order (11, 22, 33, 23, 13, 12) on write.

The `_ff_gauss.vtk` is the file to open when you want to look at the result rather than read it. Figure 5.2 is the shipped table recovered and rendered straight out of it.

5.2.5 What it is validated against

The accuracy table in `src/opensg_solid/rm_plate_1D/USER_GUIDE.md` records the recovery errors against reference elasticity solutions, falling at $\mathcal{O}(S^{-2})$ in the slenderness S .

component (chain)	$S = 4$	$S = 10$	thin
σ_{13} (cross-ply)	21 %	4.4 %	0.18 % at $S = 50$
σ_{33} equilibrium	5.5 %	1.2 %	0.05 %
σ_{11} second-order	27 %	2.0 %	0.04 % at $S = 64$
σ_{12} angle-ply	18 %	—	0.07 % at $S = 64$
U_1	30 %	1.4 %	0.02 % at $S = 64$

Table 5.9: Recovery accuracy against reference elasticity solutions, from `USER_GUIDE.md`.

Soft-core sandwiches need a larger S for the in-plane second-order recovery; their σ_{13} , σ_{33} and displacements stay accurate at $S = 10$. Separately, `dehom_README.md` in the example folder records that the finite-difference gradient path and the analytically differentiated known-mode path were run side by side on the same field and agree to better than 1 % on every component in the interior

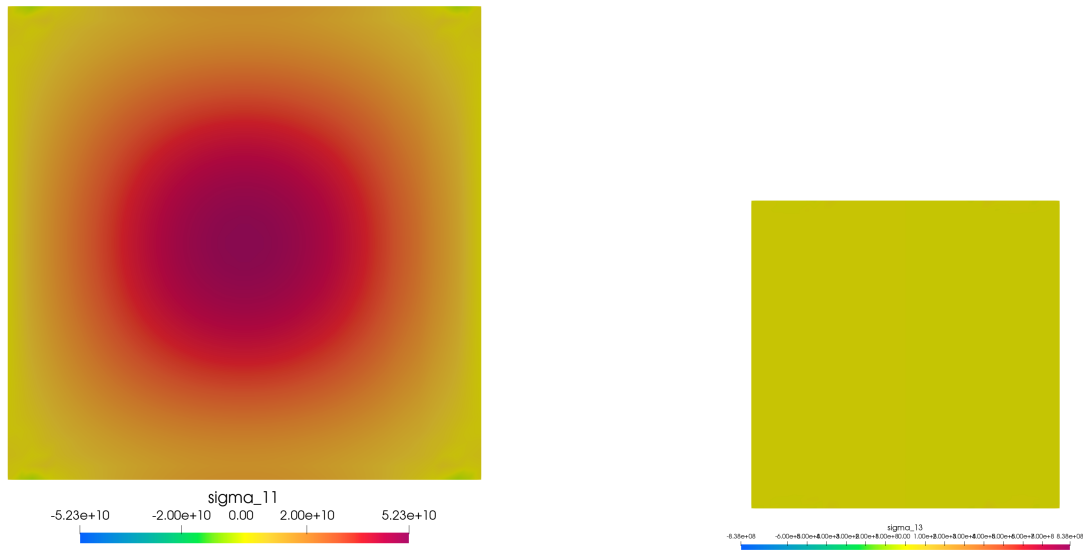


Figure 5.2: ParaView renders of the recovered field on the SG, produced from the shipped `ex5_shell_S8R_field_ff_gauss.vtk` by `docs/latex/render_pv.py`. Left: the `S11` array, σ_{11} in Pa, colour scale $\pm 5.23 \times 10^{10}$; right: the `S13` array, σ_{13} in Pa, colour scale $\pm 8.38 \times 10^8$. The grid points *are* the recovery stations — the 40×40 in-plane lattice times the 45 SG-element Gauss depths — so each coloured value is one recovered Gauss-point value, drawn with flat shading and never averaged across an element boundary. The camera looks down the thickness axis, so the visible face is the outermost Gauss depth. σ_{11} there follows the sin-sin pressure of the `dehom: loads:` block and peaks at the plate centre; σ_{13} is near zero on the full-field scale, because the transverse shear must vanish at the traction-free face, and changes sign across mid-span in x following the $\cos(\pi x/a)$ gradient of that same load.

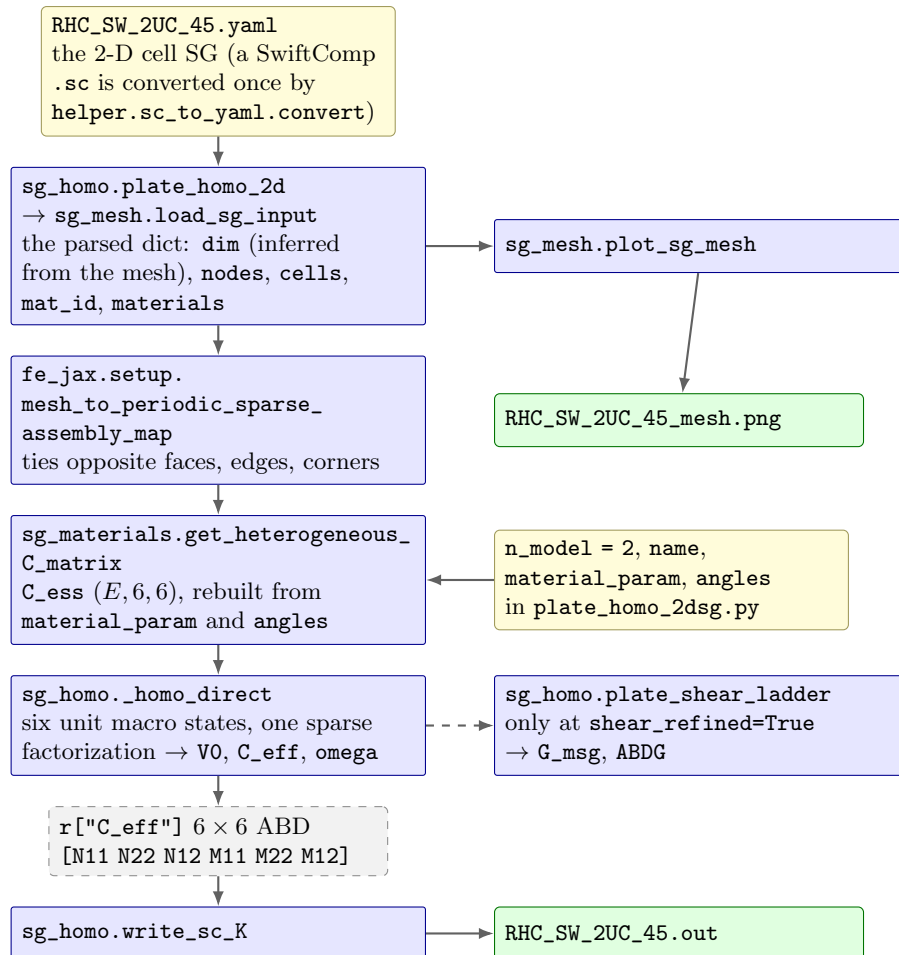
(four integration points trimmed per edge); the 11–15 % whole-field figure is entirely the one-sided stencil band about two elements wide at the boundary. That same README derives the full chain equation by equation and states which term is Yu's and which is this project's. The second-order recovery gate itself is `examples/OpenSG-solid/CS_OpenSG_exampels/static/validate_v2.py`, which drives four Pagano cylindrical-bending cases with exact closed-form harmonic drivers; its reference `.dat` profiles are produced by the per-case `run_case.py` and are not checked in.

5.3 Plate properties from a 2-D cell SG

5.3.1 What it computes

When the wall has in-plane microstructure — a honeycomb core, a corrugation, a truss — the gene is a 2-D mesh of one periodic cell, and `plate_homo_2d` homogenizes it to the plate ABD with `n_model=2`. Opposite faces, edges and corners are tied through the sparse periodic map; `boundary="periodic"` is the default and is the correct treatment for a periodic medium.

Everything below the runner happens inside the single `plate_homo_2d` call, in this order.



5.3.2 The example SG and the input you edit

`RHC_SW_2UC_45.yaml` is a two-unit-cell composite honeycomb sandwich section: 4251 nodes, 6640 three-node triangles, three materials. Coordinates are in millimetres and moduli in MPa. The yaml

is the whole problem: a short scalar header above the mesh blocks says what to run, and the mesh and material blocks say what to run it on.

```
n_model: 2      # 1 = beam, 2 = plate, 3 = solid -- the macro model
refined: 0      # 0 = classical (plate ABD / beam EB); 1 = shear-refined
analysis: H     # H = homogenization / D = dehomogenization
```

Listing 5.3: The header of a solid SG yaml. Every key is defaulted; a headerless mesh still runs.

key	values	meaning
msg	shell, solid	which engine owns the file; omit and the mesh dialect decides
n_model	1, 2, 3	macro model: 1 beam, 2 plate, 3 3-D solid
refined	0, 1	0 classical (plate ABD 6×6 , beam EB 4×4), 1 shear-refined (plate ABDG 8×8 , beam Timoshenko 6×6); ignored for the solid model
analysis	H, D	optional; the command-line argument wins
epsilon_bar	6 floats	the macro state for D
omega	float	optional user SG measure, overriding the one measured from the mesh
aperiodic	1	optional; zero fluctuation on every bounding-box-face node instead of periodicity

Table 5.10: The header contract of the solid SG yaml. **There is no dim: key and no scale: key.** The SG dimension is read from the mesh — the element node count, with ties broken by the leading node-coordinate columns that actually vary — and the SG measure ω is measured from the mesh. A legacy **dim:/scale:** left in an old file is still tolerated by the loader, but neither is part of the input contract and neither should be written into a new file. (The **junction:** family of keys belongs to **msg-shell** recovery and is not read on this route.)

The **material_param** and **angles** arguments of **plate_homo_2d** are an *override* of the material blocks stored in the file, kept for scripting; examples 4 and 7 drive the same cell that way:

```
n_model = 2      # 1: Beam; 2: Plate; 3: 3D elastic
name = "RHC_SW_2UC_45" # reads <name>.yaml

material_param = jnp.array([
    (108e3, 8e3, 8e3, 4e3, 4e3, 3e3, 0.32, 0.32, 0.30), # ply +45
    (108e3, 8e3, 8e3, 4e3, 4e3, 3e3, 0.32, 0.32, 0.30), # ply -45
    (69e3, 69e3, 69e3, 26.54e3, 26.54e3, 26.54e3, 0.30, 0.30, 0.30)])
angles = jnp.array([45.0, -45.0, 0.0]) # deg per material; 0.0 = none
```

Listing 5.4: The material override block, as examples 4 and 7 carry it.

material_param is one row of engineering constants per material, $(E_1, E_2, E_3, G_{12}, G_{13}, G_{23}, \nu_{12}, \nu_{13}, \nu_{23})$, and **angles** one rotation per material in degrees. Passing them *overrides* the material blocks stored in the SG file. The shipped **RHC_SW_2UC_45.yaml** keeps the same data in exactly one place instead: all three materials are **type: 1**, the nine engineering constants together with their ply rotation — **angle: 45.0** and **angle: -45.0** for the two face laminae, and the isotropic aluminium core ($E = 69000$ MPa, $G = 26540$ MPa, $\nu = 0.3$) at **angle: 0.0**. A file that stores its plies *pre-rotated* instead, as a **type: 2** full 6×6 **C** block, must not be given **angles** as well, or the plies are rotated twice. The same numbers are recorded next to the runner in **mat_override.yaml** for reference; omit both arguments to use the file's own blocks unchanged.

The solver reads `<name>.yaml`. A SwiftComp `.sc` is converted once with the packaged helper. That helper is a library function, not a command-line tool: the module `sc_to_yaml` has no `__main__` block, so `python -m opensg_solid.helper.sc_to_yaml file.sc` does nothing. Call `convert` instead, which writes `<base>.yaml` and `<base>.msh` and returns the parsed dictionary:

```
from opensg_solid.helper.sc_to_yaml import convert

sc = convert("RHC_SW_2UC_45.sc")    # writes RHC_SW_2UC_45.yaml + .msh
```

Listing 5.5: Converting a SwiftComp `.sc` once.

5.3.3 Run it

The yaml says what to do, so the run is one command with one argument:

```
cd examples/OpenSG-solid/3_get_plate_props_from_2D_SG
opensg RHC_SW_2UC_45.yaml
```

`opensg` is the unified entry point: it resolves the engine from the yaml’s `msg:` key — or, absent that key, from the mesh dialect — and hands the arguments on unchanged. `opensg_solid RHC_SW_2UC_45.yaml` and `python -m opensg_solid RHC_SW_2UC_45.yaml` are the same run through the engine-specific entry point. `plate_homo_2dsg.py` in the folder is the identical call through the Python API, `plate_homo_2d(name + ".yaml")`, for driving it from your own script.

5.3.4 What comes out

The run writes `RHC_SW_2UC_45.out`, the timed Classical Plate stiffness and compliance in the `.K` layout, and `RHC_SW_2UC_45_mesh.png`, the cell mesh with elements coloured by material. It also prints the same 6×6 to the terminal, under the macro law’s own title. The `.out` banner records the SG measure and the boundary treatment:

```
OpenSG msg-solid plate model, omega 35.248001, periodic

The Effective Classical Plate Stiffness Matrix
-----
3.3585973E+005    7.7500043E+004    -1.4366892E+002    4.1488404E-001    1.4861902E+000    3.6602858E-002
```

closing with `Time taken: 7.83 sec` for this 4251-node, 6640-triangle cell.

5.3.5 What the numbers mean

$\omega = 35.248001$ is the cell’s in-plane period, and it is directly checkable against the mesh: the nodes run from $y_1 = -17.624$ to $+17.624$ mm. It is *measured*, never declared — the yaml carries no normalization key, because the homogenizer always measures its own SG. For a cell converted from SwiftComp the measurement is also checkable against the source: the trailing line of `RHC_SW_2UC_45.sc` reads 35.248, and the measured ω agrees with it to the digits that line carries, which is the check that the mesh and the `.sc` header describe the same cell. The diagonal, in the mesh’s own units:

The membrane law is strongly anisotropic, $A_{11}/A_{22} = 2.97$, while the bending law is much closer to isotropic, $D_{11}/D_{22} = 1.59$: the two directions see the cell very differently in stretching and much less differently in bending. The largest entry of the whole B block is 4.954 against A terms of order 10^5 and D terms of order 10^7 , which is what a cell symmetric about its mid-plane should give and is the first health check to make on a run of your own.

term	value	units
A_{11}	3.3585973×10^5	N/mm
A_{22}	1.1291139×10^5	N/mm
A_{66}	1.0551654×10^5	N/mm
D_{11}	9.0367521×10^7	N-mm
D_{22}	5.7011050×10^7	N-mm
D_{66}	4.6026320×10^7	N-mm

Table 5.11: Diagonal of the honeycomb cell's plate ABD.

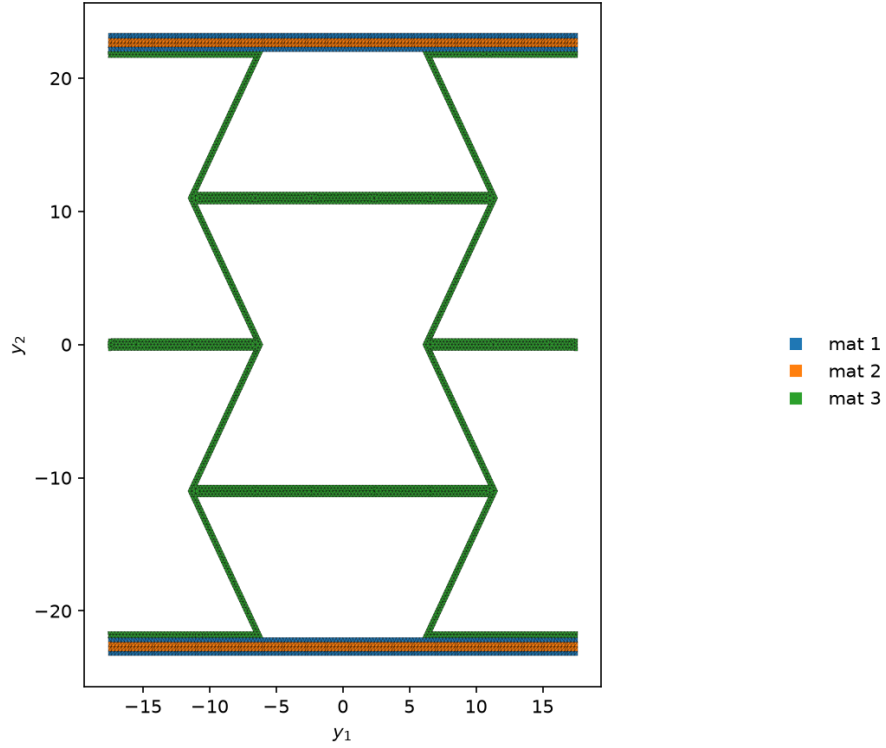


Figure 5.3: The two-unit-cell honeycomb sandwich structure gene, elements coloured by material, as written to `RHC_SW_2UC_45_mesh.png`. The $\pm 45^\circ$ face laminae (materials 1 and 2) are the thin horizontal bands at $y_2 = \pm 23$ mm and the isotropic core web (material 3) folds between them over the in-plane period. The through-thickness coordinate is y_2 , spanning ± 23.35 mm.

5.3.6 The Reissner-Mindlin option

Setting `refined: 1` in the yaml header — or passing `refined=1` to `plate_homo_2d`, of which `shear_refined=True` is the legacy plate-only spelling — additionally runs the RM first-order warping ladder and returns `r["G_msg"]` (the 2×2 transverse-shear block), `r["ABDG"]` (the same 8×8 block form as Eq. (5.2)), `r["A6_ladder"]` and `r["Ustar_rel"]`. When the ladder produces an SPD fit the `.out` is written from the 8×8 and its matrices are titled **Reissner-Mindlin** instead of **Classical Plate**; otherwise the run falls back to the 6×6 . The same header switch serves the beam route: `n_model: 1` with `refined: 0` reports the classical Euler-Bernoulli Beam 4×4 , and with `refined: 1` the Timoshenko Beam 6×6 . The solid law has no refined variant.

```
from opensg_solid.sg_homo import plate_homo_2d

r = plate_homo_2d("RHC_SW_2UC_45.yaml", material_param=material_param,
                  angles=angles, n_model=2, shear_refined=True)
print(r["ABDG"])
```

Listing 5.6: Requesting the 8x8 from a 2-D cell SG.

5.3.7 The 1-D gene in the same folder, and a naming trap

The folder also ships `Plate_1D_SG_2UC_45.yaml`, a **1-D** structure gene — two-node line elements through a stack, its dimension read from the mesh rather than declared in the header — which runs through the identical command by naming that file instead. `plate_homo_2d` reads 1-D, 2-D and 3-D genes from the same code path, so a stack of plies can be homogenized either here or through the dedicated through-thickness route of Section 5.1.

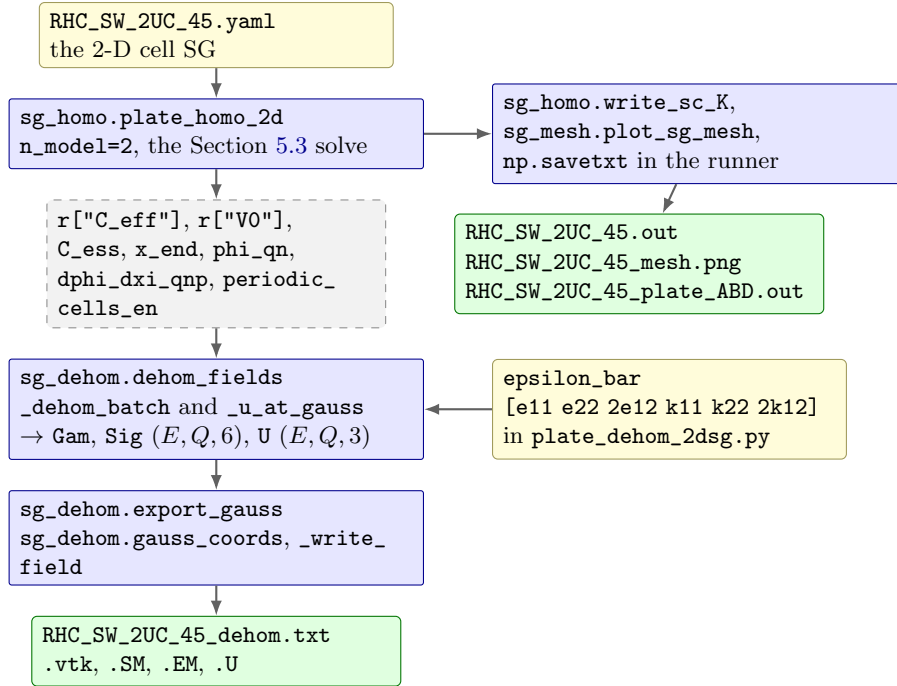
Be careful about the output name. `plate_homo_2d` always derives its report path from the *input* basename, so setting `name = "Plate_1D_SG_2UC_45"` writes `Plate_1D_SG_2UC_45.out` — the .K-layout report — and nothing else. The file `Plate_1D_SG_2UC_45_plate_ABD.out` sitting in that folder is not produced by `plate_homo_2dsg.py` at all: it is an `np.savetxt` leftover from an older script style, a bare 6×6 under a single comment line (`# plate 6x6 [N11 N22 N12 M11 M22 M12] from the 1D SG Plate_1D_SG_2UC_45.sc`) with no banner, no compliance block and no timing. Its contents are still a valid record of that gene — $A_{11} = A_{22} = 1.65452926 \times 10^6$, $A_{66} = 6.42527474 \times 10^5$, $D_{11} = D_{22} = 7.86727254 \times 10^7$, $D_{66} = 3.12496023 \times 10^7$ in N/mm and N·mm — but do not expect your run to reproduce that file name or that layout.

5.4 Plate recovery from a 2-D SG

5.4.1 What it computes

Given a converged plate solution you know the macro plate strain at the point of interest. Feeding it back into the homogenized cell recovers the local 3-D strain and stress at **every Gauss point of the gene**, which is where a honeycomb actually fails — in the web, not in the smeared plate. `dehom_fields` returns the strain, the stress and the fluctuation displacement in one pass, using the warping columns V_0 that the homogenization already solved. Nothing is re-solved.

That reuse is the whole point of the tree below: the homogenization node runs once and its result dictionary carries the warping modes, the element stiffness table and the quadrature data straight into the recovery.



5.4.2 The input you edit

The recovery needs one thing beyond the homogenization: the macro plate strain, in the order `[e11, e22, 2e12, k11, k22, 2k12]`. On the command-line route it is a header key of the SG yml, read when the analysis is D:

```
epsilon_bar: [0.0, 0.1, 0.0, 0.1, 0.0, 0.0]
```

Listing 5.7: The macro plate strain driving the recovery, in the yml header.

`plate_dehom_2dsg.py` sets the same six numbers in code, as `epsilon_bar = jnp.array([0.0, 0.1, 0.0, 0.1, 0.0, 0.0])`, alongside the `name`, `material_param` and `angles` of Section 5.3.

The shipped state combines a transverse extension $\varepsilon_{22} = 0.1$ with a bending curvature $\kappa_{11} = 0.1 \text{ mm}^{-1}$. This vector is yours: it comes from your plate solution at the station you care about, and there is no default that would be meaningful.

5.4.3 Run it

The trailing D asks for dehomogenization: the run homogenizes first, then recovers, and reports one `Time taken` covering the whole thing.

```
cd examples/OpenSG-solid/4_get_plate_dehom_from_2DSG
opensg RHC_SW_2UC_45.yml D
```

`python plate_dehom_2dsg.py` is the same chain through the Python API, and additionally writes the bare 6×6 `RHC_SW_2UC_45_plate_ABD.out` through `np.savetxt`.

5.4.4 What comes out

The 19920 rows are 6640 triangles times three quadrature points. The script prints the ABD diagonal and the peak of each stress component.

file	content
RHC_SW_2UC_45.out	the timed .K-layout Classical Plate report written by plate_homo_2d inside the script; regenerated on each run
RHC_SW_2UC_45_plate_ABD.out	the plain-text 6×6 ABD via np.savetxt , header plate 6x6 [N11 N22 N12 M11 M22 M12] from the 2D SG RHC_SW_2UC_45
RHC_SW_2UC_45_dehom.txt	one row per Gauss point: GP_Index, Coord_X, Coord_Y , then Eps_xx Eps_yy Eps_zz Eps_yz Eps_xz Eps_xy and Sig_xx through Sig_xy — SwiftComp component order; one header line plus 19920 rows
RHC_SW_2UC_45_dehom.vtk	ASCII UNSTRUCTURED_GRID point cloud of the same 19920 points, each strain and stress component as its own SCALARS array, ready for ParaView
RHC_SW_2UC_45_dehom.SM / .EM / .U	the same fields in the rm_plate_1D layout: x y z then S11 S22 S33 S12 S13 S23, E11 through E23 , and U1 U2 U3
RHC_SW_2UC_45_mesh.png	the mesh figure, from the plate_homo_2d call the script makes first

Table 5.12: Outputs of `plate_dehom_2dsg.py`.

Two conventions are worth knowing before reading the files. In the **.SM/.EM/.U** tables the SG coordinates are placed in the **last** slots of the **x y z** triple, so a 2-D gene leaves the **x** column at zero and puts y_1 in **y** and the through-thickness y_2 in **z**. And **.U** is the **fluctuation-only** displacement — its own header says so — because a unit-state SG recovery has no macro displacement to add.

5.4.5 Reading the result

The first Gauss point of the shipped run sits at $(y_1, y_2) = (17.54067, 23.29375)$ mm, right at the top face of the cell whose half-thickness is 23.35 mm, and reports

$$\varepsilon_{11} = 2.329375, \quad \sigma_{11} = 8.755003 \times 10^4, \quad \sigma_{22} = 7.102971 \times 10^4, \quad \sigma_{12} = -6.542276 \times 10^4 \quad (5.4)$$

in MPa. That ε_{11} is exactly $\kappa_{11}y_2 = 0.1 \times 23.29375$, the macro part of the strain reproduced to every printed digit, which is the cheapest possible check that the strain state you passed is the strain state that arrived. The other five components are what the recovery adds: $\varepsilon_{22} = 0.2643$ and $\varepsilon_{33} = -0.9187$ are the fluctuation response of the cell, and σ_{12} is large even though the applied macro shear strain $2\varepsilon_{12}$ is zero, which is consistent with the off-axis face laminae the mesh figure shows at that height.

Over the whole gene the field maxima are:

component	max $ \sigma $ (MPa)	max $ \varepsilon $
<i>xx</i>	1.641632×10^5	2.329375
<i>yy</i>	9.199997×10^4	1.499038
<i>zz</i>	2.535821×10^4	1.032184
<i>yz</i>	1.083274×10^4	1.153314
<i>xz</i>	5.210321×10^3	8.807437×10^{-1}
<i>xy</i>	7.369978×10^4	7.881358×10^{-1}

Table 5.13: Component maxima over the 19920 Gauss points of the shipped plate recovery.

The *xx* strain maximum is again $\kappa_{11}y_2^{\max} = 2.329375$, as it must be at the extreme fibre. Local states like the σ_{zz} and σ_{yz} columns are exactly what a smeared plate model cannot show you, and

the reason to run a dehomogenization at all. The driver here is a demonstration magnitude, not a service load; scale it to your own macro state before reading the stresses as failure indices.

5.5 Equivalent 3-D solid from a 2-D SG

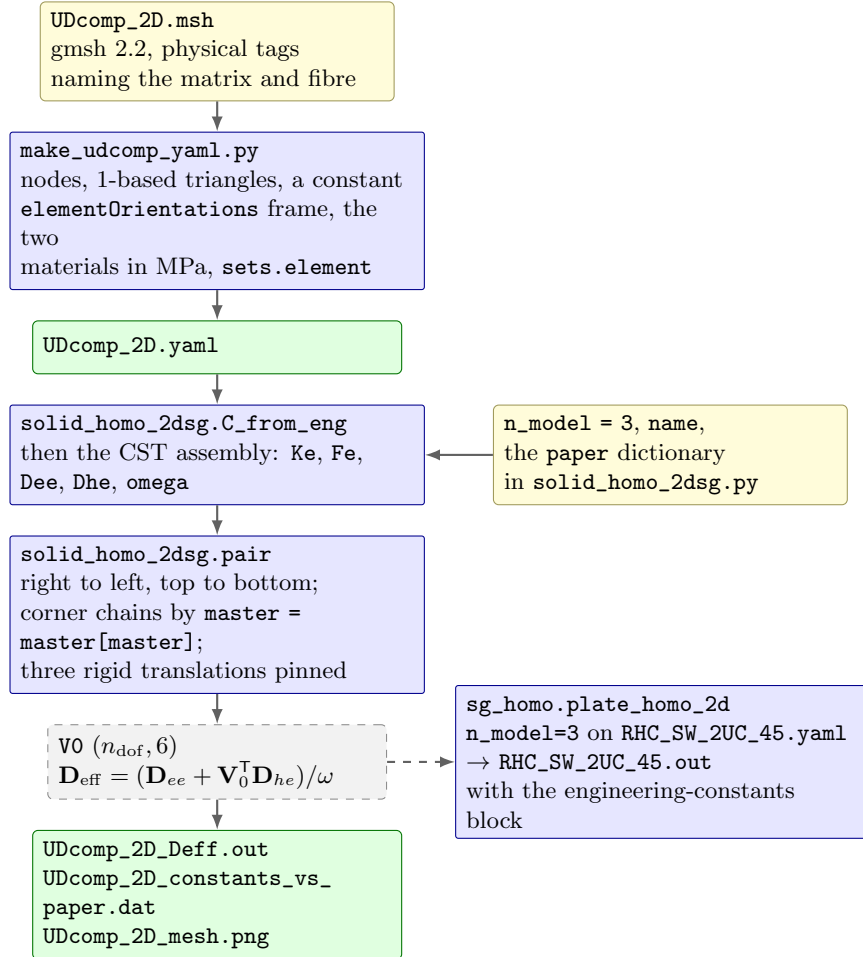
5.5.1 What it computes

`n_model=3` reduces an SG to an equivalent **3-D solid** stiffness: the 6×6 material law of the homogenized continuum, in Voigt order [11, 22, 33, 23, 13, 12] with engineering shears. On a **2-D** SG this is the generalized-plane-strain problem: the cell is periodic in its two in-plane directions and uniform out of plane, six unit macro strains are applied, the fluctuation field $\mathbf{V}_0$ is solved for each, and the effective law is

$$\mathbf{D}_{\text{eff}} = \frac{\mathbf{D}_{ee} + \mathbf{V}_0^T \mathbf{D}_{he}}{\omega}, \quad (5.5)$$

with ω the cell area. This is the classical micromechanics unit-cell problem, which makes it the natural place to validate the route against published numbers.

The shipped runner takes the self-contained path on the left of the tree; the engine path on the right is the same macro model reached through `plate_homo_2d`, and is what a 2-D SG in the ordinary `sg_mesh.load_sg_input` dialect goes through.



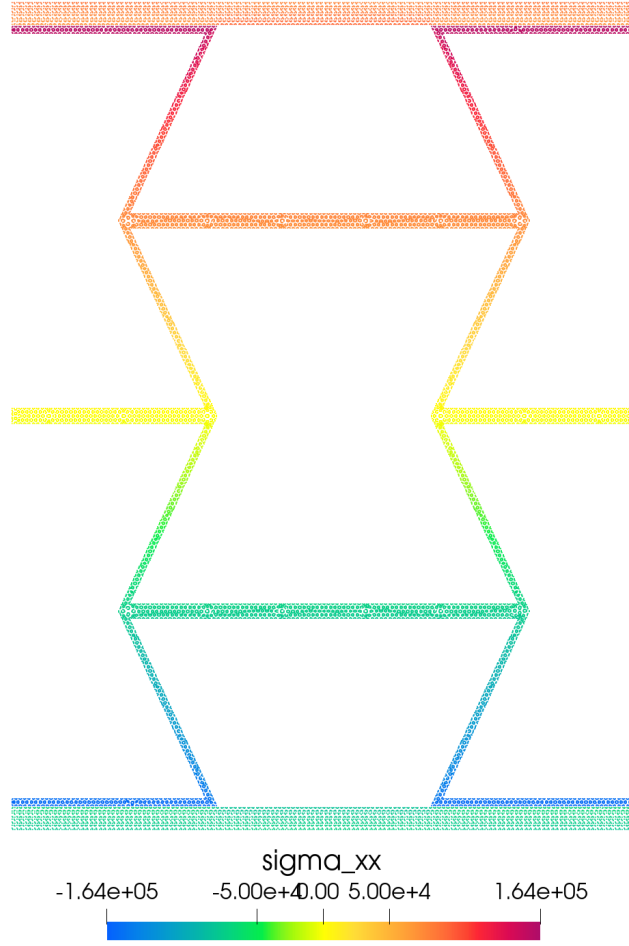


Figure 5.4: ParaView render of the recovered field on the SG mesh, produced from the `RHC_SW_2UC_45_dehom.vtk` written by this example, using `docs/latex/render_pv.py`. The array is `Sig_xx`, the axial stress σ_{xx} in MPa (units follow the SG input, millimetres and megapascals), on a symmetric colour scale of $\pm 1.64 \times 10^5$ MPa — the xx row of Table 5.13. Every one of the 19 920 points is a single Gauss point carried by its own vertex cell and coloured flat, so no value has been averaged across an element boundary. The field varies linearly with the through-thickness coordinate y_2 , which is the macro part $\kappa_{11}y_2$ of the driver, and each face pack splits into two sub-bands of visibly different σ_{xx} at almost the same height, consistent with the $+45^\circ$ and -45° laminae differing in the sign of their C_{16} coupling.

5.5.2 The example SG

`examples/OpenSG-solid/5_get_solid_props_from_SG` homogenizes a unidirectional fibre/matrix square unit cell: 801 nodes, 1536 three-node triangles, 512 matrix elements and 1024 fibre elements, cell area $\omega = 1$.

material	E_1, E_2, E_3 (MPa)	G_{12}, G_{13}, G_{23} (MPa)	$\nu_{12}, \nu_{13}, \nu_{23}$	density
matrix	4760, 4760, 4760	1737.226 each	0.37, 0.37, 0.37	1200.0
fibre	276000, 19500, 19500	70000, 70000, 5735	0.28, 0.28, 0.70	1800.0

Table 5.14: The two phases of the UD unit cell. The matrix shear modulus is written by the converter as $4760/(2 \cdot 1.37)$.

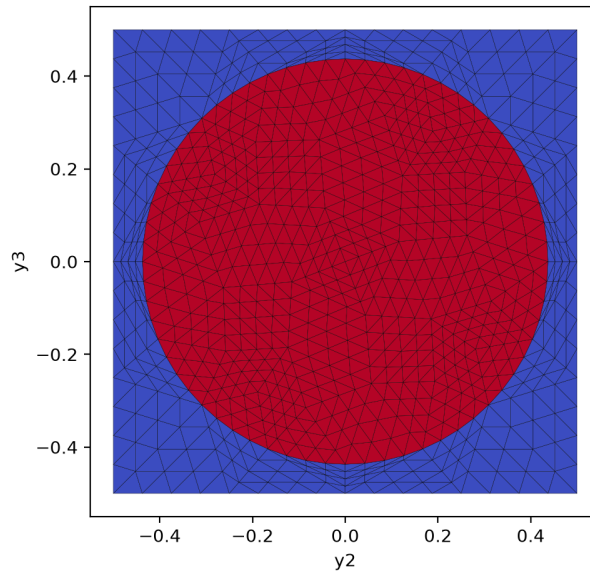


Figure 5.5: The unidirectional composite unit cell written to `UDcomp_2D_mesh.png`, fibre and matrix elements coloured by phase, with the SG coordinates labelled y_2 and y_3 .

5.5.3 Step 1: build the SG yaml from the gmsh mesh

`make_udcomp_yaml.py` is a one-time converter from `UDcomp_2D.msh` (gmsh 2.2 format) to `UDcomp_2D.yaml`. It writes the nodes as - `[x y z]` rows, the triangles as 1-based - `[n1 n2 n3]` rows, a constant `elementOrientations` frame ($e_1 = \text{axial } z, e_2 = +x, e_3 = +y$), the two materials as engineering constants in MPa, and `sets.element` splitting the mesh into the matrix and fibre phases by gmsh physical tag. The yaml is already checked in, so this step is only needed if you remesh.

```
cd examples/OpenSG-solid/5_get_solid_props_from_SG
python make_udcomp_yaml.py
```

5.5.4 Step 2: homogenize

```
python solid_homo_2dsg.py
```

The script is kept rather than replaced by the unified command because it does more than homogenize: it writes the comparison table against the published constants and the mesh figure. For a 2-D SG in the ordinary solid dialect the same macro law is one command, `opensg <name>.yaml` with `n_model: 3` in the header.

The User Input block is three lines: `n_model = 3`, `name = "UDcomp_2D"` (which reads `<name>.yaml`), and the `paper` dictionary of reference constants to compare against.

This particular runner is deliberately self-contained. It assembles constant-strain triangles, the periodic master-slave node map (right to left matching y , top to bottom matching x , with corner chains resolved by three passes of `master = master[master]`) and the three pinned rigid translations in plain NumPy, so the entire `n_model=3` algebra is readable in one file. It therefore writes its own `_Deff.out` through `np.savetxt` rather than the SwiftComp-layout report, and it consumes the `nodes/elements/elementOrientations/materials/sets` yaml dialect that `make_udcomp_yaml.py` emits, not the `nodes/cells/mat_id/materials` dialect that `opensg_solid.sg_mesh.load_sg_input` parses. The engine path itself is exercised on the honeycomb SG: `n_model=3` on `RHC_SW_2UC_45.yaml` runs through `plate_homo_2d` and writes the full SwiftComp `.out` with its engineering-constants block.

5.5.5 What comes out

file	content
UDcomp_2D_DeFF.out	the 6×6 $\mathbf{D}_{\text{eff}}$ in MPa, Voigt order [e11 e22 e33 2e23 2e13 2e12], with ω in the header
UDcomp_2D_constants_vs_paper.dat	the nine effective engineering constants next to the published values and the percentage difference
UDcomp_2D_mesh.png	the unit cell, elements coloured by phase

Table 5.15: Outputs of `solid_homo_2dsg.py`.

5.5.6 What the numbers mean, and what they were validated against

The nine constants are read off the compliance $\mathbf{S} = \mathbf{D}_{\text{eff}}^{-1}$ as $E_i = 1/S_{ii}$, $G_{ij} = 1/S_{kk}$ and $\nu_{ij} = -S_{ij}/S_{ii}$, and compared against Table II Model-3 of the ASC-23 OpenSG-Solid paper — the same case as the FEniCS `3DModel.ipynb` in `wenbinyugroup/OpenSG`. The shipped `UDcomp_2D_constants_vs_paper.dat` reads:

Every constant agrees with the published value to within 0.002%, which is the round-off of the paper's own printed precision. The route is exact against its reference, not merely close.

The stiffness itself, from `UDcomp_2D_DeFF.out` (MPa, $\omega = 1$):

$$\mathbf{D}_{\text{eff}} = \begin{bmatrix} 1.726544 \times 10^5 & 8.658488 \times 10^3 & 8.658488 \times 10^3 & 0 & 0 & 0 \\ 8.658488 \times 10^3 & 1.750725 \times 10^4 & 1.026390 \times 10^4 & 0 & 0 & 0 \\ 8.658488 \times 10^3 & 1.026390 \times 10^4 & 1.750725 \times 10^4 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3.095555 \times 10^3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 6.801763 \times 10^3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 6.801763 \times 10^3 \end{bmatrix}$$

constant	this code	paper	% difference
E_1 (GPa)	167.255337	167.255300	+0.0000
E_2 (GPa)	11.413657	11.413600	+0.0005
E_3 (GPa)	11.413657	11.413600	+0.0005
G_{23} (GPa)	3.095555	3.095500	+0.0018
G_{13} (GPa)	6.801763	6.801700	+0.0009
G_{12} (GPa)	6.801763	6.801700	+0.0009
ν_{12}	0.311780	0.311780	-0.0000
ν_{13}	0.311780	0.311780	-0.0000
ν_{23}	0.575743	0.575740	+0.0006

Table 5.16: UDcomp_2D_constants_vs_paper.dat, verbatim.

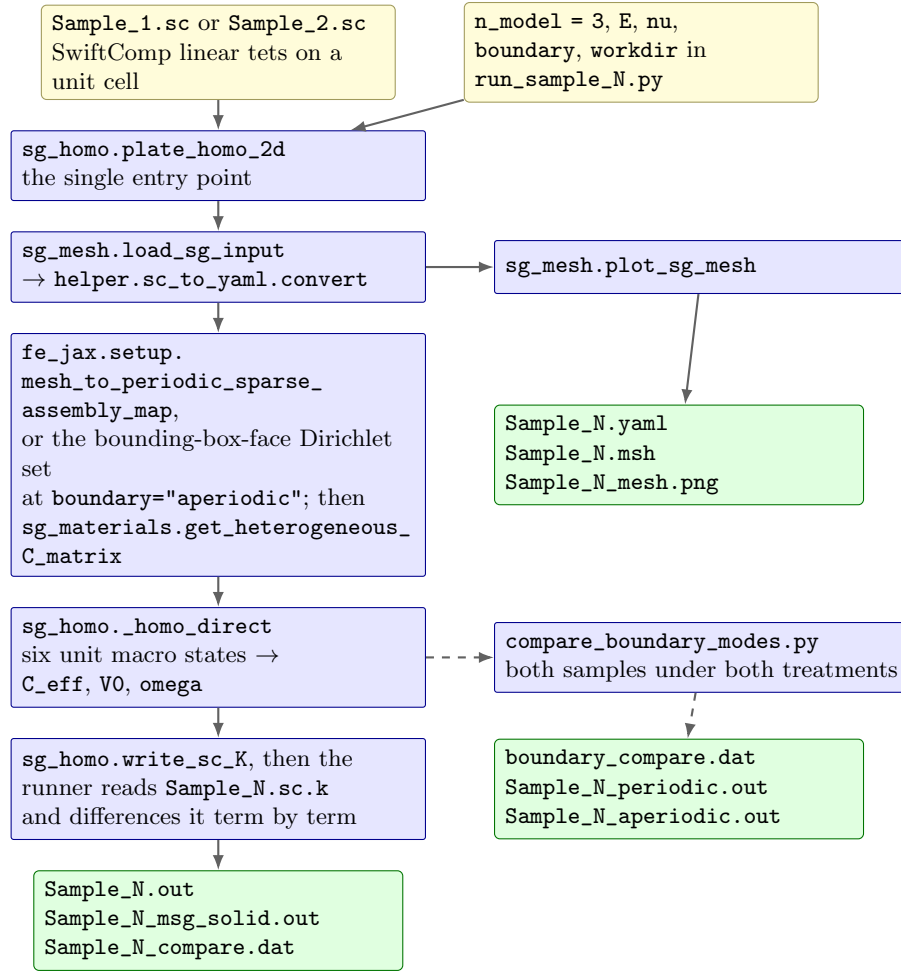
Two health checks are worth making a habit of. First, the symmetry the geometry demands is recovered: $D_{22} = D_{33}$ and $D_{55} = D_{66}$ to all printed digits, and every normal-shear coupling term in the shipped file is either exactly zero or below 3.4×10^{-2} MPa, six to seven orders under the diagonal, which is the numerical signature of a correctly tied periodic cell. Second, the transverse plane is **not** isotropic: a transversely isotropic material would satisfy $D_{44} = (D_{22} - D_{23})/2 = 3621.7$ MPa, whereas the computed D_{44} is 3095.6 MPa, 14.5 % lower. A square fibre array is tetragonal, so G_{23} genuinely has to be homogenized rather than inferred from E_2 and ν_{23} , which is precisely why a unit-cell solve is needed instead of a rule of mixtures.

5.6 Equivalent 3-D solid from a 3-D SG

5.6.1 What it computes

When the SG is periodic in all three directions — a triply-periodic minimal surface (TPMS) unit cell, a lattice block, any genuine 3-D microstructure — the same macro model is reached with `plate_homo_2d(..., n_model=3, boundary="periodic")` on a tet or hex mesh. "periodic" is the default on every route; `boundary="aperiodic"` maps the boundary solution onto the bounding-box-face nodes as Dirichlet data (solid elements carry only translations, so this clamps every degree of freedom of those nodes) and is the explicit request for SGs that genuinely are not periodic. Aperiodic supports the plate and solid models with `solver="direct"` only; asking for it with `n_model=1` or `solver="cg"` raises.

The tree is the sample runner and the boundary comparison, the two paths every number in this section comes from. Note the loader branch: a SwiftComp .sc is converted on the way in, so the .yaml and .msh appear in the working directory as a side effect of the homogenization.



5.6.2 The samples and the runners

Two TPMS samples are supplied as SwiftComp `.sc` meshes of linear tets on a unit cell, in aluminium. Reading the `.sc` meta line directly gives their sizes.

sample	surface	nodes	tets	relative density
Sample_1	Schwarz-P	116 851	545 741	0.3000
Sample_2	Schwarz-P	54 874	191 957	0.0857

Table 5.17: The two TPMS solid structure genes. The relative density is the fraction of the cell filled by aluminium; it is *not* the SG measure. Both genes sit on a unit cell, so for both $\omega = 1$.

Four runners sit in `6_get_solid_props_from_3D_SG`.

```
cd examples/OpenSG-solid/6_get_solid_props_from_3D_SG/sample_1
python run_sample_1.py
```

```
cd examples/OpenSG-solid/6_get_solid_props_from_3D_SG
python compare_boundary_modes.py
```

Both scripts are kept as scripts because both do something the solver does not: the sample runner differences the result against the vendor's `.K` and writes `Sample_N_compare.dat`, and `compare_`

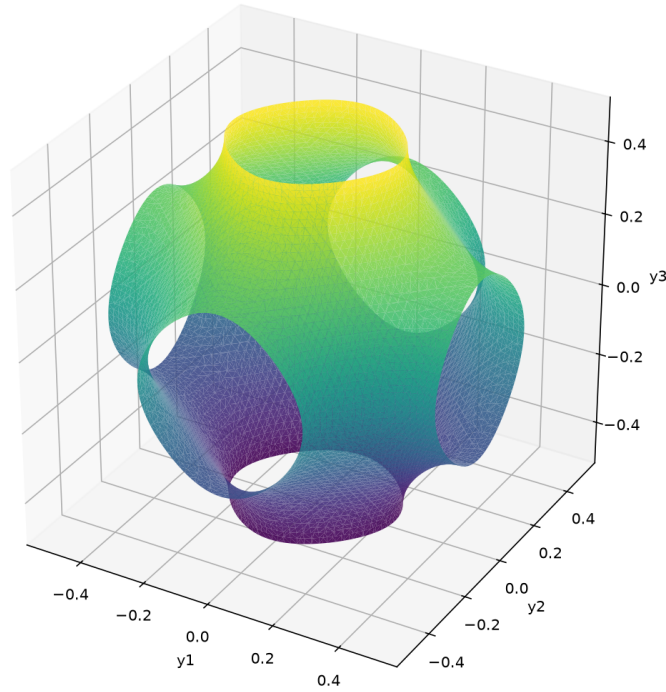


Figure 5.6: The Schwarz-P triply-periodic minimal surface on its unit cell, drawn from the real mesh of `schwarz_p_3Dshell.yaml` by `examples/OpenSG_shell/4_get_solid_props_from_shell_3D_SG/schwarz_solid_props.py`, with the SG coordinates labelled y_1 , y_2 and y_3 . This is the surface itself, meshed as a `msg-shell` SG; the two solid genes of Table 5.17 discretise the same surface as a volume of linear tets, which is why `tpms_all_compare.dat` can put the two engines side by side at matched thickness.

script	what it does
<code>sample_1/run_sample_1.py</code> , <code>sample_2/run_sample_2.py</code>	homogenize one sample at $E = 69$ GPa, $\nu = 0.3$ with <code>boundary="periodic"</code> and compare term by term against the vendor's own <code>Sample_N.sc.k</code> ; writes <code>Sample_N.out</code> , <code>Sample_N_msg_solid.out</code> , <code>Sample_N_compare.dat</code> and <code>Sample_N_mesh.png</code>
<code>compare_boundary_modes.py</code>	runs both samples under both boundary treatments and writes <code>boundary_compare.dat</code> , plus per-mode copies <code>Sample_N_periodic.out</code> and <code>Sample_N_aperiodic.out</code>
<code>tpms_solid_props.py</code>	the plain homogenization driver, taking the sample number as an optional argument
<code>sc_to_3dyaml.py</code>	converts the <code>.sc</code> tets to the OpenSG 3-D solid yaml and reports the tet-volume check that distinguishes a solid mesh from a shell surface

Table 5.18: The runners of example 6.

`boundary_modes.py` runs each sample twice and tabulates the two treatments against each other. The homogenization on its own is one command — `opensg Sample_1.yaml`, whose header already carries `msg: solid, n_model: 3` and `refined: 0`.

One detail to check before comparing numbers across scripts: the two sample runners and `compare_boundary_modes.py` use $E = 69$ GPa, matched to the SwiftComp .K, whereas `tpms_solid_props.py` and `sc_to_3dyaml.py` use $E = 70$ GPa. Only the 69 GPa runs are comparable with the benchmark below. The `Sample_N_msg_solid.out` files that `tpms_solid_props.py` writes at the top level of the folder are regenerated on each run.

5.6.3 Normalization: the .out is already per unit cell

For a 3-D solid SG the measure ω is the **node bounding-box volume**, not the summed element (material) volume. Both these genes sit on a unit cell, so $\omega = 1$ and the .out is written per unit cell — exactly the normalization a SwiftComp .K uses. `Sample_1.out` and `Sample_1_periodic.out` both open with `OpenSG msg-solid 3D elastic model, omega 1, periodic` and report $C_{11} = 1.0190158 \times 10^{10}$ Pa = 10190.1580 MPa, against the 1.0190158E+004 MPa of `Sample_1.sc.k`; Sample 2 reports 2.3031598×10^9 Pa = 2303.1598 MPa against 2.3031598E+003. The two files are therefore *directly* comparable, with no rescale in between, and the comparison scripts’ surviving `C x omega` line is a no-op at $\omega = 1$.

This is a change from the older behaviour, in which ω was the material volume — 0.3 and 0.0857 for these two cells — so the .out came out 1/relative density too stiff and only the comparison table’s rescale made the numbers line up. Any note to that effect is obsolete: the figures printed in the .out now are the figures in the tables below. Every comparison table below is per unit cell, in MPa.

5.6.4 The SwiftComp digit-parity benchmark

term	Sample 1			Sample 2		
	msg-solid	SwiftComp	err %	msg-solid	SwiftComp	err %
C_{11}	10190.158	10190.158	−0.000	2303.160	2303.160	−0.000
C_{12}	5646.069	5646.068	+0.000	1612.130	1612.130	−0.000
C_{13}	5646.092	5646.091	+0.000	1612.141	1612.141	+0.000
C_{22}	10190.131	10190.131	+0.000	2303.067	2303.067	+0.000
C_{23}	5646.012	5646.012	+0.000	1612.120	1612.120	+0.000
C_{33}	10189.958	10189.958	+0.000	2303.019	2303.018	+0.000
C_{44}	4519.640	4519.640	−0.000	1106.734	1106.735	−0.000
C_{55}	4519.637	4519.637	−0.000	1106.668	1106.668	+0.000
C_{66}	4519.755	4519.755	−0.000	1106.664	1106.664	−0.000

Table 5.19: `Sample_1_compare.dat` and `Sample_2_compare.dat`, per unit cell in MPa, at $E = 69$ GPa and $\nu = 0.3$. Sample 1 is 545 741 tets and solved in 165.9 s; Sample 2 is 191 957 tets and solved in 61.4 s.

Digit-for-digit on both samples, to seven significant figures on every entry. The effective law is cubic to five digits, and every symmetry-forbidden coupling sits four to five orders below the diagonal, which is the health check on the all-directions periodic tie. The engineering constants that the .out prints for Sample 2 — $E_1 = 9.7551133 \times 10^8$ Pa, $G_{12} = 1.1066643 \times 10^9$ Pa, $\nu_{12} = 0.41174$ — are per unit cell in the same way, and only the two moduli carry that normalization; Poisson’s ratio is a ratio.

5.6.5 Periodic versus aperiodic

`compare_boundary_modes.py` runs the same `.sc` through both treatments, so the percentage column measures the aperiodic boundary layer of a single cell directly against the exact periodic answer.

term	Sample 1			Sample 2		
	aperiodic	periodic	% diff	aperiodic	periodic	% diff
C_{11}	12977.036	10190.158	+27.349	2810.329	2303.160	+22.021
C_{22}	12977.030	10190.131	+27.349	2810.363	2303.067	+22.027
C_{33}	12976.815	10189.958	+27.349	2810.251	2303.019	+22.025
C_{12}	5423.565	5646.069	−3.941	1548.627	1612.130	−3.939
C_{13}	5423.674	5646.092	−3.939	1548.954	1612.141	−3.919
C_{23}	5423.151	5646.012	−3.947	1549.049	1612.120	−3.912
C_{44}	5545.997	4519.640	+22.709	1300.352	1106.734	+17.494
C_{55}	5546.044	4519.637	+22.710	1300.151	1106.668	+17.483
C_{66}	5546.133	4519.755	+22.709	1300.048	1106.664	+17.474
solve	146.3 s	150.2 s		52.4 s	51.5 s	

Table 5.20: `boundary_compare.dat`, per unit cell in MPa. The aperiodic run clamps 17610 boundary nodes on Sample 1 and 4456 on Sample 2.

Read the table this way. Clamping the fluctuation on the cube faces forbids the affine fluctuation modes, so the cell can only come out stiffer: the normal and shear diagonals rise by 17 to 27 %, while the off-diagonal C_{12} family falls by about 4 %. The bias is large because the clamped set is large — 17 610 of 116 851 nodes, or 15 %, for Sample 1, and 4456 of 54 874, or 8 %, for Sample 2 — since the solid sheet meets the cube faces in two-dimensional patches. On the solid route aperiodic is also **not** faster: the periodic tie merges only face pairs, so the factorized size barely changes, and the two timings are within a few percent of each other in both directions. Periodic is the exact treatment for a periodic medium and is therefore the default; aperiodic is the right model only when the SG genuinely is not periodic.

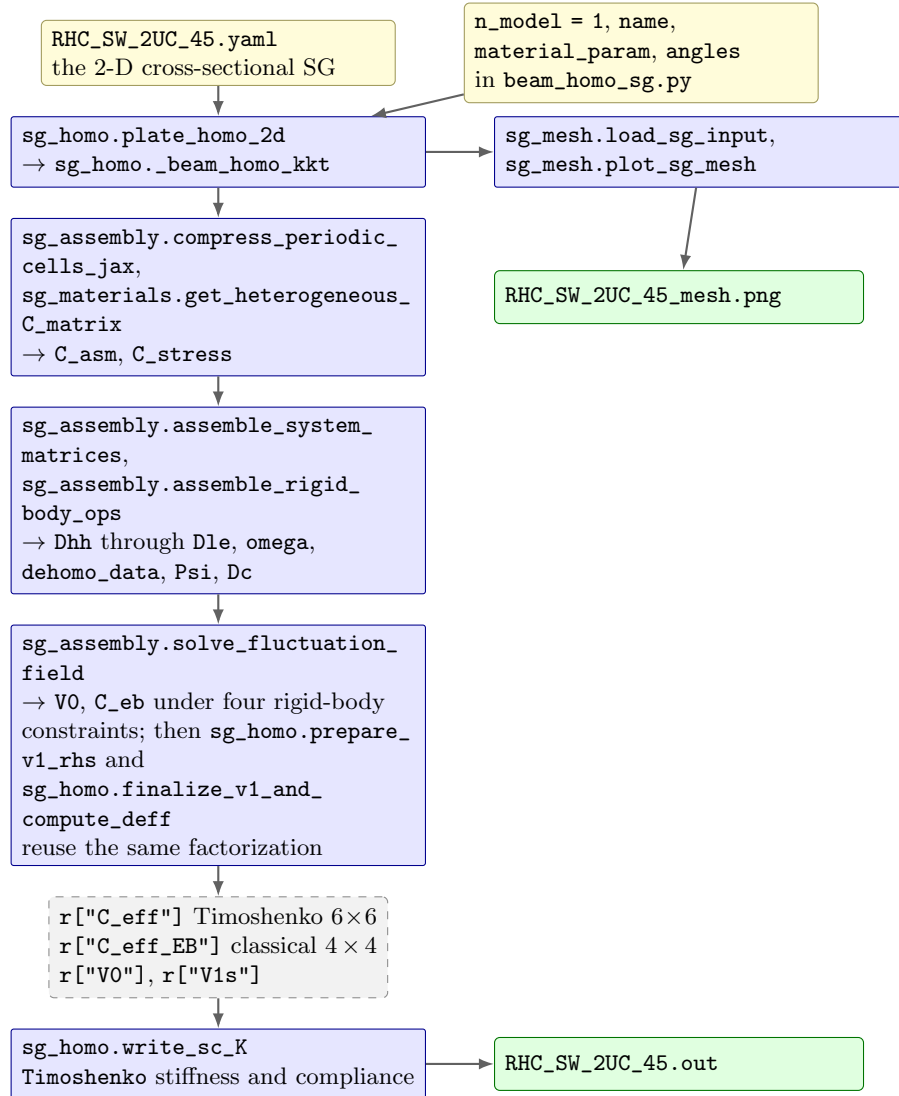
The folder carries two further comparison tables, both regenerated by their own scripts: `tpms_all_compare.dat`, which lines the solid results up against the same Schwarz-P surface homogenized by the `msg-shell` route at matched thickness, and `solid_vs_shell_compare.dat`, which adds the normalized stiffness $C/(E\rho)$ and the Zener ratio $2C_{44}/(C_{11} - C_{12})$ — 1.989 for Sample 1, 3.203 for Sample 2, and 3.151 for the shell run at the matched thickness $t = 0.0365$. Those three rows are the cleanest available statement that the two engines describe the same structure.

5.7 Beam properties from a 2-D SG

5.7.1 What it computes

`n_model=1` reduces a cross-sectional SG to the Timoshenko beam constitutive law, relating the axial force, two transverse shear forces, torque and two bending moments to the six generalized strains $[\epsilon_{11}, \gamma_{12}, \gamma_{13}, \kappa_1, \kappa_2, \kappa_3]$. Internally this is the Beam/solid KKT engine merged into `sg_assembly/sg_homo`: the zeroth-order warping solve V_0 is run under four rigid-body Lagrange constraints, the l -chain first-order solve V_{1s} reuses the same factorization, and the pair is reduced to the 6×6 . The classical Euler-Bernoulli 4×4 in $[\epsilon_{11}, \kappa_1, \kappa_2, \kappa_3]$ falls out of the same V_0 solve and rides along in `r["C_eff_EB"]` at no extra cost. The beam route requires an SG of dimension 2 or 3.

The entry point is the same `plate_homo_2d` as every other route, but `n_model=1` branches to a different engine at the first fork, and that engine is where the two shear rows come from.



5.7.2 The example SG and the input you edit

`examples/OpenSG-solid/7_get_beam_props_from_SG` homogenizes the same `RHC_SW_2UC_45.yaml` honeycomb cross-section as Section 5.3: 4251 nodes, 6640 three-node triangles. Coordinates are in mm and moduli in MPa, so the resulting stiffnesses are in N and N·mm².

The runner has no command-line arguments — the SG file and the materials are the **User Input** block at the top of the script, identical to the plate runner except for `n_model`.

```

import numpy as np
import jax.numpy as jnp

from opensg_solid.sg_homo import plate_homo_2d

n_model = 1                                # 1: Beam; 2: Plate; 3: 3D elastic
name = "RHC_SW_2UC_45"                     # reads <name>.yaml

```


id	material	E_1, E_2, E_3 (MPa)	G_{12}, G_{13}, G_{23} (MPa)	angle
1	ply	108000, 8000, 8000	4000, 4000, 3000	$+45^\circ$
2	ply	108000, 8000, 8000	4000, 4000, 3000	-45°
3	aluminium core	69000, 69000, 69000	26540, 26540, 26540	none

Table 5.21: The three materials of the honeycomb section; $\nu_{12}, \nu_{13}, \nu_{23}$ are 0.32, 0.32, 0.30 for the plies and 0.30 throughout for the core.

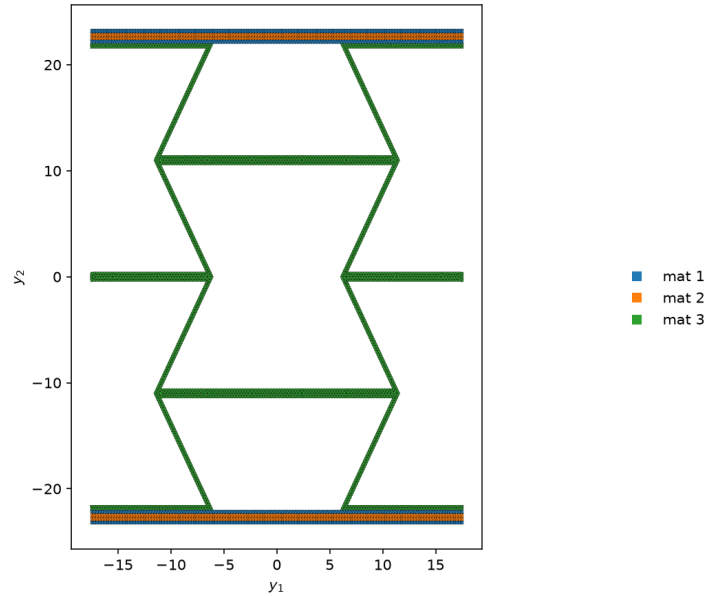


Figure 5.7: The mesh the beam route runs on, written by `sg_mesh.plot_sg_mesh` to `RHC_SW_2UC_45_mesh.png`. It is the same 4251-node, 6640-triangle mesh as Figure 5.3, because the plate and beam routes read the identical file; only `n_model` differs. What changes is the reading: with `n_model=1` the mesh is a *cross-section*, its two coordinates are y_2 and y_3 , and the beam axis y_1 is normal to the page — which is why the recovery of Section 5.8 reports its extreme fibre at $y_3 = 23.29375$ mm where the plate recovery of Section 5.4 reports the same point as y_2 .

```

material_param = jnp.array([
    (108e3, 8e3, 8e3, 4e3, 4e3, 3e3, 0.32, 0.32, 0.30),
    (108e3, 8e3, 8e3, 4e3, 4e3, 3e3, 0.32, 0.32, 0.30),
    (69e3, 69e3, 69e3, 26.54e3, 26.54e3, 26.54e3, 0.30, 0.30, 0.30)])
angles = jnp.array([45.0, -45.0, 0.0])

r = plate_homo_2d(name + ".yaml", material_param=material_param,
                  angles=angles, n_model=n_model)
np.set_printoptions(precision=5)
print("beam 6x6 [eps11 gam12 gam13 kappa1 kappa2 kappa3] (Timoshenko):")
print(r["C_eff"])
print("beam 4x4 [eps11 kappa1 kappa2 kappa3] (EB, same run):")
print(r["C_eff_EB"])

```

Listing 5.8: beam_homo_sg.py, complete.

5.7.3 Run it

The homogenization itself is one command; the yaml in this folder carries `n_model: 1` and `refined: 1` in its header, so it asks for the Timoshenko 6×6 without any argument:

```

cd examples/OpenSG-solid/7_get_beam_props_from_SG
opensg RHC_SW_2UC_45.yaml

```

`python beam_homo_sg.py` runs the same solve through the Python API and additionally prints the classical Euler-Bernoulli 4×4 that rides along in `r["C_eff_EB"]`, which the command-line route does not show.

5.7.4 What comes out

The run writes `RHC_SW_2UC_45.out`, the timed SwiftComp-layout report containing The Effective Timoshenko Beam Stiffness Matrix followed by The Effective Timoshenko Beam Compliance Matrix, with banner `OpenSG msg-solid beam model, omega 1` and closing line `Time taken: 8.94 sec`; and `RHC_SW_2UC_45.mesh.png`, the SG mesh coloured by material.

The banner reports $\omega = 1$: for a beam model on a 2-D SG the measure is unity, so the 6×6 is the **total sectional stiffness**, not a per-area density. No engineering-constants block is printed, because a beam stiffness is not a material stiffness.

5.7.5 What the numbers mean

The shipped `RHC_SW_2UC_45.out` stiffness block reads:

9.8992866E+006	6.9973096E+003	1.2577025E+001	-7.7065215E-001	2.2740495E+000	-8.6717860E-001
6.9973096E+003	1.9074524E+006	5.9324366E+001	-3.4044009E+001	-1.6298115E+000	-6.0670378E-002
1.2577025E+001	5.9324366E+001	9.3619028E+005	-8.4787043E+001	-9.3250231E-001	8.5948371E-001
-7.7065184E-001	-3.4044009E+001	-8.4787043E+001	2.9774561E+008	-3.1862083E+006	-7.3057137E+003
2.2740506E+000	-1.6298115E+000	-9.3250231E-001	-3.1862083E+006	2.1895745E+009	-9.5964309E+004
-8.6717875E-001	-6.0670378E-002	8.5948371E-001	-7.3057137E+003	-9.5964309E+004	9.2899273E+008

Read the diagonal only as a first orientation. When couplings are present the engineering stiffnesses are the inverses of the **compliance** diagonal, which is why the same `.out` prints the compliance: here $1/S_{11} = 9.89926 \times 10^6$ against $C_{11} = 9.89929 \times 10^6$ (0.0003 % apart) and $1/S_{44} = 2.97741 \times 10^8$ against $C_{44} = 2.97746 \times 10^8$ (0.0015 % apart), so on this section the couplings shift the engineering values by less than 0.002 %.

entry	conjugate pair	value	reads as
C_{11}	ϵ_{11}	9.8992866×10^6 N	axial (extensional) stiffness
C_{22}	γ_{12}	1.9074524×10^6 N	transverse shear stiffness, direction 2
C_{33}	γ_{13}	9.3619028×10^5 N	transverse shear stiffness, direction 3
C_{44}	κ_1	2.9774561×10^8 N·mm ²	torsional stiffness
C_{55}	κ_2	2.1895745×10^9 N·mm ²	bending stiffness about axis 2
C_{66}	κ_3	9.2899273×10^8 N·mm ²	bending stiffness about axis 3
C_{45}	$\kappa_1 - \kappa_2$	-3.1862083×10^6 N·mm ²	twist-bend coupling from the $\pm 45^\circ$ plies

Table 5.22: The Timoshenko 6×6 of the honeycomb section, entry by entry.

The twist-bend term C_{45} is the physically meaningful signature of the $\pm 45^\circ$ layup. Quantify it rather than eyeballing it: at 3.1862×10^6 N·mm² it is a factor 93 below C_{44} and a factor 690 below C_{55} — two to three orders down, not three.

5.7.6 The built-in cross-check: Euler-Bernoulli against Timoshenko

`r["C_eff_EB"]` is the classical 4×4 from the same V_0 solve, printed by `beam_homo_sg.py`. Its four entries must reproduce the classical rows of the Timoshenko 6×6 , since the two differ only by the l -chain refinement that produces the shear terms. On this SG they agree to at least five significant figures on every classical diagonal.

term	EB 4×4	Timoshenko 6×6
extension	9.89926177×10^6	9.89928657×10^6
torsion	2.97745363×10^8	2.97745606×10^8
bending 2	2.18957447×10^9	2.18957452×10^9
bending 3	9.28992811×10^8	9.28992729×10^8

Table 5.23: The two classical laws from one run. The Timoshenko column is `RHC_SW_2UC_45_beam_Timo.out`; the EB column is the same four numbers, recorded in `RHC_SW_2UC_45_beam_EB.out` in example 8, a file left over from an earlier version of that script.

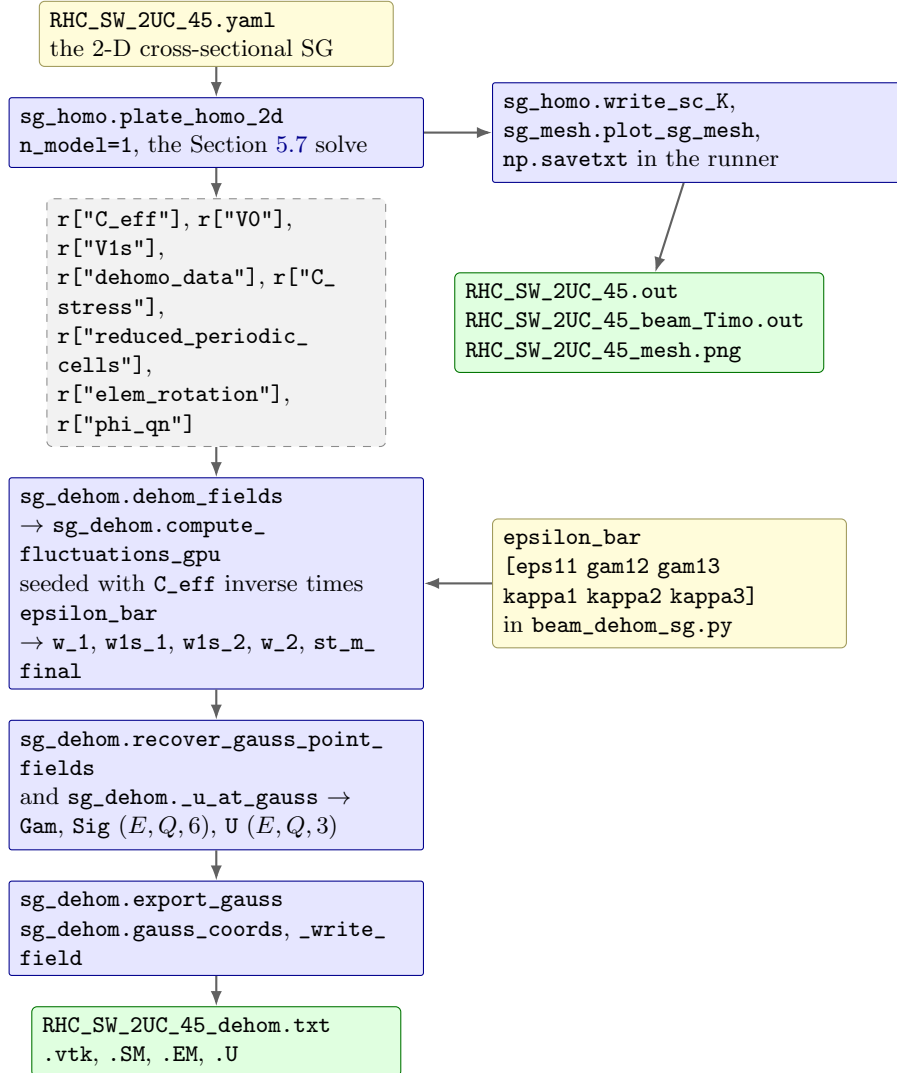
The two transverse shear stiffnesses C_{22} and C_{33} exist **only** in the Timoshenko 6×6 ; that is the entire reason to run `n_model=1` rather than read the EB block.

5.8 Beam recovery

5.8.1 What it computes

Dehomogenization takes a macro beam state and recovers the pointwise 3-D strain and stress inside the SG, at every element Gauss point. For `n_model=1` this runs the generalized-Timoshenko recovery ladder (`sg_dehom.compute_fluctuations_gpu`): a successive-derivative product-rule chain $\mathbf{F}' = \mathbf{R}\mathbf{F}$ seeded with $\mathbf{F}_{1d} = \mathbf{C}_{\text{eff}}^{-1}\bar{\epsilon}$, which builds the shear-corrected states and the zeroth- and first-order warping displacement fields. Nothing from the homogenization is recomputed: the result dictionary `r` carries the warping modes, the material tables and the quadrature data, so you homogenize once and recover as often as you like.

The tree shows exactly what rides along. Everything below the dashed data node is a contraction of quantities that already exist.



5.8.2 The input you edit

examples/OpenSG-solid/8_get_beam_dehom_from_SG runs the same honeycomb SG. Beyond the material block, the one new user input is `epsilon_bar`, the **macro beam strain** $[\epsilon_{11}, \gamma_{12}, \gamma_{13}, \kappa_1, \kappa_2, \kappa_3]$ — a strain six-vector, not a load vector. The shipped driver is $\epsilon_{11} = 0.001$ combined with $\kappa_2 = 0.01 \text{ mm}^{-1}$.

```

from opensg_solid.sg_homo import plate_homo_2d
from opensg_solid.sg_dehom import dehomo_fields, export_gauss

# the MACRO beam state [eps11, gam12, gam13, kappa1, kappa2, kappa3]
epsilon_bar = jnp.array([0.001, 0.0, 0.0, 0.0, 0.01, 0.0])

r = plate_homo_2d(name + ".yaml", material_param=material_param,
                  angles=angles, n_model=n_model)
np.savetxt(name + "_beam_Timo.out", r["C_eff"], fmt="%16.8e",

```

```

        header="beam Timoshenko 6x6 [eps11 gam12 gam13 kappa1 kappa2"
            " kappa3] (Beam_solid KKT engine) from the %dD SG %s"
            % (r["n_sg"], name))

Gam, Sig, U = dehom_fields(r, epsilon_bar)
export_gauss(r, Gam, Sig, name + "_dehom", U_eqd=U)
for i, nm in enumerate(("xx", "yy", "zz", "yz", "xz", "xy")):
    print("  max|Sig_%s| = %.5e" % (nm, np.abs(Sig[... , i]).max()))

```

Listing 5.9: The body of beam_dehom_sg.py after the shared material block.

Only **classical-channel** states are valid drivers: extension, twist, bending and their combinations. The recovery chain is kept verbatim from its validated reference, and its seed is the compliance-scaled state $\mathbf{C}_{\text{eff}}^{-1}\bar{\epsilon}$, which makes the two transverse shear entries `epsilon_bar[1]` and `epsilon_bar[2]` **recovery-inert** — a shear-only driver recovers essentially zero stress. This is a documented deviation from the refined theory stated in the docstring of `compute_fluctuations_gpu`, not a switch to flip.

5.8.3 Run it

The yml in this folder carries the same six numbers as its `epsilon_bar`: header key, so the whole chain — homogenize, then recover — is one command with the D argument:

```

cd examples/OpenSG-solid/8_get_beam_dehom_from_SG
opensg RHC_SW_2UC_45.yml D

```

`python beam_dehom_sg.py` is the same chain through the Python API, and additionally writes the bare 6×6 RHC_SW_2UC_45_beam_Timo.out through `np.savetxt`.

5.8.4 What comes out

`dehom_fields` returns three arrays shaped $(E, Q, \cdot)$ over elements and Gauss points: `Gam` $(E, Q, 6)$ strain, `Sig` $(E, Q, 6)$ stress, and `U` $(E, Q, 3)$ the **fluctuation-only** displacement (for the beam route, the zeroth- plus first-order warping $w_1 + w_{1s}$; there is no macro contribution in a unit-state SG recovery). `export_gauss` writes them out.

The `.txt/.vtk` pair uses the SwiftComp component order `xx yy zz yz xz xy`; the `.SM/.EM/.U` files reorder to 11 22 33 12 13 23 on write only. Units follow the SG input system throughout, MPa here.

5.8.5 What the numbers mean

The shipped table is the run of the driver above. Its component maxima are:

The axial field is the one to sanity-check first, because beam kinematics predict it directly: $\epsilon_{xx} \approx \bar{\epsilon}_{11} + \kappa_2 y_3$ plus a warping correction. The first row of the shipped `.txt` sits at $y_3 = 23.29375$ mm and reports $\epsilon_{xx} = 2.339375 \times 10^{-1}$, against $0.001 + 0.01 \times 23.29375 = 0.2339375$: an exact match, and the same value is the table maximum, as it must be at the extreme fibre.

The useful content of the recovery is everything *else*: the transverse and shear components, which exist purely because the cross-section is heterogeneous and warps, and which no beam theory can supply. The shipped driver is a demonstration magnitude, not a service load; scale it to your own macro state before reading the stresses as failure indices.

file	content
RHC_SW_2UC_45.out	the Timoshenko report written by <code>plate_homo_2d</code> itself, as in Section 5.7; regenerated on each run
RHC_SW_2UC_45_beam_Timo.out	the bare 6×6 written by the script through <code>np.savetxt</code> , one commented header line — convenient to <code>np.loadtxt</code>
RHC_SW_2UC_45_dehom.txt	tab-separated Gauss table: <code>GP_Index</code> , <code>Coord_X</code> , <code>Coord_Y</code> , six <code>Eps_*</code> , six <code>Sig_*</code> ; one header line plus 19 920 rows (6640 triangles times three quadrature points)
RHC_SW_2UC_45_dehom.vtk	the same cloud as an ASCII unstructured grid of vertex cells, each strain and stress component a <code>SCALARS</code> field — open in ParaView
RHC_SW_2UC_45_dehom.SM / .EM / .U	the OpenSG dehom files: stress, strain and fluctuation displacement in the <code>rm_plate_1D</code> layout, <code>x y z</code> columns then components printed in the order 11 22 33 12 13 23, with the SG coordinates in the last <code>n_sg</code> of the <code>x</code> , <code>y</code> , <code>z</code> slots; regenerated on each run
RHC_SW_2UC_45_mesh.png	the mesh figure

Table 5.24: Outputs of `beam_dehom_sg.py`.

component	$\max \sigma $ (MPa)	$\max \varepsilon $
<i>xx</i>	1.529334×10^4	2.339375×10^{-1}
<i>yy</i>	3.460157×10^3	2.265195×10^{-1}
<i>zz</i>	5.539976×10^2	8.787251×10^{-2}
<i>yz</i>	6.034080×10^2	1.344338×10^{-1}
<i>xz</i>	1.176515×10^3	1.704428×10^{-1}
<i>xy</i>	2.959409×10^3	1.028684×10^{-1}

Table 5.25: Component maxima over the 19 920 Gauss points of the shipped beam recovery.

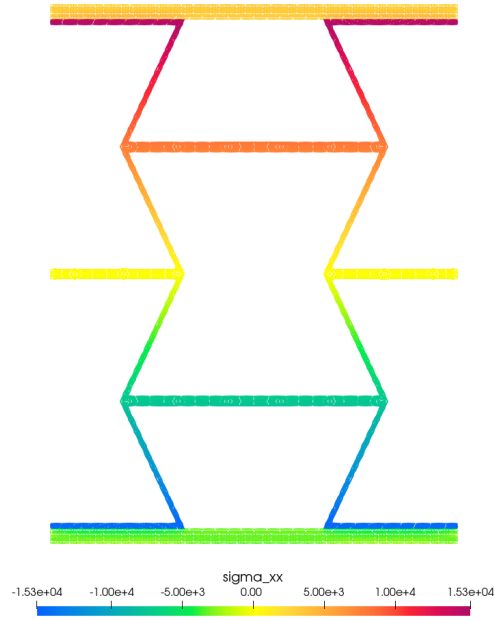


Figure 5.8: ParaView render of the recovered field on the SG mesh, produced from the `RHC_SW_2UC_45_dehom.vtk` written by this example, using `docs/latex/render_pv.py`. The array is `Sig_xx`, the axial stress σ_{xx} in MPa (units follow the SG input, millimetres and megapascals), on a symmetric colour scale of $\pm 1.53 \times 10^4$ MPa — the xx row of Table 5.25. Each of the 19920 points is one Gauss point carried by its own vertex cell and coloured flat, so nothing is averaged across an element boundary. The gene is the same honeycomb as Figure 5.4 and the pattern is again linear in the through-thickness coordinate, but the peak is an order of magnitude lower, tracking this driver’s extreme-fibre strain of 0.2339 against the 2.3294 of the plate recovery.

5.9 Files at a glance

Table 5.26: Every msg-solid capability in one table. `<name>` is the SG file’s own basename — the `name` variable where a runner still sets one — and `<ff stem>` the basename of the `.ff` table. `opensg` is the unified entry point; `opensg_solid` and `python -m opensg_solid` are the same run.

ex	you edit, then you run	it writes
1	layup_db.yaml; python 1d_sg.py	1dsg.yaml, 1dsg.png, layup_db_plate_homo.out
2	the USER INPUT block; python rm_dehom.py	1dsg_dehom.SM, .EM, .U, .vtk
2	a .ff table and the dehom: block; python rm_dehom_ff.py ex5_shell_S8R_field.ff	<ff stem>.ff.SM, .EM, .U, <ff stem>.ff_gauss.vtk
3	the yaml header (n_model, refined, analysis); opensg <name>.yaml	<name>.out, <name>.mesh.png
4	the above plus epsilon_bar;; opensg <name>.yaml D	<name>.out, <name>.dehom.txt, .vtk, .SM, .EM, .U, <name>.mesh.png; python plate_dehom_2dsg.py adds <name>.plate_ABD.out
5	name and the paper dictionary; python solid_homo_2dsg.py	<name>.Deff.out, <name>.constants_vs_paper.dat, <name>.mesh.png
6	the .sc path, E and ν ; python run_sample_1.py (the homogenization alone is opensg Sample_1.yaml)	Sample_1.out, Sample_1_msg_solid.out, Sample_1_compare.dat, Sample_1_mesh.png
6	nothing; python compare_boundary_modes.py	boundary_compare.dat, Sample_1_periodic.out, Sample_1_aperiodic.out and the Sample 2 pair
7	the yaml header (n_model: 1, refined: 1); opensg <name>.yaml	<name>.out, <name>.mesh.png
8	the above plus epsilon_bar;; opensg <name>.yaml D	<name>.out, <name>.dehom.txt, .vtk, .SM, .EM, .U, <name>.mesh.png; python beam_dehom_sg.py adds <name>.beam_Timo.out

Chapter 6

Full Blade Pipeline: windIO to BeamDyn and Back

A wind-turbine blade is designed as a three-dimensional laminated shell but analysed, aeroelastically, as a one-dimensional beam. This chapter walks the whole round trip on the IEA-22 blade: a windIO blade description becomes one 1-D shell structure gene per span station, each station is homogenized to a Timoshenko 6×6 , the spanwise table of those matrices is what a 1-D beam solver needs, the beam run returns sectional force and moment resultants station by station, and those resultants drive the dehomogenization back down to three-dimensional stress in the wall.

Three separate tools are involved, and the boundary between them is the single most important thing to understand before running anything. OpenSG never runs the blade dynamics. It produces the sectional constitutive law that the beam solver needs, and it consumes the sectional loads that the beam solver returns.

Figure 6.1 is the entire round trip drawn as one data flow: which file you edit, which function consumes it, what that function hands on, and which file it leaves behind. Read it once before running anything – the two dashed edges in the middle are the only place where the chain leaves OpenSG, and everything above and below them is reproducible from the shipped example directory. The rest of this chapter walks the same graph stage by stage, with a smaller tree at the head of each stage that opens up the box.

Table 6.1: The five stages of the round trip, and which tool owns each one.

Stage	What happens	Tool	Driver
1	windIO blade $\rightarrow$ per-station 1-D shell SG YAML	OpenSG'io	scripts/opensg_io.py
2	1-D shell SG $\rightarrow$ Timoshenko 6×6	OpenSG-2.0	1_homo_beam_props.py 4_sweep_51_stations.py
3	spanwise 6×6 table $\rightarrow$ blade dynamics	BeamDyn / OpenFAST	run by you
4	sectional loads returned as a table	BeamDyn / OpenFAST	ff51_rmc_reform.dat
5	loads + $6 \times 6 \rightarrow$ 3-D wall stress	OpenSG-2.0	2_dehom_v2_vabs.py 4_sweep_51_stations.py
	correctness check on transverse shear	OpenSG-2.0	3_q_consistency_check.py

6.1 The worked example

Everything in this chapter runs in the directory `examples/OpenSG_shell/IEA_blade_beam/`. Run the scripts from inside that directory: the package holds no path assumptions, and the example

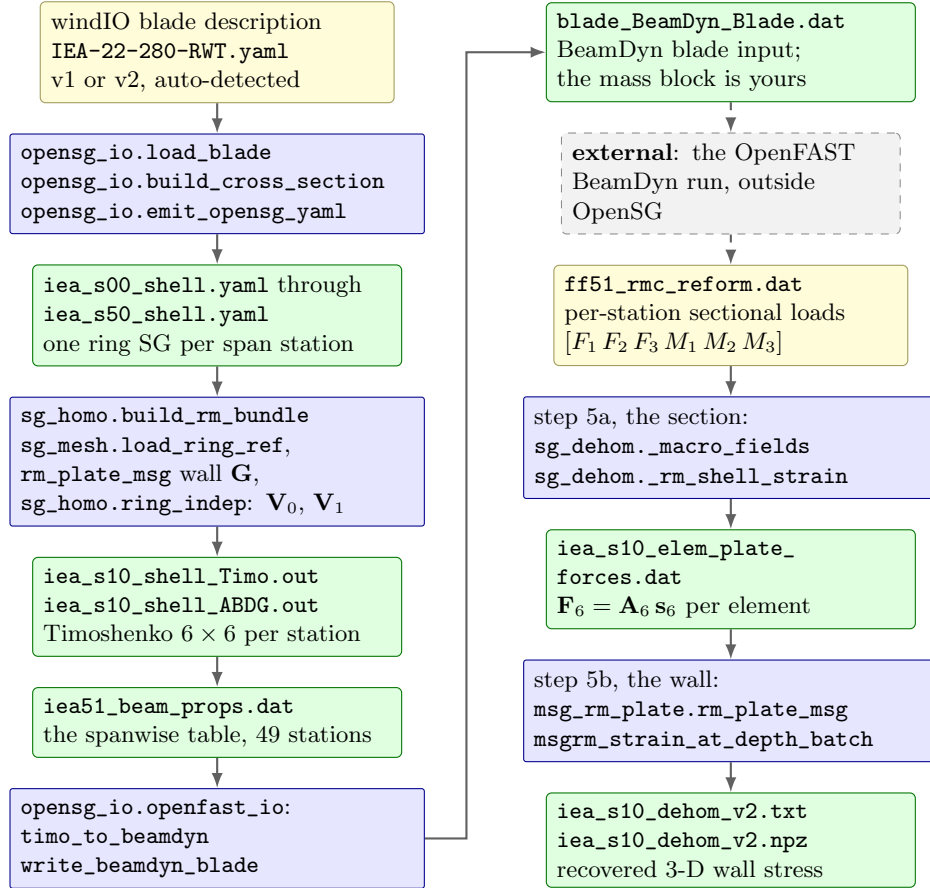


Figure 6.1: The whole blade round trip as one data flow. The left column is stages 1 to 3, the right column stages 3 to 5, and the two dashed edges are the OpenFAST BeamDyn run, which OpenSG neither contains nor launches. Yellow nodes are files you edit, blue nodes are functions that run, grey nodes are quantities that pass between them, and green nodes are files written to disk.

owns every path it uses.

The three single-station scripts all analyse the same section, `iea_s10`, the IEA-22 station at $r/R = 0.20$. Its 1-D shell SG, `iea_s10_shell.yaml`, is a ring of **310 wall elements** carrying **6 layups** (`layup_0` through `layup_5`) built from **6 materials** (`gelcoat`, `glass_triax`, `glass_uniax`, `medium_density_foam`, `carbon_uniax`, `glass_biax`). The element count is confirmed by the recovery output: `iea_s10_elem_plate_forces.dat` has two header lines and 310 data rows, one per ring element. The fourth script, `4_sweep_51_stations.py`, repeats the same computation for all 51 stations of the blade.

6.2 Stage 1: windIO blade to per-station 1-D shell SG YAMLS

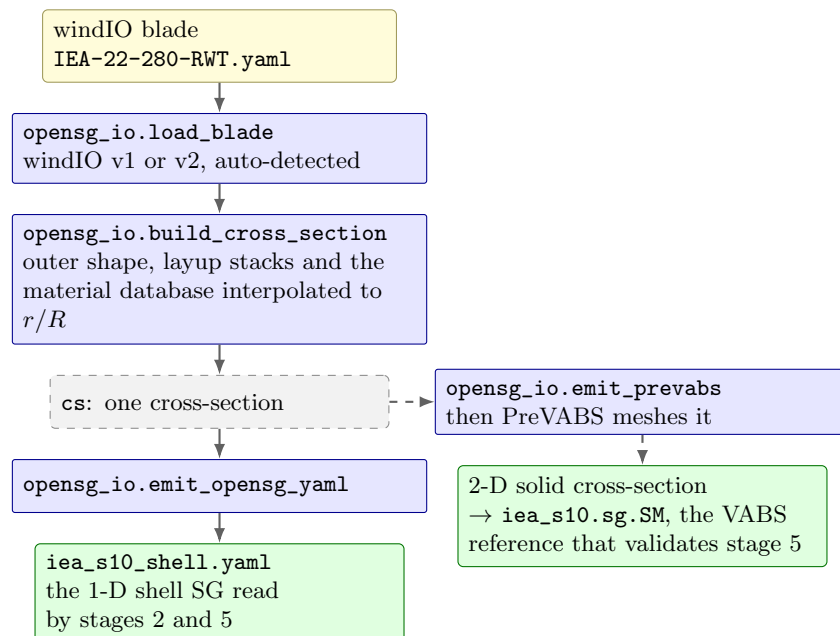
What it does

`OpenSG_io` is the input layer, and it is a *separate repository*: https://github.com/bagla0/OpenSG_io, documented at https://bagla0.github.io/OpenSG_io/. It reads a windIO blade description – auto-detecting windIO v1 and v2 – interpolates the outer shape, the layup stacks and the material database to a requested span fraction, and writes the cross-section in the two forms an MSG analysis can use:

- the **1-D shell SG YAML** that `opensg_shell` reads directly, which is what stages 2 and 5 of this chapter consume; and
- a **PreVABS XML** that meshes to a 2-D solid cross-section, which is the route that produces the independent VABS reference used to validate the recovery in stage 5.

It also reads PreVABS XML cross-sections and OpenFAST ElastoDyn/BeamDyn blade data, so published beam properties can be pulled in as a validation reference. Its own README is explicit that it does not run windIO or OpenFAST; it reads those formats so you do not have to hand-build the structure gene.

The four functions and the fork between the two output forms:



Installing it

```
git clone --recurse-submodules https://github.com/bagla0/OpenSG_io
cd OpenSG_io
pip install -r requirements.txt
python scripts/fetch_prevabs.py
```

The `requirements.txt` in that repository lists `numpy`, `scipy`, `pyyaml` and `meshio` as the core dependencies, plus `gmsh`, `tetgen` and `pyvista` for the mesh generators. A windIO *v2* input file additionally needs the windIO package installed; *v1* files do not. The `fetch_prevabs.py` script downloads the PreVABS release binary for your operating system, which is required only for the 2-D solid route – the 1-D shell YAML needed for stages 2 through 5 is written by pure Python.

The command line

One CLI handles any of the supported inputs. It writes a set of stations in a single call:

```
python scripts/opensg_io.py BAR_URC.yaml out --name bar --stations 0.3 0.5 0.7 --solid
```

Here `BAR_URC.yaml` is the windIO blade, `out` is the output directory, `--name bar` is the file-name stem, `--stations` takes the span fractions r/R , and `--solid` additionally emits the PreVABS XML and runs PreVABS to build the 2-D solid section. Omit `--solid` if you only want the 1-D shell YAMLs.

The Python API

The API is what you script a 51-station blade with, and it is what generated the station files used in the rest of this chapter:

```
from opensg_io import load_blade, build_cross_section, emit_opensg_yaml, emit_prevabs

blade = load_blade("IEA-22-280-RWT.yaml")      # windIO v1 or v2, auto-detected
cs = build_cross_section(blade, r=0.5)         # one span fraction
emit_opensg_yaml(cs, "shell_r050.yaml")        # the 1-D shell SG for opensg_shell
emit_prevabs(cs, "prevabs_r050", name="r050")  # PreVABS XML -> 2-D solid route
```

Listing 6.1: Building one station’s 1-D shell SG and its PreVABS twin.

Loop over your station list and you have the `iea_s00_shell.yaml` through `iea_s50_shell.yaml` directory that stage 2 points at.

What the 1-D shell SG YAML contains

`iea_s10_shell.yaml` is the reference for the format. Its top-level keys, in the order they appear in the file, are listed in Table 6.2.

A concrete section block from that file, the spar-cap laminate:

```
sections:
- type: shell
  elementSet: layup_0
  layup:
  - - gelcoat
    - 0.0005
    - 0.0
```

Table 6.2: Top-level keys of the 1-D shell SG YAML, in file order.

Key	Content
<code>nodes</code>	contour node coordinates [<code>y1</code> , <code>y2</code> , <code>y3</code>] in metres, one per line
<code>elements</code>	two-node line segments along the section contour
<code>sets</code>	<code>element</code> sets, one per layup, holding the element labels
<code>sections</code>	<code>type</code> : <code>shell</code> , <code>elementSet</code> : <code>layup_k</code> , and the <code>layup</code> as [<code>material</code> , <code>thickness</code> (m), <code>angle</code> (deg)] triples, outermost ply first
<code>elementOrientations</code>	one 3×3 frame per element, flattened; the first row is $\mathbf{e}_1$, out of the section plane, along the beam axis
<code>materials</code>	<code>density</code> in kg/m^3 and orthotropic <code>elastic</code> : { <code>E</code> , <code>G</code> , <code>nu</code> } triples in Pa
<code>reference</code>	which surface the section geometry refers to

```
- - glass_triax
- 0.00300294
- 0.0
- - glass_uniax
- 0.02609467
- 0.0
- - glass_triax
- 0.00300294
- 0.0
```

The `reference` field is load-bearing and is the single source of truth for the whole chapter. `build_rm_bundle` reads it when no override is passed, and it fixes the through-thickness offset `frac` used by the ring laminate law, the plate SG’s reference plane, the emitted ABD and the recovery depth conversion alike. The IEA stations all carry `reference`: `center`.

Table 6.3: The `reference` field and the through-thickness fraction it sets.

<code>reference</code>	<code>frac</code>	Meaning
<code>center</code>	0.5	mid-surface (what the IEA stations use)
<code>oml</code>	0.0	outer mould line
<code>oml_flip</code>	1.0	outer mould line, reversed stacking
<code>iml</code>	1.0	inner mould line

6.3 Stage 2: each station to a Timoshenko 6×6

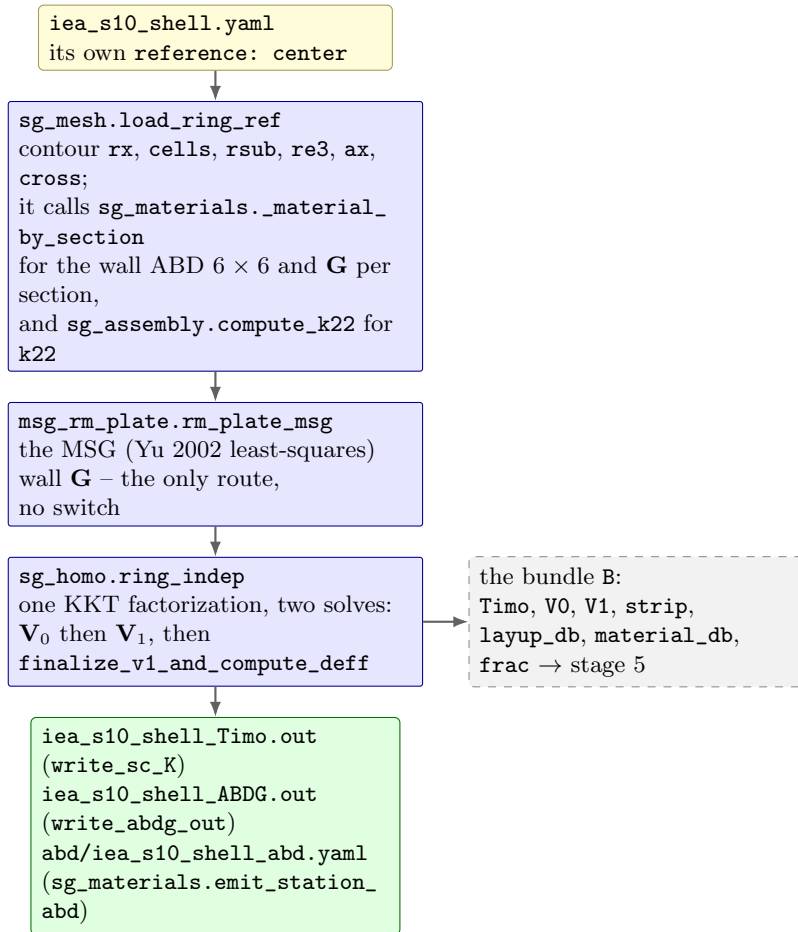
What it computes

The RM shell ring homogenization: a six-degree-of-freedom-per-node shell element with an element-wise drilling Lagrange multiplier, γ_{23} tied by MITC, and the MSG (Yu 2002 least-squares) wall transverse-shear law $\mathbf{G}$. The ring warping problem is solved for the four Euler–Bernoulli modes $\mathbf{V}_0$ and the Timoshenko correction $\mathbf{V}_1$, and the energy is condensed to the beam 6×6 in the VABS strain order

$$\boldsymbol{\varepsilon} = [\varepsilon_{11} \quad \gamma_{12} \quad \gamma_{13} \quad \kappa_1 \quad \kappa_2 \quad \kappa_3]^\top,$$

whose diagonal is $[EA \quad GA_2 \quad GA_3 \quad GJ \quad EI_2 \quad EI_3]$.

One call does all of it, but it is worth knowing what that call chains together, because every intermediate in the following tree is an entry of the returned bundle that stage 5 reads back:



What you edit

One line at the top of `1_homo_beam_props.py`:

```
##### User Input #####
name = "iea_s10"                # IEA-22 station at r/R = 0.20
#####
```

The command

```
python 1_homo_beam_props.py
```

The script is kept here because it does more than homogenize: it also writes the bare 6×6 `iea_s10_beam_Timo.out` and the ring mesh figure. The homogenization on its own is the unified command,

```
opensg iea_s10_shell.yaml
```

which dispatches on the yaml's `msg:` `shell` key — or, absent that key, on the mesh dialect — and hands the argument to `opensg_shell` unchanged. The macro model comes from the yaml

header, and *that* header is what decides which beam law you get: `refined: 1` asks for the Timoshenko 6×6 and `iea_s10_shell_Timo.out`, while the default `refined: 0` reports the classical Euler-Bernoulli 4×4 into `iea_s10_shell_EB.out`.

The API, if you drive it yourself

```
import numpy as np
from opensg_shell import build_rm_bundle

B = build_rm_bundle("iea_s10_shell.yaml")
C6 = np.asarray(B["Timo"])           # (6,6) Timoshenko stiffness
print(B["ref"], B["g_source"])       # 'center' 'msg'
```

Listing 6.2: The whole homogenization is one call.

The full signature is `build_rm_bundle(shell_yaml, ref=None, shear="mitc4_g23")`. Passing `ref=None` means the YAML's own `reference` field decides, which is why homogenization and dehomogenization cannot silently disagree; pass an explicit `ref` only to override.

There is no wall-**G** selector. The 2×2 transverse-shear block of every wall plate law is built by one route: the MSG/VAM second-order-energy (Yu 2002 least-squares) projection computed by `opensg_solid.rm_plate_1D.msg_rm_plate.rm_plate_msg`. The retired `g_source` argument is still *accepted* by `build_rm_bundle`, `build_solid_bundle`, `shell_sg3d` and `emit_station_abd` so that old call sites do not break, but an explicit value raises a `DeprecationWarning` and is ignored. The complementary-energy (Whitney 1973) shear-flow **G** survives only as the internal numerical fallback on a laminate whose MSG U^* fit is not positive definite; it is not a user-selectable alternative.

What comes back is a bundle, not just a matrix. It carries `Timo`, the warping modes `V0` and `V1`, the ring geometry (`corners`, `red_cells`, `rx3`, `re3`, `k22`, `ax`, `cross`, `strip`), `layup_per_elem`, the geometry-free `layup_db` and `material_db`, and `frac`, `ref` and `g_source` — the last a record, always `"msg"`, not a choice. Stage 5 consumes every one of those entries, so do not throw the bundle away after reading the matrix out of it.

What comes out

Two details about the ABD YAML are worth knowing before you reuse it. It is written by `emit_station_abd` with `ref="mid"` when the station is `center`-referenced, so its `reference: mid` field means the same surface as the ring's `center`. And it records `g_source: msg`, because there is only one wall-**G** route: the two transverse-shear rows of the cached 8×8 are the same MSG **G** the ring homogenization itself used, so the cache and the ring agree block for block and a recovery driven from the cache cannot silently disagree with the 6×6 . For `iea_s10` the file reports `n_layups: 6`, and for `layup_0` a total thickness of 0.03260055 m and a `mass_per_area` of 62.89 kg/m².

What the numbers mean

Table 6.5 is the content of `iea_s10_beam_Timo.out`, in SI units. The YAML is in metres and pascals, so EA , GA_2 and GA_3 are in N and GJ , EI_2 , EI_3 in N·m².

Read the matrix as physics rather than as six numbers on a diagonal. The extension–bending entries 1.67×10^9 and -2.08×10^9 say that the section's tension centre does not sit on the reference axis. The bending–bending coupling -5.56×10^9 couples flap and edge bending, so the principal bending axes are not the reference axes either. The shear–twist entries -1.02×10^8 and -1.62×10^8

Table 6.4: Files written by a single-station homogenization run.

File	Written by	Content
iea_s10_shell_Timo.out	build_rm_bundle	the timed SwiftComp .K layout: the banner OpenSG msg-shell beam model [ext sh2 sh3 twist bend2 bend3], The Effective Timoshenko Stiffness Matrix, its compliance, and a Time taken footer
iea_s10_shell_ABDG.out	build_rm_bundle	one block per section: The Effective Reissner- Mindlin Plate Stiffness Matrix, the 8×8 ABD plus $\mathbf{G}$, and its compliance
abd/iea_s10_shell_abd.yaml	build_rm_bundle	the per-station ABD YAML, emitted once and cached; carries <code>mass_per_area</code> per layup
iea_s10_beam_Timo.out	the script	the bare 6×6 as text, with the strain order and the reference recorded in the header
iea_s10_ring_mesh.png	the script	the real computed ring, elements coloured by layup

Table 6.5: The Timoshenko 6×6 of IEA-22 station `iea_s10` at $r/R = 0.20$, in the VABS strain order, from `iea_s10_beam_Timo.out`.

	ε_{11}	γ_{12}	γ_{13}	κ_1	κ_2	κ_3
ε_{11}	2.76606e+10	-1.72307e-05	4.66505e-06	5.40202e-06	1.66864e+09	-2.07730e+09
γ_{12}	-1.72307e-05	7.18613e+08	5.75204e+07	-1.01920e+08	2.30800e-06	5.27387e-06
γ_{13}	4.66505e-06	5.75204e+07	4.21744e+08	-1.62178e+08	8.48168e-06	2.79314e-07
κ_1	5.40202e-06	-1.01920e+08	-1.62178e+08	2.43757e+09	1.64496e-06	-8.11406e-06
κ_2	1.66864e+09	2.30800e-06	8.48168e-06	1.64496e-06	3.51439e+10	-5.55712e+09
κ_3	-2.07730e+09	5.27387e-06	2.79314e-07	-8.11406e-06	-5.55712e+09	6.81297e+10

locate the shear centre away from the reference axis as well. The entries printed at 10^{-5} against diagonals of 10^{10} are numerically zero: extension does not couple to transverse shear or to twist in this section, and the solver reproduces that null result to fifteen orders of magnitude, which is a useful sanity check that the ring assembled and condensed correctly.

The spanwise table: all 51 stations

`4_sweep_51_stations.py` repeats the homogenization for every station of the blade and, in the same loop, runs the first-order recovery of stage 5. What you edit is the block at the top:

```
##### User Input #####
yaml_dir = os.path.expanduser(
    "~/OpenSG-TW-claude/examples/data/iea_all_stations/shell51/1d_yaml")
ff_file = "ff51_rmc_reform.dat"
n_depth = 9 # through-thickness recovery points per element
stations = range(51) # s00 .. s50
#####
```

`yaml_dir` is where `iea_s00_shell.yaml` through `iea_s50_shell.yaml` live, which is the OpenSG'io output of stage 1. Point it at your own directory. The script builds each path as `iea_s%02d_shell.yaml` against that directory, and reads the load table `ff_file` from the current directory.

```
python 4_sweep_51_stations.py
```

A station whose YAML is missing prints `s%02d: yaml missing -- skipped` and is passed over, and any station that raises is caught and reported as `s%02d: FAILED` so that one bad section cannot kill a sweep that has already spent minutes on the others. In the shipped run, **49 of the 51 stations completed**: both `iea51_beam_props.dat` and `iea51_peak_stress.dat` carry $\eta = 0.00$ through 0.94 in steps of 0.02 plus $\eta = 1.00$, with $\eta = 0.96$ and $\eta = 0.98$ absent.

One indexing rule matters here. The sweep takes station i 's load vector as row i of the FF table, `FF = ffall[i, 1:]`, and its span fraction as `eta = ffall[i, 0]`. The row order of the load table must therefore match the station numbering of the YAML files. In the shipped data, station `s10` is row 10 and $\eta = 0.20$.

`iea51_beam_props.dat` holds one row per completed station: η followed by the six diagonal terms, written with the header IEA-22 spanwise Timoshenko beam properties (RM shell ring). Selected rows, verbatim from the file:

Table 6.6: Selected rows of `iea51_beam_props.dat`. Stiffnesses in N (EA , GA_2 , GA_3) and N·m² (GJ , EI_2 , EI_3).

$\eta = r/R$	EA	GA_2	GA_3	GJ	EI_2	EI_3
0.00	6.44275512e+10	7.96306621e+09	8.36135930e+09	1.27413308e+11	2.97295270e+11	2.19213722e+11
0.10	2.89212767e+10	1.11237808e+09	1.04545606e+09	1.14245218e+10	8.40070906e+10	1.21502270e+11
0.20	2.76605989e+10	7.18613262e+08	4.21744284e+08	2.43756534e+09	3.51438873e+10	6.81297238e+10
0.50	2.15596413e+10	5.34263266e+08	2.02128533e+08	4.95011412e+08	7.26524060e+09	1.82791086e+10
0.80	9.28038409e+09	3.90243799e+08	7.82870454e+07	6.27349440e+07	5.76401363e+08	2.07536066e+09
0.94	2.64020156e+09	1.80834082e+08	3.43479097e+07	1.35795150e+07	6.21667876e+07	4.27156667e+08
1.00	2.65581077e+08	1.56743220e+07	1.88135914e+06	5.15174420e+04	3.04364786e+05	1.60428557e+06

The $\eta = 0.20$ row is digit-for-digit the diagonal of the standalone `iea_s10` run of Table 6.5: EA is `2.76605989e+10` in both, GJ is `2.43756534e+09` in both, and so on for all six. The sweep and

the single-station script are the same computation, which is the cheapest available check that your sweep is wired correctly – run `1_homo_beam_props.py` on one station and match its row.

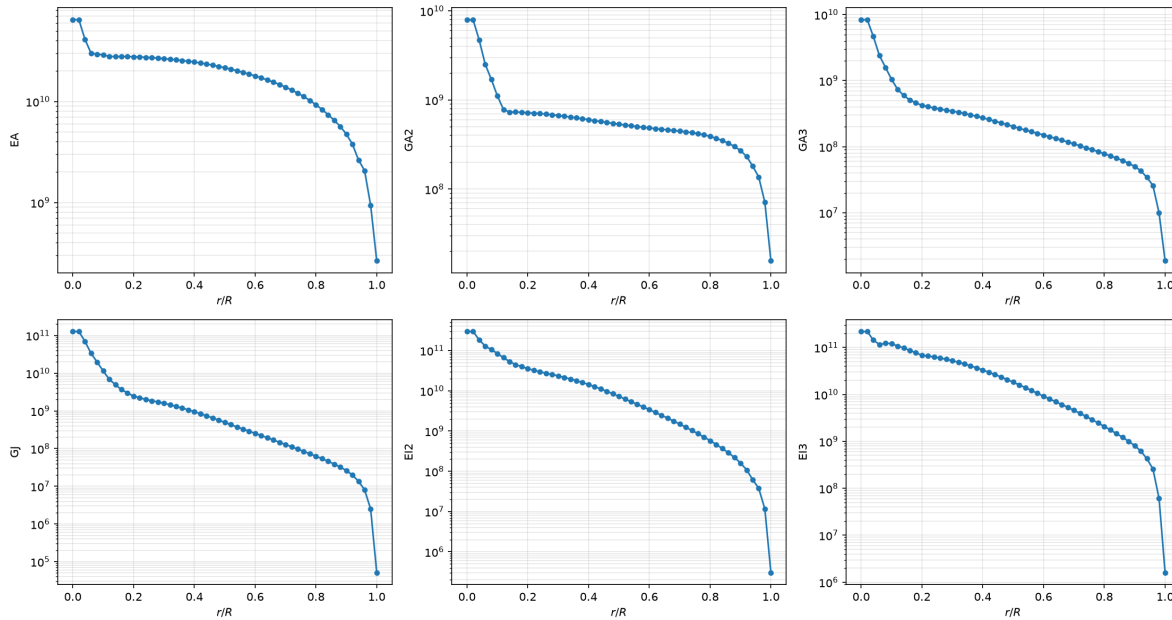


Figure 6.2: Spanwise Timoshenko diagonal of the IEA-22 blade: the six terms EA , GA_2 , GA_3 , GJ , EI_2 , EI_3 on log axes against r/R , one panel each, produced by `4_sweep_51_stations.py`.

Figure 6.2 plots those six columns. Three regimes are visible. The root collapses fast as the circular root transitions into an aerofoil: GJ falls from 1.274×10^{11} at $\eta = 0$ to 1.142×10^{10} N·m² at $\eta = 0.10$, an order of magnitude in the first tenth of span, and GA_2 falls from 7.96×10^9 to 1.11×10^9 N over the same interval. The mid-span then decays smoothly as the section thins. The tip drops off a cliff. Across the whole blade GJ goes from 1.27×10^{11} to 5.15×10^4 N·m², six and a half decades, which is why the figure is drawn on log axes and why a beam solver must be given the table rather than an average.

6.4 Stage 3: handing the table to BeamDyn

Be explicit about what changes hands here. **OpenSG produces the sectional 6×6 stiffness matrix, and nothing else.** BeamDyn, the geometrically exact beam module of OpenFAST, is a separate program that you install, configure and run yourself. OpenSG-2.0 ships no BeamDyn input file and no BeamDyn results, and this chapter does not attempt to document BeamDyn’s input format: for the meaning, ordering and units of every field in a BeamDyn blade input file, and for the choice of quadrature and the number of input stations, consult the OpenFAST BeamDyn documentation, which is the authority on that file.

The bridge that OpenSG’io provides

OpenSG_io contains an `openfast_io` module whose job is exactly this hand-off. Two functions write the OpenFAST side:

```
import numpy as np
from opensg_io.openfast_io import timo_to_beamdyn, write_beamdyn_blade
```

```
# etas: (n,) span fractions; Ks: list of n OpenSG Timoshenko 6x6 matrices
Kbd = timo_to_beamdyn(Ks[0])          # one matrix, reordered
write_beamdyn_blade(etas, Ks, M_list=None,
                    out_path="blade_BeamDyn_Blade.dat")
```

Listing 6.3: Writing a BeamDyn blade input from a spanwise table of OpenSG 6x6 matrices.

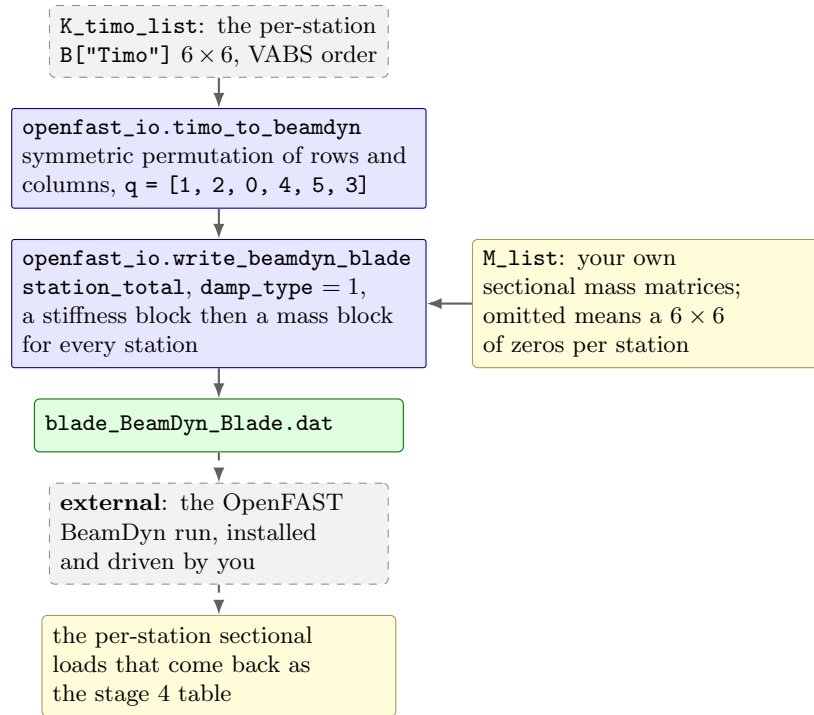
`timo_to_beamdyn(Ktimo)` reorders an OpenSG Timoshenko 6×6 , which is stored in the VABS order $[EA, GA_2, GA_3, GJ, EI_2, EI_3]$, into the BeamDyn degree-of-freedom order. It applies a symmetric permutation `Ktimo[np.ix_(q, q)]` with `q = [1, 2, 0, 4, 5, 3]`, which is the mapping in Table 6.7. Because the permutation is applied to both rows and columns, every off-diagonal coupling is carried across intact.

Table 6.7: The reordering performed by `timo_to_beamdyn`: which OpenSG diagonal term lands in which BeamDyn slot.

BeamDyn slot	Meaning	From OpenSG term
1	shear`x	GA_2
2	shear`y	GA_3
3	axial	EA
4	bend`x	EI_2
5	bend`y	EI_3
6	torsion	GJ

`write_beamdyn_blade(etas, K_timo_list, M_list=None, out_path="blade_BeamDyn_Blade.dat", damp=(0.003, 0.002, 0.002, 0.002, 0.003, 0.002))` takes the span fractions and the list of OpenSG matrices, reorders each one internally with `timo_to_beamdyn`, and writes a BeamDyn blade input with a `station_total` equal to the number of entries you supplied, `damp_type` set to 1, the six damping coefficients from `damp`, and then a stiffness block followed by a mass block for each station. Verify the resulting file against the BeamDyn documentation for the OpenFAST version you are running before trusting it.

Drawn as a data flow, with the two dashed edges marking the part OpenSG does not perform:



The mass matrix is yours

`write_beamdyn_blade` takes `M_list` as an *optional* argument and **writes a 6×6 block of zeros for every station when it is omitted**. That is a placeholder, not a mass model. Nothing in this pipeline computes a sectional mass matrix: the only mass quantity OpenSG-2.0 produces here is `mass_per_area` per laminate in the emitted ABD YAML (Section 6.3). Building the sectional inertia matrix, assembling the complete BeamDyn input, and running the aeroelastic simulation are all yours, outside OpenSG.

Use the full matrix, not the table's diagonal

`iea51_beam_props.dat` records only the six diagonal terms, for plotting and inspection. The couplings displayed in Table 6.5 are precisely what a composite blade beam model exists to carry, so feed the beam solver the full 6×6 of each station – either from that station's `<station>_Timo.out` or, more conveniently, straight from `B["Timo"]` in your own driver script, collected into the `K_timo_list` above.

Reading OpenFAST data back as a reference

The same module reads in the other direction, which is how you validate an OpenSG table against a published blade. `read_beamdyn_blade(path)` parses a BeamDyn blade input and returns (`etas`, `K_list`, `M_list`) with each matrix in BeamDyn order; `beamdyn_to_timo(Kbd)` reorders one back into the OpenSG order so it can be compared term by term. `read_elastodyn_blade(path)` parses an ElastoDyn blade file and returns arrays of `B1Fract`, `PitchAxis`, `StrcTwst` in degrees, `BMassDen` in kg/m, and `FlpStff` and `EdgStff` in N·m², which map to OpenSG's EI_2 (flap) and EI_3 (edge); `elastodyn_at(ed, r)` interpolates those to any span fraction.

6.5 Stage 4: the loads come back as the FF table

A BeamDyn or GEBT run returns, at every station, the sectional force and moment resultants the blade actually carries in a given load case. That table is the input to recovery. `ff51_rmc_reform.dat` is such a table. It has one comment header line and 51 data rows, one per station. The header and the first data rows, verbatim:

```
# eta F1 F2 F3 M1 M2 M3 (RM CENTER-ref BeamDyn, VABS order)
0.00000000e+00 -6.44900592e-06 -2.39422988e-08 2.23607775e+06 1.21251850e+06 -1.12030544e+08 -4.11329552e-07
2.00000000e-02 2.07495361e+03 7.56198425e+01 2.15497000e+06 1.21427150e+06 -1.05899552e+08 2.54692651e+03
4.00000000e-02 4.74173926e+03 1.68900665e+02 2.07209938e+06 1.23365012e+06 -9.99422000e+07 5.32342041e+03
```

Column 1 is $\eta = r/R$, running from 0.00 to 1.00 in steps of 0.02. Columns 2 through 4 are the forces F_1 , F_2 , F_3 in N; columns 5 through 7 are the moments M_1 , M_2 , M_3 in N·m. The row used by the single-station scripts is row 10:

```
2.00000000e-01 4.19268867e+04 8.26739746e+03 1.46733512e+06 1.21914500e+06 -6.01849720e+07 3.04264594e+05
```

The frame and the component order stated in that header are not decoration. The resultants must be expressed about the same reference axis and in the same component order as the 6×6 that produced them, or the round trip does not close and the recovered stress is silently wrong. Here both are *RM center-ref* and VABS order, matching **reference: center** in the station YAMLS. Note also the $\eta = 0$ row: its axial and in-plane entries print as -6.4×10^{-6} and -2.4×10^{-8} N, a numerically zero root row rather than a physical load.

A converter turns the table into an annotated YAML and pulls one station out of it:

```
from opensg_shell.helper.ff_to_yaml import convert, station_ff

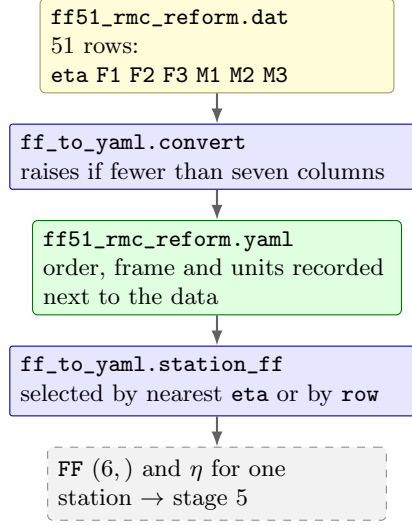
d = convert("ff51_rmc_reform.dat") # writes ff51_rmc_reform.yaml
FF, eta = station_ff("ff51_rmc_reform.yaml", eta=0.20) # (6,) resultants, and 0.2
```

Listing 6.4: `ff51_rmc_reform.yaml`: the FF table as a self-describing file.

`convert(dat_path, out_base=None, frame="RM center-ref")` reads the seven-column table, raising if it has fewer than seven columns, and writes `<base>.yaml`. `station_ff(yaml_path, eta=None, row=None)` returns the (6,) resultant vector and the η of one station, selected either by nearest η or by row index. The resulting `ff51_rmc_reform.yaml` records the convention explicitly next to the data, which is what makes the file readable months later:

```
source: ff51_rmc_reform.dat
convention:
  order: [F1, F2, F3, M1, M2, M3]
  frame: RM center-ref
  units: {force: N, moment: N.m}
stations:
- row: 0
  eta: 0.0
  FF: [-6.44900592e-06, -2.39422988e-08, 2236077.75, 1212518.5, -112030544.0, -4.11329552e-07]
- row: 10
  eta: 0.2
  FF: [41926.8867, 8267.39746, 1467335.12, 1219145.0, -60184972.0, 304264.594]
```

The whole of stage 4 is therefore a short chain, and the only thing it decides is which row of the table stage 5 will use:



6.6 Stage 5: dehomogenization back to 3-D wall stress

What it computes

A two-step chain, because a blade wall is a laminate inside a section:

$$\text{beam } \mathbf{FF} \xrightarrow{\text{RM shell ring}} \text{per-element plate forces} \xrightarrow{\text{RM plate SG}} \boldsymbol{\sigma}(z) \text{ through the wall.}$$

Step 5a, the section (`opensg_shell`). The macro beam strain is $\bar{\boldsymbol{\epsilon}} = \mathbf{C}_6^{-1} \mathbf{FF}$; `_macro_fields(B, beam_force_vabs=FF)` solves it and recombines the nodal RM warping fields $\mathbf{w} = \mathbf{V}_0 \bar{\boldsymbol{\epsilon}} + \mathbf{V}_1 \boldsymbol{\epsilon}'$ and their derivatives, and `_rm_shell_strain(B, e, xi, st_m, aA, aB)` evaluates, for ring element $\mathbf{e}$ at arc coordinate $\mathbf{xi}$, the six shell strains

$$\mathbf{s}_6 = \begin{bmatrix} \varepsilon_{11} & \varepsilon_{22} & 2\varepsilon_{12} & \kappa_{11} & \kappa_{22} & 2\kappa_{12} \end{bmatrix}.$$

That wall's own MSG ABD turns them into the **plate reaction forces**

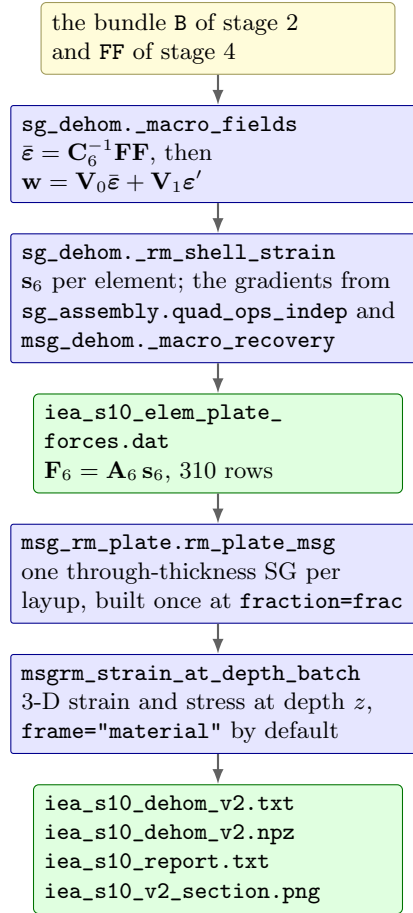
$$\mathbf{F}_6 = \mathbf{A}_6 \mathbf{s}_6 = \begin{bmatrix} N_{11} & N_{22} & N_{12} & M_{11} & M_{22} & M_{12} \end{bmatrix},$$

the interface quantity handed down to the plate SG.

Step 5b, the wall (`opensg_solid.rm_plate_1D`). `rm_plate_msg(thick, angles_deg, mat_names, material_db, fraction=frac)` builds the through-thickness MSG-RM structure gene for each layup once, and `msgrm_strain_at_depth` – or its vectorized twin `msgrm_strain_at_depth_batch`, which evaluates many points of one laminate in a handful of einsums – returns the 3-D Voigt strain and stress at depth z . Both default to `frame="material"`, meaning the result is rotated into the ply material frame where failure criteria live; pass `frame="plate"` to stay in the wall frame.

Recovery needs strain *gradients*, and they come from the ring itself. The span gradient $\partial \mathbf{E} / \partial x_1$ comes from the macro recovery ladder, computed per element as `BDe @ st_c11 + BDh @ aB[g]` with the element operators from `quad_ops_indep`. The arc gradient $\partial \mathbf{E} / \partial x_2$ comes from nodal differencing of $\mathbf{s}_6$ between the two elements sharing a node – but only where those two elements carry the *same* layup, because a layup change is a physical jump and must never be differenced across. Passing the second gradients as well (`dE11, dE12, dE22`) selects the second-order (V2) recovery, the rung that can in principle carry the transverse pair σ_{33}, σ_{13} that first order cannot.

The two steps and the interface file between them:



What you edit

The user block at the top of `2_dehom_v2_vabs.py`:

```
##### User Input #####
name = "iea_s10" # IEA-22 station at r/R = 0.20
ff_file = "ff51_rmc_reform.dat" # BeamDyn FF (RM center-ref, VABS order)
station_row = 10 # row 10 -> eta = 0.20
vabs_sm = "iea_s10.sg.SM" # the VABS 2-D solid reference cloud
jpath_file = "iea_s10.lp_sparcap_left_thickness.coords"
#####
```

```
python 2_dehom_v2_vabs.py
```

This one stays a script: beyond the recovery itself it projects the result onto the VABS gauss cloud, writes the validation report and draws four figures, none of which the solver does. The bare recovery of a single station is the unified command with the `D` argument, `opensg <station>_shell.yaml D`, which homogenizes and then recovers in one run and takes its macro state from the `<station>_shell.ff` file next to the `yaml` rather than from a row of the sweep table.

What comes out

The first two data rows of `iea_s10_elem_plate_forces.dat` show what the interface quantity looks like in practice:

Table 6.8: Files written by `2_dehom_v2_vabs.py`.

File	Content
<code>iea_s10_report.txt</code>	the validation report: the FF vector, the Timo diagonal, rms differences against VABS split web/skin, the V2 payload block, and the through-thickness path tables
<code>iea_s10_elem_plate_forces.dat</code>	310 rows, one per ring element: element index, y_2 y_3 , the six shell strains, and the six plate reaction forces N_{11}, N_{22}, N_{12} [N/m] and M_{11}, M_{22}, M_{12} [N]
<code>iea_s10_dehom_v2.txt</code>	166 302 rows, one per VABS gauss point: y_2 y_3 elem z , then the six recovered V2 stresses and the six VABS stresses, all MPa
<code>iea_s10_dehom_v2.npz</code>	the same fields plus <code>C6</code> , <code>FF</code> , <code>s6mid</code> , <code>F6mid</code> and every strain gradient, for re-plotting without re-solving
<code>iea_s10_v2_section.png</code>	section contours, VABS beside RM V2, for $\sigma_{11}, \sigma_{12}, \sigma_{13}, \sigma_{33}$
<code>iea_s10_parity.png</code>	parity clouds, first order and V2 against VABS
<code>iea_s10_thickness_paths.png</code>	through-thickness profiles on a spar-cap wall, a web, and the station's own reference path

```
# per-element PLATE REACTION FORCES from the RM shell beam model (Timo), eta=0.20
# elem  y2(m) y3(m)  e11 e22 2e12 k11 k22 2k12  N11 N22 N12(N/m) M11 M22 M12(N)
      0  4.677080e+00  1.048036e-01  5.426575e-04 -1.683049e-04  5.685206e-04 ...
```

What it was validated against

`iea_s10.sg.SM` is the VABS stress cloud of the *same* cross-section meshed as a 2-D solid, which is the independent reference: the recovery is projected onto its gauss points and differenced there, 166 302 points in all. Every point is assigned to its nearest ring element, a local arc coordinate and a signed wall depth, and the VABS columns are permuted into the (11, 22, 33, 23, 13, 12) order and converted from Pa to MPa before the comparison. Table 6.9 is copied from `iea_s10_report.txt`, with the VABS rms alongside so each difference has a scale.

Table 6.9: rms difference against VABS in MPa, web / skin, from `iea_s10_report.txt` at $\eta = 0.20$.

Component	first order (web / skin)	V2 (web / skin)	VABS rms (web / skin)
σ_{11}	43.073 / 9.529	43.073 / 9.529	44.440 / 96.676
σ_{22}	0.612 / 0.482	0.612 / 0.482	0.704 / 0.998
σ_{12}	6.799 / 1.255	6.798 / 1.255	22.026 / 6.009
σ_{33}	0.081 / 0.049	0.081 / 0.049	0.081 / 0.049
σ_{13}	0.394 / 0.192	0.394 / 0.192	0.394 / 0.193
σ_{23}	0.035 / 0.028	0.035 / 0.028	0.035 / 0.028

Read this honestly, because the table says three different things at once.

The in-plane skin stress is good. σ_{11} in the skin differs by 9.53 MPa rms against a VABS rms of 96.68 MPa, and σ_{12} by 1.26 against 6.01. The report also tabulates three through-thickness paths, and those sharpen the picture considerably. On the station's own reference thickness path, whose 15 points sit **0.00 mm** from the nearest VABS gauss point and are therefore exactly in the reference frame, the σ_{11} difference is **1.265 MPa against a VABS rms of 286.837 MPa**, better than half a percent. The two paths generated from the ring geometry sit 2.44 mm and 2.43 mm from the nearest VABS point, and their larger differences – 38.355 MPa rms on the spar cap, 2.299

MPa on the web – are dominated by that geometric offset rather than by the recovery itself. The printed nearest-VABS distance is the frame check: millimetres means the same frame, hundreds of millimetres means you are comparing against a foreign mesh.

The web is the weak spot. For σ_{11} in the web the difference, 43.07, is essentially the VABS rms itself, 44.44, which means the web axial stress is not being reproduced. The same pattern holds for the three transverse components everywhere: the difference equals the VABS rms to three digits, which is what you get when the recovered field is very nearly zero. This is the known transverse recovery gap of the shell-to-plate route, not a tuning problem.

The second-order rung is inert at this load state. The report's V2 payload block measures what V2 *changes*, rather than a rounded difference that could hide a small effect. At this station σ_{11} moves by 1.950×10^{-4} MPa rms against a first-order rms of 8.851×10^1 , σ_{12} by 5.685×10^{-3} against 1.068×10^1 , σ_{33} by 5.190×10^{-5} MPa against a VABS rms of 5.576×10^{-2} , and σ_{13} and σ_{23} by exactly 0.000×10^0 . For this constant-force blade state the V2 drivers carry no payload, and the first-order σ_{33} rms is 3.603×10^{-13} MPa, machine zero.

The correctness check on transverse shear

`3_q_consistency_check.py` tests an identity that any recovered transverse shear must satisfy, wall by wall: the thickness integral of the recovered stress must equal the wall's own transverse-shear resultant, which the RM shell already knows.

$$I_a = \int_h \sigma_{a3} dz \stackrel{!}{=} Q_a = (\mathbf{G}_{\text{msg}} \mathbf{s}_2)_a, \quad \mathbf{s}_2 = [2\gamma_{13} \quad 2\gamma_{23}].$$

The plate pipeline enforces this by construction – shape from the strain gradients, amplitude from the resultant, σ_{a3} rescaled by Q/I . The shell-to-plate chain never uses $\mathbf{s}_2$, so nothing pins the amplitude, and the ratio $r = I/Q$ measures how far off it is. Consistency means $r = 1$.

```
##### User Input #####
name = "iea_s10"
ff_file = "ff51_rmc_reform.dat"
station_row = 10          # eta = r/R = 0.20
n_z = 41                  # integration points through each wall
#####
```

```
python 3_q_consistency_check.py
```

The script writes `iea_s10_q_consistency.txt` and `iea_s10_q_consistency.png`, the latter plotting recovered $|I_{13}|$ against wall $|Q_1|$ on log axes with the consistency line, beside the histogram of $\log_{10}(I_{13}/Q_1)$. The measured result:

Table 6.10: Q-consistency ratios at $\eta = 0.20$, from `iea_s10_q_consistency.txt`.

Ratio	median	mean	min	max	Verdict
I_{13}/Q_1	5.228e−01	−6.974e+00	−1.299e+03	2.260e+01	target invalid
I_{23}/Q_2	9.895e−01	1.096e+00	−1.217e+01	1.194e+01	target valid

σ_{23} **passes**. The median I_{23}/Q_2 is 0.9895, a **1.05% Q-consistency error**, with a wall Q_2 rms of 8.9686 N/m against a recovered I_{23} rms of 8.3085 N/m.

σ_{13} **is not measurable in a cross-section model**, and the script explains why rather than papering over it. This ring is homogenized with `shear="mitc4_g23"`: γ_{23} is tied by MITC while

γ_{13} is raw and carries the flat-wall drilling mode, so a Q_1 built from it is not a physical wall shear and cannot serve as a recovery target. The fingerprint is in the magnitudes – Q_1 rms is 9.9064×10^2 N/m against a Q_2 rms of 8.9686 N/m, a factor of about 110, which is implausible for a thin-walled section that carries its load as in-plane shear flow. The ring is a one-quad-deep prismatic strip, so the γ_{13} tying interpolates along a direction in which the row is already linear and reproduces it exactly: the script measures `rms |Q1(g23) - Q1(both)| = 0.0000e+00` N/m and the same for Q_2 , so `mitc4_g23` and `mitc4_both` are bit-identical here even though both flags are plumbed and branch. A physical γ_{13} needs the surface-quad route, where both rows are genuinely tied, not a rescale of this ring against a contaminated Q_1 .

The practical rule that follows: trust σ_{11} , σ_{22} and σ_{12} from this pipeline, treat σ_{23} as good to about one percent in amplitude, and do not use the cross-section route’s σ_{13} or σ_{33} as a design quantity.

The same recovery, all 51 stations

`4_sweep_51_stations.py` runs the *first-order* recovery – it passes only the two first gradients to `msgmr_strain_at_depth_batch` – at `n_depth = 9` depths per wall element. The depths are mid-bin in the normalized thickness coordinate, $\zeta = (k + \frac{1}{2})/9$ for $k = 0 \dots 8$, converted to a physical depth by $z = (\zeta - \text{frac})h$ with h the total thickness of that element’s layup and `frac` taken from the bundle. The peak $|\sigma|$ per component over the whole section, in the ply material frame, goes to `iea51_peak_stress.dat`.

Table 6.11: Selected rows of `iea51_peak_stress.dat`: peak $|\sigma|$ per component over the whole section, ply frame, in MPa. Column order follows the file header.

$\eta = r/R$	σ_{11}	σ_{22}	σ_{33}	σ_{23}	σ_{13}	σ_{12}
0.00	1.49271417e+02	1.53636678e+00	4.13972884e-13	5.39458803e-03	4.12768841e-02	2.24186054e+00
0.16	3.25793641e+02	4.14543750e+00	8.97794962e-13	3.91335971e-02	3.51941097e-02	7.01052831e+00
0.20	3.18177433e+02	3.08795353e+00	9.12696123e-13	1.57149557e-02	3.45770096e-02	5.66031217e+00
0.54	2.14382707e+02	1.62953363e+00	8.13510269e-13	1.74810053e-02	8.23774625e-02	5.64234617e+01
0.80	1.11316749e+02	1.94984342e+00	1.93947926e-13	1.15167225e-02	5.58043894e-02	3.85749086e+01
1.00	4.88165692e-01	2.71026227e-02	2.03726813e-15	4.70632777e-05	5.13380478e-02	7.89278421e+00

Reading the whole file rather than these six rows: the peak axial stress of the entire blade in this load case is **325.79 MPa at $\eta = 0.16$** , and the peak in-plane shear is **56.42 MPa at $\eta = 0.54$** , where the shear webs take over from the caps. The σ_{33} column never leaves the range 2×10^{-15} to 2×10^{-12} MPa over all 49 stations – identically zero to round-off, the first-order limit noted above – so do not read it as a physical through-thickness stress. The σ_{13} column, which stays between 0.033 and 0.105 MPa over the whole blade, is subject to the same caveat as in the Q-consistency check and is not a design quantity either.

Figure 6.3 shows both halves. The left panel plots the spanwise peaks of σ_{11} and σ_{12} : the axial peak climbs steeply out of the root, crests near $\eta \approx 0.16$ and decays to almost nothing at the tip, while the shear peak rises in distinct steps rather than smoothly, near $\eta \approx 0.36$ and $\eta \approx 0.46$, where the peak jumps from 4.85 to 25.75 MPa and from 24.40 to 54.30 MPa respectively as a different wall takes over the shear. The right panel records the per-station wall time of the two halves separately, so you can see where a sweep’s cost actually sits before scaling it to more stations or more blades. The same arrays are saved to `iea51_sweep.npz` under the keys `eta`, `props`, `peak`, `t_homo`, `t_dehom` and `stations`, so a sweep never has to be repeated just to redraw a figure.

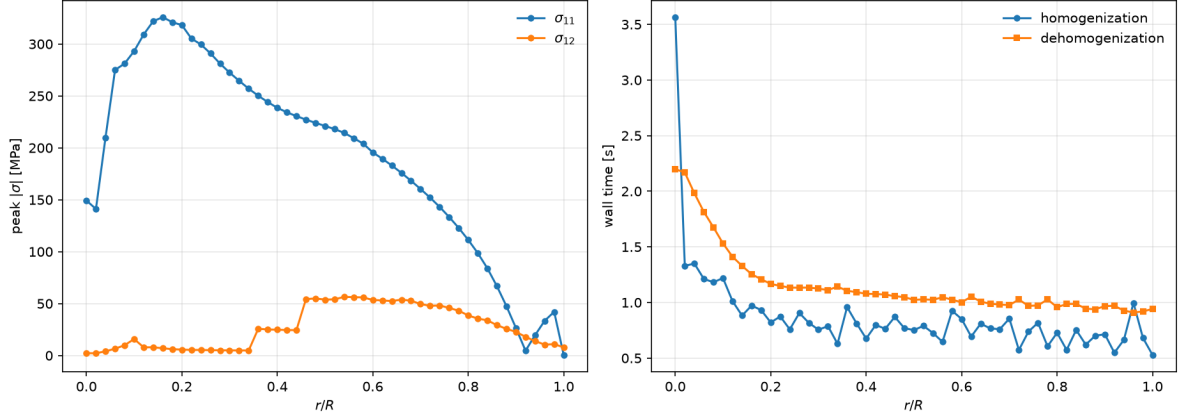


Figure 6.3: The two halves of the 51-station sweep. Left: the spanwise peaks of σ_{11} and σ_{12} against r/R . Right: the recorded per-station wall time, homogenization against dehomogenization.

6.7 Where each tool ends

Table 6.12: Who does what across the three tools in the round trip.

OpenSG'io	OpenSG-2.0	OpenFAST / BeamDyn
Reads windIO v1 and v2, PreVABS XML, and OpenFAST ElastoDyn/BeamDyn blade data.	Homogenizes each 1-D shell SG to the Timoshenko 6×6 and writes <code><station>_Timo.out</code> .	Runs the blade dynamics. Installed, configured and run by you, outside OpenSG.
Writes the per-station 1-D shell SG YAMLs that <code>opensg_shell</code> consumes.	Writes the RM wall law per section, <code><station>_ABDG.out</code> , and the ABD YAML with <code>mass_per_area</code> .	Returns the per-station sectional force and moment resultants that come back as the FF table.
Writes the PreVABS XML for the 2-D solid route, which supplies the independent VABS reference.	Recovers the 3-D wall stress from the FF table and validates it against that VABS cloud.	Owens the blade input file format; consult the BeamDyn documentation for it.
Bridges the hand-off with <code>timo_to_beamdyn</code> and <code>write_beamdyn_blade</code> , and reads OpenFAST data back as a validation reference.	Runs the Q-consistency diagnostic on the recovered transverse shear.	Sectional mass matrices are not produced by either OpenSG tool and remain your responsibility.

The chain closes only because the conventions match at every hand-off: the same reference surface (**center**), the same component order (VABS), and the same units (SI, metres and pascals in, newtons and newton-metres back). Those three lines are the first things to check when a recovered stress field looks wrong, and they are checkable without rerunning anything – the station YAML records its **reference**, the FF table records its frame and order in the header line, and the ABD YAML records the reference it was emitted at.